Context: Component Controller:

1. **HEADER_DATA**
   - LOAD_DATE TYPE BUDAT
   - NAME1 TYPE NAME1_GP
   - KUNNR TYPE KUNNR
2. **Z_GET_COMP_CAPACITY**
   - **IMPORTING**
     - **VEHICLE TYPE OIGCC-TU_NUMBER**
   - **EXPORTING**
     - **CAPACITY1 TYPE CHAR012**
     - **CAPACITY2 TYPE CHAR012**
     - **CAPACITY3 TYPE CHAR012**
     - **CAPACITY4 TYPE CHAR012**
     - **CAPACITY5 TYPE CHAR012**
     - **CAPACITY6 TYPE CHAR012**
     - **CAPACITY7 TYPE CHAR012**
     - **CAPACITY8 TYPE CHAR012**
   - **CHANGING**
     - **ITAB_OIGCC**
       - **CLIENT TYPE MANDT**
       - **TU_NUMBER TYPE OIG_TRUNMR**
       - **COM_NUMBER TYPE OIG_CMPNMR**
       - **SEQ_NMBR TYPE OIG_SEQNR**
       - **CMP_MINVOL TYPE OIG_CMPUVL**
       - **CMP_MAXVOL TYPE OIG_CMOMVL**
       - **LOAD_SEQ TYPE OIG_LOASEQ**
       - **GROUPNAME TYPE OIG_CPCG**
       - **DOUB_HULL TYPE OIG_HULLID**
       - **COM_IDTEXT TYPE OIG_CMPTXT**
       - **CRE_NAME TYPE ERNAM**
       - **CRE_DATE TYPE ERSDA**
       - **CHA_NAME TYPE AENAM**
       - **CHA_DATE TYPE LAEDA**

3. **ENABLE**
    - EN1 TYPE WDY_BOOLEAN
    - EN2 TYPE WDY_BOOLEAN
    - EN3 TYPE WDY_BOOLEAN
    - EN4 TYPE WDY_BOOLEAN
    - EN5 TYPE WDY_BOOLEAN
    - QN1 TYPE WDY_BOOLEAN
    - QN2 TYPE WDY_BOOLEAN
    - QN3 TYPE WDY_BOOLEAN
    - QN4 TYPE WDY_BOOLEAN
    - QN5 TYPE WDY_BOOLEAN
    - SAVE TYPE WDY_BOOLEAN
    - DELETE TYPE WDY_BOOLEAN
    - SAVE_ALL TYPE WDY_BOOLEAN
    - SUBMIT TYPE WDY_BOOLEAN
    - SUBMIT_ALL TYPE WDY_BOOLEAN
    - CONT TYPE WDY_BOOLEAN
    - EN6 TYPE WDY_BOOLEAN
    - EN7 TYPE WDY_BOOLEAN
    - EN8 TYPE WDY_BOOLEAN
    - QN6 TYPE WDY_BOOLEAN
    - QN7 TYPE WDY_BOOLEAN
    - QN8 TYPE WDY_BOOLEAN
4. **PROD1**
    - PRODUCT TYPE CHAR100
5. **PROD2**
    - PRODUCT TYPE CHAR100
6. **PROD3**
    - PRODUCT TYPE CHAR100
7. **PROD4**
    - PRODUCT TYPE CHAR100
8. **PROD5**
    - PRODUCT TYPE CHAR100

9. **Z_GET_CUSTOMER_INDEN**

- **IMPORTING_1**
    - VEHICLE TYPE OIG_VHLNMR
    - IND_DATE TYPE BEGDA
- **CHANGING_1**
    - **ITAB_INDENT**
        - MANDT TYPE MANDT
        - BEGDA TYPE BEGDA
        - ENDDA TYPE ENDDA
        - SHNUMBER TYPE TKNUM
        - VEHICLE TYPE OIG_VHLNMR
        - DEPOT TYPE WERKS_D
        - INDENT_DATE TYPE ZSD_CUST_INDENT-INDENT_DATE
        - INDENT_TIME TYPE TIMS
        - CUST_USER_ID TYPE ZSD_CUST_INDENT-CUST_USER_ID
        - KUNNR TYPE KUNWE
        - KUNNR_DESC TYPE CHAR30
        - SHTYPE TYPE OIG_TDSTYP
        - KONDM TYPE KONDM
        - PROD_CMP1 TYPE MATNR
        - VAL_COMP1 TYPE BWTAR_D
        - QUAN_COMP1 TYPE ZSD_CUST_INDENT-QUAN_COMP1
        - PROD_CMP2 TYPE MATNR
        - VAL_COMP2 TYPE BWTAR_D
        - QUAN_COMP2 TYPE ZSD_CUST_INDENT-QUAN_COMP2
        - PROD_CMP3 TYPE MATNR
        - VAL_COMP3 TYPE BWTAR_D
        - QUAN_COMP3 TYPE ZSD_CUST_INDENT-QUAN_COMP3
        - PROD_CMP4 TYPE MATNR
        - VAL_COMP4 TYPE BWTAR_D
        - QUAN_COMP4 TYPE ZSD_CUST_INDENT-QUAN_COMP4
        - PROD_CMP5 TYPE MATNR
        - VAL_COMP5 TYPE BWTAR_D
        - QUAN_COMP5 TYPE ZSD_CUST_INDENT-QUAN_COMP5
        - PROD_CMP6 TYPE MATNR

        - VAL_COMP6 TYPE BWTAR_D
        - QUAN_COMP6 TYPE ZSD_CUST_INDENT-QUAN_COMP6
        - PROD_CMP7 TYPE MATNR
        - VAL_COMP7 TYPE BWTAR_D
        - QUAN_COMP7 TYPE ZSD_CUST_INDENT-QUAN_COMP7
        - PROD_CMP8 TYPE MATNR
        - VAL_COMP8 TYPE BWTAR_D
        - QUAN_COMP8 TYPE ZSD_CUST_INDENT-QUAN_COMP8
        - SUMMARY TYPE ZSD_CUST_INDENT-SUMMARY
        - ZTT_STATUS TYPE ZTT_STATUS
        - ZTT_STATUS_DESC TYPE ZTT_STATUS_DESC
        - ATF_FLUSH TYPE CHAR1
        - CHK TYPE ZSD_CUST_INDENT-CHK
        - CONTRACT TYPE ZSD_CUST_INDENT-CONTRACT
        - TPT_GSTN TYPE ZSD_CUST_INDENT-TPT_GSTN
        - ERROR TYPE ZSD_CUST_INDENT-ERROR
        - ZDELETE TYPE ZSD_CUST_INDENT-ZDELETE
10. **ATF_FLUSH**
    - FLUSH TYPE CHAR1
    - DESC TYPE CHAR3
11. **EXPORT**
    - VBELN TYPE VBELN
12. **IND_MODE**
    - IND_TYPE TYPE ZIND_MODE
    - DEL_IND TYPE CHAR1
13. **Z_CHECK**
    - Z_ERROR TYPE CHAR1
14. **GSTN**
    - TPT_GSTN TYPE ZSD_CUST_INDENT-TPT_GSTN
15. **ENABLE1**
    - ET1 TYPE WDY_BOOLEAN
    - ET2 TYPE WDY_BOOLEAN
16. **FLUSH_REASON**
    - FLSH_REASON TYPE ZSD_CUST_INDENT-FLUSH_REASON

17. **ENABLE2**
    - ZT1 TYPE WDY_BOOLEAN
    - ZT2 TYPE WDY_BOOLEAN
    - ZT3 TYPE WDY_BOOLEAN DEFAULT VALUE 'X'
18. **MATERIAL1**
    - MATNR TYPE CHAR100
19. **MATERIAL2**
    - MATNR TYPE CHAR100
20. **WV_ENABLE**
    - VWEN1 TYPE WDY_BOOLEAN
    - VWEN2 TYPE WDY_BOOLEAN
    - VWEN3 TYPE WDY_BOOLEAN
    - VWEN4 TYPE WDY_BOOLEAN
    - VW_SV1 TYPE WDY_BOOLEAN
    - VW_SV2 TYPE WDY_BOOLEAN
    - VW_SV3 TYPE WDY_BOOLEAN
    - VW_SV4 TYPE WDY_BOOLEAN
    - CH_INDT TYPE WDY_BOOLEAN
21. **NO_VEH_CONTRACT**
    - VBELN1 TYPE VBELN
    - VBELN2 TYPE VBELN
22. **ZSD_INDENT_NON_VEH**
    - **IMPORTING_2**
        - INDENT_DATE TYPE ZSD_INDENT_NVEH-BEGDA
        - CREATED_BY TYPE ZSD_INDENT_NVEH-CUST_USER_ID
        - CUSTOMER_CODE TYPE ZSD_INDENT_NVEH-KUNNR
    - **CHANGING_2**
        - ORDER_NO TYPE ZSD_INDENT_NVEH-ORDER_NO
        - BEGDA TYPE ZSD_INDENT_NVEH-BEGDA
        - VEHICLE TYPE ZSD_INDENT_NVEH-VEHICLE
        - DEPOT TYPE ZSD_INDENT_NVEH-DEPOT
        - INDENT_DATE TYPE ZSD_INDENT_NVEH-INDENT_DATE
        - INDENT_TIME ZSD_INDENT_NVEH-INDENT_TIME
        - CUST_USER_ID TYPE ZSD_INDENT_NVEH-CUST_USER_ID

- KUNNR TYPE ZSD_INDENT_NVEH-KUNNR
- KUNNR_DESC TYPE ZSD_INDENT_NVEH-KUNNR_DESC
- MATNR1 TYPE ZSD_INDENT_NVEH-MATNR1
- QUANTITY1 TYPE ZSD_INDENT_NVEH-QUANTITY1
- MATNR2 TYPE ZSD_INDENT_NVEH-MATNR2
- QUANTITY2 TYPE ZSD_INDENT_NVEH-QUANTITY2
- CONTRACT1 TYPE ZSD_INDENT_NVEH-CONTRACT1
- CONTRACT2 TYPE ZSD_INDENT_NVEH-CONTRACT2
- CHK TYPE ZSD_INDENT_NVEH-CHK
- TPT_GSTN TYPE ZSD_INDENT_NVEH-TPT_GSTN
- ERROR TYPE ZSD_INDENT_NVEH-ERROR
- BL_QTY1 TYPE ZSD_INDENT_NVEH-BL_QTY1
- BL_QTY2 ZSD_INDENT_NVEH-BL_QTY2

23. **IND_DEL**
    - DEL_IND TYPE CHAR1
24. **PROD6**
    - PRODUCT TYPE CHAR100
25. **PROD7**
    - PRODUCT TYPE CHAR100
26. **PROD8**
    - PRODUCT TYPE CHAR100
27. **INBND_HEADER**
    - UNLD_DATE TYPE BUDAT
    - UNLD_VEHICLE TYPE CHAR012
    - UNLD_NAME TYPE NAME1_GP
    - UNLD_KUNNR TYPE KUNNR
    - UNLOAD_INV TYPE CHAR0016
    - UNLD_QTY TYPE CHAR012
    - UNLD_PLANT TYPE WERKS_D
    - UNLD_MATNR TYPE MATNR
    - UNLD_ACT1 TYPE WDY_BOOLEAN
    - UNLD_ACT2 TYPE WDY_BOOLEAN
    - UNLD_ACT3 TYPE WDY_BOOLEAN
    - UNLD_ACT4 TYPE WDY_BOOLEAN
    - UNLD_STOCK TYPE CHAR012

- UNLD_QTY_L15 TYPE CHAR012
- UNLD_QTY_KG TYPE CHAR012
- MOD_QTY_KL TYPE CHAR012
- MOD_QTY_KL15 TYPE CHAR012
- MOD_QTY_MT TYPE CHAR012
- UNLD_TDATE TYPE BUDAT
- UNLD_STOCK15 TYPE CHAR012
- UNLD_STOCKMT TYPE CHAR012
- MOD_VEHICLE TYPE CHAR012
- MOD_INV TYPE CHAR0016
- LN_HLD_KL TYPE CHAR012
- LN_HLD_KL15 TYPE CHAR012
- LN_HLD_MT TYPE CHAR012
- OPEN_INDT TYPE CHAR012
- BAL_QTY TYPE CHAR012

28. **ZINBOUND_INDENT_LIST**
    - **UNLD_IMPORTING**
        - UN_MATNR TYPE MATNR
    - **UNLD_CHANGING**
        - MANDT TYPE ZINB_INDENT_TAB-MANDT
        - IND_DATE TYPE ZINB_INDENT_TAB-IND_DATE
        - IND_MATNR TYPE ZINB_INDENT_TAB-IND_MATNR
        - IND_VEH TYPE ZINB_INDENT_TAB-IND_VEH
        - IND_QTY TYPE ZINB_INDENT_TAB-IND_QTY
        - IND_CUST TYPE ZINB_INDENT_TAB-IND_CUST
        - IND_PLANT TYPE ZINB_INDENT_TAB-IND_PLANT
        - IND_INV TYPE ZINB_INDENT_TAB-IND_INV
        - CRT_DATE TYPE ZINB_INDENT_TAB-CRT_DATE
        - CRT_USR TYPE ZINB_INDENT_TAB-CRT_USR
        - GR_NO TYPE ZINB_INDENT_TAB-GR_NO
        - CHK TYPE ZINB_INDENT_TAB-CHK
        - ZSTATUS TYPE ZINB_INDENT_TAB-ZSTATUS

29. **Z_GET_CUST_UNLOAD_IN**
    - **CHANGING_3**
        - **ZINDENT_DATA**
            - MANDT TYPE MANDT
            - IND_DATE TYPE DATUM
            - IND_MATNR TYPE MATNR
            - IND_VEH TYPE OIG_VHLNMR
            - IND_QTY TYPE LABST
            - IND_CUST TYPE KUNNR
            - IND_PLANT TYPE WERKS_D
            - IND_INV TYPE ZINB_INDENT_TAB-IND_INV
            - CRT_DATE TYPE DATUM
            - CRT_USR TYPE UNAME
            - GR_NO TYPE MBLNR
            - CHK TYPE ZINBINDENT_TAB-CHK
            - ZSTATUS TYPE ZINB_INDENT_TAB-ZSTATUS
30. **IND_CAT**
    - IND_CATEGORY TYPE CHAR100
31. **IND_END_USE**
    - IND_USE_MS TYPE CHAR100
    - IND_USE_HSD TYPE CHAR100

Attributes (Component Controller):

- WD_CONTEXT TYPE IF_WD_CONTEXT_NODE "Reference to Local Controller Context
- WD_THIS TYPE IF_COMPONENTCONTROLLER "Self-Reference to Local Controller Interface
- CUST_ID TYPE CHAR10 "Character Field Length = 10
- IND_DATE TYPE BEGDA "Start Date
- KUNNR TYPE KUNNR "Customer Number
- LV_PERNR TYPE XUBNAME ""
- VEHICLE TYPE OIG_VHLNMR "TD Vehicle Number

Method List(Component Controller):

- **METHOD EXECUTE_Z_GET_COMP_CAPACITY .**

```
* declarations for context navigation
 DATA LO_Z_GET_COMP_CAPACITY TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_CHANGING TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_ITAB_OIGCC TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_EXPORTING TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_IMPORTING TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_ELEMENT TYPE REF TO IF_WD_CONTEXT_ELEMENT.
 DATA LT_ELEMENTS TYPE WDR_CONTEXT_ELEMENT_SET.

* declarations for parameters
 DATA LV_CAPACITY1 TYPE CHAR012.

 DATA LV_CAPACITY2 TYPE CHAR012.

 DATA LV_CAPACITY3 TYPE CHAR012.

 DATA LV_CAPACITY4 TYPE CHAR012.

 DATA LV_CAPACITY5 TYPE CHAR012.

 DATA LV_CAPACITY6 TYPE CHAR012.

 DATA LV_CAPACITY7 TYPE CHAR012.

 DATA LV_CAPACITY8 TYPE CHAR012.

 DATA LV_VEHICLE TYPE OIGCC-TU_NUMBER.

 DATA LT_C_ITAB_OIGCC TYPE IF_COMPONENTCONTROLLER=>ELEMENTS_ITAB_OIGCC.
 DATA LS_C_ITAB_OIGCC LIKE LINE OF LT_C_ITAB_OIGCC.
 DATA LT_C_ITAB_OIGCC_CP TYPE IF_COMPONENTCONTROLLER=>ELEMENTS_ITAB_OIGCC.


* get all involved child nodes
 LO_Z_GET_COMP_CAPACITY = WD_CONTEXT->GET_CHILD_NODE( WD_THIS-
>WDCTX_Z_GET_COMP_CAPACITY ).
 LO_CHANGING = LO_Z_GET_COMP_CAPACITY->GET_CHILD_NODE( WD_THIS->WDCTX_CHANGING ).
 LO_ITAB_OIGCC = LO_CHANGING->GET_CHILD_NODE( WD_THIS->WDCTX_ITAB_OIGCC ).
 LO_EXPORTING = LO_Z_GET_COMP_CAPACITY->GET_CHILD_NODE( WD_THIS->WDCTX_EXPORTING ).
 LO_IMPORTING = LO_Z_GET_COMP_CAPACITY->GET_CHILD_NODE( WD_THIS->WDCTX_IMPORTING ).

* get input from context
 LO_IMPORTING->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `VEHICLE`
  IMPORTING
   VALUE = LV_VEHICLE ).
 LT_ELEMENTS = LO_ITAB_OIGCC->GET_ELEMENTS( ).
 LOOP AT LT_ELEMENTS[] INTO LO_ELEMENT.
  LO_ELEMENT->GET_STATIC_ATTRIBUTES( IMPORTING STATIC_ATTRIBUTES = LS_C_ITAB_OIGCC ).
  INSERT LS_C_ITAB_OIGCC INTO TABLE LT_C_ITAB_OIGCC[].
 ENDLOOP.

 LT_C_ITAB_OIGCC_CP = LT_C_ITAB_OIGCC[].


* the invocation - errors are always fatal !!!
```

```
 CALL FUNCTION 'Z_GET_COMP_CAPACITY'
  EXPORTING
    VEHICLE =                LV_VEHICLE
  IMPORTING
    CAPACITY1 =              LV_CAPACITY1
    CAPACITY2 =              LV_CAPACITY2
    CAPACITY3 =              LV_CAPACITY3
    CAPACITY4 =              LV_CAPACITY4
    CAPACITY5 =              LV_CAPACITY5
    CAPACITY6 =              LV_CAPACITY6
    CAPACITY7 =              LV_CAPACITY7
    CAPACITY8 =              LV_CAPACITY8
  TABLES
    ITAB_OIGCC =             LT_C_ITAB_OIGCC
 .

* store output to context
 LO_EXPORTING->SET_ATTRIBUTE(
  EXPORTING
    NAME  = `CAPACITY1`
    VALUE = LV_CAPACITY1 ).
 LO_EXPORTING->SET_ATTRIBUTE(
  EXPORTING
    NAME  = `CAPACITY2`
    VALUE = LV_CAPACITY2 ).
 LO_EXPORTING->SET_ATTRIBUTE(
  EXPORTING
    NAME  = `CAPACITY3`
    VALUE = LV_CAPACITY3 ).
 LO_EXPORTING->SET_ATTRIBUTE(
  EXPORTING
    NAME  = `CAPACITY4`
    VALUE = LV_CAPACITY4 ).
 LO_EXPORTING->SET_ATTRIBUTE(
  EXPORTING
    NAME  = `CAPACITY5`
    VALUE = LV_CAPACITY5 ).
 LO_EXPORTING->SET_ATTRIBUTE(
  EXPORTING
    NAME  = `CAPACITY6`
    VALUE = LV_CAPACITY6 ).
 LO_EXPORTING->SET_ATTRIBUTE(
  EXPORTING
    NAME  = `CAPACITY7`
    VALUE = LV_CAPACITY7 ).
 LO_EXPORTING->SET_ATTRIBUTE(
  EXPORTING
    NAME  = `CAPACITY8`
    VALUE = LV_CAPACITY8 ).
 IF LT_C_ITAB_OIGCC[] NE LT_C_ITAB_OIGCC_CP[].
  LO_ITAB_OIGCC->BIND_TABLE( LT_C_ITAB_OIGCC[] ).
 ENDIF.
 CLEAR LT_C_ITAB_OIGCC_CP[].
 CLEAR LT_C_ITAB_OIGCC[].

ENDMETHOD.
```

- **METHOD EXECUTE_Z_GET_CUSTOMER_INDENT .**

```
  DATA LO_ND_IND_DEL TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LO_EL_IND_DEL TYPE REF TO IF_WD_CONTEXT_ELEMENT.
  DATA LS_IND_DEL TYPE WD_THIS->ELEMENT_IND_DEL.
  DATA LV_DEL_IND TYPE WD_THIS->ELEMENT_IND_DEL-DEL_IND.
  LO_ND_IND_DEL = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_IND_DEL ).
  LO_EL_IND_DEL = LO_ND_IND_DEL->GET_ELEMENT( ).
  IF LO_EL_IND_DEL IS NOT INITIAL.
   LO_EL_IND_DEL->GET_ATTRIBUTE(
    EXPORTING
     NAME = `DEL_IND`
    IMPORTING
     VALUE = LV_DEL_IND ).
  ENDIF.


 DATA LO_ND_IND_MODE TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_EL_IND_MODE TYPE REF TO IF_WD_CONTEXT_ELEMENT.
 DATA LS_IND_MODE TYPE WD_THIS->ELEMENT_IND_MODE.
 DATA LV_IND_TYPE TYPE WD_THIS->ELEMENT_IND_MODE-IND_TYPE.
* DATA LV_DEL_IND TYPE WD_THIS->ELEMENT_IND_MODE-DEL_IND.
 LO_ND_IND_MODE = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_IND_MODE ).

 LO_EL_IND_MODE = LO_ND_IND_MODE->GET_ELEMENT( ).
 IF LO_EL_IND_MODE IS NOT INITIAL.
 LO_EL_IND_MODE->GET_ATTRIBUTE(
  EXPORTING
   NAME = `IND_TYPE`
  IMPORTING
   VALUE = LV_IND_TYPE ).
 ENDIF.



* declarations for context navigation
 DATA LO_Z_GET_CUSTOMER_INDEN TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_CHANGING_1 TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_ITAB_INDENT TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_IMPORTING_1 TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_ELEMENT TYPE REF TO IF_WD_CONTEXT_ELEMENT.
 DATA LT_ELEMENTS TYPE WDR_CONTEXT_ELEMENT_SET.

* declarations for parameters
 DATA LV_IND_DATE TYPE BEGDA.

 DATA LV_VEHICLE TYPE OIG_VHLNMR.

 DATA LT_C_ITAB_INDENT TYPE IF_COMPONENTCONTROLLER=>ELEMENTS_ITAB_INDENT.
 DATA LS_C_ITAB_INDENT LIKE LINE OF LT_C_ITAB_INDENT.
 DATA LT_C_ITAB_INDENT_CP TYPE IF_COMPONENTCONTROLLER=>ELEMENTS_ITAB_INDENT.


* get all involved child nodes
 LO_Z_GET_CUSTOMER_INDEN = WD_CONTEXT->GET_CHILD_NODE( WD_THIS-
>WDCTX_Z_GET_CUSTOMER_INDEN ).
 LO_CHANGING_1 = LO_Z_GET_CUSTOMER_INDEN->GET_CHILD_NODE( WD_THIS-
>WDCTX_CHANGING_1 ).
 LO_ITAB_INDENT = LO_CHANGING_1->GET_CHILD_NODE( WD_THIS->WDCTX_ITAB_INDENT ).
 LO_IMPORTING_1 = LO_Z_GET_CUSTOMER_INDEN->GET_CHILD_NODE( WD_THIS-
>WDCTX_IMPORTING_1 ).

* get input from context
 LO_IMPORTING_1->GET_ATTRIBUTE(
```

```
  EXPORTING
   NAME  = `IND_DATE`
  IMPORTING
   VALUE = LV_IND_DATE ).
 LO_IMPORTING_1->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `VEHICLE`
  IMPORTING
   VALUE = LV_VEHICLE ).
 LT_ELEMENTS = LO_ITAB_INDENT->GET_ELEMENTS( ).
 LOOP AT LT_ELEMENTS[] INTO LO_ELEMENT.
  LO_ELEMENT-
>GET_STATIC_ATTRIBUTES( IMPORTING STATIC_ATTRIBUTES = LS_C_ITAB_INDENT ).  "#EC CI_FLDEXT_OK[2
215424] " #S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
  INSERT LS_C_ITAB_INDENT INTO TABLE LT_C_ITAB_INDENT[].
 ENDLOOP.

 LT_C_ITAB_INDENT_CP = LT_C_ITAB_INDENT[].

*** Get Loading Date from Header START ***
  DATA LO_ND_HEADER_DATA TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LO_EL_HEADER_DATA TYPE REF TO IF_WD_CONTEXT_ELEMENT.
  DATA LS_HEADER_DATA TYPE WD_THIS->ELEMENT_HEADER_DATA.
  DATA LV_LOAD_DATE TYPE WD_THIS->ELEMENT_HEADER_DATA-LOAD_DATE.
  LO_ND_HEADER_DATA = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS-
>WDCTX_HEADER_DATA ).
  LO_EL_HEADER_DATA = LO_ND_HEADER_DATA->GET_ELEMENT( ).

  IF LO_EL_HEADER_DATA IS NOT INITIAL.
   LO_EL_HEADER_DATA->GET_ATTRIBUTE(
   EXPORTING
    NAME =  `LOAD_DATE`
   IMPORTING
    VALUE = LV_LOAD_DATE ).
  ENDIF.

  LV_IND_DATE = LV_LOAD_DATE.


*** Get Loading Date from Header END ***



* the invocation - errors are always fatal !!!
 CALL FUNCTION 'Z_GET_CUSTOMER_INDENT'
  EXPORTING
    VEHICLE    =  LV_VEHICLE
    IND_DATE   =  LV_IND_DATE
    ZUNAME     =  SY-UNAME
  TABLES
    ITAB_INDENT =   LT_C_ITAB_INDENT.


 DATA: ZIND_TYP TYPE ZSD_CUST_INDENT.
 DATA ZTABCUST TYPE TABLE OF ZSD_CUST_USR_MAP.
 DATA ZTABCUST1 LIKE LINE OF ZTABCUST.
 DATA: ZKDGRP TYPE KNVV-KDGRP,
     ZKNVV TYPE TABLE OF KNVV,
     ITKNVV LIKE LINE OF ZKNVV.
 DATA: ZKNVP TYPE TABLE OF KNVP,
     ZKNVP1 TYPE TABLE OF KNVP,
     ITKNVP LIKE LINE OF ZKNVP.
```

```
 SELECT * FROM ZSD_CUST_USR_MAP INTO CORRESPONDING FIELDS OF TABLE ZTABCUST
 WHERE CUST_USER_ID = SY-UNAME.

 LOOP AT ZTABCUST INTO ZTABCUST1.
  SELECT SINGLE KDGRP FROM KNVV INTO ZKDGRP WHERE KUNNR = ZTABCUST1-
KUNNR.  "#EC CI_NOORDER " #S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
 IF ZKDGRP <> 'DI' AND ZKDGRP <> 'RE' AND ZKDGRP <> 'TG' AND ZKDGRP <> 'OI'
   AND ZKDGRP <> 'EX'.

   SELECT * FROM KNVV INTO TABLE ZKNVV WHERE KDGRP = ZKDGRP AND SPART <> '11'
    AND KDGRP <> 'DI' AND KDGRP <> 'TG' AND KDGRP <> 'OI' AND KDGRP <> 'RE'.
   SORT ZKNVV BY KUNNR.

  ELSE.
   SELECT * FROM KNVV INTO TABLE ZKNVV WHERE KUNNR = ZTABCUST1-KUNNR AND SPART <> '11'.
   SORT ZKNVV BY KUNNR.
  ENDIF.

  DELETE ADJACENT DUPLICATES FROM ZKNVV COMPARING KUNNR.

  LOOP AT ZKNVV INTO ITKNVV.
    SELECT * FROM KNVP INTO TABLE ZKNVP WHERE KUNNR = ITKNVV-KUNNR AND PARVW = 'WE'
     AND VTWEG <> '11'.
    LOOP AT ZKNVP INTO ITKNVP.
     APPEND ITKNVP TO ZKNVP1.
    ENDLOOP.
  ENDLOOP.
  CLEAR ZKNVV. REFRESH ZKNVV.
 ENDLOOP.
 SORT ZKNVP1 BY KUNN2.
 DELETE ADJACENT DUPLICATES FROM ZKNVP1 COMPARING KUNN2.

 CLEAR ZTABCUST1.
 READ TABLE ZTABCUST INTO ZTABCUST1 INDEX 1. "#EC CI_NOORDER " #S/4 - OSS Note# - Added by - Z_S
HSHRUKHA – 2021/08/18
  IF LV_IND_TYPE IS NOT INITIAL OR LV_DEL_IND IS NOT INITIAL.
   LOOP AT LT_C_ITAB_INDENT INTO ZIND_TYP.
    IF ( ZIND_TYP-INDENT_TYPE <> LV_IND_TYPE AND LV_IND_TYPE <> ' ' ) OR
     ( ZIND_TYP-ZDELETE <> LV_DEL_IND AND LV_DEL_IND <> ' ' ).
     DELETE LT_C_ITAB_INDENT.
    ENDIF.
    IF ZIND_TYP-DEPOT <> ZTABCUST1-DEPOT.
     DELETE LT_C_ITAB_INDENT.
    ENDIF.
    READ TABLE ZKNVP1 INTO ITKNVP WITH KEY KUNN2 = ZIND_TYP-KUNNR.
    IF SY-SUBRC <> 0.
     DELETE LT_C_ITAB_INDENT.
    ENDIF.
   ENDLOOP.
  ENDIF.

TYPES: BEGIN OF ZWERKS1 ,
       SIGN(1),
       OPTION(2),
       LOW(4),
       HIGH(4),
       END OF ZWERKS1.

DATA: ZWERKS TYPE TABLE OF ZWERKS1.
DATA: ZWERKS2 TYPE ZWERKS1.

 CLEAR ZTABCUST1.
 LOOP AT ZTABCUST INTO ZTABCUST1.
```

```
   ZWERKS2-SIGN = 'I'.
   ZWERKS2-OPTION = 'EQ'.
   ZWERKS2-LOW  = ZTABCUST1-DEPOT.
   APPEND ZWERKS2 TO ZWERKS.
  ENDLOOP.
  SORT ZWERKS BY LOW.
  DELETE ADJACENT DUPLICATES FROM ZWERKS COMPARING LOW.

  IF LV_IND_TYPE IS INITIAL OR LV_DEL_IND IS INITIAL.
   LOOP AT LT_C_ITAB_INDENT INTO ZIND_TYP.
*    IF ZIND_TYP-DEPOT <> ZTABCUST1-DEPOT.
     IF ZIND_TYP-DEPOT NOT IN ZWERKS.
      DELETE LT_C_ITAB_INDENT.
     ENDIF.
     READ TABLE ZKNVP1 INTO ITKNVP WITH KEY KUNN2 = ZIND_TYP-KUNNR.
     IF SY-SUBRC <> 0.
      DELETE LT_C_ITAB_INDENT.
     ENDIF.
   ENDLOOP.
  ENDIF.

* store output to context
  SORT LT_C_ITAB_INDENT BY ZTT_STATUS DESCENDING.
  IF LT_C_ITAB_INDENT[] NE LT_C_ITAB_INDENT_CP[].
   LO_ITAB_INDENT->BIND_TABLE( LT_C_ITAB_INDENT[] ).
  ENDIF.
  CLEAR LT_C_ITAB_INDENT_CP[].
  CLEAR LT_C_ITAB_INDENT[].

ENDMETHOD.
```

- **METHOD EXECUTE_Z_GET_CUST_INDENT_NO_V .**

```
 DATA LO_ND_HEADER_DATA TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_EL_HEADER_DATA TYPE REF TO IF_WD_CONTEXT_ELEMENT.
 DATA LS_HEADER_DATA TYPE WD_THIS->ELEMENT_HEADER_DATA.
 DATA LV_LOAD_DATE TYPE WD_THIS->ELEMENT_HEADER_DATA-LOAD_DATE.
 DATA LV_KUNNR TYPE WD_THIS->ELEMENT_HEADER_DATA-KUNNR.
 LO_ND_HEADER_DATA = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS-
>WDCTX_HEADER_DATA ).
 LO_EL_HEADER_DATA = LO_ND_HEADER_DATA->GET_ELEMENT( ).

 IF LO_EL_HEADER_DATA IS NOT INITIAL.
  LO_EL_HEADER_DATA->GET_ATTRIBUTE(
   EXPORTING
    NAME =  `LOAD_DATE`
   IMPORTING
    VALUE = LV_LOAD_DATE ).

  LO_EL_HEADER_DATA->GET_ATTRIBUTE(
   EXPORTING
    NAME =  `KUNNR`
   IMPORTING
    VALUE = LV_KUNNR ).

 ENDIF.



DATA LO_ZSD_INDENT_NON_VEH TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_CHANGING_2 TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_IMPORTING_2 TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_ELEMENT TYPE REF TO IF_WD_CONTEXT_ELEMENT.
DATA LT_ELEMENTS TYPE WDR_CONTEXT_ELEMENT_SET.
DATA LT_C_CHANGING_2 TYPE IF_COMPONENTCONTROLLER=>ELEMENTS_CHANGING_2.
DATA LS_C_CHANGING_2 LIKE LINE OF LT_C_CHANGING_2.
DATA LT_C_CHANGING_2_CP TYPE IF_COMPONENTCONTROLLER=>ELEMENTS_CHANGING_2.

* get all involved child nodes
LO_ZSD_INDENT_NON_VEH = WD_CONTEXT->GET_CHILD_NODE( WD_THIS-
>WDCTX_ZSD_INDENT_NON_VEH ).
LO_CHANGING_2 = LO_ZSD_INDENT_NON_VEH->GET_CHILD_NODE( WD_THIS->WDCTX_CHANGING_2 ).
LO_IMPORTING_2 = LO_ZSD_INDENT_NON_VEH->GET_CHILD_NODE( WD_THIS->WDCTX_IMPORTING_2 ).
LT_ELEMENTS = LO_CHANGING_2->GET_ELEMENTS( ).

LOOP AT LT_ELEMENTS[] INTO LO_ELEMENT.
  LO_ELEMENT-
>GET_STATIC_ATTRIBUTES( IMPORTING STATIC_ATTRIBUTES = LS_C_CHANGING_2 ).  "#EC CI_FLDEXT_OK[
2215424] " #S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
  INSERT LS_C_CHANGING_2 INTO TABLE LT_C_CHANGING_2[].
ENDLOOP.


LT_C_CHANGING_2_CP = LT_C_CHANGING_2[].
DATA LV_CUST_ID TYPE CHAR10.
MOVE SY-UNAME TO LV_CUST_ID.

CLEAR LT_C_CHANGING_2. REFRESH LT_C_CHANGING_2.
LO_CHANGING_2->BIND_TABLE( LT_C_CHANGING_2[] ).
** the invocation - errors are always fatal !!!
 CALL FUNCTION 'Z_GET_CUST_INDENT_NO_VEHICLE'
  EXPORTING
*    VEHICLE =              LV_VEHICLE
    IND_DATE =              LV_LOAD_DATE
```

```
    KUNNR =                 LV_KUNNR
    CUST_ID =               LV_CUST_ID
  TABLES
    ZINDENT_DATA =          LT_C_CHANGING_2.


* store output to context
IF LT_C_CHANGING_2[] IS NOT INITIAL.
 IF LT_C_CHANGING_2[] NE LT_C_CHANGING_2_CP[].
  LO_CHANGING_2->BIND_TABLE( LT_C_CHANGING_2[] ).
 ENDIF.
ENDIF.
 CLEAR LT_C_CHANGING_2_CP[].
 CLEAR LT_C_CHANGING_2[].

ENDMETHOD.
```

- **METHOD EXECUTE_Z_GET_CUST_UNLOAD_INDE .**

```
* declarations for context navigation
 DATA LO_Z_GET_CUST_UNLOAD_IN TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_CHANGING_3 TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_ZINDENT_DATA TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_ELEMENT TYPE REF TO IF_WD_CONTEXT_ELEMENT.
 DATA LT_ELEMENTS TYPE WDR_CONTEXT_ELEMENT_SET.

* declarations for parameters
 DATA LT_C_ZINDENT_DATA TYPE IF_COMPONENTCONTROLLER=>ELEMENTS_ZINDENT_DATA.
 DATA LS_C_ZINDENT_DATA LIKE LINE OF LT_C_ZINDENT_DATA.
 DATA LT_C_ZINDENT_DATA_CP TYPE IF_COMPONENTCONTROLLER=>ELEMENTS_ZINDENT_DATA.

 DATA LO_ND_INBND_HEADER TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_EL_INBND_HEADER TYPE REF TO IF_WD_CONTEXT_ELEMENT.
 DATA LS_INBND_HEADER TYPE WD_THIS->Element_inbnd_header.
 DATA LV_UNLD_DATE TYPE WD_THIS->Element_inbnd_header-UNLD_DATE.
 DATA LV_UNLD_TDATE TYPE WD_THIS->Element_inbnd_header-UNLD_TDATE.
 DATA FT_DATE TYPE STRING.
 LO_ND_INBND_HEADER = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS-
>WDCTX_INBND_HEADER ).
 LO_EL_INBND_HEADER = LO_ND_INBND_HEADER->GET_ELEMENT( ).

*  get single attribute
  LO_EL_INBND_HEADER->GET_ATTRIBUTE(
   EXPORTING
    NAME =  `UNLD_DATE`
   IMPORTING
    VALUE = LV_UNLD_DATE ).

  LO_EL_INBND_HEADER->GET_ATTRIBUTE(
   EXPORTING
    NAME =  `UNLD_TDATE`
   IMPORTING
    VALUE = LV_UNLD_TDATE ).

  CONCATENATE LV_UNLD_DATE LV_UNLD_TDATE INTO FT_DATE.
  CONDENSE FT_DATE.



* get all involved child nodes
 LO_Z_GET_CUST_UNLOAD_IN = WD_CONTEXT->GET_CHILD_NODE( WD_THIS-
```

```
>WDCTX_Z_GET_CUST_UNLOAD_IN ).
 LO_CHANGING_3 = LO_Z_GET_CUST_UNLOAD_IN->GET_CHILD_NODE( WD_THIS-
>WDCTX_CHANGING_3 ).
 LO_ZINDENT_DATA = LO_CHANGING_3->GET_CHILD_NODE( WD_THIS->WDCTX_ZINDENT_DATA ).

 CLEAR LT_C_ZINDENT_DATA_CP[]. REFRESH LT_C_ZINDENT_DATA_CP[].
 CLEAR LT_C_ZINDENT_DATA[]. REFRESH LT_C_ZINDENT_DATA[].
 CLEAR LS_C_ZINDENT_DATA.
* get input from context
 LT_ELEMENTS = LO_ZINDENT_DATA->GET_ELEMENTS( ).
 LOOP AT LT_ELEMENTS[] INTO LO_ELEMENT.
  LO_ELEMENT->GET_STATIC_ATTRIBUTES( IMPORTING STATIC_ATTRIBUTES = LS_C_ZINDENT_DATA ).
  INSERT LS_C_ZINDENT_DATA INTO TABLE LT_C_ZINDENT_DATA[].
 ENDLOOP.

 LT_C_ZINDENT_DATA_CP = LT_C_ZINDENT_DATA[].




* the invocation - errors are always fatal !!!
 CALL FUNCTION 'Z_GET_CUST_UNLOAD_INDENT'
  EXPORTING
    IND_DATE     = IND_DATE
    KUNNR        = KUNNR
    CUST_ID      = CUST_ID
    MATNR        = MATNR
    FR_TO_DATE    = FT_DATE
  TABLES
    ZINDENT_DATA  = LT_C_ZINDENT_DATA.

  sort LT_C_ZINDENT_DATA by ind_date ascending.
* store output to context
 IF LT_C_ZINDENT_DATA[] NE LT_C_ZINDENT_DATA_CP[].
  LO_ZINDENT_DATA->BIND_TABLE( LT_C_ZINDENT_DATA[] ).
 ENDIF.
 CLEAR LT_C_ZINDENT_DATA_CP[].
 CLEAR LT_C_ZINDENT_DATA[].

ENDMETHOD.
```

- **method EXECUTE_Z_INDENT_QTY_CHECK .**

```
  DATA LO_ND_ITAB_INDENT TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LT_ITAB_INDENT TYPE WD_THIS->ELEMENTS_ITAB_INDENT.
  LO_ND_ITAB_INDENT = WD_CONTEXT-
>PATH_GET_NODE( PATH = `Z_GET_CUSTOMER_INDEN.CHANGING_1.ITAB_INDENT` ).
  LO_ND_ITAB_INDENT-
>GET_STATIC_ATTRIBUTES_TABLE( IMPORTING TABLE = LT_ITAB_INDENT ).  "#EC CI_FLDEXT_OK[2215424]
" #S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18

TYPES: BEGIN OF ITAB,
    PROD TYPE MATNR,
    QTY(16),
    SPART TYPE SPART,
    END OF ITAB.

DATA ITAB_1 TYPE STANDARD TABLE OF ITAB.
DATA ITAB_2 TYPE STANDARD TABLE OF ITAB.
DATA ITAB_3 TYPE STANDARD TABLE OF ITAB.
DATA ITABS TYPE ITAB.
DATA ITABS1 TYPE ITAB.
DATA: ZKDGRP TYPE KDGRP.
DATA: LV_ERCODE4 TYPE CHAR1.

DATA: TCAP1(16) TYPE C,
   TCAP2(16) TYPE C,
   TCAP3(16) TYPE C,
   TCAP4(16) TYPE C,
   TCAP5(16) TYPE C,
   TCAP6(16) TYPE C,
   TCAP7(16) TYPE C,
   TCAP8(16) TYPE C.

DATA: NCAP1 TYPE ZSD_CUST_INDENT-HSD,
   NCAP1C TYPE ZSD_CUST_INDENT-HSD,
   NCAP2 TYPE ZPROD_GRP_INDENT-QUANTITY,
   NCAP2C TYPE ZPROD_GRP_INDENT-QUANTITY,
   NCAPT TYPE ZSD_CUST_INDENT-HSD,
   NCAP2T TYPE ZPROD_GRP_INDENT-QUANTITY.
DATA LNG TYPE I.
DATA CUST_INDENT TYPE ZSD_CUST_INDENT.

DATA LO_WINDOW_MANAGER TYPE REF TO IF_WD_WINDOW_MANAGER.
DATA LO_API_COMPONENT  TYPE REF TO IF_WD_COMPONENT.
DATA LO_WINDOW       TYPE REF TO IF_WD_WINDOW.
DATA: lt_string type table of string,
   ls_string like line of lt_string.
DATA l_api          TYPE REF TO if_wd_view_controller.

DATA LO_ND_IMPORTING_1 TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_EL_IMPORTING_1 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
DATA LS_IMPORTING_1 TYPE WD_THIS->ELEMENT_IMPORTING_1.
DATA LV_BEGDA TYPE WD_THIS->ELEMENT_IMPORTING_1-IND_DATE.

* navigate from <CONTEXT> to <IMPORTING_1> via lead selection
 LO_ND_IMPORTING_1 = WD_CONTEXT-
>PATH_GET_NODE( PATH = `Z_GET_CUSTOMER_INDEN.IMPORTING_1` ).
* get element via lead selection
 LO_EL_IMPORTING_1 = LO_ND_IMPORTING_1->GET_ELEMENT( ).
 IF LO_EL_IMPORTING_1 IS INITIAL.
 ENDIF.

* get single attribute
 LO_EL_IMPORTING_1->GET_ATTRIBUTE(
```

```
  EXPORTING
    NAME = `IND_DATE`
  IMPORTING
    VALUE = LV_BEGDA ).


READ TABLE LT_ITAB_INDENT INTO CUST_INDENT INDEX 1.
CLEAR LNG. CLEAR ITAB_1. REFRESH ITAB_1. CLEAR ITABS.
 LNG = STRLEN( CUST_INDENT-QUAN_COMP1 ).
 LNG = LNG - 2.
 TCAP1 = CUST_INDENT-QUAN_COMP1+0(LNG).
 CLEAR LNG.
 LNG = STRLEN( CUST_INDENT-QUAN_COMP2 ).
 LNG = LNG - 2.
 TCAP2 = CUST_INDENT-QUAN_COMP2+0(LNG).
 CLEAR LNG.
 LNG = STRLEN( CUST_INDENT-QUAN_COMP3 ).
 LNG = LNG - 2.
 TCAP3 = CUST_INDENT-QUAN_COMP3+0(LNG).
 CLEAR LNG.
 LNG = STRLEN( CUST_INDENT-QUAN_COMP4 ).
 LNG = LNG - 2.
 TCAP4 = CUST_INDENT-QUAN_COMP4+0(LNG).
 CLEAR LNG.
 LNG = STRLEN( CUST_INDENT-QUAN_COMP5 ).
 LNG = LNG - 2.
 TCAP5 = CUST_INDENT-QUAN_COMP5+0(LNG).
 CLEAR LNG.
 LNG = STRLEN( CUST_INDENT-QUAN_COMP6 ).
 LNG = LNG - 2.
 TCAP6 = CUST_INDENT-QUAN_COMP6+0(LNG).
 CLEAR LNG.
 LNG = STRLEN( CUST_INDENT-QUAN_COMP7 ).
 LNG = LNG - 2.
 TCAP7 = CUST_INDENT-QUAN_COMP7+0(LNG).
 CLEAR LNG.
 LNG = STRLEN( CUST_INDENT-QUAN_COMP8 ).
 LNG = LNG - 2.
 TCAP8 = CUST_INDENT-QUAN_COMP8+0(LNG).


 CLEAR ITABS.
 ITABS-PROD = CUST_INDENT-PROD_CMP1.
 ITABS-QTY = TCAP1.
 SELECT SINGLE SPART FROM MARA INTO ITABS-SPART WHERE MATNR = ITABS-PROD.
 APPEND ITABS TO ITAB_1.

 CLEAR ITABS.
 ITABS-PROD = CUST_INDENT-PROD_CMP2.
 ITABS-QTY = TCAP2.
 SELECT SINGLE SPART FROM MARA INTO ITABS-SPART WHERE MATNR = ITABS-PROD.
 APPEND ITABS TO ITAB_1.

 CLEAR ITABS.
 ITABS-PROD = CUST_INDENT-PROD_CMP3.
 ITABS-QTY = TCAP3.
 SELECT SINGLE SPART FROM MARA INTO ITABS-SPART WHERE MATNR = ITABS-PROD.
 APPEND ITABS TO ITAB_1.

 CLEAR ITABS.
 ITABS-PROD = CUST_INDENT-PROD_CMP4.
 ITABS-QTY = TCAP4.
```

```
SELECT SINGLE SPART FROM MARA INTO ITABS-SPART WHERE MATNR = ITABS-PROD.
APPEND ITABS TO ITAB_1.

CLEAR ITABS.
ITABS-PROD = CUST_INDENT-PROD_CMP5.
ITABS-QTY = TCAP5.
SELECT SINGLE SPART FROM MARA INTO ITABS-SPART WHERE MATNR = ITABS-PROD.
APPEND ITABS TO ITAB_1.

CLEAR ITABS.
ITABS-PROD = CUST_INDENT-PROD_CMP6.
ITABS-QTY = TCAP6.
SELECT SINGLE SPART FROM MARA INTO ITABS-SPART WHERE MATNR = ITABS-PROD.
APPEND ITABS TO ITAB_1.

CLEAR ITABS.
ITABS-PROD = CUST_INDENT-PROD_CMP7.
ITABS-QTY = TCAP7.
SELECT SINGLE SPART FROM MARA INTO ITABS-SPART WHERE MATNR = ITABS-PROD.
APPEND ITABS TO ITAB_1.

CLEAR ITABS.
ITABS-PROD = CUST_INDENT-PROD_CMP8.
ITABS-QTY = TCAP8.
SELECT SINGLE SPART FROM MARA INTO ITABS-SPART WHERE MATNR = ITABS-PROD.
APPEND ITABS TO ITAB_1.

SORT ITAB_1 BY SPART.
DELETE ITAB_1 WHERE PROD IS INITIAL.
CLEAR: TCAP1, TCAP2, TCAP3, TCAP4, TCAP5, TCAP6, TCAP7, TCAP8.
ITAB_2[] = ITAB_1[].

CLEAR ITABS.
LOOP AT ITAB_1 INTO ITABS.
 CLEAR TCAP1.
 AT NEW SPART.
  LOOP AT ITAB_2 INTO ITABS1 WHERE SPART = ITABS-SPART.
   TCAP1 = TCAP1 + ITABS1-QTY.
  ENDLOOP.
  CONDENSE TCAP1.
  ITABS-QTY = TCAP1.
  APPEND ITABS TO ITAB_3.
  IF ITABS-SPART = '10'.
   CUST_INDENT-LPG = TCAP1.
   ELSEIF ITABS-SPART = '25'.
   CUST_INDENT-MS = TCAP1.
   ELSEIF ITABS-SPART = '30'.
   CUST_INDENT-ATF = TCAP1.
   ELSEIF ITABS-SPART = '35'.
   CUST_INDENT-SKO = TCAP1.
   ELSEIF ITABS-SPART = '40' AND ITABS-PROD+0(5) <> 'HSDAR'.
   CUST_INDENT-HSD = TCAP1.
   ELSEIF ITABS-PROD+0(5) = 'HSDAR'.
   CUST_INDENT-ARHSD = TCAP1.
   ELSEIF ITABS-SPART = '50'.
   CUST_INDENT-PWAX = TCAP1.
   ELSEIF ITABS-SPART = '60'.
   CUST_INDENT-MTO = TCAP1.
   ELSEIF ITABS-SPART = '80'.
   CUST_INDENT-RPC = TCAP1.
   ELSEIF ITABS-SPART = '90'.
   CUST_INDENT-SUL = TCAP1.
   ELSEIF ITABS-SPART = '85'.
```

```
      CUST_INDENT-NTG = TCAP1.
      ELSE.
        CUST_INDENT-OTHER = TCAP1.
     ENDIF.
   ENDAT.
   CLEAR ITABS.
 ENDLOOP.

 SORT ITAB_3 BY SPART.
 DELETE ADJACENT DUPLICATES FROM ITAB_3 COMPARING SPART.
 CLEAR: TCAP1, TCAP2, TCAP3, TCAP4, TCAP5, NCAP1, NCAPT, TCAP6, TCAP7, TCAP8.
 CLEAR ITABS. CLEAR NCAP2.

LOOP AT ITAB_3 INTO ITABS.

 CLEAR: NCAP1, NCAPT, NCAP1C, NCAP2, NCAP2C, NCAP2T.
 IF ITABS-SPART = '10'.
   SELECT SUM( LPG ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
    KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
   SELECT SUM( LPG ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
    KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
   SELECT SUM( LPG ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
    BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.

   ELSEIF ITABS-SPART = '25'.
    SELECT SUM( MS ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
    KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
    SELECT SUM( MS ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
     KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
    SELECT SUM( MS ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
     BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.

   ELSEIF ITABS-SPART = '30'.
    SELECT SUM( ATF ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
    KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
    SELECT SUM( ATF ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
     KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
    SELECT SUM( ATF ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
     BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.

   ELSEIF ITABS-SPART = '35'.
    SELECT SUM( SKO ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
    KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
    SELECT SUM( SKO ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
     KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
    SELECT SUM( SKO ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
     BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.

   ELSEIF ITABS-SPART = '40' AND ITABS-PROD+0(5) <> 'HSDAR'.
    SELECT SUM( HSD ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
    KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
    SELECT SUM( HSD ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
     KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
```

```
   SELECT SUM( HSD ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
    BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.

  ELSEIF ITABS-PROD+0(5) = 'HSDAR'.
   SELECT SUM( ARHSD ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
   KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
   SELECT SUM( ARHSD ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
     KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
   SELECT SUM( ARHSD ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
    BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.

  ELSEIF ITABS-SPART = '50'.
   SELECT SUM( PWAX ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
   KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
   SELECT SUM( PWAX ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
     KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
   SELECT SUM( PWAX ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
    BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.

  ELSEIF ITABS-SPART = '60'.
   SELECT SUM( MTO ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
   KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
   SELECT SUM( MTO ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
     KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
   SELECT SUM( MTO ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
    BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.

  ELSEIF ITABS-SPART = '80'.
   SELECT SUM( RPC ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
   KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
   SELECT SUM( RPC ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
     KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
   SELECT SUM( RPC ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
    BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.

  ELSEIF ITABS-SPART = '90'.
   SELECT SUM( SUL ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
   KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
   SELECT SUM( SUL ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
     KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
   SELECT SUM( SUL ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
    BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.

  ELSEIF ITABS-SPART = '85'.
   SELECT SUM( NTG ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE  "#EC CI_NOORDER " #S/4 - OSS N
ote# - Added by - Z_SHSHRUKHA – 2021/08/18
   KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
   SELECT SUM( NTG ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE  "#EC CI_NOORDER " #S/4 - OSS No
te# - Added by - Z_SHSHRUKHA – 2021/08/18
     KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
   SELECT SUM( NTG ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
```

```
    BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.
 ENDIF.

 NCAP1C = NCAP1C + ITABS-QTY.
 NCAP1  = NCAP1 + ITABS-QTY.
 NCAPT  = NCAPT + ITABS-QTY.

 CLEAR LV_ERCODE4.
 IF ITABS-SPART = '40' AND ITABS-PROD+0(5) <> 'HSDAR'.
  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE      "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'HSD' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.
  IF SY-SUBRC <> 0.
   SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE  "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'HSD' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT.
  ENDIF.

  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE
   MATERIAL = 'HSD' AND CUST_GROUP = CUST_INDENT-KDGRP AND
    LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  IF SY-SUBRC <> 0.
   SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE
   MATERIAL = 'HSD' AND CUST_GROUP = CUST_INDENT-KDGRP AND
    LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  ENDIF.

  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE
   MATERIAL = 'HSD' AND CUST_GROUP = ' ' AND CUSTOMER = ' ' AND
    LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.
  IF SY-SUBRC <> 0.
   SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE      "#EC CI_NOORDER "
#S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'HSD' AND CUST_GROUP = ' ' AND CUSTOMER = ' '
     AND LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT.
  ENDIF.

 ENDIF.

 IF ITABS-SPART = '40' AND ITABS-PROD+0(5) = 'HSDAR'.
  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE      "#EC CI_NOORDER " #S
/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'ARHSD' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.
  IF SY-SUBRC <> 0.
   SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE      "#EC CI_NOORDER " #
S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'ARHSD' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT.
  ENDIF.

   SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE      "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'ARHSD' AND CUST_GROUP = CUST_INDENT-KDGRP AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
   IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE     "#EC CI_NOORDER " #S
/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
     MATERIAL = 'ARHSD' AND CUST_GROUP = CUST_INDENT-KDGRP AND
      LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
   ENDIF.
```

```
   SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE     "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
     MATERIAL = 'ARHSD' AND CUST_GROUP = ' ' AND
      LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE    "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'ARHSD' AND CUST_GROUP = ' ' AND LDATE = '00000000'
      AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  ENDIF.
 ENDIF.


 IF ITABS-SPART = '30' .
  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE        "#EC CI_NOORDER " #
S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'ATF' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.
  IF SY-SUBRC <> 0.
   SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE         "#EC CI_NOORDER "
#S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'ATF' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT.
  ENDIF.

  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE         "#EC CI_NOORDER " #S
/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'ATF' AND CUST_GROUP = CUST_INDENT-KDGRP AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  IF SY-SUBRC <> 0.
   SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE
   MATERIAL = 'ATF' AND CUST_GROUP = CUST_INDENT-KDGRP AND
    LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  ENDIF.

  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE     "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'ATF' AND CUST_GROUP = ' ' AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  IF SY-SUBRC <> 0.
   SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE    "#EC CI_NOORDER " #S
/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'ATF' AND CUST_GROUP = ' ' AND LDATE = '00000000'
     AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  ENDIF.
 ENDIF.

IF ITABS-SPART = '35' .
 SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE     "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'SKO' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.
  IF SY-SUBRC <> 0.
   SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE   "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'SKO' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT.
  ENDIF.

  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE      "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'SKO' AND CUST_GROUP = CUST_INDENT-KDGRP AND
```

```
    LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE     "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'SKO' AND CUST_GROUP = CUST_INDENT-KDGRP AND
     LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  ENDIF.

  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE     "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'SKO' AND CUST_GROUP = ' ' AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE    "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'SKO' AND CUST_GROUP = ' ' AND LDATE = '00000000'
      AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  ENDIF.
ENDIF.

IF ITABS-SPART = '25' .
  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE  "#EC CI_NOORDER " #S/4
- OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'MS' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE   "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'MS' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT.
  ENDIF.

  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE
    MATERIAL = 'MS' AND CUST_GROUP = CUST_INDENT-KDGRP AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE
    MATERIAL = 'MS' AND CUST_GROUP = CUST_INDENT-KDGRP AND
     LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  ENDIF.

  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE
    MATERIAL = 'MS' AND CUST_GROUP = ' ' AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE    "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'MS' AND CUST_GROUP = ' ' AND LDATE = '00000000'
      AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  ENDIF.
ENDIF.

IF ITABS-SPART = '10' .
  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE    "#EC CI_NOORDER " #S/4
 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'LPG' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE    "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'LPG' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT.
  ENDIF.
```

```
  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE    "#EC CI_NOORDER " #S/4 -
 OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'LPG' AND CUST_GROUP = CUST_INDENT-KDGRP AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
   IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE  "#EC CI_NOORDER " #S/4
 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'LPG' AND CUST_GROUP = CUST_INDENT-KDGRP AND
     LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
   ENDIF.

  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE    "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'LPG' AND CUST_GROUP = ' ' AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE    "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'LPG' AND CUST_GROUP = ' ' AND LDATE = '00000000'
      AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  ENDIF.
ENDIF.


IF ITABS-SPART = '60' .
  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE      "#EC CI_NOORDER " #S
/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'MTO' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE      "#EC CI_NOORDER " #
S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'MTO' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT.
  ENDIF.

  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE    "#EC CI_NOORDER " #S/4
 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'MTO' AND CUST_GROUP = CUST_INDENT-KDGRP AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE    "#EC CI_NOORDER " #S/4
 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'MTO' AND CUST_GROUP = CUST_INDENT-KDGRP AND
     LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  ENDIF.

  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE   "#EC CI_NOORDER " #S/4
 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'MTO' AND CUST_GROUP = ' ' AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE  "#EC CI_NOORDER " #S/4
 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'MTO' AND CUST_GROUP = ' ' AND LDATE = '00000000'
      AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  ENDIF.
ENDIF.

IF ITABS-SPART = '50' .
  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE     "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
```

```
    MATERIAL = 'WAX' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.
   IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE    "#EC CI_NOORDER " #S
/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'WAX' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT.
   ENDIF.

   SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE     "#EC CI_NOORDER " #S/4
 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'WAX' AND CUST_GROUP = CUST_INDENT-KDGRP AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
   IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE   "#EC CI_NOORDER " #S/4
- OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'WAX' AND CUST_GROUP = CUST_INDENT-KDGRP AND
     LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
   ENDIF.

   SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE    "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'WAX' AND CUST_GROUP = ' ' AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
   IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE    "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'WAX' AND CUST_GROUP = ' ' AND LDATE = '00000000'
      AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
   ENDIF.
ENDIF.


IF ITABS-SPART = '90' .
  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE    "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'SUL' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.
   IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE   "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'SUL' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT.
   ENDIF.

   SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE     "#EC CI_NOORDER " #S/4
 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'SUL' AND CUST_GROUP = CUST_INDENT-KDGRP AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
   IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE  "#EC CI_NOORDER " #S/4
- OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'SUL' AND CUST_GROUP = CUST_INDENT-KDGRP AND
     LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
   ENDIF.

   SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE      "#EC CI_NOORDER " #
S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'SUL' AND CUST_GROUP = ' ' AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
   IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE    "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
```

```
    MATERIAL = 'SUL' AND CUST_GROUP = ' ' AND LDATE = '00000000'
     AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  ENDIF.
ENDIF.

IF ITABS-SPART = '80' .
  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE   "#EC CI_NOORDER " #S/4
- OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'RPC_CPC' AND CUSTOMER = CUST_INDENT-KUNNR AND
    LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE   "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'RPC_CPC' AND CUSTOMER = CUST_INDENT-KUNNR AND
    LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT.
  ENDIF.

  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE   "#EC CI_NOORDER " #S/4 -
OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'RPC_CPC' AND CUST_GROUP = CUST_INDENT-KDGRP AND
    LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE   "#EC CI_NOORDER " #S/4
- OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'RPC_CPC' AND CUST_GROUP = CUST_INDENT-KDGRP AND
    LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  ENDIF.

  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE   "#EC CI_NOORDER " #S/4
- OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'RPC_CPC' AND CUST_GROUP = ' ' AND
    LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE   "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'RPC_CPC' AND CUST_GROUP = ' ' AND LDATE = '00000000'
     AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  ENDIF.
ENDIF.


IF ITABS-SPART = '85' .
  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE   "#EC CI_NOORDER " #S/4
 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'NTG' AND CUSTOMER = CUST_INDENT-KUNNR AND
    LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE   "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'NTG' AND CUSTOMER = CUST_INDENT-KUNNR AND
    LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT.
  ENDIF.

  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE   "#EC CI_NOORDER " #S/4 -
OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'NTG' AND CUST_GROUP = CUST_INDENT-KDGRP AND
    LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE   "#EC CI_NOORDER " #S/4
- OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'NTG' AND CUST_GROUP = CUST_INDENT-KDGRP AND
    LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  ENDIF.
```

```
  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE    "#EC CI_NOORDER " #S/4
- OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'NTG' AND CUST_GROUP = ' ' AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  IF SY-SUBRC <> 0.
     SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE     "#EC CI_NOORDER " #
S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
     MATERIAL = 'NTG' AND CUST_GROUP = ' ' AND LDATE = '00000000'
      AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  ENDIF.
ENDIF.

  IF NCAP1C > NCAP2C AND NCAP2C > 0.
   LV_ERCODE4 = 'X'.
  ENDIF.

  IF NCAP1 > NCAP2 AND NCAP2 > 0.
   LV_ERCODE4 = 'X'.
  ENDIF.

  IF NCAPT > NCAP2T AND NCAP2T > 0.
   LV_ERCODE4 = 'X'.
  ENDIF.

  MODIFY LT_ITAB_INDENT FROM CUST_INDENT INDEX 1.

   LO_ND_ITAB_INDENT = WD_CONTEXT-
>PATH_GET_NODE( PATH = `Z_GET_CUSTOMER_INDEN.CHANGING_1.ITAB_INDENT` ).
   LO_ND_ITAB_INDENT-
>BIND_TABLE( NEW_ITEMS = LT_ITAB_INDENT SET_INITIAL_ELEMENTS = ABAP_TRUE ).


  IF LV_ERCODE4 = 'X'.
   CLEAR ls_string. CLEAR lt_string. REFRESH lt_string.
   CONCATENATE 'Indent quantity exceeded from plan for material' ITABS-
PROD INTO ls_string SEPARATED BY SPACE.
*    ls_string = 'Indent quantity exceeded from plan'.
    insert ls_string into table lt_string.
    LO_API_COMPONENT        = WD_THIS->WD_GET_API( ).
    LO_WINDOW_MANAGER       = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).
    LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
          text         = lt_string
          button_kind    = if_wd_window=>co_buttons_ok
*           button_kind    = 4
          message_type   = if_wd_window=>CO_MSG_TYPE_STOPP
          window_title   = 'Please check!'
          window_position = if_wd_window=>co_center
          WINDOW_WIDTH   = '20%'
          WINDOW_HEIGHT  = '20%' ).
    LO_WINDOW->open( ).

     DATA LO_ND_Z_CHECK TYPE REF TO IF_WD_CONTEXT_NODE.
     DATA LO_EL_Z_CHECK TYPE REF TO IF_WD_CONTEXT_ELEMENT.
     DATA LS_Z_CHECK TYPE WD_THIS->ELEMENT_Z_CHECK.
     DATA LV_Z_ERROR TYPE WD_THIS->ELEMENT_Z_CHECK-Z_ERROR.
     LO_ND_Z_CHECK = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_Z_CHECK ).
*    get element via lead selection
     LO_EL_Z_CHECK = LO_ND_Z_CHECK->GET_ELEMENT( ).
     IF LO_EL_Z_CHECK IS INITIAL.
     ENDIF.

*    set single attribute
```

```
      LV_Z_ERROR = 'X'.
      LO_EL_Z_CHECK->SET_ATTRIBUTE(
       NAME =  `Z_ERROR`
       VALUE = LV_Z_ERROR ).

  ENDIF.
ENDLOOP.
endmethod.
```

- **method EXECUTE_Z_IND_QTY_CHECK_SUBMIT .**

```
 DATA LO_ND_ITAB_INDENT TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LT_ITAB_INDENT TYPE WD_THIS->ELEMENTS_ITAB_INDENT.

*  navigate from <CONTEXT> to <ITAB_INDENT> via lead selection
  LO_ND_ITAB_INDENT = WD_CONTEXT-
>PATH_GET_NODE( PATH = `Z_GET_CUSTOMER_INDEN.CHANGING_1.ITAB_INDENT` ).
  LO_ND_ITAB_INDENT-
>GET_STATIC_ATTRIBUTES_TABLE( IMPORTING TABLE = LT_ITAB_INDENT ).  "#EC CI_FLDEXT_OK[2215424] "
 #S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18

TYPES: BEGIN OF ITAB,
     PROD TYPE MATNR,
     QTY(16),
     SPART TYPE SPART,
     END OF ITAB.

DATA ITAB_1 TYPE STANDARD TABLE OF ITAB.
DATA ITAB_2 TYPE STANDARD TABLE OF ITAB.
DATA ITAB_3 TYPE STANDARD TABLE OF ITAB.
DATA ITABS TYPE ITAB.
DATA ITABS1 TYPE ITAB.
DATA: ZKDGRP TYPE KDGRP.
DATA: LV_ERCODE4 TYPE CHAR1.

DATA: TCAP1(16) TYPE C,
    TCAP2(16) TYPE C,
    TCAP3(16) TYPE C,
    TCAP4(16) TYPE C,
    TCAP5(16) TYPE C,
    TCAP6(16) TYPE C,
    TCAP7(16) TYPE C,
    TCAP8(16) TYPE C.

DATA: NCAP1 TYPE ZSD_CUST_INDENT-HSD,
    NCAP1C TYPE ZSD_CUST_INDENT-HSD,
    NCAP2 TYPE ZPROD_GRP_INDENT-QUANTITY,
    NCAP2C TYPE ZPROD_GRP_INDENT-QUANTITY,
    NCAPT TYPE ZSD_CUST_INDENT-HSD,
    NCAP2T TYPE ZPROD_GRP_INDENT-QUANTITY.
DATA LNG TYPE I.
DATA CUST_INDENT TYPE ZSD_CUST_INDENT.
```

```
DATA LO_WINDOW_MANAGER TYPE REF TO IF_WD_WINDOW_MANAGER.
DATA LO_API_COMPONENT  TYPE REF TO IF_WD_COMPONENT.
DATA LO_WINDOW       TYPE REF TO IF_WD_WINDOW.
DATA: lt_string type table of string,
    ls_string like line of lt_string.
DATA l_api           TYPE REF TO if_wd_view_controller.

DATA LO_ND_IMPORTING_1 TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_EL_IMPORTING_1 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
DATA LS_IMPORTING_1 TYPE WD_THIS->ELEMENT_IMPORTING_1.
DATA LV_BEGDA TYPE WD_THIS->ELEMENT_IMPORTING_1-IND_DATE.

* navigate from <CONTEXT> to <IMPORTING_1> via lead selection
 LO_ND_IMPORTING_1 = WD_CONTEXT-
>PATH_GET_NODE( PATH = `Z_GET_CUSTOMER_INDEN.IMPORTING_1` ).
* get element via lead selection
 LO_EL_IMPORTING_1 = LO_ND_IMPORTING_1->GET_ELEMENT( ).
 IF LO_EL_IMPORTING_1 IS INITIAL.
 ENDIF.

* get single attribute
 LO_EL_IMPORTING_1->GET_ATTRIBUTE(
  EXPORTING
   NAME =  `IND_DATE`
  IMPORTING
   VALUE = LV_BEGDA ).



READ TABLE LT_ITAB_INDENT INTO CUST_INDENT INDEX 1.
CLEAR LNG.
 LNG = STRLEN( CUST_INDENT-QUAN_COMP1 ).
 LNG = LNG - 2.
 TCAP1 = CUST_INDENT-QUAN_COMP1+0(LNG).
 CLEAR LNG.
 LNG = STRLEN( CUST_INDENT-QUAN_COMP2 ).
 LNG = LNG - 2.
 TCAP2 = CUST_INDENT-QUAN_COMP2+0(LNG).
 CLEAR LNG.
 LNG = STRLEN( CUST_INDENT-QUAN_COMP3 ).
 LNG = LNG - 2.
 TCAP3 = CUST_INDENT-QUAN_COMP3+0(LNG).
 CLEAR LNG.
 LNG = STRLEN( CUST_INDENT-QUAN_COMP4 ).
 LNG = LNG - 2.
 TCAP4 = CUST_INDENT-QUAN_COMP4+0(LNG).
 CLEAR LNG.
 LNG = STRLEN( CUST_INDENT-QUAN_COMP5 ).
 LNG = LNG - 2.
 TCAP5 = CUST_INDENT-QUAN_COMP5+0(LNG).
 CLEAR LNG.
 LNG = STRLEN( CUST_INDENT-QUAN_COMP6 ).
 LNG = LNG - 2.
 TCAP6 = CUST_INDENT-QUAN_COMP6+0(LNG).
 CLEAR LNG.
 LNG = STRLEN( CUST_INDENT-QUAN_COMP7 ).
 LNG = LNG - 2.
 TCAP7 = CUST_INDENT-QUAN_COMP7+0(LNG).
 CLEAR LNG.
 LNG = STRLEN( CUST_INDENT-QUAN_COMP8 ).
 LNG = LNG - 2.
 TCAP8 = CUST_INDENT-QUAN_COMP8+0(LNG).
```

```
CLEAR ITABS.
ITABS-PROD = CUST_INDENT-PROD_CMP1.
ITABS-QTY = TCAP1.
SELECT SINGLE SPART FROM MARA INTO ITABS-SPART WHERE MATNR = ITABS-PROD.
APPEND ITABS TO ITAB_1.

CLEAR ITABS.
ITABS-PROD = CUST_INDENT-PROD_CMP2.
ITABS-QTY = TCAP2.
SELECT SINGLE SPART FROM MARA INTO ITABS-SPART WHERE MATNR = ITABS-PROD.
APPEND ITABS TO ITAB_1.

CLEAR ITABS.
ITABS-PROD = CUST_INDENT-PROD_CMP3.
ITABS-QTY = TCAP3.
SELECT SINGLE SPART FROM MARA INTO ITABS-SPART WHERE MATNR = ITABS-PROD.
APPEND ITABS TO ITAB_1.

CLEAR ITABS.
ITABS-PROD = CUST_INDENT-PROD_CMP4.
ITABS-QTY = TCAP4.
SELECT SINGLE SPART FROM MARA INTO ITABS-SPART WHERE MATNR = ITABS-PROD.
APPEND ITABS TO ITAB_1.

CLEAR ITABS.
ITABS-PROD = CUST_INDENT-PROD_CMP5.
ITABS-QTY = TCAP5.
SELECT SINGLE SPART FROM MARA INTO ITABS-SPART WHERE MATNR = ITABS-PROD.
APPEND ITABS TO ITAB_1.

CLEAR ITABS.
ITABS-PROD = CUST_INDENT-PROD_CMP6.
ITABS-QTY = TCAP6.
SELECT SINGLE SPART FROM MARA INTO ITABS-SPART WHERE MATNR = ITABS-PROD.
APPEND ITABS TO ITAB_1.

CLEAR ITABS.
ITABS-PROD = CUST_INDENT-PROD_CMP7.
ITABS-QTY = TCAP7.
SELECT SINGLE SPART FROM MARA INTO ITABS-SPART WHERE MATNR = ITABS-PROD.
APPEND ITABS TO ITAB_1.

CLEAR ITABS.
ITABS-PROD = CUST_INDENT-PROD_CMP8.
ITABS-QTY = TCAP8.
SELECT SINGLE SPART FROM MARA INTO ITABS-SPART WHERE MATNR = ITABS-PROD.
APPEND ITABS TO ITAB_1.

SORT ITAB_1 BY SPART.
DELETE ITAB_1 WHERE PROD IS INITIAL.
CLEAR: TCAP1, TCAP2, TCAP3, TCAP4, TCAP5, TCAP6, TCAP7, TCAP8.
ITAB_2[] = ITAB_1[].

LOOP AT ITAB_1 INTO ITABS.
 CLEAR TCAP1.
 AT NEW SPART.
  LOOP AT ITAB_2 INTO ITABS1 WHERE SPART = ITABS-SPART.
   TCAP1 = TCAP1 + ITABS1-QTY.
  ENDLOOP.
  CONDENSE TCAP1.
  ITABS-QTY = TCAP1.
  APPEND ITABS TO ITAB_3.
  IF ITABS-SPART = '10'.
```

```
      CUST_INDENT-LPG = TCAP1.
      ELSEIF ITABS-SPART = '25'.
      CUST_INDENT-MS = TCAP1.
      ELSEIF ITABS-SPART = '30'.
      CUST_INDENT-ATF = TCAP1.
      ELSEIF ITABS-SPART = '35'.
      CUST_INDENT-SKO = TCAP1.
      ELSEIF ITABS-SPART = '40' AND ITABS-PROD+0(5) <> 'HSDAR'.
      CUST_INDENT-HSD = TCAP1.
      ELSEIF ITABS-PROD+0(5) = 'HSDAR'.
      CUST_INDENT-ARHSD = TCAP1.
      ELSEIF ITABS-SPART = '50'.
      CUST_INDENT-PWAX = TCAP1.
      ELSEIF ITABS-SPART = '60'.
      CUST_INDENT-MTO = TCAP1.
      ELSEIF ITABS-SPART = '80'.
      CUST_INDENT-RPC = TCAP1.
      ELSEIF ITABS-SPART = '90'.
      CUST_INDENT-SUL = TCAP1.
    ENDIF.
  ENDAT.
  CLEAR ITABS.
 ENDLOOP.

 SORT ITAB_3 BY SPART.
 DELETE ADJACENT DUPLICATES FROM ITAB_3 COMPARING SPART.
 CLEAR: TCAP1, TCAP2, TCAP3, TCAP4, TCAP5, TCAP6, NCAP1, NCAPT, TCAP7, TCAP8.
 CLEAR ITABS. CLEAR NCAP2.

 DATA LO_ND_WV_ENABLE TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_EL_WV_ENABLE TYPE REF TO IF_WD_CONTEXT_ELEMENT.
 DATA LS_WV_ENABLE TYPE WD_THIS->ELEMENT_WV_ENABLE.
 DATA LV_VW_SV2 TYPE WD_THIS->ELEMENT_WV_ENABLE-VW_SV2.
 LO_ND_WV_ENABLE = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_WV_ENABLE ).
 LO_EL_WV_ENABLE = LO_ND_WV_ENABLE->GET_ELEMENT( ).
 IF LO_EL_WV_ENABLE IS NOT INITIAL.
  LO_EL_WV_ENABLE->GET_ATTRIBUTE(
   EXPORTING
    NAME = `VW_SV2`
   IMPORTING
    VALUE = LV_VW_SV2 ).
 ENDIF.

LOOP AT ITAB_3 INTO ITABS.
 CLEAR NCAP1. CLEAR NCAPT.
 IF LV_VW_SV2 = ' '.
    IF ITABS-SPART = '10'.
      SELECT SUM( LPG ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
       KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
      SELECT SUM( LPG ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
       KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
      SELECT SUM( LPG ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
       BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.

      ELSEIF ITABS-SPART = '25'.
       SELECT SUM( MS ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
       KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
       SELECT SUM( MS ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
        KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
```

```
      SELECT SUM( MS ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
       BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.

    ELSEIF ITABS-SPART = '30'.
      SELECT SUM( ATF ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
      KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
      SELECT SUM( ATF ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
       KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
      SELECT SUM( ATF ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
       BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.

    ELSEIF ITABS-SPART = '35'.
      SELECT SUM( SKO ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
      KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
      SELECT SUM( SKO ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
       KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
      SELECT SUM( SKO ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
       BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.

    ELSEIF ITABS-SPART = '40' AND ITABS-PROD+0(5) <> 'HSDAR'.
      SELECT SUM( HSD ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
      KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
      SELECT SUM( HSD ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
       KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
      SELECT SUM( HSD ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
       BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.

    ELSEIF ITABS-PROD+0(5) = 'HSDAR'.
      SELECT SUM( ARHSD ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
      KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
      SELECT SUM( ARHSD ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
       KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
      SELECT SUM( ARHSD ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
       BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.

    ELSEIF ITABS-SPART = '50'.
      SELECT SUM( PWAX ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
      KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
      SELECT SUM( PWAX ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
       KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
      SELECT SUM( PWAX ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
       BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.

    ELSEIF ITABS-SPART = '60'.
      SELECT SUM( MTO ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
      KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
      SELECT SUM( MTO ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
       KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
      SELECT SUM( MTO ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
       BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.
```

```
    ELSEIF ITABS-SPART = '80'.
      SELECT SUM( RPC ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
      KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
      SELECT SUM( RPC ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
       KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
      SELECT SUM( RPC ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
       BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.

    ELSEIF ITABS-SPART = '90'.
      SELECT SUM( SUL ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
      KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
      SELECT SUM( SUL ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
       KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
      SELECT SUM( SUL ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
       BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.

    ELSEIF ITABS-SPART = '85'.
      SELECT SUM( NTG ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
      KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
      SELECT SUM( NTG ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
       KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT.
      SELECT SUM( NTG ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
       BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.
   ENDIF.
  ELSE.
   IF ITABS-SPART = '10'.
    SELECT SUM( LPG ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
      KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT AND ZTT_STATUS <> '4'.
    SELECT SUM( LPG ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
      KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT AND ZTT_STATUS <> '4'.
    SELECT SUM( LPG ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
      BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND ZTT_STATUS <> '4'.

    ELSEIF ITABS-SPART = '25'.
      SELECT SUM( MS ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
      KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT AND ZTT_STATUS <> '4'.
      SELECT SUM( MS ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
       KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT AND ZTT_STATUS <> '4'.
      SELECT SUM( MS ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
       BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND ZTT_STATUS <> '4'.

    ELSEIF ITABS-SPART = '30'.
      SELECT SUM( ATF ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
      KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT AND ZTT_STATUS <> '4'.
      SELECT SUM( ATF ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
       KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT AND ZTT_STATUS <> '4'.
      SELECT SUM( ATF ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
       BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND ZTT_STATUS <> '4'.

    ELSEIF ITABS-SPART = '35'.
      SELECT SUM( SKO ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
```

```
      KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT AND ZTT_STATUS <> '4'.
      SELECT SUM( SKO ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
       KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT AND ZTT_STATUS <> '4'.
      SELECT SUM( SKO ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
       BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND ZTT_STATUS <> '4'.

     ELSEIF ITABS-SPART = '40' AND ITABS-PROD+0(5) <> 'HSDAR'.
      SELECT SUM( HSD ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
      KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT AND ZTT_STATUS <> '4'.
      SELECT SUM( HSD ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
       KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT AND ZTT_STATUS <> '4'.
      SELECT SUM( HSD ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
       BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND ZTT_STATUS <> '4'.

     ELSEIF ITABS-PROD+0(5) = 'HSDAR'.
      SELECT SUM( ARHSD ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
      KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT AND ZTT_STATUS <> '4'.
      SELECT SUM( ARHSD ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
       KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT AND ZTT_STATUS <> '4'.
      SELECT SUM( ARHSD ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
       BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND ZTT_STATUS <> '4'.

     ELSEIF ITABS-SPART = '50'.
      SELECT SUM( PWAX ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
      KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT AND ZTT_STATUS <> '4'.
      SELECT SUM( PWAX ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
       KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT AND ZTT_STATUS <> '4'.
      SELECT SUM( PWAX ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
       BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND ZTT_STATUS <> '4'.

     ELSEIF ITABS-SPART = '60'.
      SELECT SUM( MTO ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
      KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT AND ZTT_STATUS <> '4'.
      SELECT SUM( MTO ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
       KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT AND ZTT_STATUS <> '4'.
      SELECT SUM( MTO ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
       BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND ZTT_STATUS <> '4'.

     ELSEIF ITABS-SPART = '80'.
      SELECT SUM( RPC ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
      KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT AND ZTT_STATUS <> '4'.
      SELECT SUM( RPC ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
       KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT AND ZTT_STATUS <> '4'.
      SELECT SUM( RPC ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
       BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND ZTT_STATUS <> '4'.

     ELSEIF ITABS-SPART = '90'.
      SELECT SUM( SUL ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
      KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT AND ZTT_STATUS <> '4'.
      SELECT SUM( SUL ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
```

```
        KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT AND ZTT_STATUS <> '4'.
       SELECT SUM( SUL ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
        BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND ZTT_STATUS <> '4'.

      ELSEIF ITABS-SPART = '85'.
       SELECT SUM( NTG ) FROM ZSD_CUST_INDENT INTO NCAP1C WHERE
       KUNNR = CUST_INDENT-KUNNR AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT AND ZTT_STATUS <> '4'.
       SELECT SUM( NTG ) FROM ZSD_CUST_INDENT INTO NCAP1 WHERE
        KDGRP = CUST_INDENT-KDGRP AND BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-
DEPOT AND ZTT_STATUS <> '4'.
       SELECT SUM( NTG ) FROM ZSD_CUST_INDENT INTO NCAPT WHERE
        BEGDA = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND ZTT_STATUS <> '4'.
    ENDIF.


  ENDIF.

*  IF LV_VW_SV2 = 'X'.
*   SELECT COUNT(*) FROM ZSD_CUST_INDENT WHERE BEGDA = CUST_INDENT-
BEGDA AND DEPOT = CUST_INDENT-DEPOT
*    AND VEHICLE = CUST_INDENT-VEHICLE AND ZTT_STATUS = '4'.
*    IF SY-SUBRC <> 0.
      NCAP1C = NCAP1C + ITABS-QTY.
      NCAP1  = NCAP1  + ITABS-QTY.
      NCAPT  = NCAPT  + ITABS-QTY.
*     ENDIF.
*  ENDIF.

 CLEAR LV_ERCODE4.
 IF ITABS-SPART = '40' AND ITABS-PROD+0(5) <> 'HSDAR'.
  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE   "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'HSD' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.
  IF SY-SUBRC <> 0.
   SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE   "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'HSD' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT.
  ENDIF.

  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE   "#EC CI_NOORDER " #S/4 -
 OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'HSD' AND CUST_GROUP = CUST_INDENT-KDGRP AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  IF SY-SUBRC <> 0.
   SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE  "#EC CI_NOORDER " #S/4
- OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'HSD' AND CUST_GROUP = CUST_INDENT-KDGRP AND
     LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  ENDIF.

  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE  "#EC CI_NOORDER " #S/4
- OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'HSD' AND CUST_GROUP = ' ' AND CUSTOMER = ' ' AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.
  IF SY-SUBRC <> 0.
   SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE  "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'HSD' AND CUST_GROUP = ' ' AND CUSTOMER = ' '
     AND LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT.
```

```
  ENDIF.

 ENDIF.

 IF ITABS-SPART = '40' AND ITABS-PROD+0(5) = 'HSDAR'.
  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE   "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'ARHSD' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.
  IF SY-SUBRC <> 0.
   SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE  "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'ARHSD' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT.
  ENDIF.

   SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE  "#EC CI_NOORDER " #S/4 -
 OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
     MATERIAL = 'ARHSD' AND CUST_GROUP = CUST_INDENT-KDGRP AND
      LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
    IF SY-SUBRC <> 0.
     SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE   "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
      MATERIAL = 'ARHSD' AND CUST_GROUP = CUST_INDENT-KDGRP AND
       LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
    ENDIF.

   SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE   "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
     MATERIAL = 'ARHSD' AND CUST_GROUP = ' ' AND
      LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  IF SY-SUBRC <> 0.
   SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE "#EC CI_NOORDER " #S/4
 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'ARHSD' AND CUST_GROUP = ' ' AND LDATE = '00000000'
     AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  ENDIF.
 ENDIF.


 IF ITABS-SPART = '30' .
  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE  "#EC CI_NOORDER " #S/4
 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'ATF' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.
  IF SY-SUBRC <> 0.
   SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE  "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'ATF' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT.
  ENDIF.

  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE   "#EC CI_NOORDER " #S/4 -
 OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'ATF' AND CUST_GROUP = CUST_INDENT-KDGRP AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  IF SY-SUBRC <> 0.
   SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE   "#EC CI_NOORDER " #S/4
 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'ATF' AND CUST_GROUP = CUST_INDENT-KDGRP AND
     LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  ENDIF.
```

```
  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE    "#EC CI_NOORDER " #S/4
 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'ATF' AND CUST_GROUP = ' ' AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE   "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'ATF' AND CUST_GROUP = ' ' AND LDATE = '00000000'
      AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  ENDIF.
 ENDIF.

IF ITABS-SPART = '35' .
  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE   "#EC CI_NOORDER " #S/4
 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'SKO' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE   "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'SKO' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT.
  ENDIF.

  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE    "#EC CI_NOORDER " #S/4 -
 OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'SKO' AND CUST_GROUP = CUST_INDENT-KDGRP AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE    "#EC CI_NOORDER " #S/4
 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'SKO' AND CUST_GROUP = CUST_INDENT-KDGRP AND
     LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  ENDIF.

  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE   "#EC CI_NOORDER " #S/4
 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'SKO' AND CUST_GROUP = ' ' AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE   "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'SKO' AND CUST_GROUP = ' ' AND LDATE = '00000000'
      AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  ENDIF.
ENDIF.

IF ITABS-SPART = '25' .
  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE    "#EC CI_NOORDER " #S/4
 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'MS' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE   "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'MS' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT.
  ENDIF.

  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE  "#EC CI_NOORDER " #S/4 -
OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'MS' AND CUST_GROUP = CUST_INDENT-KDGRP AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
```

```
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE    "#EC CI_NOORDER " #S/4
- OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
     MATERIAL = 'MS' AND CUST_GROUP = CUST_INDENT-KDGRP AND
      LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  ENDIF.

  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE    "#EC CI_NOORDER " #S/4
 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
     MATERIAL = 'MS' AND CUST_GROUP = ' ' AND
      LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE   "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
     MATERIAL = 'MS' AND CUST_GROUP = ' ' AND LDATE = '00000000'
       AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  ENDIF.
ENDIF.

IF ITABS-SPART = '10' .
  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE    "#EC CI_NOORDER " #S/4
 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
     MATERIAL = 'LPG' AND CUSTOMER = CUST_INDENT-KUNNR AND
      LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE   "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
     MATERIAL = 'LPG' AND CUSTOMER = CUST_INDENT-KUNNR AND
      LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT.
  ENDIF.

  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE   "#EC CI_NOORDER " #S/4 -
OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
     MATERIAL = 'LPG' AND CUST_GROUP = CUST_INDENT-KDGRP AND
      LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
   IF SY-SUBRC <> 0.
     SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE   "#EC CI_NOORDER " #S/4
- OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
     MATERIAL = 'LPG' AND CUST_GROUP = CUST_INDENT-KDGRP AND
      LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
   ENDIF.

  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE   "#EC CI_NOORDER " #S/4
- OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
     MATERIAL = 'LPG' AND CUST_GROUP = ' ' AND
      LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE   "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
     MATERIAL = 'LPG' AND CUST_GROUP = ' ' AND LDATE = '00000000'
       AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  ENDIF.
ENDIF.


IF ITABS-SPART = '60' .
  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE "#EC CI_NOORDER " #S/4 -
 OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
     MATERIAL = 'MTO' AND CUSTOMER = CUST_INDENT-KUNNR AND
      LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE  "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
```

```
    MATERIAL = 'MTO' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT.
   ENDIF.

   SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE  "#EC CI_NOORDER " #S/4 -
OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
     MATERIAL = 'MTO' AND CUST_GROUP = CUST_INDENT-KDGRP AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
   IF SY-SUBRC <> 0.
     SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE  "#EC CI_NOORDER " #S/4 -
 OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
     MATERIAL = 'MTO' AND CUST_GROUP = CUST_INDENT-KDGRP AND
     LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
   ENDIF.

   SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE  "#EC CI_NOORDER " #S/4
- OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
     MATERIAL = 'MTO' AND CUST_GROUP = ' ' AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
   IF SY-SUBRC <> 0.
     SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE  "#EC CI_NOORDER " #S/4
 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
     MATERIAL = 'MTO' AND CUST_GROUP = ' ' AND LDATE = '00000000'
      AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
   ENDIF.
 ENDIF.

 IF ITABS-SPART = '50' .
   SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE  "#EC CI_NOORDER " #S/4
 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
     MATERIAL = 'WAX' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.
   IF SY-SUBRC <> 0.
     SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE  "#EC CI_NOORDER " #S/
 4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
     MATERIAL = 'WAX' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT.
   ENDIF.

   SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE  "#EC CI_NOORDER " #S/4 -
 OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
     MATERIAL = 'WAX' AND CUST_GROUP = CUST_INDENT-KDGRP AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
   IF SY-SUBRC <> 0.
     SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE  "#EC CI_NOORDER " #S/4
 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
     MATERIAL = 'WAX' AND CUST_GROUP = CUST_INDENT-KDGRP AND
     LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
   ENDIF.

   SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE  "#EC CI_NOORDER " #S/4
 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
     MATERIAL = 'WAX' AND CUST_GROUP = ' ' AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
   IF SY-SUBRC <> 0.
     SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE  "#EC CI_NOORDER " #S/4
 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
     MATERIAL = 'WAX' AND CUST_GROUP = ' ' AND LDATE = '00000000'
      AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
   ENDIF.
 ENDIF.
```

```
IF ITABS-SPART = '90' .
 SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE "#EC CI_NOORDER " #S/4 -
OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'SUL' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE  "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'SUL' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT.
  ENDIF.

  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE
    MATERIAL = 'SUL' AND CUST_GROUP = CUST_INDENT-KDGRP AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE
    MATERIAL = 'SUL' AND CUST_GROUP = CUST_INDENT-KDGRP AND
     LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  ENDIF.

  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE
    MATERIAL = 'SUL' AND CUST_GROUP = ' ' AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE
    MATERIAL = 'SUL' AND CUST_GROUP = ' ' AND LDATE = '00000000'
      AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  ENDIF.
ENDIF.

IF ITABS-SPART = '80' .
 SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE  "#EC CI_NOORDER " #S/4
- OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'RPC_CPC' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE  "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'RPC_CPC' AND CUSTOMER = CUST_INDENT-KUNNR AND
     LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT.
  ENDIF.

  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE  "#EC CI_NOORDER " #S/4 -
OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'RPC_CPC' AND CUST_GROUP = CUST_INDENT-KDGRP AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE   "#EC CI_NOORDER " #S/4
- OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'RPC_CPC' AND CUST_GROUP = CUST_INDENT-KDGRP AND
     LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  ENDIF.

  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE   "#EC CI_NOORDER " #S/4
- OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'RPC_CPC' AND CUST_GROUP = ' ' AND
     LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  IF SY-SUBRC <> 0.
    SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE   "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'RPC_CPC' AND CUST_GROUP = ' ' AND LDATE = '00000000'
      AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
```

```
  ENDIF.
ENDIF.


IF ITABS-SPART = '85' .
 SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE  "#EC CI_NOORDER " #S/4
- OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'NTG' AND CUSTOMER = CUST_INDENT-KUNNR AND
    LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT.
  IF SY-SUBRC <> 0.
   SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2C WHERE   "#EC CI_NOORDER " #S/
4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'NTG' AND CUSTOMER = CUST_INDENT-KUNNR AND
    LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT.
  ENDIF.

  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE  "#EC CI_NOORDER " #S/4 -
OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'NTG' AND CUST_GROUP = CUST_INDENT-KDGRP AND
    LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  IF SY-SUBRC <> 0.
   SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2 WHERE  "#EC CI_NOORDER " #S/4 -
 OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    MATERIAL = 'NTG' AND CUST_GROUP = CUST_INDENT-KDGRP AND
    LDATE = '00000000' AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  ENDIF.

  SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE
   MATERIAL = 'NTG' AND CUST_GROUP = ' ' AND
   LDATE = CUST_INDENT-BEGDA AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  IF SY-SUBRC <> 0.
   SELECT SINGLE QUANTITY FROM ZPROD_GRP_INDENT INTO NCAP2T WHERE
   MATERIAL = 'NTG' AND CUST_GROUP = ' ' AND LDATE = '00000000'
    AND DEPOT = CUST_INDENT-DEPOT AND CUSTOMER = ' '.
  ENDIF.
ENDIF.

  IF NCAP1C > NCAP2C AND NCAP2C > 0.
   LV_ERCODE4 = 'X'.
  ENDIF.

  IF NCAP1 > NCAP2 AND NCAP2 > 0.
   LV_ERCODE4 = 'X'.
  ENDIF.

  IF NCAPT > NCAP2T AND NCAP2T > 0.
   LV_ERCODE4 = 'X'.
  ENDIF.

  MODIFY LT_ITAB_INDENT FROM CUST_INDENT INDEX 1.

  LO_ND_ITAB_INDENT = WD_CONTEXT-
>PATH_GET_NODE( PATH = `Z_GET_CUSTOMER_INDEN.CHANGING_1.ITAB_INDENT` ).
  LO_ND_ITAB_INDENT-
>BIND_TABLE( NEW_ITEMS = LT_ITAB_INDENT SET_INITIAL_ELEMENTS = ABAP_TRUE ).


  IF LV_ERCODE4 = 'X'.
   CLEAR ls_string. CLEAR lt_string. REFRESH lt_string.
   CONCATENATE 'Indent quantity for material' ITABS-PROD 'exceeded from plan'
    INTO ls_string SEPARATED BY SPACE.
    insert ls_string into table lt_string.
    LO_API_COMPONENT         = WD_THIS->WD_GET_API( ).
```

```
    LO_WINDOW_MANAGER        = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).
    LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
            text         = lt_string
            button_kind    = if_wd_window=>co_buttons_ok
*            button_kind    = 4
            message_type   = if_wd_window=>CO_MSG_TYPE_STOPP
            window_title   = 'Please check!'
            window_position = if_wd_window=>co_center
            WINDOW_WIDTH   = '20%'
            WINDOW_HEIGHT  = '20%' ).
    LO_WINDOW->open( ).

     DATA LO_ND_Z_CHECK TYPE REF TO IF_WD_CONTEXT_NODE.
     DATA LO_EL_Z_CHECK TYPE REF TO IF_WD_CONTEXT_ELEMENT.
     DATA LS_Z_CHECK TYPE WD_THIS->ELEMENT_Z_CHECK.
     DATA LV_Z_ERROR TYPE WD_THIS->ELEMENT_Z_CHECK-Z_ERROR.
     LO_ND_Z_CHECK = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_Z_CHECK ).
*    get element via lead selection
     LO_EL_Z_CHECK = LO_ND_Z_CHECK->GET_ELEMENT( ).
     IF LO_EL_Z_CHECK IS INITIAL.
     ENDIF.

*    set single attribute
     LV_Z_ERROR = 'X'.
     LO_EL_Z_CHECK->SET_ATTRIBUTE(
      NAME =  `Z_ERROR`
      VALUE = LV_Z_ERROR ).

  ENDIF.
ENDLOOP.

 LO_ND_WV_ENABLE = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_WV_ENABLE ).
 LO_EL_WV_ENABLE = LO_ND_WV_ENABLE->GET_ELEMENT( ).
 IF LO_EL_WV_ENABLE IS NOT INITIAL.
  CLEAR LV_VW_SV2.
  LO_EL_WV_ENABLE->SET_ATTRIBUTE(
  NAME =  `VW_SV2`
  VALUE = LV_VW_SV2 ).
 ENDIF.
endmethod.
```

- **method WDDOINIT .**

```
wd_this->lv_pernr = SY-UNAME.
DATA ZTABCUST TYPE TABLE OF ZSD_CUST_USR_MAP.
DATA ZTABCUST1 LIKE LINE OF ZTABCUST.
DATA ZSOLD_TO TYPE KUNNR.

DATA LO_IND_DEL TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LV_IND_DEL TYPE wd_this->ELEMENTS_IND_DEL.
DATA LS_IND_DEL TYPE IG_COMPONENTCONTROLLER=>Element_IND_DEL.
CLEAR LS_IND_DEL. CLEAR LV_IND_DEL. REFRESH LV_IND_DEL.
LO_IND_DEL = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_IND_DEL ).
 APPEND LS_IND_DEL TO LV_IND_DEL.
 LS_IND_DEL-DEL_IND = 'Y'.
 APPEND LS_IND_DEL TO LV_IND_DEL.
 LS_IND_DEL-DEL_IND = 'N'.
 APPEND LS_IND_DEL TO LV_IND_DEL.
LO_IND_DEL->bind_table( new_items = LV_IND_DEL set_initial_elements = abap_true ).


DATA LO_IND_MOD TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LV_IND_MOD TYPE wd_this->ELEMENTS_IND_MODE.
DATA LS_IND_MOD TYPE IG_COMPONENTCONTROLLER=>Element_IND_MODE.
CLEAR LS_IND_MOD. CLEAR LV_IND_MOD. REFRESH LV_IND_MOD.
LO_IND_MOD = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_IND_MODE ).
 APPEND LS_IND_MOD TO LV_IND_MOD.
 SELECT DOMVALUE_L FROM DD07L INTO LS_IND_MOD-IND_TYPE WHERE
  DOMNAME = 'ZIND_MODE'.
  APPEND LS_IND_MOD TO LV_IND_MOD.
 ENDSELECT.
LO_IND_MOD->bind_table( new_items = LV_IND_MOD set_initial_elements = abap_true ).


SELECT * FROM ZSD_CUST_USR_MAP INTO CORRESPONDING FIELDS OF TABLE ZTABCUST
 WHERE CUST_USER_ID = wd_this->lv_pernr.

DATA LV_IND_DATE TYPE BEGDA.
DATA LO_IMPORTING_1 TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_Z_GET_CUSTOMER_INDEN TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LR_ELEMENT TYPE REF TO IF_WD_CONTEXT_ELEMENT.
DATA LV_CONTRACT TYPE ZSD_CUST_INDENT-CHK.
DATA LO_CHANGING_1 TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_ITAB_INDENT_1 TYPE REF TO IF_WD_CONTEXT_NODE.

LO_Z_GET_CUSTOMER_INDEN = WD_CONTEXT->GET_CHILD_NODE( WD_THIS-
>WDCTX_Z_GET_CUSTOMER_INDEN ).
LO_IMPORTING_1 = LO_Z_GET_CUSTOMER_INDEN->GET_CHILD_NODE( WD_THIS-
>WDCTX_IMPORTING_1 ).
LO_IMPORTING_1->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `IND_DATE`
  IMPORTING
   VALUE = LV_IND_DATE ).
LV_IND_DATE = sy-datum.
LR_ELEMENT = LO_IMPORTING_1->GET_ELEMENT( ).

LR_ELEMENT->SET_ATTRIBUTE(
     EXPORTING
      value =  LV_IND_DATE         " Attribute Value
      name  = 'IND_DATE' ).

DATA:  itheader_data type REF TO IF_WD_CONTEXT_NODE,
     itheader_data1 TYPE wd_this->Elements_header_data,
     lsheader_data1 LIKE LINE OF itheader_data1.
```

```
*      itdata_head TYPE wd_this->Elements_header_data.
 DATA: ZKNVP TYPE TABLE OF KNVP,
       ZKNVP1 TYPE TABLE OF KNVP,
       ITKNVP LIKE LINE OF ZKNVP.
 DATA: ZKDGRP TYPE KNVV-KDGRP,
       ZKNVV TYPE TABLE OF KNVV,
       ITKNVV LIKE LINE OF ZKNVV.

 itheader_data = wd_context->path_get_node( path = `HEADER_DATA` ).
 LOOP AT ZTABCUST INTO ZTABCUST1.
  SELECT SINGLE KDGRP FROM KNVV INTO ZKDGRP WHERE KUNNR = ZTABCUST1-
KUNNR.  "#EC CI_NOORDER " #S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
 IF ZKDGRP <> 'DI' AND ZKDGRP <> 'RE' AND ZKDGRP <> 'TG' AND ZKDGRP <> 'OI'
   AND ZKDGRP <> 'EX'.

   SELECT * FROM KNVV INTO TABLE ZKNVV WHERE KDGRP = ZKDGRP AND SPART <> '11'
    AND KDGRP <> 'DI' AND KDGRP <> 'TG' AND KDGRP <> 'OI' AND KDGRP <> 'RE'.
   SORT ZKNVV BY KUNNR.

  ELSE.
   SELECT * FROM KNVV INTO TABLE ZKNVV WHERE KUNNR = ZTABCUST1-KUNNR AND SPART <> '11'.
   SORT ZKNVV BY KUNNR.
  ENDIF.

  DELETE ADJACENT DUPLICATES FROM ZKNVV COMPARING KUNNR.

  LOOP AT ZKNVV INTO ITKNVV.
    SELECT * FROM KNVP INTO TABLE ZKNVP WHERE KUNNR = ITKNVV-KUNNR AND PARVW = 'WE'
     AND VTWEG <> '11'.
    LOOP AT ZKNVP INTO ITKNVP.
     APPEND ITKNVP TO ZKNVP1.
    ENDLOOP.
  ENDLOOP.
  CLEAR ZKNVV. REFRESH ZKNVV.
 ENDLOOP.
 SORT ZKNVP1 BY KUNN2.
 DELETE ADJACENT DUPLICATES FROM ZKNVP1 COMPARING KUNN2.

 LOOP AT ZKNVP1 INTO ITKNVP.
  lsheader_data1-KUNNR = ITKNVP-KUNN2.
  lsheader_data1-load_date = sy-datum.
  SELECT SINGLE NAME1 FROM KNA1 INTO lsheader_data1-NAME1 WHERE KUNNR = lsheader_data1-
KUNNR.
  APPEND lsheader_data1 TO ITHEADER_DATA1.
 ENDLOOP.
 CLEAR lsheader_data1.
 IF ZTABCUST1-DEPOT = '3100'.
  READ TABLE ITHEADER_DATA1 INTO lsheader_data1 WITH KEY KUNNR = '0000100036'.
  IF SY-SUBRC = 0.
   DELETE ITHEADER_DATA1 WHERE KUNNR = '0000100036'.
   INSERT lsheader_data1 INTO ITHEADER_DATA1 INDEX 1.
  ENDIF.
  CLEAR lsheader_data1.
 ENDIF.
 IF ZTABCUST1-DEPOT = '3202'.
  READ TABLE ITHEADER_DATA1 INTO lsheader_data1 WITH KEY KUNNR = '0000100192'.
  IF SY-SUBRC = 0.
   DELETE ITHEADER_DATA1 WHERE KUNNR = '0000100192'.
   INSERT lsheader_data1 INTO ITHEADER_DATA1 INDEX 1.
  ENDIF.
  CLEAR lsheader_data1.

  READ TABLE ITHEADER_DATA1 INTO lsheader_data1 WITH KEY KUNNR = '0000100193'.
```

```
 IF SY-SUBRC = 0.
  DELETE ITHEADER_DATA1 WHERE KUNNR = '0000100193'.
  INSERT lsheader_data1 INTO ITHEADER_DATA1 INDEX 2.
 ENDIF.
 CLEAR lsheader_data1.

 READ TABLE ITHEADER_DATA1 INTO lsheader_data1 WITH KEY KUNNR = '0000100194'.
 IF SY-SUBRC = 0.
  DELETE ITHEADER_DATA1 WHERE KUNNR = '0000100194'.
  INSERT lsheader_data1 INTO ITHEADER_DATA1 INDEX 3.
 ENDIF.
 CLEAR lsheader_data1.

 READ TABLE ITHEADER_DATA1 INTO lsheader_data1 WITH KEY KUNNR = '0000100195'.
 IF SY-SUBRC = 0.
  DELETE ITHEADER_DATA1 WHERE KUNNR = '0000100195'.
  INSERT lsheader_data1 INTO ITHEADER_DATA1 INDEX 4.
 ENDIF.
 CLEAR lsheader_data1.
ENDIF.
IF ZTABCUST1-DEPOT <> '3100'.
 DELETE ITHEADER_DATA1 WHERE KUNNR = '0000100036'.
ENDIF.
IF ZTABCUST1-DEPOT <> '3202'.
 DELETE ITHEADER_DATA1 WHERE KUNNR = '0000100192'.
 DELETE ITHEADER_DATA1 WHERE KUNNR = '0000100193'.
 DELETE ITHEADER_DATA1 WHERE KUNNR = '0000100194'.
 DELETE ITHEADER_DATA1 WHERE KUNNR = '0000100195'.
ENDIF.

lsheader_data1-load_date = sy-datum.
insert lsheader_data1 into itheader_data1 index 1.
itheader_data->bind_table( new_items = itheader_data1 set_initial_elements = abap_true ).
clear lsheader_data1.

*** To disable the compartment capacities START ***
DATA LO_EXPORTING TYPE REF TO IF_WD_CONTEXT_NODE.

LO_EXPORTING = wd_context->get_child_node( name = wd_this->WDCTX_ENABLE ).
LO_EXPORTING->set_attribute(
   name = 'EN1'
   value = ' ' ).
LO_EXPORTING->set_attribute(
   name = 'EN2'
   value = ' ' ).
LO_EXPORTING->set_attribute(
   name = 'EN3'
   value = ' ' ).
LO_EXPORTING->set_attribute(
   name = 'EN4'
   value = ' ' ).
LO_EXPORTING->set_attribute(
   name = 'EN5'
   value = ' ' ).

LO_EXPORTING->set_attribute(
   name = 'QN1'
   value = ' ' ).
LO_EXPORTING->set_attribute(
   name = 'QN2'
   value = ' ' ).
LO_EXPORTING->set_attribute(
   name = 'QN3'
```

```
    value = ' ' ).
 LO_EXPORTING->set_attribute(
    name = 'QN4'
    value = ' ' ).
 LO_EXPORTING->set_attribute(
    name = 'QN5'
    value = ' ' ).
*** To disable the compartment capacities END ***

DATA:  itinbnd_header type REF TO IF_WD_CONTEXT_NODE,
       itinbnd_header1 TYPE wd_this->Elements_inbnd_header,
       lsinbnd_header1 LIKE LINE OF itinbnd_header1.

  read table ZTABCUST into ZTABCUST1 index 1.
  if ZTABCUST1-EBMS = 'Y'.
   itinbnd_header = wd_context->path_get_node( path = 'INBND_HEADER' ).
*   lsinbnd_header1-unld_matnr  = 'ETHNL01'.
   select single imat2 from ZEBMS_BLEND into lsinbnd_header1-unld_matnr.
   APPEND lsinbnd_header1 TO itinbnd_header1.
   clear lsinbnd_header1.
   lsinbnd_header1-UNLD_KUNNR = ZTABCUST1-KUNNR.
   lsinbnd_header1-UNLD_PLANT = ZTABCUST1-DEPOT.
   insert lsinbnd_header1 into itinbnd_header1 index 1.
   itinbnd_header->bind_table( new_items = itinbnd_header1 set_initial_elements = abap_true ).
   clear lsinbnd_header1.
  endif.

endmethod.
```

Context INDMAIN

1. IMPORTING
   - VEHICLE TYPE OIGCC-TU_NUMBER
2. EXPORTING
   - CAPACITY1 TYPE CHAR012
   - CAPACITY2 TYPE CHAR012
   - CAPACITY3 TYPE CHAR012
   - CAPACITY4 TYPE CHAR012
   - CAPACITY5 TYPE CHAR012
   - CAPACITY6 TYPE CHAR012
   - CAPACITY7 TYPE CHAR012
   - CAPACITY8 TYPE CHAR012
3. PROD1
   - PRODUCT TYPE CHAR100
4. PROD2
   - PRODUCT TYPE CHAR100
5. PROD3
   - PRODUCT TYPE CHAR100

6. PROD4
    - PRODUCT TYPE CHAR100
7. PROD5
    - PRODUCT TYPE CHAR100
8. HEADER_DATA
    - LOAD_DATE TYPE BUDAT
    - NAME1 TYPE NAME1_GP
    - KUNNR TYPE KUNNR
9. ATF_FLUSH
    - FLUSH TYPE CHAR1
    - DESC TYPE CHAR3
10. ERROR
    - ERCODE TYPE CHAR1
    - ALL_SUBMIT TYPE CHAR1
11. ENABLE
    - EN1 TYPE WDY_BOOLEAN
    - EN2 TYPE WDY_BOOLEAN
    - EN3 TYPE WDY_BOOLEAN
    - EN4 TYPE WDY_BOOLEAN
    - EN5 TYPE WDY_BOOLEAN
    - QN1 TYPE WDY_BOOLEAN
    - QN2 TYPE WDY_BOOLEAN
    - QN3 TYPE WDY_BOOLEAN
    - QN4 TYPE WDY_BOOLEAN
    - QN5 TYPE WDY_BOOLEAN
    - SAVE TYPE WDY_BOOLEAN
    - DELETE TYPE WDY_BOOLEAN
    - SAVE_ALL TYPE WDY_BOOLEAN
    - SUBMIT TYPE WDY_BOOLEAN
    - SUBMIT_ALL TYPE WDY_BOOLEAN
    - CONT TYPE WDY_BOOLEAN
    - EN6 TYPE WDY_BOOLEAN
    - EN7 TYPE WDY_BOOLEAN
    - EN8 TYPE WDY_BOOLEAN

- QN6 TYPE WDY_BOOLEAN
- QN7 TYPE WDY_BOOLEAN
- QN8 TYPE WDY_BOOLEAN

12. EXPORT
    - VBELN TYPE VBELN
13. IND_MODE
    - IND_TYPE TYPE ZIND_MODE
    - DEL_IND TYPE CHAR1
14. Z_CHECK
    - Z_ERROR TYPE CHAR1
15. GSTN
    - TPT_GSTN TYPE ZSD_CUST_INDENT-TPT_GSTN
16. ENABLE1
    - ET1 TYPE WDY_BOOLEAN
    - ET2 TYPE WDY_BOOLEAN
17. FLUSH_REASON
    - FLSH_REASON TYPE ZSD_CUST_INDENT-FLUSH_REASON
18. ENABLE2
    - ZT1 TYPE WDY_BOOLEAN
    - ZT2 TYPE WDY_BOOLEAN
    - ZT3 TYPE WDY_BOOLEAN *(DEFAULT VALUE 'X')*
19. Z_GET_CUSTOMER_INDEN
    - IMPORTING_1
        - VEHICLE TYPE OIG_VHLNMR
        - IND_DATE TYPE BEGDA
    - CHANGING_1
        - ITAB_INDENT
            - MANDT TYPE MANDT
            - BEGDA TYPE BEGDA
            - ENDDA TYPE ENDDA
            - SHNUMBER TYPE TKNUM
            - VEHICLE TYPE OIG_VHLNMR
            - DEPOT TYPE WERKS_D
            - INDENT_DATE TYPE ZSD_CUST_INDENT-INDENT_DATE
            - INDENT_TIME TYPE TIMS

- CUST_USER_ID TYPE ZSD_CUST_INDENT-CUST_USER_ID
- KUNNR TYPE KUNWE
- KUNNR_DESC TYPE CHAR30
- SHTYPE TYPE OIG_TDSTYP
- KONDM TYPE KONDM
- PROD_CMP1 TYPE MATNR
- VAL_COMP1 TYPE BWTAR_D
- QUAN_COMP1 TYPE ZSD_CUST_INDENT-QUAN_COMP1
- PROD_CMP2 TYPE MATNR
- VAL_COMP2 TYPE BWTAR_D
- QUAN_COMP2 TYPE ZSD_CUST_INDENT-QUAN_COMP2
- PROD_CMP3 TYPE MATNR
- VAL_COMP3 TYPE BWTAR_D
- QUAN_COMP3 TYPE ZSD_CUST_INDENT-QUAN_COMP3
- PROD_CMP4 TYPE MATNR
- VAL_COMP4 TYPE BWTAR_D
- QUAN_COMP4 TYPE ZSD_CUST_INDENT-QUAN_COMP4
- PROD_CMP5 TYPE MATNR
- VAL_COMP5 TYPE BWTAR_D
- QUAN_COMP5 TYPE ZSD_CUST_INDENT-QUAN_COMP5
- PROD_CMP6 TYPE MATNR
- VAL_COMP6 TYPE BWTAR_D
- QUAN_COMP6 TYPE ZSD_CUST_INDENT-QUAN_COMP6
- PROD_CMP7 TYPE MATNR
- VAL_COMP7 TYPE BWTAR_D
- QUAN_COMP7 TYPE ZSD_CUST_INDENT-QUAN_COMP7
- PROD_CMP8 TYPE MATNR
- VAL_COMP8 TYPE BWTAR_D
- QUAN_COMP8 TYPE ZSD_CUST_INDENT-QUAN_COMP8
- SUMMARY TYPE ZSD_CUST_INDENT-SUMMARY
- ZTT_STATUS TYPE ZTT_STATUS
- ZTT_STATUS_DESC TYPE ZTT_STATUS_DESC
- ATF_FLUSH TYPE CHAR1
- CHK TYPE ZSD_CUST_INDENT-CHK
- CONTRACT TYPE ZSD_CUST_INDENT-CONTRACT
- TPT_GSTN TYPE ZSD_CUST_INDENT-TPT_GSTN

        - ERROR TYPE ZSD_CUST_INDENT-ERROR
        - ZDELETE TYPE ZSD_CUST_INDENT-ZDELETE

20. MATERIAL1
    - MATNR TYPE CHAR100
21. MATERIAL2
    - MATNR TYPE CHAR100
22. WV_ENABLE
    - VWEN1 TYPE WDY_BOOLEAN
    - VWEN2 TYPE WDY_BOOLEAN
    - VWEN3 TYPE WDY_BOOLEAN
    - VWEN4 TYPE WDY_BOOLEAN
    - VW_SV1 TYPE WDY_BOOLEAN
    - VW_SV2 TYPE WDY_BOOLEAN
    - VW_SV3 TYPE WDY_BOOLEAN
    - VW_SV4 TYPE WDY_BOOLEAN
    - CH_INDT TYPE WDY_BOOLEAN
23. NO_VEH_CONTRACT
    - VBELN1 TYPE VBELN
    - VBELN2 TYPE VBELN
24. ZSD_INDENT_NON_VEH
    - IMPORTING_2
        - INDENT_DATE TYPE ZSD_INDENT_NVEH-BEGDA
        - CREATED_BY TYPE ZSD_INDENT_NVEH-CUST_USER_ID
        - CUSTOMER_CODE TYPE ZSD_INDENT_NVEH-KUNNR
    - CHANGING_2
        - ORDER_NO TYPE ZSD_INDENT_NVEH-ORDER_NO
        - BEGDA TYPE ZSD_INDENT_NVEH-BEGDA
        - VEHICLE TYPE ZSD_INDENT_NVEH-VEHICLE
        - DEPOT TYPE ZSD_INDENT_NVEH-DEPOT
        - INDENT_DATE TYPE ZSD_INDENT_NVEH-INDENT_DATE
        - INDENT_TIME TYPE ZSD_INDENT_NVEH-INDENT_TIME
        - CUST_USER_ID TYPE ZSD_INDENT_NVEH-CUST_USER_ID
        - KUNNR TYPE ZSD_INDENT_NVEH-KUNNR
        - KUNNR_DESC TYPE ZSD_INDENT_NVEH-KUNNR_DESC
        - MATNR1 TYPE ZSD_INDENT_NVEH-MATNR1

        - QUANTITY1 TYPE ZSD_INDENT_NVEH-QUANTITY1
        - MATNR2 TYPE ZSD_INDENT_NVEH-MATNR2
        - QUANTITY2 TYPE ZSD_INDENT_NVEH-QUANTITY2
        - CONTRACT1 TYPE ZSD_INDENT_NVEH-CONTRACT1
        - CONTRACT2 TYPE ZSD_INDENT_NVEH-CONTRACT2
        - CHK TYPE ZSD_INDENT_NVEH-CHK
        - TPT_GSTN TYPE ZSD_INDENT_NVEH-TPT_GSTN
        - ERROR TYPE ZSD_INDENT_NVEH-ERROR
        - BL_QTY1 TYPE ZSD_INDENT_NVEH-BL_QTY1
        - BL_QTY2 TYPE ZSD_INDENT_NVEH-BL_QTY2
25. IND_DEL
    - DEL_IND TYPE CHAR1
26. PROD6
    - PRODUCT TYPE CHAR100
27. PROD7
    - PRODUCT TYPE CHAR100
28. PROD8
    - PRODUCT TYPE CHAR100
29. INBND_HEADER
    - UNLD_DATE TYPE BUDAT
    - UNLD_VEHICLE TYPE CHAR012
    - UNLD_NAME TYPE NAME1_GP
    - UNLD_KUNNR TYPE KUNNR
    - UNLOAD_INV TYPE CHAR0016
    - UNLD_QTY TYPE CHAR012
    - UNLD_PLANT TYPE WERKS_D
    - UNLD_MATNR TYPE MATNR
    - UNLD_ACT1 TYPE WDY_BOOLEAN
    - UNLD_ACT2 TYPE WDY_BOOLEAN
    - UNLD_ACT3 TYPE WDY_BOOLEAN
    - UNLD_ACT4 TYPE WDY_BOOLEAN
    - UNLD_STOCK TYPE CHAR012
    - UNLD_QTY_L15 TYPE CHAR012
    - UNLD_QTY_KG TYPE CHAR012
    - MOD_QTY_KL TYPE CHAR012

- MOD_QTY_KL15 TYPE CHAR012
- MOD_QTY_MT TYPE CHAR012
- UNLD_TDATE TYPE BUDAT
- UNLD_STOCK15 TYPE CHAR012
- UNLD_STOCKMT TYPE CHAR012
- MOD_VEHICLE TYPE CHAR012
- MOD_INV TYPE CHAR0016
- LN_HLD_KL TYPE CHAR012
- LN_HLD_KL15 TYPE CHAR012
- LN_HLD_MT TYPE CHAR012
- OPEN_INDT TYPE CHAR012
- BAL_QTY TYPE CHAR012

30. ZINBOUND_INDENT_LIST
    - UNLD_IMPORTING
        - UN_MATNR TYPE MATNR
    - UNLD_CHANGING
        - MANDT TYPE ZINB_INDENT_TAB-MANDT
        - IND_DATE TYPE ZINB_INDENT_TAB-IND_DATE
        - IND_MATNR TYPE ZINB_INDENT_TAB-IND_MATNR
        - IND_VEH TYPE ZINB_INDENT_TAB-IND_VEH
        - IND_QTY TYPE ZINB_INDENT_TAB-IND_QTY
        - IND_CUST TYPE ZINB_INDENT_TAB-IND_CUST
        - IND_PLANT TYPE ZINB_INDENT_TAB-IND_PLANT
        - IND_INV TYPE ZINB_INDENT_TAB-IND_INV
        - CRT_DATE TYPE ZINB_INDENT_TAB-CRT_DATE
        - CRT_USR TYPE ZINB_INDENT_TAB-CRT_USR
        - GR_NO TYPE ZINB_INDENT_TAB-GR_NO
        - CHK TYPE ZINB_INDENT_TAB-CHK
        - ZSTATUS TYPE ZINB_INDENT_TAB-ZSTATUS
31. Z_GET_CUST_UNLOAD_IN
    - CHANGING_3
        - ZINDENT_DATA
            - MANDT TYPE MANDT
            - IND_DATE TYPE DATUM
            - IND_MATNR TYPE MATNR

        - IND_VEH TYPE OIG_VHLNMR
        - IND_QTY TYPE LABST
        - IND_CUST TYPE KUNNR
        - IND_PLANT TYPE WERKS_D
        - IND_INV TYPE ZINB_INDENT_TAB-IND_INV
        - CRT_DATE TYPE DATUM
        - CRT_USR TYPE UNAME
        - GR_NO TYPE MBLNR
        - CHK TYPE ZINBINDENT_TAB-CHK
        - ZSTATUS TYPE ZINB_INDENT_TAB-ZSTATUS

32. IND_CAT
    - IND_CATEGORY TYPE CHAR100
33. IND_END_USE
    - IND_USE_MS TYPE CHAR100
    - IND_USE_HSD TYPE CHAR100

Attributes:

- WD_CONTEXT TYPE IF_WD_CONTEXT_NODE "Reference to Local Controller Context"
- WD_THIS TYPE IF_INDMAIN "Self-Reference to Local Controller Interface"
- WD_COMP_CONTROLLER TYPE IG_COMPONENTCONTROLLER "Reference to Component Controller"

Methods: View INDMAIN

- **method ONACTIONACTIVATE_GSTN .**

```
  DATA LO_ND_HEADER_DATA TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LO_EL_HEADER_DATA TYPE REF TO IF_WD_CONTEXT_ELEMENT.
  DATA LS_HEADER_DATA TYPE WD_THIS->ELEMENT_HEADER_DATA.
  DATA LV_KUNNR TYPE WD_THIS->ELEMENT_HEADER_DATA-KUNNR.
  DATA: ZKDGRP TYPE KDGRP.

* navigate from <CONTEXT> to <HEADER_DATA> via lead selection
  LO_ND_HEADER_DATA = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS-
>WDCTX_HEADER_DATA ).
* get element via lead selection
  LO_EL_HEADER_DATA = LO_ND_HEADER_DATA->GET_ELEMENT( ).
  IF LO_EL_HEADER_DATA IS INITIAL.
  ENDIF.
* get single attribute
  LO_EL_HEADER_DATA->GET_ATTRIBUTE(
   EXPORTING
    NAME = `KUNNR`
```

```
    IMPORTING
      VALUE = LV_KUNNR ).

  DATA LO_ND_ENABLE1 TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LO_EL_ENABLE1 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
  DATA LS_ENABLE1 TYPE WD_THIS->ELEMENT_ENABLE1.
  DATA LV_ET1 TYPE WD_THIS->ELEMENT_ENABLE1-ET1.

   CLEAR ZKDGRP.
   SELECT SINGLE KDGRP FROM KNVV INTO ZKDGRP WHERE KUNNR = LV_KUNNR. "#EC CI_NOORDER " #S
/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
    IF ZKDGRP = 'DI' OR ZKDGRP = 'EX'.
     LV_ET1 = 'X'.
     ELSE.
      CLEAR LV_ET1.
    ENDIF.


*     navigate from <CONTEXT> to <ENABLE1> via lead selection
     LO_ND_ENABLE1 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_ENABLE1 ).
*     get element via lead selection
     LO_EL_ENABLE1 = LO_ND_ENABLE1->GET_ELEMENT( ).

     IF LO_EL_ENABLE1 IS INITIAL.
     ENDIF.
*     set single attribute
     LO_EL_ENABLE1->SET_ATTRIBUTE(
      NAME = `ET1`
      VALUE = LV_ET1 ).
*    ENDIF.


   DATA LO_GSTNID TYPE REF TO IF_WD_CONTEXT_NODE.
   DATA LV_GSTNID TYPE wd_this->ELEMENTS_GSTN.
   DATA LS_GSTNID TYPE IG_COMPONENTCONTROLLER=>Element_GSTN.
   DATA: CT_GST TYPE CHAR18,
      CT_GSTNM TYPE CHAR50.

   CLEAR LS_GSTNID. CLEAR LV_GSTNID. REFRESH LV_GSTNID.

   LO_GSTNID = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_GSTN ).
   APPEND LS_GSTNID TO LV_GSTNID.

   SELECT GSTN NAME FROM ZTRANS_GSTN INTO (CT_GST, CT_GSTNM).
   CLEAR LS_GSTNID-TPT_GSTN.
   CONCATENATE CT_GST CT_GSTNM INTO LS_GSTNID-TPT_GSTN RESPECTING BLANKS.
*    CONDENSE LS_GSTNID-TPT_GSTN.
    APPEND LS_GSTNID TO LV_GSTNID.
    ENDSELECT.

   LO_GSTNID->bind_table( new_items = LV_GSTNID set_initial_elements = abap_true ).


endmethod.
```

- method ONACTIONCHECK_QUANTITY .

```
DATA LO_WINDOW_MANAGER TYPE REF TO IF_WD_WINDOW_MANAGER.
DATA LO_API_COMPONENT  TYPE REF TO IF_WD_COMPONENT.
DATA LO_WINDOW       TYPE REF TO IF_WD_WINDOW.
DATA: lt_string type table of string,
    ls_string like line of lt_string.
DATA l_api          TYPE REF TO if_wd_view_controller.
DATA: ERR.
DATA LO_EXPORTING TYPE REF TO IF_WD_CONTEXT_NODE.

 DATA LO_IMPORTING TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LV_CAPACITY1 TYPE CHAR012.
 DATA LV_CAPACITY2 TYPE CHAR012.
 DATA LV_CAPACITY3 TYPE CHAR012.
 DATA LV_CAPACITY4 TYPE CHAR012.
 DATA LV_CAPACITY5 TYPE CHAR012.
 DATA LV_VEHICLE TYPE OIGCC-TU_NUMBER.
 DATA: LT_C_ITAB_OIGCC TYPE TABLE OF OIGCC.

 DATA: LV_QTY1 TYPE OIGCC-CMP_MAXVOL,
     LV_QTY2 TYPE OIGCC-CMP_MAXVOL,
     LV_QTY3 TYPE OIGCC-CMP_MAXVOL,
     LV_QTY4 TYPE OIGCC-CMP_MAXVOL,
     LV_QTY5 TYPE OIGCC-CMP_MAXVOL.
 DATA: W_NUMBER(10) TYPE P DECIMALS 3.
 DATA: CAP1(16) TYPE C,
     CAP2(16) TYPE C,
     CAP3(16) TYPE C,
     CAP4(16) TYPE C,
     CAP5(16) TYPE C.

 DATA: LO_ERROR TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA: LV_ERCODE TYPE CHAR1.

 LO_ERROR = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_ERROR ).
 CLEAR LV_ERCODE.
 LO_ERROR->SET_ATTRIBUTE(
  EXPORTING
   NAME  = `ERCODE`
   VALUE = LV_ERCODE ).

LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_IMPORTING ).
* get input from context
 LO_IMPORTING->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `VEHICLE`
  IMPORTING
   VALUE = LV_VEHICLE ).

CALL FUNCTION 'Z_GET_COMP_CAPACITY'
  EXPORTING
   VEHICLE =                LV_VEHICLE
  IMPORTING
   CAPACITY1 =               LV_CAPACITY1
```

```
    CAPACITY2 =              LV_CAPACITY2
    CAPACITY3 =              LV_CAPACITY3
    CAPACITY4 =              LV_CAPACITY4
    CAPACITY5 =              LV_CAPACITY5
  TABLES
    ITAB_OIGCC =             LT_C_ITAB_OIGCC.

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_EXPORTING ).
LO_IMPORTING->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `CAPACITY1`
  IMPORTING
   VALUE = LV_QTY1 ).
CALL FUNCTION 'CEVA_CONVERT_FLOAT_TO_CHAR'
  EXPORTING
   float_imp = LV_QTY1
   format_imp = W_NUMBER "Field Format
   round_imp = ' ' "Round off
  IMPORTING
   char_exp = CAP1.
CONDENSE CAP1.

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_EXPORTING ).
LO_IMPORTING->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `CAPACITY2`
  IMPORTING
   VALUE = LV_QTY2 ).
CALL FUNCTION 'CEVA_CONVERT_FLOAT_TO_CHAR'
  EXPORTING
   float_imp = LV_QTY2
   format_imp = W_NUMBER "Field Format
   round_imp = ' ' "Round off
  IMPORTING
   char_exp = CAP2.
CONDENSE CAP2.

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_EXPORTING ).
LO_IMPORTING->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `CAPACITY3`
  IMPORTING
   VALUE = LV_QTY3 ).
CALL FUNCTION 'CEVA_CONVERT_FLOAT_TO_CHAR'
  EXPORTING
   float_imp = LV_QTY3
   format_imp = W_NUMBER "Field Format
   round_imp = ' ' "Round off
  IMPORTING
   char_exp = CAP3.
CONDENSE CAP3.
```

```
CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_EXPORTING ).
LO_IMPORTING->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `CAPACITY4`
  IMPORTING
   VALUE = LV_QTY4 ).
CALL FUNCTION 'CEVA_CONVERT_FLOAT_TO_CHAR'
  EXPORTING
   float_imp = LV_QTY4
   format_imp = W_NUMBER "Field Format
   round_imp = ' ' "Round off
  IMPORTING
   char_exp = CAP4.
CONDENSE CAP4.

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_EXPORTING ).
LO_IMPORTING->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `CAPACITY5`
  IMPORTING
   VALUE = LV_QTY5 ).
CALL FUNCTION 'CEVA_CONVERT_FLOAT_TO_CHAR'
  EXPORTING
   float_imp = LV_QTY5
   format_imp = W_NUMBER "Field Format
   round_imp = ' ' "Round off
  IMPORTING
   char_exp = CAP5.
CONDENSE CAP5.
IF CAP1 = 0. CLEAR CAP1. ENDIF.
IF CAP2 = 0. CLEAR CAP2. ENDIF.
IF CAP3 = 0. CLEAR CAP3. ENDIF.
IF CAP4 = 0. CLEAR CAP4. ENDIF.
IF CAP5 = 0. CLEAR CAP5. ENDIF.
DATA: lvc_num1   type p,
   lvc_num2   type p,
   lvc_num3   type p,
   lvc_num4   type p,
   lvc_num5   type p,
   lvm_num1   type p,
   lvm_num2   type p,
   lvm_num3   type p,
   lvm_num4   type p,
   lvm_num5   type p.

CALL FUNCTION 'MOVE_CHAR_TO_NUM'
 EXPORTING
  CHR         = LV_CAPACITY1
 IMPORTING
  NUM         = lvc_num1.
```

```
CALL FUNCTION 'MOVE_CHAR_TO_NUM'
 EXPORTING
  CHR         = CAP1
 IMPORTING
  NUM         = lvm_num1.

CALL FUNCTION 'MOVE_CHAR_TO_NUM'
 EXPORTING
  CHR         = LV_CAPACITY2
 IMPORTING
  NUM         = lvc_num2.

CALL FUNCTION 'MOVE_CHAR_TO_NUM'
 EXPORTING
  CHR         = CAP2
 IMPORTING
  NUM         = lvm_num2.

CALL FUNCTION 'MOVE_CHAR_TO_NUM'
 EXPORTING
  CHR         = LV_CAPACITY3
 IMPORTING
  NUM         = lvc_num3.

CALL FUNCTION 'MOVE_CHAR_TO_NUM'
 EXPORTING
  CHR         = CAP3
 IMPORTING
  NUM         = lvm_num3.

CALL FUNCTION 'MOVE_CHAR_TO_NUM'
 EXPORTING
  CHR         = LV_CAPACITY4
 IMPORTING
  NUM         = lvc_num4.

CALL FUNCTION 'MOVE_CHAR_TO_NUM'
 EXPORTING
  CHR         = CAP4
 IMPORTING
  NUM         = lvm_num4.

CALL FUNCTION 'MOVE_CHAR_TO_NUM'
 EXPORTING
  CHR         = LV_CAPACITY5
 IMPORTING
  NUM         = lvc_num5.

CALL FUNCTION 'MOVE_CHAR_TO_NUM'
 EXPORTING
  CHR         = CAP5
 IMPORTING
  NUM         = lvm_num5.
```

```
IF lvc_num1 < lvm_num1 OR lvc_num2 < lvm_num2 OR lvc_num3 < lvm_num3
 OR lvc_num4 < lvm_num4 OR lvc_num5 < lvm_num5.

CLEAR LO_ERROR.
LO_ERROR = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_ERROR ).
LV_ERCODE = 'X'.
LO_ERROR->SET_ATTRIBUTE(
  EXPORTING
   NAME  = `ERCODE`
   VALUE = LV_ERCODE ).


 ERR = 'X'.
 ls_string = 'Indent quantity cannot be more than compartment capacity'.
 insert ls_string into table lt_string.
 LO_API_COMPONENT        = WD_COMP_CONTROLLER->WD_GET_API( ).
 LO_WINDOW_MANAGER       = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).

 LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
         text         = lt_string
         button_kind    = if_wd_window=>co_buttons_ok
*         button_kind    = 4
         message_type   = if_wd_window=>CO_MSG_TYPE_STOPP
         window_title   = 'Please check!'
         window_position = if_wd_window=>co_center
         WINDOW_WIDTH   = '20%'
         WINDOW_HEIGHT  = '20%' ).
 LO_WINDOW->open( ).

 LO_EXPORTING = wd_context->get_child_node( name = wd_this->WDCTX_ENABLE ).
 LO_EXPORTING->set_attribute(
       name = 'SAVE'
       value = ' ' ).

ELSE.
 CLEAR ERR.
 CLEAR LO_EXPORTING.
 LO_EXPORTING = wd_context->get_child_node( name = wd_this->WDCTX_ENABLE ).
 LO_EXPORTING->set_attribute(
       name = 'SAVE'
       value = 'X' ).

ENDIF.

 DATA LO_ND_PROD1 TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_EL_PROD1 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
 DATA LS_PROD1 TYPE WD_THIS->Element_prod1.
 DATA LV_PRODUCT TYPE WD_THIS->Element_prod1-PRODUCT.
 LO_ND_PROD1 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD1 ).
 LO_EL_PROD1 = LO_ND_PROD1->GET_ELEMENT( ).

* get single attribute
 LO_EL_PROD1->GET_ATTRIBUTE(
```

```
 EXPORTING
  NAME =  `PRODUCT`
 IMPORTING
  VALUE = LV_PRODUCT ).
IF LV_PRODUCT+0(4) = 'PWAX' OR LV_PRODUCT+0(4) = 'CPC0'
 OR LV_PRODUCT+0(4) = 'SUL0' OR LV_PRODUCT+0(4) = 'RPC0'
 OR LV_PRODUCT+0(4) = 'GWAX'.

  DATA LO_ND_EXPORT TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LT_EXPORT TYPE WD_THIS->Elements_export.
  DATA LO_EL_EXPORT TYPE REF TO IF_WD_CONTEXT_ELEMENT.
  DATA LS_EXPORT TYPE WD_THIS->Element_export.
  DATA LV_VBELN TYPE WD_THIS->Element_export-VBELN.
  DATA: ZIND_NVEH TYPE ZSD_INDENT_NVEH.


  LO_ND_EXPORT = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_EXPORT ).
  LO_ND_EXPORT->GET_STATIC_ATTRIBUTES_TABLE( IMPORTING TABLE = LT_EXPORT ).

  LO_ND_EXPORT = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_EXPORT ).
  LO_EL_EXPORT = LO_ND_EXPORT->GET_ELEMENT( ).

  IF LO_EL_EXPORT IS INITIAL.
  ENDIF.

  LO_EL_EXPORT->GET_ATTRIBUTE(
   EXPORTING
    NAME =  `VBELN`
   IMPORTING
    VALUE = LV_VBELN ).


  CLEAR ZIND_NVEH.
  IF LV_VBELN IS NOT INITIAL.
   SELECT SINGLE * FROM ZSD_INDENT_NVEH INTO ZIND_NVEH WHERE
    ORDER_NO = LV_VBELN AND ORD_CLOSED = ' '.
   IF SY-SUBRC = 0.
    ZIND_NVEH-BL_QTY1 = ZIND_NVEH-BL_QTY1 * 1000.
    IF LVM_NUM1 > ZIND_NVEH-BL_QTY1.
     CLEAR LV_VBELN.
     LV_VBELN = '        '.
     LO_EL_EXPORT->SET_ATTRIBUTE(
      NAME =  `VBELN`
      VALUE = LV_VBELN ).

     LO_ND_EXPORT->bind_table( new_items = LT_EXPORT set_initial_elements = abap_true ).

     ERR = 'X'.
     ZIND_NVEH-BL_QTY1 = ZIND_NVEH-BL_QTY1 / 1000.
     ls_string = ZIND_NVEH-BL_QTY1.
     condense ls_string.
     CONCATENATE 'Order balance =' ls_string 'is less than indent quantity. '
      'You may close the order.' into ls_string separated by space.
     insert ls_string into table lt_string.
```

```
        LO_API_COMPONENT        = WD_COMP_CONTROLLER->WD_GET_API( ).
        LO_WINDOW_MANAGER       = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).

        LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
              text         = lt_string
              button_kind    = if_wd_window=>co_buttons_ok
*              button_kind    = 4
              message_type   = if_wd_window=>CO_MSG_TYPE_STOPP
              window_title   = 'Please check!'
              window_position = if_wd_window=>co_center
              WINDOW_WIDTH   = '20%'
              WINDOW_HEIGHT  = '20%' ).
        LO_WINDOW->open( ).

        LO_EXPORTING = wd_context->get_child_node( name = wd_this->WDCTX_ENABLE ).
        LO_EXPORTING->set_attribute(
             name = 'SAVE'
             value = ' ' ).
       ENDIF.
      ENDIF.
     ENDIF.



    ENDIF.



    endmethod.
```

- method ONACTIONDELETE_INDENT .

```
DATA lt_indent TYPE ig_componentcontroller=>Elements_itab_indent.
DATA ls_indent TYPE ig_componentcontroller=>Element_itab_indent.
DATA LO_Z_GET_CUSTOMER_INDEN TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_Z_GET_CUSTOMER_INDEN_CHANG TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_IMPORTING TYPE REF TO IF_WD_CONTEXT_NODE.
DATA: CT TYPE I,
   CT1 TYPE I,
   CT2 TYPE I.

LO_Z_GET_CUSTOMER_INDEN = WD_CONTEXT->GET_CHILD_NODE( WD_THIS-
>WDCTX_Z_GET_CUSTOMER_INDEN ).
LO_Z_GET_CUSTOMER_INDEN_CHANG = LO_Z_GET_CUSTOMER_INDEN->GET_CHILD_NODE( WD_THIS-
>WDCTX_CHANGING_1 ).
LO_IMPORTING = LO_Z_GET_CUSTOMER_INDEN_CHANG->GET_CHILD_NODE( WD_THIS-
>WDCTX_ITAB_INDENT ).
LO_IMPORTING-
>get_static_attributes_table( IMPORTING table = lt_indent ).  "#EC CI_FLDEXT_OK[2215424] " #S/4 - OSS No
te# - Added by - Z_SHSHRUKHA – 2021/08/18



LOOP AT lt_indent INTO ls_indent.
 if ls_indent-CHK = 'X' and ( ls_indent-ZTT_STATUS = '4' or ls_indent-ZTT_STATUS = '5' ).
  CT = CT + 1.
 endif.
 if ls_indent-CHK = 'X' and ls_indent-ZTT_STATUS <> '4' and ls_indent-ZTT_STATUS <> '5'.
```

```
  CT1 = CT1 + 1.
 endif.

 if ls_indent-CHK = 'X'.
  CT2 = CT2 + 1.
 endif.
ENDLOOP.

*** Deletion check POPUP message START ***
DATA LO_WINDOW_MANAGER TYPE REF TO IF_WD_WINDOW_MANAGER.
DATA LO_API_COMPONENT  TYPE REF TO IF_WD_COMPONENT.
DATA LO_WINDOW       TYPE REF TO IF_WD_WINDOW.
data: lt_string type table of string,
    ls_string like line of lt_string.
DATA l_api           TYPE REF TO if_wd_view_controller.

 LO_API_COMPONENT        = WD_COMP_CONTROLLER->WD_GET_API( ).
 LO_WINDOW_MANAGER       = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).

 if CT2 = 0.
  ls_string = 'Please select the indents to be deleted'.
  insert ls_string into table lt_string.
  LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
              text         = lt_string
              button_kind    = if_wd_window=>co_buttons_ok
*              button_kind    = 4
              message_type   = if_wd_window=>CO_MSG_TYPE_QUESTION
              window_title   = 'Please check!'
              window_position = if_wd_window=>co_center ).

  LO_WINDOW->open( ).
 endif.

 if CT = 0 and CT1 > 0.
  ls_string = 'Selected indents already processed and cannot be deleted'.
  insert ls_string into table lt_string.
  LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
              text         = lt_string
              button_kind    = if_wd_window=>co_buttons_ok
*              button_kind    = 4
              message_type   = if_wd_window=>CO_MSG_TYPE_QUESTION
              window_title   = 'Please check!'
              window_position = if_wd_window=>co_center ).

  LO_WINDOW->open( ).
 endif.

 if CT > 0 and CT1 = 0.
   ls_string = 'Do you want to delete the selected indents?'.
   insert ls_string into table lt_string.
 endif.

 if CT > 0 and CT1 > 0.
  ls_string = 'Only indents not yet processed will be deleted?'.
```

```
  insert ls_string into table lt_string.
 endif.

 if CT > 0.
  LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
              text         = lt_string
 *             button_kind    = if_wd_window=>co_buttons_ok
              button_kind    = 4
              message_type   = if_wd_window=>CO_MSG_TYPE_QUESTION
              window_title   = 'Please check!'
              window_position = if_wd_window=>co_center ).

  DATA LO_API_START_VIEW TYPE REF TO IF_WD_VIEW_CONTROLLER.
  LO_API_START_VIEW = WD_THIS->WD_GET_API( ).

  CALL METHOD LO_WINDOW->SUBSCRIBE_TO_BUTTON_EVENT
   EXPORTING
    BUTTON    = if_wd_window=>co_button_yes
    ACTION_NAME = 'REACT_TO_YES'
    ACTION_VIEW = LO_API_START_VIEW.

  LO_WINDOW->open( ).

  CALL METHOD LO_WINDOW_MANAGER->CREATE_POPUP_TO_CONFIRM
   EXPORTING
    TEXT             = lt_string
    BUTTON_KIND        = 4
 *    MESSAGE_TYPE       =
 *    CLOSE_BUTTON       = ABAP_TRUE
 *    WINDOW_TITLE       =
 *    WINDOW_LEFT_POSITION =
 *    WINDOW_TOP_POSITION  =
 *    WINDOW_POSITION    =
 *    WINDOW_WIDTH       =
 *    WINDOW_HEIGHT      =
 *    DEFAULT_BUTTON     =
   RECEIVING
    RESULT           = lo_window.
 endif.
 *** Deletion check POPUP message END ***

 endmethod.
```

- method ONACTIONDELETE_INDENT .

```
DATA lt_indent TYPE ig_componentcontroller=>Elements_itab_indent.
DATA ls_indent TYPE ig_componentcontroller=>Element_itab_indent.
DATA LO_Z_GET_CUSTOMER_INDEN TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_Z_GET_CUSTOMER_INDEN_CHANG TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_IMPORTING TYPE REF TO IF_WD_CONTEXT_NODE.
DATA: CT TYPE I,
    CT1 TYPE I,
    CT2 TYPE I.

LO_Z_GET_CUSTOMER_INDEN = WD_CONTEXT->GET_CHILD_NODE( WD_THIS-
>WDCTX_Z_GET_CUSTOMER_INDEN ).
```

```
LO_Z_GET_CUSTOMER_INDEN_CHANG = LO_Z_GET_CUSTOMER_INDEN->GET_CHILD_NODE( WD_THIS-
>WDCTX_CHANGING_1 ).
LO_IMPORTING = LO_Z_GET_CUSTOMER_INDEN_CHANG->GET_CHILD_NODE( WD_THIS-
>WDCTX_ITAB_INDENT ).
LO_IMPORTING-
>get_static_attributes_table( IMPORTING table = lt_indent ).  "#EC CI_FLDEXT_OK[2215424] " #S/4 - OSS No
te# - Added by - Z_SHSHRUKHA – 2021/08/18


LOOP AT lt_indent INTO ls_indent.
 if ls_indent-CHK = 'X' and ( ls_indent-ZTT_STATUS = '4' or ls_indent-ZTT_STATUS = '5' ).
  CT = CT + 1.
 endif.
 if ls_indent-CHK = 'X' and ls_indent-ZTT_STATUS <> '4' and ls_indent-ZTT_STATUS <> '5'.
  CT1 = CT1 + 1.
 endif.

 if ls_indent-CHK = 'X'.
  CT2 = CT2 + 1.
 endif.
ENDLOOP.

*** Deletion check POPUP message START ***
DATA LO_WINDOW_MANAGER TYPE REF TO IF_WD_WINDOW_MANAGER.
DATA LO_API_COMPONENT  TYPE REF TO IF_WD_COMPONENT.
DATA LO_WINDOW       TYPE REF TO IF_WD_WINDOW.
data: lt_string type table of string,
    ls_string like line of lt_string.
DATA l_api          TYPE REF TO if_wd_view_controller.

 LO_API_COMPONENT        = WD_COMP_CONTROLLER->WD_GET_API( ).
 LO_WINDOW_MANAGER       = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).

 if CT2 = 0.
  ls_string = 'Please select the indents to be deleted'.
  insert ls_string into table lt_string.
  LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
             text         = lt_string
             button_kind    = if_wd_window=>co_buttons_ok
*             button_kind    = 4
             message_type   = if_wd_window=>CO_MSG_TYPE_QUESTION
             window_title   = 'Please check!'
             window_position = if_wd_window=>co_center ).

  LO_WINDOW->open( ).
 endif.

 if CT = 0 and CT1 > 0.
  ls_string = 'Selected indents already processed and cannot be deleted'.
  insert ls_string into table lt_string.
  LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
             text         = lt_string
             button_kind    = if_wd_window=>co_buttons_ok
*             button_kind    = 4
```

```
            message_type    = if_wd_window=>CO_MSG_TYPE_QUESTION
            window_title    = 'Please check!'
            window_position = if_wd_window=>co_center ).

  LO_WINDOW->open( ).
endif.

if CT > 0 and CT1 = 0.
  ls_string = 'Do you want to delete the selected indents?'.
  insert ls_string into table lt_string.
endif.

if CT > 0 and CT1 > 0.
  ls_string = 'Only indents not yet processed will be deleted?'.
  insert ls_string into table lt_string.
endif.

if CT > 0.
 LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
            text         = lt_string
*             button_kind    = if_wd_window=>co_buttons_ok
            button_kind    = 4
            message_type    = if_wd_window=>CO_MSG_TYPE_QUESTION
            window_title    = 'Please check!'
            window_position = if_wd_window=>co_center ).

 DATA LO_API_START_VIEW TYPE REF TO IF_WD_VIEW_CONTROLLER.
 LO_API_START_VIEW = WD_THIS->WD_GET_API( ).

 CALL METHOD LO_WINDOW->SUBSCRIBE_TO_BUTTON_EVENT
  EXPORTING
   BUTTON    = if_wd_window=>co_button_yes
   ACTION_NAME = 'REACT_TO_YES'
   ACTION_VIEW = LO_API_START_VIEW.

 LO_WINDOW->open( ).

 CALL METHOD LO_WINDOW_MANAGER->CREATE_POPUP_TO_CONFIRM
  EXPORTING
   TEXT            = lt_string
   BUTTON_KIND        = 4
*    MESSAGE_TYPE       =
*    CLOSE_BUTTON       = ABAP_TRUE
*    WINDOW_TITLE       =
*    WINDOW_LEFT_POSITION =
*    WINDOW_TOP_POSITION  =
*    WINDOW_POSITION    =
*    WINDOW_WIDTH       =
*    WINDOW_HEIGHT      =
*    DEFAULT_BUTTON     =
  RECEIVING
   RESULT          = lo_window.
endif.
*** Deletion check POPUP message END ***
```

endmethod.

- method ONACTIONFLUSHING_REASON .

```
DATA LO_ND_ATF_FLUSH TYPE REF TO IF_WD_CONTEXT_NODE.

DATA LO_EL_ATF_FLUSH TYPE REF TO IF_WD_CONTEXT_ELEMENT.
DATA LS_ATF_FLUSH TYPE WD_THIS->ELEMENT_ATF_FLUSH.
DATA LV_DESC TYPE WD_THIS->ELEMENT_ATF_FLUSH-DESC.
LO_ND_ATF_FLUSH = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_ATF_FLUSH ).
LO_EL_ATF_FLUSH = LO_ND_ATF_FLUSH->GET_ELEMENT( ).
LO_EL_ATF_FLUSH->GET_ATTRIBUTE(
 EXPORTING
  NAME = `DESC`
 IMPORTING
  VALUE = LV_DESC ).

DATA LO_ND_ENABLE1 TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_EL_ENABLE1 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
DATA LS_ENABLE1 TYPE WD_THIS->ELEMENT_ENABLE1.
DATA LV_ET2 TYPE WD_THIS->ELEMENT_ENABLE1-ET2.
LO_ND_ENABLE1 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_ENABLE1 ).
LO_EL_ENABLE1 = LO_ND_ENABLE1->GET_ELEMENT( ).
IF LV_DESC = 'YES'. LV_ET2 = 'X'. ELSE. CLEAR LV_ET2. ENDIF.
LO_EL_ENABLE1->SET_ATTRIBUTE(
 NAME = `ET2`
 VALUE = LV_ET2 ).


DATA LO_ATF_REAS TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LV_ATF_REAS TYPE wd_this->ELEMENTS_FLUSH_REASON.
DATA LS_ATF_REAS TYPE IG_COMPONENTCONTROLLER=>Element_FLUSH_REASON.

CLEAR LS_ATF_REAS. CLEAR LV_ATF_REAS. REFRESH LV_ATF_REAS.
LO_ATF_REAS = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_FLUSH_REASON ).
 APPEND LS_ATF_REAS TO LV_ATF_REAS.
 SELECT DDTEXT FROM DD07T INTO LS_ATF_REAS-FLSH_REASON WHERE
   DOMNAME = 'ZATF_FLUSH_REASON'.
   APPEND LS_ATF_REAS TO LV_ATF_REAS.
   ENDSELECT.
  LO_ATF_REAS->bind_table( new_items = LV_ATF_REAS set_initial_elements = abap_true ).
```

endmethod.

- method ONACTIONGET_DIR_CUST_INDENTS .

```
 DATA LO_COMPONENTCONTROLLER TYPE REF TO IG_COMPONENTCONTROLLER .
  LO_COMPONENTCONTROLLER =   WD_THIS->GET_COMPONENTCONTROLLER_CTR( ).

   LO_COMPONENTCONTROLLER->EXECUTE_Z_GET_CUST_INDENT_NO_V(
   ).
```

endmethod.

- method ONACTIONGET_INDENTS .

```
DATA LO_COMPONENTCONTROLLER TYPE REF TO IG_COMPONENTCONTROLLER .
```

```
LO_COMPONENTCONTROLLER =  WD_THIS->GET_COMPONENTCONTROLLER_CTR( ).

 LO_COMPONENTCONTROLLER->EXECUTE_Z_GET_CUSTOMER_INDENT(
 ).

DATA LO_EXPORTING TYPE REF TO IF_WD_CONTEXT_NODE.
LO_EXPORTING = wd_context->get_child_node( name = wd_this->WDCTX_ENABLE ).
LO_EXPORTING->set_attribute(
    name = 'DELETE'
    value = 'X' ).

LO_EXPORTING->set_attribute(
    name = 'SUBMIT'
    value = 'X' ).

LO_EXPORTING->set_attribute(
    name = 'SUBMIT_ALL'
    value = 'X' ).
endmethod.
```

- method ONACTIONGET_VEHICLE .

```
  DATA LO_ND_IND_END_USE TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LO_EL_IND_END_USE TYPE REF TO IF_WD_CONTEXT_ELEMENT.
  DATA LS_IND_END_USE TYPE WD_THIS->Element_ind_end_use.
  DATA LV_IND_USE_MS TYPE WD_THIS->Element_ind_end_use-IND_USE_MS.
  DATA LV_IND_USE_HSD TYPE WD_THIS->Element_ind_end_use-IND_USE_HSD.

  LO_ND_IND_END_USE = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS-
>WDCTX_IND_END_USE ).
  LO_EL_IND_END_USE = LO_ND_IND_END_USE->GET_ELEMENT( ).
  IF LO_EL_IND_END_USE IS INITIAL.
  ENDIF.
* set single attribute
  CLEAR: LV_IND_USE_MS, LV_IND_USE_HSD.
  LO_EL_IND_END_USE->SET_ATTRIBUTE(
   NAME = `IND_USE_MS`
   VALUE = LV_IND_USE_MS ).
  LO_EL_IND_END_USE->SET_ATTRIBUTE(
   NAME = `IND_USE_HSD`
   VALUE = LV_IND_USE_HSD ).



*** Call function module in method to check TT licenses START **
 DATA LO_ND_HEADER_DATA TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_EL_HEADER_DATA TYPE REF TO IF_WD_CONTEXT_ELEMENT.
 DATA LS_HEADER_DATA TYPE WD_THIS->ELEMENT_HEADER_DATA.
 DATA LV_NAME1 LIKE LS_HEADER_DATA-NAME1.
 DATA LV_KUNNR LIKE LS_HEADER_DATA-KUNNR.
 DATA: CUST_NAME TYPE KNA1-NAME1.
 DATA: IT_NOPROD TYPE TABLE OF ZSD_CUST_NO_PRD,
     WA_NOPROD LIKE LINE OF IT_NOPROD,
     ZUSR TYPE ZSD_CUST_USR_MAP.
```

```
 LO_ND_HEADER_DATA = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS-
>WDCTX_HEADER_DATA ).
 LO_EL_HEADER_DATA = LO_ND_HEADER_DATA->GET_ELEMENT( ).
 LO_EL_HEADER_DATA->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `KUNNR`
  IMPORTING
   VALUE = LV_KUNNR ).
 CLEAR CUST_NAME.
 SELECT SINGLE NAME1 FROM KNA1 INTO CUST_NAME WHERE KUNNR = LV_KUNNR.

 LO_EL_HEADER_DATA->SET_ATTRIBUTE(
  EXPORTING
   NAME =  `NAME1`
   VALUE =  CUST_NAME ).


 DATA LO_ND_ENABLE2 TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_EL_ENABLE2 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
 DATA LS_ENABLE2 TYPE WD_THIS->ELEMENT_ENABLE2.
 DATA LV_ZT1 TYPE WD_THIS->ELEMENT_ENABLE2-ZT1.
 LO_ND_ENABLE2 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_ENABLE2 ).
 LO_EL_ENABLE2 = LO_ND_ENABLE2->GET_ELEMENT( ).
 LO_EL_ENABLE2->GET_ATTRIBUTE(
  EXPORTING
   NAME =  `ZT1`
  IMPORTING
   VALUE = LV_ZT1 ).

 IF LV_ZT1 = 'X'.
     DATA LV_ZT3 TYPE WD_THIS->ELEMENT_ENABLE2-ZT3.
     LO_ND_ENABLE2 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_ENABLE2 ).
     LO_EL_ENABLE2 = LO_ND_ENABLE2->GET_ELEMENT( ).
     CLEAR LV_ZT3.
     LO_EL_ENABLE2->SET_ATTRIBUTE(
      NAME =  `ZT3`
      VALUE = LV_ZT3 ).

     DATA LO_ND_PROD1 TYPE REF TO IF_WD_CONTEXT_NODE.
     DATA LO_EL_PROD1 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
     DATA LS_PROD1 TYPE WD_THIS->ELEMENT_PROD1.
     DATA LV_PRODUCT1 TYPE WD_THIS->ELEMENT_PROD1-PRODUCT.
     LO_ND_PROD1 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD1 ).
     LO_EL_PROD1 = LO_ND_PROD1->GET_ELEMENT( ).
     LO_EL_PROD1->GET_ATTRIBUTE(
      EXPORTING
       NAME =  `PRODUCT`
      IMPORTING
       VALUE = LV_PRODUCT1 ).

     DATA LO_ND_PROD2 TYPE REF TO IF_WD_CONTEXT_NODE.
     DATA LO_EL_PROD2 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
     DATA LS_PROD2 TYPE WD_THIS->ELEMENT_PROD2.
     DATA LV_PRODUCT2 TYPE WD_THIS->ELEMENT_PROD2-PRODUCT.
```

```
LO_ND_PROD2 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD2 ).
LO_EL_PROD2 = LO_ND_PROD2->GET_ELEMENT( ).
LO_EL_PROD2->GET_ATTRIBUTE(
 EXPORTING
  NAME = `PRODUCT`
 IMPORTING
  VALUE = LV_PRODUCT2 ).

DATA LO_ND_PROD3 TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_EL_PROD3 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
DATA LS_PROD3 TYPE WD_THIS->ELEMENT_PROD3.
DATA LV_PRODUCT3 TYPE WD_THIS->ELEMENT_PROD3-PRODUCT.
LO_ND_PROD3 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD3 ).
LO_EL_PROD3 = LO_ND_PROD3->GET_ELEMENT( ).
LO_EL_PROD3->GET_ATTRIBUTE(
 EXPORTING
  NAME = `PRODUCT`
 IMPORTING
  VALUE = LV_PRODUCT3 ).

DATA LO_ND_PROD4 TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_EL_PROD4 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
DATA LS_PROD4 TYPE WD_THIS->ELEMENT_PROD4.
DATA LV_PRODUCT4 TYPE WD_THIS->ELEMENT_PROD4-PRODUCT.
LO_ND_PROD4 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD4 ).
LO_EL_PROD4 = LO_ND_PROD4->GET_ELEMENT( ).
LO_EL_PROD4->GET_ATTRIBUTE(
 EXPORTING
  NAME = `PRODUCT`
 IMPORTING
  VALUE = LV_PRODUCT4 ).

DATA LO_ND_PROD5 TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_EL_PROD5 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
DATA LS_PROD5 TYPE WD_THIS->ELEMENT_PROD5.
DATA LV_PRODUCT5 TYPE WD_THIS->ELEMENT_PROD5-PRODUCT.
LO_ND_PROD5 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD5 ).
LO_EL_PROD5 = LO_ND_PROD5->GET_ELEMENT( ).
LO_EL_PROD5->GET_ATTRIBUTE(
 EXPORTING
  NAME = `PRODUCT`
 IMPORTING
  VALUE = LV_PRODUCT5 ).

DATA LO_ND_PROD6 TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_EL_PROD6 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
DATA LS_PROD6 TYPE WD_THIS->ELEMENT_PROD6.
DATA LV_PRODUCT6 TYPE WD_THIS->ELEMENT_PROD6-PRODUCT.
LO_ND_PROD6 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD6 ).
LO_EL_PROD6 = LO_ND_PROD6->GET_ELEMENT( ).
LO_EL_PROD6->GET_ATTRIBUTE(
 EXPORTING
  NAME = `PRODUCT`
 IMPORTING
```

```
      VALUE = LV_PRODUCT6 ).

    DATA LO_ND_PROD7 TYPE REF TO IF_WD_CONTEXT_NODE.
    DATA LO_EL_PROD7 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
    DATA LS_PROD7 TYPE WD_THIS->ELEMENT_PROD7.
    DATA LV_PRODUCT7 TYPE WD_THIS->ELEMENT_PROD7-PRODUCT.
    LO_ND_PROD7 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD7 ).
    LO_EL_PROD7 = LO_ND_PROD7->GET_ELEMENT( ).
    LO_EL_PROD7->GET_ATTRIBUTE(
     EXPORTING
      NAME = `PRODUCT`
     IMPORTING
      VALUE = LV_PRODUCT7 ).

    DATA LO_ND_PROD8 TYPE REF TO IF_WD_CONTEXT_NODE.
    DATA LO_EL_PROD8 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
    DATA LS_PROD8 TYPE WD_THIS->ELEMENT_PROD8.
    DATA LV_PRODUCT8 TYPE WD_THIS->ELEMENT_PROD8-PRODUCT.
    LO_ND_PROD8 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD8 ).
    LO_EL_PROD8 = LO_ND_PROD8->GET_ELEMENT( ).
    LO_EL_PROD8->GET_ATTRIBUTE(
     EXPORTING
      NAME = `PRODUCT`
     IMPORTING
      VALUE = LV_PRODUCT8 ).


   DATA lr_event1 TYPE REF TO cl_wd_custom_event.
   DATA lr_event TYPE REF TO cl_wd_custom_event.
    CREATE OBJECT lr_event
      EXPORTING
       name = 'SELECT_EXPORT'.

    wd_this->ONACTIONSELECT_EXPORT(
     wdevent = lr_event1                    " ref to cl_wd_custom_event
    ).

    CLEAR: lr_event, lr_event1.
    CREATE OBJECT lr_event
      EXPORTING
       name = 'ACTIVATE_GSTN'.

    wd_this->ONACTIONACTIVATE_GSTN(
     wdevent = lr_event1                    " ref to cl_wd_custom_event
    ).


  ENDIF.

DATA LO_WINDOW_MANAGER TYPE REF TO IF_WD_WINDOW_MANAGER.
DATA LO_API_COMPONENT  TYPE REF TO IF_WD_COMPONENT.
DATA LO_WINDOW       TYPE REF TO IF_WD_WINDOW.
data: lt_string type table of string,
    ls_string like line of lt_string.
```

```
DATA l_api          TYPE REF TO if_wd_view_controller.

DATA LO_IMPORTING TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LV_VEHICLE TYPE OIGCC-TU_NUMBER.
DATA LV_ERROR TYPE CHAR1.
DATA LV_ERROR2 TYPE CHAR1.
DATA: LT_C_RETURN TYPE TABLE OF BAPIRET2.
DATA: ITAB_RETURN LIKE LINE OF LT_C_RETURN.
DATA VEH_TYPE TYPE OIGV-VEH_TYPE.
DATA CARRIER TYPE OIGV-CARRIER.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_IMPORTING ).
* get input from context
 LO_IMPORTING->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `VEHICLE`
  IMPORTING
   VALUE = LV_VEHICLE ).

 DATA LO_ND_WV_ENABLE TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_EL_WV_ENABLE TYPE REF TO IF_WD_CONTEXT_ELEMENT.
 DATA LS_WV_ENABLE TYPE WD_THIS->ELEMENT_WV_ENABLE.
 DATA LV_CH_INDT TYPE WD_THIS->ELEMENT_WV_ENABLE-CH_INDT.
 LO_ND_WV_ENABLE = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_WV_ENABLE ).
 LO_EL_WV_ENABLE = LO_ND_WV_ENABLE->GET_ELEMENT( ).
 IF LO_EL_WV_ENABLE IS NOT INITIAL.
  LO_EL_WV_ENABLE->GET_ATTRIBUTE(
   EXPORTING
    NAME =  `CH_INDT`
   IMPORTING
    VALUE = LV_CH_INDT ).
 ENDIF.


DATA: IDATE TYPE BEGDA.
DATA: ZSHIPMNT TYPE OIGSI-SHNUMBER.
DATA: ZTTSTST TYPE OIGS-OIG_SSTSF.


CLEAR IDATE. CLEAR ZSHIPMNT. CLEAR ZTTSTST.
IF LV_CH_INDT = ' '.
 SELECT MAX( BEGDA ) FROM ZSD_CUST_INDENT INTO IDATE
  WHERE VEHICLE = LV_VEHICLE AND ( ZTT_STATUS = '5' OR ZTT_STATUS = '1' OR ZTT_STATUS = '2'
  OR ZTT_STATUS = '4' ) AND ZDELETE = 'Y'.
  IF IDATE IS NOT INITIAL.
   SELECT SINGLE SHNUMBER FROM ZSD_CUST_INDENT INTO ZSHIPMNT WHERE VEHICLE = LV_VEHICLE
AND BEGDA = IDATE  "#EC CI_NOORDER " #S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
     AND ZTT_STATUS <> '1' AND ZTT_STATUS <> '2' AND ZTT_STATUS <> '4' AND ZTT_STATUS <> '5'.
   IF  ZSHIPMNT <> ' '.
    SELECT SINGLE OIG_SSTSF FROM OIGS INTO ZTTSTST WHERE SHNUMBER = ZSHIPMNT.
     IF ZTTSTST = '2' OR ZTTSTST = '3' OR ZTTSTST = '4' OR ZTTSTST = '5'.
      LV_ERROR = 'X'. LV_ERROR2 = 'X'.
      CONCATENATE 'Please delete the open indent for the vehicle on -
'  IDATE+6(2) '.' IDATE+4(2) '.' IDATE+0(4)
       into ls_string .
```

```
      APPEND ls_string TO lt_string.
      LO_API_COMPONENT        = WD_COMP_CONTROLLER->WD_GET_API( ).
       LO_WINDOW_MANAGER       = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).
       LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
                   text         = lt_string
                   button_kind    = if_wd_window=>co_buttons_ok
*                   button_kind    = 4
                   message_type   = if_wd_window=>CO_MSG_TYPE_STOPP
                   window_title   = 'Please check!'
                   window_position = if_wd_window=>co_center
                   WINDOW_WIDTH   = '20%'
                   WINDOW_HEIGHT  = '20%' ).
       LO_WINDOW->open( ).
    ENDIF.
   ENDIF.
  ENDIF.
ENDIF.

CLEAR LV_CH_INDT.
LO_EL_WV_ENABLE->SET_ATTRIBUTE(
  NAME =  `CH_INDT`
  VALUE = LV_CH_INDT ).

 IF LV_ERROR <> 'X'.
  SELECT SINGLE CARRIER FROM OIGV INTO CARRIER WHERE VEHICLE = LV_VEHICLE.
  CALL FUNCTION 'Z_CHECK_VEHICLE_LICENSE'
    EXPORTING
      VEHICLE =                 LV_VEHICLE
    IMPORTING
      ERROR =                  LV_ERROR
    TABLES
      RETURN =                  LT_C_RETURN.
 ENDIF.

CLEAR LO_IMPORTING.

if LV_ERROR = 'X' and LV_ERROR2 <> 'X'.
 LOOP AT LT_C_RETURN INTO ITAB_RETURN.
  ls_string = ITAB_RETURN-MESSAGE.
  insert ls_string into table lt_string.
 ENDLOOP.

   LO_API_COMPONENT        = WD_COMP_CONTROLLER->WD_GET_API( ).
   LO_WINDOW_MANAGER       = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).
   LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
               text         = lt_string
               button_kind    = if_wd_window=>co_buttons_ok
               message_type   = if_wd_window=>CO_MSG_TYPE_STOPP
               window_title   = 'Please check!'
               window_position = if_wd_window=>co_center
               WINDOW_WIDTH   = '50%'
               WINDOW_HEIGHT  = '50%' ).
   LO_WINDOW->open( ).
```

```
 endif.

*** Call function module in method to check TT licenses END **

*** Call function module in method to get compartment details START **
DATA LO_COMPONENTCONTROLLER TYPE REF TO IG_COMPONENTCONTROLLER .
CLEAR LO_COMPONENTCONTROLLER.
if LV_ERROR <> 'X'.
 LO_COMPONENTCONTROLLER =   WD_THIS->GET_COMPONENTCONTROLLER_CTR( ).

   LO_COMPONENTCONTROLLER->EXECUTE_Z_GET_COMP_CAPACITY(
   ).
 else. clear LO_COMPONENTCONTROLLER.

DATA VEH_EXPORTING TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LV_CAPACITY1 TYPE CHAR012.
DATA LV_CAPACITY2 TYPE CHAR012.
DATA LV_CAPACITY3 TYPE CHAR012.
DATA LV_CAPACITY4 TYPE CHAR012.
DATA LV_CAPACITY5 TYPE CHAR012.
DATA LV_CAPACITY6 TYPE CHAR012.
DATA LV_CAPACITY7 TYPE CHAR012.
DATA LV_CAPACITY8 TYPE CHAR012.

VEH_EXPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_EXPORTING ).
clear: LV_CAPACITY1, LV_CAPACITY2, LV_CAPACITY3, LV_CAPACITY4, LV_CAPACITY5, LV_CAPACITY6, LV_C
APACITY7, LV_CAPACITY8.

VEH_EXPORTING->SET_ATTRIBUTE(
   EXPORTING
    NAME  = `CAPACITY1`
    VALUE = LV_CAPACITY1 ).

VEH_EXPORTING->SET_ATTRIBUTE(
   EXPORTING
    NAME  = `CAPACITY2`
    VALUE = LV_CAPACITY2 ).

VEH_EXPORTING->SET_ATTRIBUTE(
   EXPORTING
    NAME  = `CAPACITY3`
    VALUE = LV_CAPACITY3 ).

VEH_EXPORTING->SET_ATTRIBUTE(
   EXPORTING
    NAME  = `CAPACITY4`
    VALUE = LV_CAPACITY4 ).

VEH_EXPORTING->SET_ATTRIBUTE(
   EXPORTING
    NAME  = `CAPACITY5`
    VALUE = LV_CAPACITY5 ).

VEH_EXPORTING->SET_ATTRIBUTE(
```

```
  EXPORTING
   NAME  = `CAPACITY6`
   VALUE = LV_CAPACITY6 ).

VEH_EXPORTING->SET_ATTRIBUTE(
  EXPORTING
   NAME  = `CAPACITY7`
   VALUE = LV_CAPACITY7 ).

VEH_EXPORTING->SET_ATTRIBUTE(
  EXPORTING
   NAME  = `CAPACITY8`
   VALUE = LV_CAPACITY8 ).
endif.
*** Call function module in method to get compartment details END **


*** Get Vehicle type and Enable Compartments START **
DATA LO_EXPORTING TYPE REF TO IF_WD_CONTEXT_NODE.
DATA CNT TYPE I.
DATA: VL1(1) TYPE C,
   VL2(1) TYPE C,
   VL3(1) TYPE C,
   VL4(1) TYPE C,
   VL5(1) TYPE C,
   VL6(1) TYPE C,
   VL7(1) TYPE C,
   VL8(1) TYPE C.
CLEAR: ls_string, lt_string.
REFRESH lt_string.

LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_IMPORTING ).
  select count(*) from oigcc into cnt where tu_number = LV_VEHICLE.
  select single veh_type from oigv into veh_type where
   vehicle = LV_VEHICLE.
LO_EXPORTING = wd_context->get_child_node( name = wd_this->WDCTX_ENABLE ).

  clear: VL1, VL2, VL3, VL4, VL5, VL6, VL7, VL8.
  if cnt > 0 and LV_ERROR <> 'X'. VL1 = 'X'. else. clear VL1. endif.
  LO_EXPORTING->set_attribute(
      name = 'QN1'
      value = VL1 ).

  if cnt > 1 and LV_ERROR <> 'X'. VL2 = 'X'. else. clear VL2. endif.
  LO_EXPORTING->set_attribute(
      name = 'QN2'
      value = VL2 ).

  if cnt > 2 and LV_ERROR <> 'X'. VL3 = 'X'. else. clear VL3. endif.
  LO_EXPORTING->set_attribute(
      name = 'QN3'
      value = VL3 ).

  if cnt > 3 and LV_ERROR <> 'X'. VL4 = 'X'. else. clear VL4. endif.
```

```
LO_EXPORTING->set_attribute(
    name = 'QN4'
    value = VL4 ).

if cnt > 4 and LV_ERROR <> 'X'. VL5 = 'X'. else. clear VL5. endif.
LO_EXPORTING->set_attribute(
    name = 'QN5'
    value = VL5 ).

if cnt > 5 and LV_ERROR <> 'X'. VL6 = 'X'. else. clear VL6. endif.
LO_EXPORTING->set_attribute(
    name = 'QN6'
    value = VL6 ).

if cnt > 6 and LV_ERROR <> 'X'. VL7 = 'X'. else. clear VL7. endif.
LO_EXPORTING->set_attribute(
    name = 'QN7'
    value = VL7 ).

if cnt > 7 and LV_ERROR <> 'X'. VL8 = 'X'. else. clear VL8. endif.
LO_EXPORTING->set_attribute(
    name = 'QN8'
    value = VL8 ).

if ( veh_type = 'TTV' or veh_type = 'ATF' or veh_type = 'WOIL' or veh_type = 'TTVM' ) and LV_ERROR <> 'X'.
 clear: VL1, VL2, VL3, VL4, VL5, VL6.
 IF CNT = 1. VL1 = 'X'. ENDIF.
 IF CNT = 2. VL1 = 'X'. VL2 = 'X'. ENDIF.
 IF CNT = 3. VL1 = 'X'. VL2 = 'X'. VL3 = 'X'. ENDIF.
 IF CNT = 4. VL1 = 'X'. VL2 = 'X'. VL3 = 'X'. VL4 = 'X'. ENDIF.
 IF CNT = 5. VL1 = 'X'. VL2 = 'X'. VL3 = 'X'. VL4 = 'X'. VL5 = 'X'. ENDIF.
 IF CNT = 6. VL1 = 'X'. VL2 = 'X'. VL3 = 'X'. VL4 = 'X'. VL5 = 'X'. VL6 = 'X'. ENDIF.
 IF CNT = 7. VL1 = 'X'. VL2 = 'X'. VL3 = 'X'. VL4 = 'X'. VL5 = 'X'. VL6 = 'X'. VL7 = 'X'. ENDIF.
 IF CNT = 8. VL1 = 'X'. VL2 = 'X'. VL3 = 'X'. VL4 = 'X'. VL5 = 'X'. VL6 = 'X'. VL7 = 'X'. VL8 = 'X'. ENDIF.

 LO_EXPORTING->set_attribute(
    name = 'EN1'
    value = VL1 ).
 LO_EXPORTING->set_attribute(
    name = 'EN2'
    value = VL2 ).
 LO_EXPORTING->set_attribute(
    name = 'EN3'
    value = VL3 ).
 LO_EXPORTING->set_attribute(
    name = 'EN4'
    value = VL4 ).
 LO_EXPORTING->set_attribute(
    name = 'EN5'
    value = VL5 ).
 LO_EXPORTING->set_attribute(
    name = 'EN6'
    value = VL6 ).
 LO_EXPORTING->set_attribute(
```

```
      name = 'EN7'
      value = VL7 ).
    LO_EXPORTING->set_attribute(
      name = 'EN8'
      value = VL8 ).

    LO_EXPORTING->set_attribute(
      name = 'QN1'
      value = ' ' ).
    LO_EXPORTING->set_attribute(
      name = 'QN2'
      value = ' ' ).
    LO_EXPORTING->set_attribute(
      name = 'QN3'
      value = ' ' ).
    LO_EXPORTING->set_attribute(
      name = 'QN4'
      value = ' ' ).
    LO_EXPORTING->set_attribute(
      name = 'QN5'
      value = ' ' ).
    LO_EXPORTING->set_attribute(
      name = 'QN6'
      value = ' ' ).
    LO_EXPORTING->set_attribute(
      name = 'QN7'
      value = ' ' ).
    LO_EXPORTING->set_attribute(
      name = 'QN8'
      value = ' ' ).

  endif.

  clear: VL1, VL2, VL3, VL4, VL5, VL6, VL7, VL8.
  if veh_type <> 'TTV' and veh_type <> 'ATF' and veh_type <> 'WOIL' and veh_type <> 'TTVM' and LV_ERROR <
> 'X'.
   if cnt > 0. VL1 = 'X'. else. clear VL1. endif.
   LO_EXPORTING->set_attribute(
      name = 'EN1'
      value = VL1 ).

   if cnt > 1 and LV_ERROR <> 'X'. VL2 = 'X'. else. clear VL2. endif.
   LO_EXPORTING->set_attribute(
      name = 'EN2'
      value = VL2 ).

   if cnt > 2 and LV_ERROR <> 'X'. VL3 = 'X'. else. clear VL3. endif.
   LO_EXPORTING->set_attribute(
      name = 'EN3'
      value = VL3 ).

   if cnt > 3 and LV_ERROR <> 'X'. VL4 = 'X'. else. clear VL4. endif.
   LO_EXPORTING->set_attribute(
      name = 'EN4'
```

```
        value = VL4 ).

    if cnt > 4 and LV_ERROR <> 'X'. VL5 = 'X'. else. clear VL5. endif.
    LO_EXPORTING->set_attribute(
        name = 'EN5'
        value = VL5 ).

    if cnt > 5 and LV_ERROR <> 'X'. VL6 = 'X'. else. clear VL6. endif.
    LO_EXPORTING->set_attribute(
        name = 'EN6'
        value = VL6 ).

    if cnt > 6 and LV_ERROR <> 'X'. VL7 = 'X'. else. clear VL7. endif.
    LO_EXPORTING->set_attribute(
        name = 'EN7'
        value = VL7 ).

    if cnt > 7 and LV_ERROR <> 'X'. VL8 = 'X'. else. clear VL8. endif.
    LO_EXPORTING->set_attribute(
        name = 'EN8'
        value = VL8 ).

   endif.

if LV_ERROR = 'X'.
  LO_EXPORTING->set_attribute(
        name = 'EN1'
        value = ' ' ).
  LO_EXPORTING->set_attribute(
        name = 'EN2'
        value = ' ' ).
  LO_EXPORTING->set_attribute(
        name = 'EN3'
        value = ' ' ).
  LO_EXPORTING->set_attribute(
        name = 'EN4'
        value = ' ' ).
  LO_EXPORTING->set_attribute(
        name = 'EN5'
        value = ' ' ).
  LO_EXPORTING->set_attribute(
        name = 'EN6'
        value = ' ' ).
  LO_EXPORTING->set_attribute(
        name = 'EN7'
        value = ' ' ).
  LO_EXPORTING->set_attribute(
        name = 'EN8'
        value = ' ' ).

  LO_EXPORTING->set_attribute(
        name = 'QN1'
        value = ' ' ).
  LO_EXPORTING->set_attribute(
```

```
        name = 'QN2'
        value = ' ' ).
  LO_EXPORTING->set_attribute(
        name = 'QN3'
        value = ' ' ).
  LO_EXPORTING->set_attribute(
        name = 'QN4'
        value = ' ' ).
  LO_EXPORTING->set_attribute(
        name = 'QN5'
        value = ' ' ).
  LO_EXPORTING->set_attribute(
        name = 'QN6'
        value = ' ' ).
  LO_EXPORTING->set_attribute(
        name = 'QN7'
        value = ' ' ).
  LO_EXPORTING->set_attribute(
        name = 'QN8'
        value = ' ' ).
endif.
*** Get Vehicle type and Enable Compartments END **

*** Get Product drop-down list START ***
clear: IT_NOPROD, IT_NOPROD[], WA_NOPROD, ZUSR.
 select single * from ZSD_CUST_USR_MAP into zusr where CUST_USER_ID = sy-uname.
 select * from ZSD_CUST_NO_PRD into WA_NOPROD where kunnr = zusr-kunnr.
  append WA_NOPROD to IT_NOPROD.
 endselect.

DATA:  imat_data type REF TO IF_WD_CONTEXT_NODE,
     imat_data1 TYPE wd_this->Elements_prod1,
     lmat_data1 LIKE LINE OF imat_data1,
     imat_data2 TYPE wd_this->Elements_prod2,
     imat_data3 TYPE wd_this->Elements_prod3,
     imat_data4 TYPE wd_this->Elements_prod4,
     imat_data5 TYPE wd_this->Elements_prod5,
     imat_data6 TYPE wd_this->Elements_prod6,
     imat_data7 TYPE wd_this->Elements_prod7,
     imat_data8 TYPE wd_this->Elements_prod8.

if LV_ERROR <> 'X'.
 imat_data = wd_context->path_get_node( path = ` PROD1 ` ).
 if veh_type = 'TTV' or veh_type = 'WOIL' or veh_type = 'TTVM'.
  select * into corresponding fields of table imat_data1 from ZSD_INDENT_PROD where category = 1 and pr
oduct <> 'ATF01'.
    elseif veh_type = 'ATF'.
     select * into corresponding fields of table imat_data1 from ZSD_INDENT_PROD where category = 1 and
product = 'ATF01'.
       elseif veh_type = 'LPGB'.
        select * into corresponding fields of table imat_data1 from ZSD_INDENT_PROD where category = 3.
         else.
          select * into corresponding fields of table imat_data1 from ZSD_INDENT_PROD where category = 5.
       endif.
```

```
      loop at IT_NOPROD into WA_NOPROD.
       delete imat_data1 where PRODUCT = WA_NOPROD-matnr.
      endloop.
 IF LV_ZT1 = 'X'.
  move LV_PRODUCT1 to lmat_data1-PRODUCT.
 ENDIF.
 IF veh_type <> 'ATF'.
  insert lmat_data1 into imat_data1 index 1.
 ENDIF.
 imat_data->bind_table( new_items = imat_data1 set_initial_elements = abap_true ).


 clear imat_data.
 imat_data = wd_context->path_get_node( path = `PROD2` ).
 if veh_type = 'TTV' or veh_type = 'WOIL' or veh_type = 'TTVM'.
  select * into corresponding fields of table imat_data2 from ZSD_INDENT_PROD where category = 1 and pr
oduct <> 'ATF01'.
   elseif veh_type = 'ATF'.
    select * into corresponding fields of table imat_data2 from ZSD_INDENT_PROD where category = 1 and
product = 'ATF01'.
      elseif veh_type = 'LPGB'.
       select * into corresponding fields of table imat_data2 from ZSD_INDENT_PROD where category = 3.
        else.
         select * into corresponding fields of table imat_data2 from ZSD_INDENT_PROD where category = 5.
      endif.
      loop at IT_NOPROD into WA_NOPROD.
       delete imat_data2 where PRODUCT = WA_NOPROD-matnr.
      endloop.
 clear lmat_data1.
 IF LV_ZT1 = 'X'.
  move LV_PRODUCT2 to lmat_data1-PRODUCT.
 ENDIF.
 IF veh_type <> 'ATF'.
  insert lmat_data1 into imat_data2 index 1.
 ENDIF.

 IF ( veh_type = 'ATF' OR veh_type = 'LPGB' ) AND CNT < 2.
  insert lmat_data1 into imat_data2 index 1.
 ENDIF.
 imat_data->bind_table( new_items = imat_data2 set_initial_elements = abap_true ).

 clear imat_data.
 imat_data = wd_context->path_get_node( path = `PROD3` ).
 if veh_type = 'TTV' or veh_type = 'WOIL' or veh_type = 'TTVM'.
  select * into corresponding fields of table imat_data3 from ZSD_INDENT_PROD where category = 1 and pr
oduct <> 'ATF01'.
   elseif veh_type = 'ATF'.
    select * into corresponding fields of table imat_data3 from ZSD_INDENT_PROD where category = 1 and
product = 'ATF01'.
      elseif veh_type = 'LPGB'.
       select * into corresponding fields of table imat_data3 from ZSD_INDENT_PROD where category = 3.
        else.
         select * into corresponding fields of table imat_data3 from ZSD_INDENT_PROD where category = 5.
      endif.
```

```
      loop at IT_NOPROD into WA_NOPROD.
       delete imat_data3 where PRODUCT = WA_NOPROD-matnr.
      endloop.
 clear lmat_data1.
 IF LV_ZT1 = 'X'.
  move LV_PRODUCT3 to lmat_data1-PRODUCT.
 ENDIF.
 IF veh_type <> 'ATF' .
  insert lmat_data1 into imat_data3 index 1.
 ENDIF.

 IF ( veh_type = 'ATF' OR veh_type = 'LPGB' ) AND CNT < 3.
  insert lmat_data1 into imat_data3 index 1.
 ENDIF.
 imat_data->bind_table( new_items = imat_data3 set_initial_elements = abap_true ).

 clear imat_data.
 imat_data = wd_context->path_get_node( path = `PROD4` ).
 if veh_type = 'TTV' or veh_type = 'WOIL' or veh_type = 'TTVM'.
  select * into corresponding fields of table imat_data4 from ZSD_INDENT_PROD where category = 1 and pr
oduct <> 'ATF01'.
    elseif veh_type = 'ATF'.
     select * into corresponding fields of table imat_data4 from ZSD_INDENT_PROD where category = 1 and
product = 'ATF01'.
      elseif veh_type = 'LPGB'.
       select * into corresponding fields of table imat_data4 from ZSD_INDENT_PROD where category = 3.
        else.
         select * into corresponding fields of table imat_data4 from ZSD_INDENT_PROD where category = 5.
      endif.
      loop at IT_NOPROD into WA_NOPROD.
       delete imat_data4 where PRODUCT = WA_NOPROD-matnr.
      endloop.
 clear lmat_data1.
 IF LV_ZT1 = 'X'.
  move LV_PRODUCT4 to lmat_data1-PRODUCT.
 ENDIF.
 IF veh_type <> 'ATF' AND veh_type <> 'LPGB'.
  insert lmat_data1 into imat_data4 index 1.
 ENDIF.

 IF ( veh_type = 'ATF' OR veh_type = 'LPGB' ) AND CNT < 4.
  insert lmat_data1 into imat_data4 index 1.
 ENDIF.
 imat_data->bind_table( new_items = imat_data4 set_initial_elements = abap_true ).

 clear imat_data.
 imat_data = wd_context->path_get_node( path = `PROD5` ).
 if veh_type = 'TTV' or veh_type = 'WOIL' or veh_type = 'TTVM'.
  select * into corresponding fields of table imat_data5 from ZSD_INDENT_PROD where category = 1 and pr
oduct <> 'ATF01'.
    elseif veh_type = 'ATF'.
     select * into corresponding fields of table imat_data5 from ZSD_INDENT_PROD where category = 1 and
product = 'ATF01'.
      elseif veh_type = 'LPGB'.
```

```
        select * into corresponding fields of table imat_data5 from ZSD_INDENT_PROD where category = 3.
         else.
          select * into corresponding fields of table imat_data5 from ZSD_INDENT_PROD where category = 5.
       endif.
       loop at IT_NOPROD into WA_NOPROD.
        delete imat_data5 where PRODUCT = WA_NOPROD-matnr.
       endloop.
 clear lmat_data1.
 IF LV_ZT1 = 'X'.
  move LV_PRODUCT5 to lmat_data1-PRODUCT.
 ENDIF.
 IF veh_type <> 'ATF' AND veh_type <> 'LPGB'.
  insert lmat_data1 into imat_data5 index 1.
 ENDIF.

 IF ( veh_type = 'ATF' OR veh_type = 'LPGB' ) AND CNT < 5.
  insert lmat_data1 into imat_data5 index 1.
 ENDIF.
 imat_data->bind_table( new_items = imat_data5 set_initial_elements = abap_true ).


 clear imat_data.
 imat_data = wd_context->path_get_node( path = `PROD6` ).
 if veh_type = 'TTV' or veh_type = 'WOIL' or veh_type = 'TTVM'.
  select * into corresponding fields of table imat_data6 from ZSD_INDENT_PROD where category = 1 and pr
oduct <> 'ATF01'.
    elseif veh_type = 'ATF'.
     select * into corresponding fields of table imat_data6 from ZSD_INDENT_PROD where category = 1 and
product = 'ATF01'.
       elseif veh_type = 'LPGB'.
        select * into corresponding fields of table imat_data6 from ZSD_INDENT_PROD where category = 3.
         else.
          select * into corresponding fields of table imat_data6 from ZSD_INDENT_PROD where category = 5.
       endif.
       loop at IT_NOPROD into WA_NOPROD.
        delete imat_data6 where PRODUCT = WA_NOPROD-matnr.
       endloop.
 clear lmat_data1.
 IF LV_ZT1 = 'X'.
  move LV_PRODUCT6 to lmat_data1-PRODUCT.
 ENDIF.
 IF veh_type <> 'ATF' AND veh_type <> 'LPGB'.
  insert lmat_data1 into imat_data6 index 1.
 ENDIF.

 IF ( veh_type = 'ATF' OR veh_type = 'LPGB' ) AND CNT < 6.
  insert lmat_data1 into imat_data6 index 1.
 ENDIF.
 imat_data->bind_table( new_items = imat_data6 set_initial_elements = abap_true ).


 clear imat_data.
 imat_data = wd_context->path_get_node( path = `PROD7` ).
 if veh_type = 'TTV' or veh_type = 'WOIL' or veh_type = 'TTVM'.
```

```
  select * into corresponding fields of table imat_data7 from ZSD_INDENT_PROD where category = 1 and pr
oduct <> 'ATF01'.
    elseif veh_type = 'ATF'.
     select * into corresponding fields of table imat_data7 from ZSD_INDENT_PROD where category = 1 and
product = 'ATF01'.
      elseif veh_type = 'LPGB'.
       select * into corresponding fields of table imat_data7 from ZSD_INDENT_PROD where category = 3.
        else.
         select * into corresponding fields of table imat_data7 from ZSD_INDENT_PROD where category = 5.
      endif.
      loop at IT_NOPROD into WA_NOPROD.
       delete imat_data7 where PRODUCT = WA_NOPROD-matnr.
      endloop.
 clear lmat_data1.
 IF LV_ZT1 = 'X'.
  move LV_PRODUCT7 to lmat_data1-PRODUCT.
 ENDIF.
 IF veh_type <> 'ATF' AND veh_type <> 'LPGB'.
  insert lmat_data1 into imat_data7 index 1.
 ENDIF.

 IF ( veh_type = 'ATF' OR veh_type = 'LPGB' ) AND CNT < 7.
  insert lmat_data1 into imat_data7 index 1.
 ENDIF.
 imat_data->bind_table( new_items = imat_data7 set_initial_elements = abap_true ).


 clear imat_data.
 imat_data = wd_context->path_get_node( path = `PROD8` ).
 if veh_type = 'TTV' or veh_type = 'WOIL' or veh_type = 'TTVM'.
  select * into corresponding fields of table imat_data8 from ZSD_INDENT_PROD where category = 1 and pr
oduct <> 'ATF01'.
    elseif veh_type = 'ATF'.
     select * into corresponding fields of table imat_data8 from ZSD_INDENT_PROD where category = 1 and
product = 'ATF01'.
      elseif veh_type = 'LPGB'.
       select * into corresponding fields of table imat_data8 from ZSD_INDENT_PROD where category = 3.
        else.
         select * into corresponding fields of table imat_data8 from ZSD_INDENT_PROD where category = 5.
      endif.
      loop at IT_NOPROD into WA_NOPROD.
       delete imat_data8 where PRODUCT = WA_NOPROD-matnr.
      endloop.
 clear lmat_data1.
 IF LV_ZT1 = 'X'.
  move LV_PRODUCT8 to lmat_data1-PRODUCT.
 ENDIF.
 IF veh_type <> 'ATF' AND veh_type <> 'LPGB'.
  insert lmat_data1 into imat_data8 index 1.
 ENDIF.

 IF ( veh_type = 'ATF' OR veh_type = 'LPGB' ) AND CNT < 8.
  insert lmat_data1 into imat_data8 index 1.
 ENDIF.
```

```
 imat_data->bind_table( new_items = imat_data8 set_initial_elements = abap_true ).


 LO_ND_ENABLE2 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_ENABLE2 ).
 LO_EL_ENABLE2 = LO_ND_ENABLE2->GET_ELEMENT( ).
 CLEAR LV_ZT1.
 LO_EL_ENABLE2->SET_ATTRIBUTE(
    NAME = `ZT1`
    VALUE = LV_ZT1 ).
endif.
*** Get Product drop-down list END ***


DATA LO_FLUSH TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_FLUSH_INDENT1 TYPE wd_this->Elements_ATF_FLUSH.
DATA LS_FLUSH TYPE ig_componentcontroller=>Element_ATF_FLUSH.
LO_FLUSH = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_ATF_FLUSH ).
CLEAR LO_FLUSH_INDENT1. REFRESH LO_FLUSH_INDENT1.
CLEAR LS_FLUSH.

if veh_type <> 'ATF' or LV_ERROR = 'X'.
 LO_FLUSH->bind_table( new_items = LO_FLUSH_INDENT1 set_initial_elements = abap_true ).
 clear LO_FLUSH.
endif.

if LV_ERROR <> 'X' and veh_type = 'ATF'.
  APPEND LS_FLUSH TO LO_FLUSH_INDENT1.
  LS_FLUSH-FLUSH = 'Y'.
  LS_FLUSH-DESC = 'YES'.
  APPEND LS_FLUSH TO LO_FLUSH_INDENT1.
  LS_FLUSH-FLUSH = 'N'.
  LS_FLUSH-DESC = 'NO'.
  APPEND LS_FLUSH TO LO_FLUSH_INDENT1.

  LO_FLUSH->bind_table( new_items = LO_FLUSH_INDENT1 set_initial_elements = abap_true ).
  clear LS_FLUSH.
endif.

if LV_ERROR <> 'X'.
  LO_EXPORTING->set_attribute(
       name = 'SAVE'
       value = 'X' ).
 else.
   LO_EXPORTING->set_attribute(
      name = 'SAVE'
      value = ' ' ).
 endif.




endmethod.
```

- method ONACTIONMODIFY_INDENT .

```
DATA lt_indent TYPE ig_componentcontroller=>Elements_itab_indent.
```

```
DATA ls_indent TYPE ig_componentcontroller=>Element_itab_indent.
DATA LO_Z_GET_CUSTOMER_INDEN TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_Z_GET_CUSTOMER_INDEN_CHANG TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_IMPORTING TYPE REF TO IF_WD_CONTEXT_NODE.
DATA: CT TYPE I,
    CT1 TYPE I,
    CT2 TYPE I.

LO_Z_GET_CUSTOMER_INDEN = WD_CONTEXT->GET_CHILD_NODE( WD_THIS-
>WDCTX_Z_GET_CUSTOMER_INDEN ).
LO_Z_GET_CUSTOMER_INDEN_CHANG = LO_Z_GET_CUSTOMER_INDEN->GET_CHILD_NODE( WD_THIS-
>WDCTX_CHANGING_1 ).
LO_IMPORTING = LO_Z_GET_CUSTOMER_INDEN_CHANG->GET_CHILD_NODE( WD_THIS-
>WDCTX_ITAB_INDENT ).
LO_IMPORTING-
>get_static_attributes_table( IMPORTING table = lt_indent ).  "#EC CI_FLDEXT_OK[2215424] " #S/4 - OSS No
te# - Added by - Z_SHSHRUKHA – 2021/08/18



LOOP AT lt_indent INTO ls_indent.
 if ls_indent-CHK = 'X' and ls_indent-ZTT_STATUS = '4'.
  CT = CT + 1.
 endif.
 if ls_indent-CHK = 'X' and ls_indent-ZTT_STATUS <> '4'.
  CT1 = CT1 + 1.
 endif.

 if ls_indent-CHK = 'X'.
  CT2 = CT2 + 1.
 endif.
ENDLOOP.

*** Deletion check POPUP message START ***
DATA LO_WINDOW_MANAGER TYPE REF TO IF_WD_WINDOW_MANAGER.
DATA LO_API_COMPONENT  TYPE REF TO IF_WD_COMPONENT.
DATA LO_WINDOW       TYPE REF TO IF_WD_WINDOW.
data: lt_string type table of string,
    ls_string like line of lt_string.
DATA l_api          TYPE REF TO if_wd_view_controller.

 LO_API_COMPONENT        = WD_COMP_CONTROLLER->WD_GET_API( ).
 LO_WINDOW_MANAGER       = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).

DATA: LV_ERR1 TYPE CHAR1.
CLEAR LV_ERR1.

if CT2 = 0.
 LV_ERR1 = 'X'.
  ls_string = 'Please select indent to be modified.'.
  insert ls_string into table lt_string.
  LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
            text         = lt_string
            button_kind    = if_wd_window=>co_buttons_ok
*             button_kind    = 4
```

```
              message_type    = if_wd_window=>CO_MSG_TYPE_STOPP
              window_title    = 'Please check!'
              window_position = if_wd_window=>co_center ).

   LO_WINDOW->open( ).
endif.

if CT2 > 1 AND LV_ERR1 = ' '.
 LV_ERR1 = 'X'.
  ls_string = 'Please select only one indent.'.
  insert ls_string into table lt_string.
  LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
              text         = lt_string
              button_kind    = if_wd_window=>co_buttons_ok
              message_type    = if_wd_window=>CO_MSG_TYPE_STOPP
              window_title    = 'Please check!'
              window_position = if_wd_window=>co_center ).

   LO_WINDOW->open( ).
endif.

if CT2 > 0 AND LV_ERR1 = ' '.
 if CT1 <> 0.
  LV_ERR1 = 'X'.
  ls_string = 'Already submitted indent cannot be modified.'.
  insert ls_string into table lt_string.
  LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
              text         = lt_string
              button_kind    = if_wd_window=>co_buttons_ok
              message_type    = if_wd_window=>CO_MSG_TYPE_STOPP
              window_title    = 'Please check!'
              window_position = if_wd_window=>co_center ).

   LO_WINDOW->open( ).
 endif.
endif.

 if LV_ERR1 = ' ' and CT = 1.
  ls_string = 'Do you want to modify the selected indent?'.
  insert ls_string into table lt_string.

 LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
             text         = lt_string
*             button_kind    = if_wd_window=>co_buttons_ok
             button_kind    = 4
             message_type    = if_wd_window=>CO_MSG_TYPE_QUESTION
             window_title    = 'Please check!'
             window_position = if_wd_window=>co_center ).

 DATA LO_API_START_VIEW TYPE REF TO IF_WD_VIEW_CONTROLLER.
 LO_API_START_VIEW = WD_THIS->WD_GET_API( ).

 CALL METHOD LO_WINDOW->SUBSCRIBE_TO_BUTTON_EVENT
  EXPORTING
```

```
    BUTTON    = if_wd_window=>co_button_yes
    ACTION_NAME = 'REACT_TO_MODIFY'
    ACTION_VIEW = LO_API_START_VIEW.

  LO_WINDOW->open( ).

  CALL METHOD LO_WINDOW_MANAGER->CREATE_POPUP_TO_CONFIRM
   EXPORTING
    TEXT             = lt_string
    BUTTON_KIND        = 4
   RECEIVING
    RESULT           = lo_window.
 endif.
endmethod.
```

- method ONACTIONREACT_TO_CLOSE_ORDER .

```
 DATA LO_ND_CHANGING_2 TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LT_CHANGING_2 TYPE WD_THIS->ELEMENTS_CHANGING_2.
  DATA LS_CHANGING_2 TYPE WD_THIS->ELEMENT_CHANGING_2.
  LO_ND_CHANGING_2 = WD_CONTEXT-
>PATH_GET_NODE( PATH = `ZSD_INDENT_NON_VEH.CHANGING_2` ).
  LO_ND_CHANGING_2->GET_STATIC_ATTRIBUTES_TABLE( IMPORTING TABLE = LT_CHANGING_2 ).

  LOOP AT LT_CHANGING_2 INTO LS_CHANGING_2 WHERE CHK = 'X'.
   LS_CHANGING_2-ORD_CLOSED = 'X'.
   CLEAR LS_CHANGING_2-CHK.
   MODIFY ZSD_INDENT_NVEH FROM LS_CHANGING_2.
  ENDLOOP.

  DATA LO_API_CONTROLLER    TYPE REF TO IF_WD_CONTROLLER.
  DATA LO_MESSAGE_MANAGER   TYPE REF TO IF_WD_MESSAGE_MANAGER.

  LO_API_CONTROLLER ?= WD_THIS->WD_GET_API( ).

  CALL METHOD LO_API_CONTROLLER->GET_MESSAGE_MANAGER
   RECEIVING
    MESSAGE_MANAGER = LO_MESSAGE_MANAGER.


   CALL METHOD LO_MESSAGE_MANAGER->REPORT_MESSAGE
    EXPORTING
     MESSAGE_TEXT          = 'Selected order has been closed'
     MESSAGE_TYPE          = 4.


  DATA LO_COMPONENTCONTROLLER TYPE REF TO IG_COMPONENTCONTROLLER .
    LO_COMPONENTCONTROLLER =   WD_THIS->GET_COMPONENTCONTROLLER_CTR( ).

    LO_COMPONENTCONTROLLER->EXECUTE_Z_GET_CUST_INDENT_NO_V(
    ).
endmethod.
```

- method ONACTIONREACT_TO_DEL_ORDER .

```
  DATA LO_ND_CHANGING_2 TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LT_CHANGING_2 TYPE WD_THIS->ELEMENTS_CHANGING_2.
  DATA LS_CHANGING_2 TYPE WD_THIS->ELEMENT_CHANGING_2.
```

```
  LO_ND_CHANGING_2 = WD_CONTEXT-
>PATH_GET_NODE( PATH = `ZSD_INDENT_NON_VEH.CHANGING_2` ).
  LO_ND_CHANGING_2-
>GET_STATIC_ATTRIBUTES_TABLE( IMPORTING TABLE = LT_CHANGING_2 ).  "#EC CI_FLDEXT_OK[2215424]
" #S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
  DATA: DEL(1).

  CLEAR DEL.
  LOOP AT LT_CHANGING_2 INTO LS_CHANGING_2 WHERE CHK = 'X'.
   IF LS_CHANGING_2-QUANTITY1 = LS_CHANGING_2-BL_QTY1 AND LS_CHANGING_2-
QUANTITY2 = LS_CHANGING_2-BL_QTY2.
     DELETE ZSD_INDENT_NVEH FROM LS_CHANGING_2.
     ELSE.
      DEL = 'N'.
      ENDIF.
  ENDLOOP.

  DATA LO_API_CONTROLLER    TYPE REF TO IF_WD_CONTROLLER.
  DATA LO_MESSAGE_MANAGER   TYPE REF TO IF_WD_MESSAGE_MANAGER.

  LO_API_CONTROLLER ?= WD_THIS->WD_GET_API( ).

  CALL METHOD LO_API_CONTROLLER->GET_MESSAGE_MANAGER
   RECEIVING
    MESSAGE_MANAGER = LO_MESSAGE_MANAGER.

  IF DEL = ' '.
   CALL METHOD LO_MESSAGE_MANAGER->REPORT_MESSAGE
    EXPORTING
     MESSAGE_TEXT          = 'Selected order has been deleted'
     MESSAGE_TYPE          = 4.
   ELSE.
    CALL METHOD LO_MESSAGE_MANAGER->REPORT_MESSAGE
     EXPORTING
      MESSAGE_TEXT          = 'Selected order has been processed and cannot be deleted'
      MESSAGE_TYPE          = 4.
  ENDIF.

  DATA LO_COMPONENTCONTROLLER TYPE REF TO IG_COMPONENTCONTROLLER .
    LO_COMPONENTCONTROLLER =   WD_THIS->GET_COMPONENTCONTROLLER_CTR( ).

    LO_COMPONENTCONTROLLER->EXECUTE_Z_GET_CUST_INDENT_NO_V(
    ).


endmethod.
```

- method ONACTIONREACT_TO_MODIFY .

```
 DATA lt_indent TYPE ig_componentcontroller=>Elements_itab_indent.
 DATA ls_indent TYPE ig_componentcontroller=>Element_itab_indent.
 DATA LO_Z_GET_CUSTOMER_INDEN TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_Z_GET_CUSTOMER_INDEN_CHANG TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_IMPORTING TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA: CT TYPE I.
```

```
 LO_Z_GET_CUSTOMER_INDEN = WD_CONTEXT->GET_CHILD_NODE( WD_THIS-
>WDCTX_Z_GET_CUSTOMER_INDEN ).
 LO_Z_GET_CUSTOMER_INDEN_CHANG = LO_Z_GET_CUSTOMER_INDEN->GET_CHILD_NODE( WD_THIS-
>WDCTX_CHANGING_1 ).
 LO_IMPORTING = LO_Z_GET_CUSTOMER_INDEN_CHANG->GET_CHILD_NODE( WD_THIS-
>WDCTX_ITAB_INDENT ).
 LO_IMPORTING-
>get_static_attributes_table( IMPORTING table = lt_indent ).  "#EC CI_FLDEXT_OK[2215424] " #S/4 - OSS No
te# - Added by - Z_SHSHRUKHA – 2021/08/18

 DATA lr_event TYPE REF TO cl_wd_custom_event.
 READ TABLE lt_indent INTO ls_indent WITH KEY CHK = 'X' ZTT_STATUS = '4'.

   DATA LO_ND_IMPORTING TYPE REF TO IF_WD_CONTEXT_NODE.
   DATA LO_EL_IMPORTING TYPE REF TO IF_WD_CONTEXT_ELEMENT.
   DATA LS_IMPORTING TYPE WD_THIS->ELEMENT_IMPORTING.
   DATA LV_VEHICLE TYPE WD_THIS->ELEMENT_IMPORTING-VEHICLE.
   LV_VEHICLE = ls_indent-VEHICLE.
   LO_ND_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_IMPORTING ).
   LO_EL_IMPORTING = LO_ND_IMPORTING->GET_ELEMENT( ).
   IF LO_EL_IMPORTING IS INITIAL.
   ENDIF.
   LO_EL_IMPORTING->SET_ATTRIBUTE(
    NAME = `VEHICLE`
    VALUE = LV_VEHICLE ).

    DATA LO_ND_HEADER_DATA TYPE REF TO IF_WD_CONTEXT_NODE.
    DATA LO_EL_HEADER_DATA TYPE REF TO IF_WD_CONTEXT_ELEMENT.
    DATA LS_HEADER_DATA TYPE WD_THIS->ELEMENT_HEADER_DATA.
    DATA LV_KUNNR TYPE WD_THIS->ELEMENT_HEADER_DATA-KUNNR.
    LO_ND_HEADER_DATA = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS-
>WDCTX_HEADER_DATA ).
    LO_EL_HEADER_DATA = LO_ND_HEADER_DATA->GET_ELEMENT( ).
    IF LO_EL_HEADER_DATA IS INITIAL.
    ENDIF.
    LV_KUNNR = ls_indent-KUNNR.
    LO_EL_HEADER_DATA->SET_ATTRIBUTE(
     NAME = `KUNNR`
     VALUE = LV_KUNNR ).


DATA:  imat_data type REF TO IF_WD_CONTEXT_NODE,
     imat_data1 TYPE wd_this->Elements_prod1,
     lmat_data1 LIKE LINE OF imat_data1,
     imat_data2 TYPE wd_this->Elements_prod2,
     imat_data3 TYPE wd_this->Elements_prod3,
     imat_data4 TYPE wd_this->Elements_prod4,
     imat_data5 TYPE wd_this->Elements_prod5.
imat_data = wd_context->path_get_node( path = `PROD1` ).
lmat_data1-PRODUCT = ls_indent-PROD_CMP1.
APPEND lmat_data1 TO imat_data1.
imat_data->bind_table( new_items = imat_data1 set_initial_elements = abap_true ).

clear: imat_data, lmat_data1-PRODUCT.
```

```
imat_data = wd_context->path_get_node( path = `PROD2` ).
lmat_data1-PRODUCT = ls_indent-PROD_CMP2.
APPEND lmat_data1 TO imat_data2.
imat_data->bind_table( new_items = imat_data2 set_initial_elements = abap_true ).

clear: imat_data, lmat_data1-PRODUCT.
imat_data = wd_context->path_get_node( path = `PROD3` ).
lmat_data1-PRODUCT = ls_indent-PROD_CMP3.
APPEND lmat_data1 TO imat_data3.
imat_data->bind_table( new_items = imat_data3 set_initial_elements = abap_true ).

clear: imat_data, lmat_data1-PRODUCT.
imat_data = wd_context->path_get_node( path = `PROD4` ).
lmat_data1-PRODUCT = ls_indent-PROD_CMP4.
APPEND lmat_data1 TO imat_data4.
imat_data->bind_table( new_items = imat_data4 set_initial_elements = abap_true ).

clear: imat_data, lmat_data1-PRODUCT.
imat_data = wd_context->path_get_node( path = `PROD5` ).
lmat_data1-PRODUCT = ls_indent-PROD_CMP5.
APPEND lmat_data1 TO imat_data5.
imat_data->bind_table( new_items = imat_data5 set_initial_elements = abap_true ).

 DATA LO_ND_ENABLE2 TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_EL_ENABLE2 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
 DATA LS_ENABLE2 TYPE WD_THIS->ELEMENT_ENABLE2.
 DATA LV_ZT1 TYPE WD_THIS->ELEMENT_ENABLE2-ZT1.
 DATA LV_ZT2 TYPE WD_THIS->ELEMENT_ENABLE2-ZT2.
 LO_ND_ENABLE2 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_ENABLE2 ).
 LO_EL_ENABLE2 = LO_ND_ENABLE2->GET_ELEMENT( ).
 LV_ZT1 = 'X'. LV_ZT2 = 'X'.
 LO_EL_ENABLE2->SET_ATTRIBUTE(
  NAME =  `ZT1`
  VALUE = LV_ZT1 ).

 LO_EL_ENABLE2->SET_ATTRIBUTE(
  NAME =  `ZT2`
  VALUE = LV_ZT2 ).

 DATA LO_ND_WV_ENABLE TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_EL_WV_ENABLE TYPE REF TO IF_WD_CONTEXT_ELEMENT.
 DATA LS_WV_ENABLE TYPE WD_THIS->ELEMENT_WV_ENABLE.
 DATA LV_CH_INDT TYPE WD_THIS->ELEMENT_WV_ENABLE-CH_INDT.
 LO_ND_WV_ENABLE = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_WV_ENABLE ).
 LO_EL_WV_ENABLE = LO_ND_WV_ENABLE->GET_ELEMENT( ).
 IF LO_EL_WV_ENABLE IS NOT INITIAL.
 LV_CH_INDT = 'X'.
  LO_EL_WV_ENABLE->SET_ATTRIBUTE(
    NAME =  `CH_INDT`
    VALUE = LV_CH_INDT ).
 ENDIF.


 CREATE OBJECT lr_event
```

```
  EXPORTING
    name = 'GET_VEHICLE'.

 WD_THIS->ONACTIONGET_VEHICLE(
  WDEVENT = lr_event                  " ref to cl_wd_custom_event
 ).


* get message manager
 DATA LO_API_CONTROLLER    TYPE REF TO IF_WD_CONTROLLER.
 DATA LO_MESSAGE_MANAGER   TYPE REF TO IF_WD_MESSAGE_MANAGER.

 LO_API_CONTROLLER ?= WD_THIS->WD_GET_API( ).

 CALL METHOD LO_API_CONTROLLER->GET_MESSAGE_MANAGER
  RECEIVING
    MESSAGE_MANAGER = LO_MESSAGE_MANAGER
    .

* report message
 CALL METHOD LO_MESSAGE_MANAGER->REPORT_MESSAGE
  EXPORTING
    MESSAGE_TEXT          = 'Selected Indent has been modified'
    MESSAGE_TYPE          = 4.

 DATA LO_COMPONENTCONTROLLER TYPE REF TO IG_COMPONENTCONTROLLER .
 LO_COMPONENTCONTROLLER =   WD_THIS->GET_COMPONENTCONTROLLER_CTR( ).

 LO_COMPONENTCONTROLLER->EXECUTE_Z_GET_CUSTOMER_INDENT(
 ).
endmethod.
```

- method ONACTIONREACT_TO_YES .

```
 DATA lt_indent TYPE ig_componentcontroller=>Elements_itab_indent.
 DATA ls_indent TYPE ig_componentcontroller=>Element_itab_indent.
 DATA LO_Z_GET_CUSTOMER_INDEN TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_Z_GET_CUSTOMER_INDEN_CHANG TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_IMPORTING TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA: CT TYPE I.
 DATA: LV_ER1(1),
     LV_ER2(1).

 LO_Z_GET_CUSTOMER_INDEN = WD_CONTEXT->GET_CHILD_NODE( WD_THIS-
>WDCTX_Z_GET_CUSTOMER_INDEN ).
 LO_Z_GET_CUSTOMER_INDEN_CHANG = LO_Z_GET_CUSTOMER_INDEN->GET_CHILD_NODE( WD_THIS-
>WDCTX_CHANGING_1 ).
 LO_IMPORTING = LO_Z_GET_CUSTOMER_INDEN_CHANG->GET_CHILD_NODE( WD_THIS-
>WDCTX_ITAB_INDENT ).
 LO_IMPORTING-
>get_static_attributes_table( IMPORTING table = lt_indent ). "#EC CI_FLDEXT_OK[2215424] " #S/4 - OSS Not
e# - Added by - Z_SHSHRUKHA – 2021/08/18

 DATA: ZDEL(1).
 DATA: ZIND_BULK TYPE ZSD_INDENT_NVEH.
 LOOP AT lt_indent INTO ls_indent where CHK = 'X'.
```

```
  CLEAR ZDEL. CLEAR LV_ER1. CLEAR LV_ER2.
  select single * from ZSD_INDENT_NVEH into ZIND_BULK where order_no = LS_INDENT-BULK_ORDER.
  ZIND_BULK-BL_QTY1 = ZIND_BULK-BL_QTY1 + LS_INDENT-BULK_QTY.
  if ls_indent-ZTT_STATUS = '4'.
   delete ZSD_CUST_INDENT from ls_indent.
   if sy-subrc = 0.
    if ZIND_BULK-ORDER_NO is not initial.
     MODIFY ZSD_INDENT_NVEH FROM ZIND_BULK.
    endif.
    delete from ZSAUTOMATETT_TBL WHERE begda = ls_indent-begda and depot = ls_indent-depot
    and vehicle = ls_indent-vehicle and ( ztt_status = '4' or SHNUMBER = ' ' ).
    delete from ZSAUTOMATETT_LPG WHERE begda = ls_indent-begda and depot = ls_indent-depot
    and vehicle = ls_indent-vehicle and ( ztt_status = '4' or SHNUMBER = ' ' ).
   endif.
  endif.
  if ls_indent-ZTT_STATUS = '5'.
   select single zdelete from zsd_cust_indent into zdel   "#EC CI_NOORDER " #S/4 - OSS Note# - Added by -
Z_SHSHRUKHA – 2021/08/18
    where BEGDA = LS_INDENT-BEGDA and ZTT_STATUS = '5'
    and VEHICLE = LS_INDENT-VEHICLE and DEPOT = LS_INDENT-DEPOT and KUNNR = LS_INDENT-
KUNNR.
    if zdel = 'Y'.
     LV_ER2 = 'X'.
     delete ZSD_CUST_INDENT from ls_indent.
     IF SY-SUBRC = 0.
      if ZIND_BULK-ORDER_NO is not initial.
       MODIFY ZSD_INDENT_NVEH FROM ZIND_BULK.
      endif.
      delete from ZSAUTOMATETT_TBL WHERE begda = ls_indent-begda and depot = ls_indent-depot
      and vehicle = ls_indent-vehicle and ztt_status = '5'.
      delete from ZSAUTOMATETT_LPG WHERE begda = ls_indent-begda and depot = ls_indent-depot
      and vehicle = ls_indent-vehicle and ztt_status = '5'.
     ENDIF.
     else.
      LV_ER1 = 'X'.
  endif.
  endif.
 ENDLOOP.
*  get message manager
DATA LO_WINDOW_MANAGER TYPE REF TO IF_WD_WINDOW_MANAGER.
DATA LO_API_COMPONENT  TYPE REF TO IF_WD_COMPONENT.
DATA LO_WINDOW       TYPE REF TO IF_WD_WINDOW.
data: lt_string type table of string,
    ls_string like line of lt_string.
DATA l_api           TYPE REF TO if_wd_view_controller.

 DATA LO_API_CONTROLLER    TYPE REF TO IF_WD_CONTROLLER.
 DATA LO_MESSAGE_MANAGER   TYPE REF TO IF_WD_MESSAGE_MANAGER.
 LO_API_CONTROLLER ?= WD_THIS->WD_GET_API( ).

 CALL METHOD LO_API_CONTROLLER->GET_MESSAGE_MANAGER
  RECEIVING
   MESSAGE_MANAGER = LO_MESSAGE_MANAGER.
```

```
IF LV_ER1 = 'X'.
  MOVE 'Indents with deletion indicator = N could not be deleted'  TO ls_string .
  APPEND ls_string TO lt_string.
  LO_API_COMPONENT        = WD_COMP_CONTROLLER->WD_GET_API( ).
   LO_WINDOW_MANAGER       = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).
   LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
               text         = lt_string
               button_kind    = if_wd_window=>co_buttons_ok
*               button_kind    = 4
               message_type   = if_wd_window=>CO_MSG_TYPE_STOPP
               window_title   = 'Please check!'
               window_position = if_wd_window=>co_center
               WINDOW_WIDTH   = '20%'
               WINDOW_HEIGHT  = '20%' ).
   LO_WINDOW->open( ).
ENDIF.


IF LV_ER2 = 'X'.
 CALL METHOD LO_MESSAGE_MANAGER->REPORT_MESSAGE
  EXPORTING
   MESSAGE_TEXT          = 'Selected unprocessed indents have been deleted'
   MESSAGE_TYPE          = 4
*   PARAMS              =
*   MSG_USER_DATA         =
*   IS_PERMANENT          = ABAP_FALSE
*   SCOPE_PERMANENT_MSG     = CO_MSG_SCOPE_CONTROLLER
*   VIEW                =
*   SHOW_AS_POPUP         =
*   CONTROLLER_PERMANENT_MSG  =
*   MSG_INDEX           =
*   CANCEL_NAVIGATION       =
*   ENABLE_MESSAGE_NAVIGATION =
*   COMPONENT           =
*  RECEIVING
*   MESSAGE_ID          =
   .
ENDIF.

DATA LO_COMPONENTCONTROLLER TYPE REF TO IG_COMPONENTCONTROLLER .
LO_COMPONENTCONTROLLER =   WD_THIS->GET_COMPONENTCONTROLLER_CTR( ).

 LO_COMPONENTCONTROLLER->EXECUTE_Z_GET_CUSTOMER_INDENT(
 ).

endmethod.
```

- method ONACTIONSAVE_INDENT .

```
  DATA LO_ND_IND_END_USE TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LO_EL_IND_END_USE TYPE REF TO IF_WD_CONTEXT_ELEMENT.
  DATA LS_IND_END_USE TYPE WD_THIS->Element_ind_end_use.
  DATA LV_IND_USE_MS TYPE WD_THIS->Element_ind_end_use-IND_USE_MS.
  DATA LV_IND_USE_HSD TYPE WD_THIS->Element_ind_end_use-IND_USE_HSD.
  LO_ND_IND_END_USE = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS-
>WDCTX_IND_END_USE ).
```

```
  LO_EL_IND_END_USE = LO_ND_IND_END_USE->GET_ELEMENT( ).
*  get single attribute
  LO_EL_IND_END_USE->GET_ATTRIBUTE(
   EXPORTING
    NAME =  `IND_USE_MS`
   IMPORTING
    VALUE = LV_IND_USE_MS ).
  LO_EL_IND_END_USE->GET_ATTRIBUTE(
   EXPORTING
    NAME =  `IND_USE_HSD`
   IMPORTING
    VALUE = LV_IND_USE_HSD ).



 DATA LO_ND_ENABLE2 TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_EL_ENABLE2 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
 DATA LS_ENABLE2 TYPE WD_THIS->ELEMENT_ENABLE2.
 DATA LV_ZT2 TYPE WD_THIS->ELEMENT_ENABLE2-ZT2.
 DATA LV_ZT3 TYPE WD_THIS->ELEMENT_ENABLE2-ZT3.
 LO_ND_ENABLE2 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_ENABLE2 ).
 LO_EL_ENABLE2 = LO_ND_ENABLE2->GET_ELEMENT( ).
 LO_EL_ENABLE2->GET_ATTRIBUTE(
  EXPORTING
   NAME =  `ZT2`
  IMPORTING
   VALUE = LV_ZT2 ).

 LV_ZT3 = 'X'.
 LO_EL_ENABLE2->SET_ATTRIBUTE(
  NAME =  `ZT3`
  VALUE = LV_ZT3 ).

DATA LO_Z_GET_CUSTOMER_INDEN TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_IMPORTING TYPE REF TO IF_WD_CONTEXT_NODE.
DATA: LV_BEGDA TYPE BEGDA.
DATA LV_VEHICLE TYPE OIGCC-TU_NUMBER.
DATA LV_KUNNR TYPE KNA1-KUNNR.
DATA: LV_PROD1 TYPE CHAR100,
    LV_PROD2 TYPE CHAR100,
    LV_PROD3 TYPE CHAR100,
    LV_PROD4 TYPE CHAR100,
    LV_PROD5 TYPE CHAR100,
    LV_PROD6 TYPE CHAR100,
    LV_PROD7 TYPE CHAR100,
    LV_PROD8 TYPE CHAR100.

DATA: LV_QTY1 TYPE OIGCC-CMP_MAXVOL,
    LV_QTY2 TYPE OIGCC-CMP_MAXVOL,
    LV_QTY3 TYPE OIGCC-CMP_MAXVOL,
    LV_QTY4 TYPE OIGCC-CMP_MAXVOL,
    LV_QTY5 TYPE OIGCC-CMP_MAXVOL,
    LV_QTY6 TYPE OIGCC-CMP_MAXVOL,
```

```
    LV_QTY7 TYPE OIGCC-CMP_MAXVOL,
    LV_QTY8 TYPE OIGCC-CMP_MAXVOL.

DATA: CAP1(16) TYPE C,
    CAP2(16) TYPE C,
    CAP3(16) TYPE C,
    CAP4(16) TYPE C,
    CAP5(16) TYPE C,
    CAP6(16) TYPE C,
    CAP7(16) TYPE C,
    CAP8(16) TYPE C.

DATA: C1_QT TYPE LABST,
    C2_QT TYPE LABST,
    C3_QT TYPE LABST,
    C4_QT TYPE LABST,
    C5_QT TYPE LABST,
    C6_QT TYPE LABST,
    C7_QT TYPE LABST,
    C8_QT TYPE LABST.

DATA: CNT TYPE I,
    LV_ERCODE3(1) TYPE C.

DATA: LV_PLANT TYPE WERKS_D.
DATA: LV_VAL TYPE BWTAR_D.

TYPES: BEGIN OF ZIND_STK.
     INCLUDE TYPE ZINDENT_CUSSTOCK.
TYPES: END OF ZIND_STK.

DATA: IT_STK type
    standard table of ZIND_STK.
DATA: LN_STK TYPE ZIND_STK.

DATA: CUST_INDENT TYPE ZSD_CUST_INDENT.
DATA: W_NUMBER(12) TYPE P DECIMALS 3.
DATA: VEH_TYPE TYPE OIGV-VEH_TYPE.
DATA LS_IMPORTING TYPE REF TO if_wd_context_element.

SELECT SINGLE DEPOT FROM ZSD_CUST_USR_MAP INTO LV_PLANT WHERE  "#EC CI_NOORDER " #S/4 - O
SS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
 CUST_USER_ID = SY-UNAME.

LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_IMPORTING ).
LO_IMPORTING->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `VEHICLE`
  IMPORTING
   VALUE = LV_VEHICLE ).

SELECT SINGLE VEH_TYPE FROM OIGV INTO VEH_TYPE WHERE VEHICLE = LV_VEHICLE.
CLEAR LO_IMPORTING.
*=====================================================================
```

```
 DATA LO_ND_ATF_FLUSH TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_EL_ATF_FLUSH TYPE REF TO IF_WD_CONTEXT_ELEMENT.
 DATA LS_ATF_FLUSH TYPE WD_THIS->ELEMENT_ATF_FLUSH.
 DATA LV_FLUSH TYPE WD_THIS->ELEMENT_ATF_FLUSH-FLUSH.
 DATA LV_DESC TYPE WD_THIS->ELEMENT_ATF_FLUSH-DESC.
 LO_ND_ATF_FLUSH = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_ATF_FLUSH ).
 LO_EL_ATF_FLUSH = LO_ND_ATF_FLUSH->GET_ELEMENT( ).
 IF LO_EL_ATF_FLUSH IS NOT INITIAL.
 LO_EL_ATF_FLUSH->GET_ATTRIBUTE(
  EXPORTING
   NAME =  `FLUSH`
  IMPORTING
   VALUE = LV_FLUSH ).
ENDIF.
*=====================================================================

CLEAR LO_IMPORTING.
LO_Z_GET_CUSTOMER_INDEN = WD_CONTEXT->GET_CHILD_NODE( WD_THIS-
>WDCTX_Z_GET_CUSTOMER_INDEN ).
LO_IMPORTING = LO_Z_GET_CUSTOMER_INDEN->GET_CHILD_NODE( WD_THIS->WDCTX_IMPORTING_1 ).
LO_IMPORTING->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `IND_DATE`
  IMPORTING
   VALUE = LV_BEGDA ).

 DATA LO_ND_HEADER_DATA TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_EL_HEADER_DATA TYPE REF TO IF_WD_CONTEXT_ELEMENT.
 DATA LS_HEADER_DATA TYPE WD_THIS->ELEMENT_HEADER_DATA.
 DATA LV_LOAD_DATE TYPE WD_THIS->ELEMENT_HEADER_DATA-LOAD_DATE.
 LO_ND_HEADER_DATA = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS-
>WDCTX_HEADER_DATA ).
 LO_EL_HEADER_DATA = LO_ND_HEADER_DATA->GET_ELEMENT( ).
 IF LO_EL_HEADER_DATA IS NOT INITIAL.
  LO_EL_HEADER_DATA->GET_ATTRIBUTE(
  EXPORTING
   NAME =  `LOAD_DATE`
  IMPORTING
   VALUE = LV_LOAD_DATE ).
  LV_BEGDA = LV_LOAD_DATE.
 ENDIF.


IF LV_ZT2 = 'X'.
 DELETE FROM ZSD_CUST_INDENT WHERE BEGDA = LV_BEGDA AND DEPOT = LV_PLANT
 AND VEHICLE = LV_VEHICLE AND ZTT_STATUS = '4'.
ENDIF.

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_HEADER_DATA ).
LO_IMPORTING->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `KUNNR`
  IMPORTING
```

```
    VALUE = LV_KUNNR ).

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_PROD1 ).
LO_IMPORTING->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `PRODUCT`
  IMPORTING
   VALUE = LV_PROD1 ).

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_PROD2 ).
LO_IMPORTING->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `PRODUCT`
  IMPORTING
   VALUE = LV_PROD2 ).

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_PROD3 ).
LO_IMPORTING->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `PRODUCT`
  IMPORTING
   VALUE = LV_PROD3 ).

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_PROD4 ).
LO_IMPORTING->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `PRODUCT`
  IMPORTING
   VALUE = LV_PROD4 ).

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_PROD5 ).
LO_IMPORTING->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `PRODUCT`
  IMPORTING
   VALUE = LV_PROD5 ).

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_PROD6 ).
LO_IMPORTING->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `PRODUCT`
  IMPORTING
   VALUE = LV_PROD6 ).

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_PROD7 ).
LO_IMPORTING->GET_ATTRIBUTE(
  EXPORTING
```

```
    NAME  = `PRODUCT`
   IMPORTING
    VALUE = LV_PROD7 ).

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_PROD8 ).
LO_IMPORTING->GET_ATTRIBUTE(
   EXPORTING
    NAME  = `PRODUCT`
   IMPORTING
    VALUE = LV_PROD8 ).

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_EXPORTING ).
LO_IMPORTING->GET_ATTRIBUTE(
   EXPORTING
    NAME  = `CAPACITY1`
   IMPORTING
    VALUE = LV_QTY1 ).

CALL FUNCTION 'CEVA_CONVERT_FLOAT_TO_CHAR'
   EXPORTING
    float_imp = LV_QTY1
    format_imp = W_NUMBER "Field Format
    round_imp = ' ' "Round off
   IMPORTING
    char_exp = CAP1.
condense CAP1.
if veh_type = 'ATF' or veh_type = 'TTV' or veh_type = 'WOIL' or veh_type = 'TTVM'.
 concatenate CAP1 'KL' into CAP1.
 elseif veh_type = 'OPTR' or veh_type = 'LPGB'.
  concatenate CAP1 'KG' into CAP1.
  endif.
condense CAP1.

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_EXPORTING ).
LO_IMPORTING->GET_ATTRIBUTE(
   EXPORTING
    NAME  = `CAPACITY2`
   IMPORTING
    VALUE = LV_QTY2 ).

CALL FUNCTION 'CEVA_CONVERT_FLOAT_TO_CHAR'
   EXPORTING
    float_imp = LV_QTY2
    format_imp = W_NUMBER "Field Format
    round_imp = ' ' "Round off
   IMPORTING
    char_exp = CAP2.
condense CAP2.
if veh_type = 'ATF' or veh_type = 'TTV' or veh_type = 'WOIL' or veh_type = 'TTVM'.
 concatenate CAP2 'KL' into CAP2.
 elseif veh_type = 'OPTR' or veh_type = 'LPGB'.
```

```
   concatenate CAP2 'KG' into CAP2.
   endif.
condense CAP2.


CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_EXPORTING ).
LO_IMPORTING->GET_ATTRIBUTE(
   EXPORTING
    NAME  = `CAPACITY3`
   IMPORTING
    VALUE = LV_QTY3 ).

CALL FUNCTION 'CEVA_CONVERT_FLOAT_TO_CHAR'
   EXPORTING
    float_imp = LV_QTY3
    format_imp = W_NUMBER "Field Format
    round_imp = ' ' "Round off
   IMPORTING
    char_exp = CAP3.
condense CAP3.
if veh_type = 'ATF' or veh_type = 'TTV' or veh_type = 'WOIL' or veh_type = 'TTVM'.
 concatenate CAP3 'KL' into CAP3.
 elseif veh_type = 'OPTR' or veh_type = 'LPGB'.
   concatenate CAP3 'KG' into CAP3.
   endif.
condense CAP3.


CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_EXPORTING ).
LO_IMPORTING->GET_ATTRIBUTE(
   EXPORTING
    NAME  = `CAPACITY4`
   IMPORTING
    VALUE = LV_QTY4 ).

CALL FUNCTION 'CEVA_CONVERT_FLOAT_TO_CHAR'
   EXPORTING
    float_imp = LV_QTY4
    format_imp = W_NUMBER "Field Format
    round_imp = ' ' "Round off
   IMPORTING
    char_exp = CAP4.
condense CAP4.
if veh_type = 'ATF' or veh_type = 'TTV' or veh_type = 'WOIL' or veh_type = 'TTVM'.
 concatenate CAP4 'KL' into CAP4.
 elseif veh_type = 'OPTR' or veh_type = 'LPGB'.
   concatenate CAP4 'KG' into CAP4.
   endif.
condense CAP4.


CLEAR LO_IMPORTING.
```

```
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_EXPORTING ).
LO_IMPORTING->GET_ATTRIBUTE(
   EXPORTING
    NAME  = `CAPACITY5`
   IMPORTING
    VALUE = LV_QTY5 ).

CALL FUNCTION 'CEVA_CONVERT_FLOAT_TO_CHAR'
   EXPORTING
    float_imp = LV_QTY5
    format_imp = W_NUMBER "Field Format
    round_imp = ' ' "Round off
   IMPORTING
    char_exp = CAP5.
condense CAP5.
if veh_type = 'ATF' or veh_type = 'TTV' or veh_type = 'WOIL' or veh_type = 'TTVM'.
 concatenate CAP5 'KL' into CAP5.
 elseif veh_type = 'OPTR' or veh_type = 'LPGB'.
  concatenate CAP5 'KG' into CAP5.
  endif.
condense CAP5.

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_EXPORTING ).
LO_IMPORTING->GET_ATTRIBUTE(
   EXPORTING
    NAME  = `CAPACITY6`
   IMPORTING
    VALUE = LV_QTY6 ).
CALL FUNCTION 'CEVA_CONVERT_FLOAT_TO_CHAR'
   EXPORTING
    float_imp = LV_QTY6
    format_imp = W_NUMBER "Field Format
    round_imp = ' ' "Round off
   IMPORTING
    char_exp = CAP6.
condense CAP6.
if veh_type = 'ATF' or veh_type = 'TTV' or veh_type = 'WOIL' or veh_type = 'TTVM'.
 concatenate CAP6 'KL' into CAP6.
 elseif veh_type = 'OPTR' or veh_type = 'LPGB'.
  concatenate CAP6 'KG' into CAP6.
  endif.
condense CAP6.

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_EXPORTING ).
LO_IMPORTING->GET_ATTRIBUTE(
   EXPORTING
    NAME  = `CAPACITY7`
   IMPORTING
    VALUE = LV_QTY7 ).
CALL FUNCTION 'CEVA_CONVERT_FLOAT_TO_CHAR'
   EXPORTING
    float_imp = LV_QTY7
```

```
    format_imp = W_NUMBER "Field Format
    round_imp = ' ' "Round off
  IMPORTING
    char_exp = CAP7.
condense CAP7.
if veh_type = 'ATF' or veh_type = 'TTV' or veh_type = 'WOIL' or veh_type = 'TTVM'.
 concatenate CAP7 'KL' into CAP7.
 elseif veh_type = 'OPTR' or veh_type = 'LPGB'.
  concatenate CAP7 'KG' into CAP7.
  endif.
condense CAP7.

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_EXPORTING ).
LO_IMPORTING->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `CAPACITY8`
  IMPORTING
   VALUE = LV_QTY8 ).
CALL FUNCTION 'CEVA_CONVERT_FLOAT_TO_CHAR'
  EXPORTING
   float_imp = LV_QTY8
   format_imp = W_NUMBER "Field Format
   round_imp = ' ' "Round off
  IMPORTING
   char_exp = CAP8.
condense CAP8.
if veh_type = 'ATF' or veh_type = 'TTV' or veh_type = 'WOIL' or veh_type = 'TTVM'.
 concatenate CAP8 'KL' into CAP8.
 elseif veh_type = 'OPTR' or veh_type = 'LPGB'.
  concatenate CAP8 'KG' into CAP8.
  endif.
condense CAP8.
CLEAR: C1_QT, C2_QT, C3_QT, C4_QT, C5_QT, C6_QT, C7_QT, C7_QT.
C1_QT = LV_QTY1. C2_QT = LV_QTY2. C3_QT = LV_QTY3. C4_QT = LV_QTY4.
C5_QT = LV_QTY5. C6_QT = LV_QTY6. C7_QT = LV_QTY7. C8_QT = LV_QTY8.

DATA LO_WINDOW_MANAGER TYPE REF TO IF_WD_WINDOW_MANAGER.
DATA LO_API_COMPONENT  TYPE REF TO IF_WD_COMPONENT.
DATA LO_WINDOW       TYPE REF TO IF_WD_WINDOW.
DATA: lt_string type table of string,
    ls_string like line of lt_string.
DATA l_api           TYPE REF TO if_wd_view_controller.
DATA: ERR.

DATA lr_event TYPE REF TO cl_wd_custom_event.
DATA: LO_ERROR TYPE REF TO IF_WD_CONTEXT_NODE.
DATA: LV_ERCODE TYPE CHAR1.
DATA: LV_ERCODE1 TYPE CHAR1.
DATA ZSHNUM TYPE ZSD_CUST_INDENT-SHNUMBER.
DATA ZSTATUS TYPE OIGS-OIG_SSTSF.

 CREATE OBJECT lr_event
  EXPORTING
```

```
    name = 'CHECK_QUANTITY'.

 WD_THIS->ONACTIONCHECK_QUANTITY(
  WDEVENT = lr_event                    " ref to cl_wd_custom_event
 ).

LO_ERROR = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_ERROR ).
LO_ERROR->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `ERCODE`
  IMPORTING
   VALUE = LV_ERCODE ).

IF VEH_TYPE = 'ATF' AND LV_FLUSH = ' '.
 ERR = 'X'.
 ls_string = 'Please select Flushing requirement for ATF tanker'.
 insert ls_string into table lt_string.
 LO_API_COMPONENT         = WD_COMP_CONTROLLER->WD_GET_API( ).
 LO_WINDOW_MANAGER        = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).

 LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
         text         = lt_string
         button_kind    = if_wd_window=>co_buttons_ok
*         button_kind    = 4
         message_type   = if_wd_window=>CO_MSG_TYPE_STOPP
         window_title   = 'Please check!'
         window_position = if_wd_window=>co_center
         WINDOW_WIDTH   = '20%'
         WINDOW_HEIGHT  = '20%' ).
 LO_WINDOW->open( ).
ENDIF.

IF VEH_TYPE = 'ATF' AND LV_FLUSH = 'Y'.
  DATA LO_ND_FLUSH_REASON TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LO_EL_FLUSH_REASON TYPE REF TO IF_WD_CONTEXT_ELEMENT.
  DATA LS_FLUSH_REASON TYPE WD_THIS->ELEMENT_FLUSH_REASON.
  DATA LV_FLSH_REASON TYPE WD_THIS->ELEMENT_FLUSH_REASON-FLSH_REASON.
  LO_ND_FLUSH_REASON = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS-
>WDCTX_FLUSH_REASON ).
*  get element via lead selection
  LO_EL_FLUSH_REASON = LO_ND_FLUSH_REASON->GET_ELEMENT( ).
  IF LO_EL_FLUSH_REASON IS INITIAL.
  ENDIF.
*  get single attribute
  LO_EL_FLUSH_REASON->GET_ATTRIBUTE(
   EXPORTING
    NAME =  `FLSH_REASON`
   IMPORTING
    VALUE = LV_FLSH_REASON ).

  IF LV_FLSH_REASON IS INITIAL.
   CLEAR ls_string. CLEAR lt_string. REFRESH lt_string.
     ERR = 'X'.
     ls_string = 'Please select Flushing reason'.
```

```
      insert ls_string into table lt_string.
      LO_API_COMPONENT        = WD_COMP_CONTROLLER->WD_GET_API( ).
      LO_WINDOW_MANAGER       = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).

      LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
            text         = lt_string
            button_kind    = if_wd_window=>co_buttons_ok
*             button_kind    = 4
            message_type   = if_wd_window=>CO_MSG_TYPE_STOPP
            window_title   = 'Please check!'
            window_position = if_wd_window=>co_center
            WINDOW_WIDTH   = '20%'
            WINDOW_HEIGHT  = '20%' ).
      LO_WINDOW->open( ).

  ENDIF.

ENDIF.

IF LV_KUNNR IS INITIAL.
 CLEAR ls_string. CLEAR lt_string. REFRESH lt_string.
 ERR = 'X'.
 ls_string = 'Please select Customer'.
 insert ls_string into table lt_string.
 LO_API_COMPONENT        = WD_COMP_CONTROLLER->WD_GET_API( ).
 LO_WINDOW_MANAGER       = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).

 LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
        text         = lt_string
        button_kind    = if_wd_window=>co_buttons_ok
*         button_kind    = 4
        message_type   = if_wd_window=>CO_MSG_TYPE_STOPP
        window_title   = 'Please check!'
        window_position = if_wd_window=>co_center
        WINDOW_WIDTH   = '20%'
        WINDOW_HEIGHT  = '20%' ).
 LO_WINDOW->open( ).
ENDIF.


IF LV_PLANT IS INITIAL.
 CLEAR ls_string. CLEAR lt_string. REFRESH lt_string.
 ERR = 'X'.
 ls_string = 'User is not mapped in ZSD_CUST_USR_MAP'.
 insert ls_string into table lt_string.
 LO_API_COMPONENT        = WD_COMP_CONTROLLER->WD_GET_API( ).
 LO_WINDOW_MANAGER       = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).

 LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
        text         = lt_string
        button_kind    = if_wd_window=>co_buttons_ok
*         button_kind    = 4
        message_type   = if_wd_window=>CO_MSG_TYPE_STOPP
        window_title   = 'Please check!'
```

```
          window_position = if_wd_window=>co_center
          WINDOW_WIDTH    = '20%'
          WINDOW_HEIGHT   = '20%' ).
 LO_WINDOW->open( ).
ENDIF.


IF ERR <> 'X'.
  SELECT COUNT(*) FROM ZSD_CUST_INDENT WHERE BEGDA = LV_BEGDA AND
   VEHICLE = LV_VEHICLE AND DEPOT = LV_PLANT AND SHNUMBER = ' '.
  IF SY-SUBRC = 0.
   LV_ERCODE1 = 'X'.
   ELSE.
    SELECT SINGLE SHNUMBER FROM ZSD_CUST_INDENT INTO ZSHNUM WHERE  "#EC CI_NOORDER " #S
/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
      BEGDA = LV_BEGDA AND VEHICLE = LV_VEHICLE AND DEPOT = LV_PLANT.
      IF ZSHNUM IS NOT INITIAL.
       SELECT SINGLE OIG_SSTSF FROM OIGS INTO ZSTATUS WHERE SHNUMBER = ZSHNUM.
        IF ZSTATUS < 6 AND SY-SUBRC = 0.
         LV_ERCODE1 = 'X'.
        ENDIF.
      ENDIF.
  ENDIF.

  IF LV_ERCODE1 = 'X'.
   CLEAR ls_string. CLEAR lt_string. REFRESH lt_string.
   ls_string = 'Indent already available for the vehicle'.
   insert ls_string into table lt_string.
   LO_API_COMPONENT         = WD_COMP_CONTROLLER->WD_GET_API( ).
   LO_WINDOW_MANAGER        = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).
   LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
           text         = lt_string
           button_kind    = if_wd_window=>co_buttons_ok
*           button_kind    = 4
           message_type   = if_wd_window=>CO_MSG_TYPE_STOPP
           window_title   = 'Please check!'
           window_position = if_wd_window=>co_center
           WINDOW_WIDTH   = '20%'
           WINDOW_HEIGHT  = '20%' ).
   LO_WINDOW->open( ).
  ENDIF.
ENDIF.

IF ERR <> 'X' AND LV_ERCODE1 <> 'X'.
 CLEAR CNT. CLEAR LV_ERCODE3.
 SELECT COUNT(*) FROM OIGCC INTO CNT WHERE TU_NUMBER = LV_VEHICLE.
  IF LV_PROD1 = ' '.
   LV_ERCODE3 = 'X'.
   ENDIF.
  IF CNT > 1 AND LV_PROD2 = ' '.
   LV_ERCODE3 = 'X'.
   ELSEIF CNT > 2 AND LV_PROD3 = ' '.
    LV_ERCODE3 = 'X'.
    ELSEIF CNT > 3 AND LV_PROD4 = ' '.
```

```
      LV_ERCODE3 = 'X'.
      ELSEIF CNT > 4 AND LV_PROD5 = ' '.
       LV_ERCODE3 = 'X'.
       ELSEIF CNT > 5 AND LV_PROD6 = ' '.
        LV_ERCODE3 = 'X'.
        ELSEIF CNT > 6 AND LV_PROD7 = ' '.
         LV_ERCODE3 = 'X'.
         ELSEIF CNT > 7 AND LV_PROD8 = ' '.
          LV_ERCODE3 = 'X'.
          ENDIF.

  IF LV_ERCODE3 = 'X'.
    CLEAR ls_string. CLEAR lt_string. REFRESH lt_string.
    ls_string = 'Please select Product for all compartments'.
    insert ls_string into table lt_string.
    LO_API_COMPONENT        = WD_COMP_CONTROLLER->WD_GET_API( ).
    LO_WINDOW_MANAGER       = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).
    LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
            text         = lt_string
            button_kind    = if_wd_window=>co_buttons_ok
*            button_kind    = 4
            message_type   = if_wd_window=>CO_MSG_TYPE_STOPP
            window_title   = 'Please check!'
            window_position = if_wd_window=>co_center
            WINDOW_WIDTH   = '20%'
            WINDOW_HEIGHT  = '20%' ).
    LO_WINDOW->open( ).
   ENDIF.
ENDIF.

**** For selection of Transporter GSTN START ****
DATA LV_GSTN TYPE CHAR100.
DATA LO_GSTN_CHECK TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_GSTN_CHECK_ELEMENT TYPE REF TO IF_WD_CONTEXT_ELEMENT.

LO_GSTN_CHECK = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_GSTN ).
LO_GSTN_CHECK_ELEMENT = LO_GSTN_CHECK->GET_ELEMENT( ).
IF LO_GSTN_CHECK_ELEMENT IS NOT INITIAL.
 LO_GSTN_CHECK_ELEMENT->GET_ATTRIBUTE(
     EXPORTING
      NAME  = `TPT_GSTN`
     IMPORTING
      VALUE = LV_GSTN ).
ENDIF.
DATA: CUST_KDGRP TYPE KDGRP,
    MAT_SPART TYPE SPART,
    ZCHK(10).

CLEAR: CUST_KDGRP, MAT_SPART, ZCHK.
SELECT SINGLE KDGRP FROM KNVV INTO CUST_KDGRP WHERE KUNNR = LV_KUNNR.  "#EC CI_NOORDER
" #S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
SELECT SINGLE SPART FROM MARA INTO MAT_SPART WHERE MATNR = LV_PROD1.  "#EC CI_NOORDER " #
S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
IF MAT_SPART <> '40' AND MAT_SPART <> '25' AND MAT_SPART <> '30'.
```

```
 ZCHK = 'X'.
ENDIF.
IF ( CUST_KDGRP = 'DI' OR CUST_KDGRP = 'EX' ) AND LV_GSTN IS INITIAL AND ZCHK = 'X'.
 LV_ERCODE3 = 'X'.
    CLEAR ls_string. CLEAR lt_string. REFRESH lt_string.
    ls_string = 'Please enter GSTN of Transporter'.
    insert ls_string into table lt_string.
    LO_API_COMPONENT         = WD_COMP_CONTROLLER->WD_GET_API( ).
    LO_WINDOW_MANAGER        = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).
    LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
            text         = lt_string
            button_kind    = if_wd_window=>co_buttons_ok
*             button_kind    = 4
            message_type    = if_wd_window=>CO_MSG_TYPE_STOPP
            window_title    = 'Please check!'
            window_position = if_wd_window=>co_center
            WINDOW_WIDTH    = '20%'
            WINDOW_HEIGHT   = '20%' ).
    LO_WINDOW->open( ).
ENDIF.

**** For selection of Export/ Domestic sale for WAX START ****
 DATA LV_VBELN TYPE VBELN.
 DATA LO_EXPORT_CHECK TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_EXPORT_CHECK_ELEMENT TYPE REF TO IF_WD_CONTEXT_ELEMENT.

 LO_EXPORT_CHECK = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_EXPORT ).
 LO_EXPORT_CHECK_ELEMENT = LO_EXPORT_CHECK->GET_ELEMENT( ).
 IF LO_EXPORT_CHECK_ELEMENT IS NOT INITIAL.
  LO_EXPORT_CHECK_ELEMENT->GET_ATTRIBUTE(
    EXPORTING
     NAME  = `VBELN`
    IMPORTING
     VALUE = LV_VBELN ).
 ENDIF.

***** START for CONTRACT CHECK ******
DATA lr_event1 TYPE REF TO cl_wd_custom_event.
IF LV_VBELN IS INITIAL.
  CREATE OBJECT lr_event
    EXPORTING
     name = 'SELECT_EXPORT'.

  wd_this->ONACTIONSELECT_EXPORT(
   wdevent = lr_event1                  " ref to cl_wd_custom_event
  ).
ENDIF.
***** END for CONTRACT CHECK *******

 IF LV_VBELN = ' ' AND  ( LV_PROD1+0(4) = 'PWAX' OR LV_PROD2+0(4) = 'PWAX' OR LV_PROD3+0(4) = 'PWAX
'
  OR LV_PROD4+0(4) = 'PWAX' OR LV_PROD5+0(4) = 'PWAX' OR LV_PROD6+0(4) = 'PWAX' OR LV_PROD1+0(
4) = 'SUL0' OR
  LV_PROD1+0(4) = 'RPC0' OR LV_PROD1+0(4) = 'CPC0' ).  "#EC CI_USAGE_OK[2270335] " #S/4 - OSS Note#
```

```
 - Added by - Z_SHSHRUKHA – 2021/08/18
   LV_ERCODE3 = 'X'.
    CLEAR ls_string. CLEAR lt_string. REFRESH lt_string.
    ls_string = 'Please select SALES CONTRACT'.
    insert ls_string into table lt_string.
    LO_API_COMPONENT        = WD_COMP_CONTROLLER->WD_GET_API( ).
    LO_WINDOW_MANAGER       = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).
    LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
           text         = lt_string
           button_kind    = if_wd_window=>co_buttons_ok
*           button_kind    = 4
           message_type   = if_wd_window=>CO_MSG_TYPE_STOPP
           window_title   = 'Please check!'
           window_position = if_wd_window=>co_center
           WINDOW_WIDTH   = '20%'
           WINDOW_HEIGHT  = '20%' ).
    LO_WINDOW->open( ).
  ENDIF.

**** For selection of Export/ Domestic sale for WAX END ****

**** For CONTRACT QTY CHECK for WAX START ****
TYPES:  BEGIN OF SCONTRACT ,
      VBELN TYPE VBAK-VBELN,
      KUNNR TYPE KUNNR,
      MATNR TYPE MATNR,
      VAL TYPE BWTAR_D,
      KONDM TYPE KONDM,
      TQTY TYPE VBFA-RFMNG,
      TUOM TYPE VBAP-ZIEME,
      POSNR TYPE VBAP-POSNR,
      EQTY TYPE VBFA-RFMNG,
      RQTY TYPE VBFA-RFMNG,
      END OF SCONTRACT.

DATA: IT_CONTRACT type
     standard table of SCONTRACT.
DATA: LN_CONTRACT TYPE SCONTRACT.

IF LV_VBELN <> ' ' AND  ( LV_PROD1+0(4) = 'PWAX' OR LV_PROD2+0(4) = 'PWAX' OR LV_PROD3+0(4) = 'PWA
X'
   OR LV_PROD4+0(4) = 'PWAX' OR LV_PROD5+0(4) = 'PWAX' OR LV_PROD6+0(4) = 'PWAX' OR LV_PROD1+0(
4) = 'SUL0' OR
   LV_PROD1+0(4) = 'RPC0' OR LV_PROD1+0(4) = 'CPC0' ).  "#EC CI_USAGE_OK[2270335] " #S/4 - OSS Note#
 - Added by - Z_SHSHRUKHA – 2021/08/18

  DATA: ZNVEH_IND TYPE ZSD_INDENT_NVEH,
      ZBLK_ORD TYPE CHAR10.
  CLEAR: LN_CONTRACT, ZNVEH_IND, ZBLK_ORD.
  SELECT SINGLE * FROM ZSD_INDENT_NVEH INTO ZNVEH_IND WHERE ORDER_NO = LV_VBELN.
  IF SY-SUBRC = 0.
   ZBLK_ORD  = LV_VBELN.
   LV_VBELN  = ZNVEH_IND-CONTRACT1.
  ENDIF.
```

```
 SELECT SINGLE ZMENG ZIEME POSNR FROM VBAP INTO ( LN_CONTRACT-TQTY, LN_CONTRACT-
TUOM, LN_CONTRACT-
POSNR ) "#EC CI_NOORDER " #S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
   WHERE VBELN = LV_VBELN AND MATNR = LV_PROD1.
 SELECT SUM( RFMNG_FLO ) FROM VBFA INTO LN_CONTRACT-EQTY WHERE VBELV = LV_VBELN
   AND VBTYP_N = 'C' AND POSNV = LN_CONTRACT-POSNR.
 LN_CONTRACT-RQTY = LN_CONTRACT-TQTY - LN_CONTRACT-EQTY.

 IF LN_CONTRACT-
RQTY < ( LV_QTY1 + LV_QTY2 + LV_QTY3 + LV_QTY4 + LV_QTY5 + LV_QTY6 + LV_QTY7 + LV_QTY8 ).
    LV_ERCODE3 = 'X'.
    CLEAR ls_string. CLEAR lt_string. REFRESH lt_string.
    ls_string = 'Balance quantity in the contract is not sufficient'.
    insert ls_string into table lt_string.
    LO_API_COMPONENT         = WD_COMP_CONTROLLER->WD_GET_API( ).
    LO_WINDOW_MANAGER        = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).
    LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
            text         = lt_string
            button_kind    = if_wd_window=>co_buttons_ok
*            button_kind    = 4
            message_type   = if_wd_window=>CO_MSG_TYPE_STOPP
            window_title   = 'Please check!'
            window_position = if_wd_window=>co_center
            WINDOW_WIDTH   = '20%'
            WINDOW_HEIGHT  = '20%' ).
    LO_WINDOW->open( ).

 ENDIF.
ENDIF.
**** For Material check for SKO and MTO ****
DATA: PRD_ERR(1).
DATA: ZPRODUCT TYPE CHAR100.
IF LV_PROD1 = 'SKO01' OR LV_PROD2 = 'SKO01' OR LV_PROD3 = 'SKO01' OR LV_PROD4 = 'SKO01' OR LV_PR
OD5 = 'SKO01'
 OR LV_PROD6 = 'SKO01' OR LV_PROD7 = 'SKO01' OR LV_PROD8 = 'SKO01'.
 CLEAR ZPRODUCT. ZPRODUCT = 'SKO01'.
ENDIF.

IF LV_PROD1 = 'SKO02' OR LV_PROD2 = 'SKO02' OR LV_PROD3 = 'SKO02' OR LV_PROD4 = 'SKO02' OR LV_PR
OD5 = 'SKO02'
 OR LV_PROD6 = 'SKO02' OR LV_PROD7 = 'SKO02' OR LV_PROD8 = 'SKO02'.
 CLEAR ZPRODUCT. ZPRODUCT = 'SKO02'.
ENDIF.

IF LV_PROD1 = 'MTO01' OR LV_PROD2 = 'MTO01' OR LV_PROD3 = 'MTO01' OR LV_PROD4 = 'MTO01' OR LV_P
ROD5 = 'MTO01'
 OR LV_PROD6 = 'MTO01' OR LV_PROD7 = 'MTO01' OR LV_PROD8 = 'MTO01'.
 CLEAR ZPRODUCT. ZPRODUCT = 'MTO01'.
ENDIF.

IF ( LV_PROD1 = ZPRODUCT OR LV_PROD2 = ZPRODUCT OR LV_PROD3 = ZPRODUCT OR LV_PROD4 = ZPRO
DUCT OR LV_PROD5 = ZPRODUCT
 OR LV_PROD6 = ZPRODUCT OR LV_PROD7 = ZPRODUCT OR LV_PROD8 = ZPRODUCT ) AND ZPRODUCT <>
' '.
```

```
 CLEAR PRD_ERR.
 IF CNT = 8 AND ( LV_PROD1 <> LV_PROD2 OR LV_PROD1 <> LV_PROD3 OR LV_PROD1 <> LV_PROD4 OR LV
_PROD1 <> LV_PROD5
   OR LV_PROD1 <> LV_PROD6 OR LV_PROD1 <> LV_PROD7 OR LV_PROD1 <> LV_PROD8 ).
   PRD_ERR = 'E'.

   ELSEIF CNT = 7 AND ( LV_PROD1 <> LV_PROD2 OR LV_PROD1 <> LV_PROD3 OR LV_PROD1 <> LV_PROD4
OR LV_PROD1 <> LV_PROD5
   OR LV_PROD1 <> LV_PROD6 OR LV_PROD1 <> LV_PROD7 ).
   PRD_ERR = 'E'.

   ELSEIF CNT = 6 AND ( LV_PROD1 <> LV_PROD2 OR LV_PROD1 <> LV_PROD3 OR LV_PROD1 <> LV_PROD4
OR LV_PROD1 <> LV_PROD5
   OR LV_PROD1 <> LV_PROD6 ).
   PRD_ERR = 'E'.

   ELSEIF CNT = 5 AND ( LV_PROD1 <> LV_PROD2 OR LV_PROD1 <> LV_PROD3 OR LV_PROD1 <> LV_PROD4
OR LV_PROD1 <> LV_PROD5 ).
    PRD_ERR = 'E'.
    ELSEIF CNT = 4 AND ( LV_PROD1 <> LV_PROD2 OR LV_PROD1 <> LV_PROD3 OR LV_PROD1 <> LV_PROD
4 ).
     PRD_ERR = 'E'.
     ELSEIF CNT = 3 AND ( LV_PROD1 <> LV_PROD2 OR LV_PROD1 <> LV_PROD3 ).
      PRD_ERR = 'E'.
      ELSEIF CNT = 2 AND LV_PROD1 <> LV_PROD2.
       PRD_ERR = 'E'.
 ENDIF.
 IF PRD_ERR <> ' '.
   LV_ERCODE3 = 'X'.
   CLEAR ls_string. CLEAR lt_string. REFRESH lt_string.
   ls_string = 'Please select the same material for all compartments'.
   insert ls_string into table lt_string.
   LO_API_COMPONENT         = WD_COMP_CONTROLLER->WD_GET_API( ).
   LO_WINDOW_MANAGER        = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).

   LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
           text         = lt_string
           button_kind    = if_wd_window=>co_buttons_ok
           message_type   = if_wd_window=>CO_MSG_TYPE_STOPP
           window_title   = 'Please check!'
           window_position = if_wd_window=>co_center
           WINDOW_WIDTH   = '20%'
           WINDOW_HEIGHT  = '20%' ).
   LO_WINDOW->open( ).
 ENDIF.
ENDIF.


TYPES: BEGIN OF ITAB,
     PROD TYPE MATNR,
     QTY(16),
     SPART TYPE SPART,
     END OF ITAB.
```

```
DATA ITAB_1 TYPE STANDARD TABLE OF ITAB.
DATA ITAB_2 TYPE STANDARD TABLE OF ITAB.
DATA ITAB_3 TYPE STANDARD TABLE OF ITAB.
DATA ITABS TYPE ITAB.
DATA ITABS1 TYPE ITAB.
DATA: ZKDGRP TYPE KDGRP.
DATA: LV_ERCODE4 TYPE CHAR1.
DATA: ZCUST_MAP TYPE ZSD_CUST_USR_MAP.
DATA: LV_VAL_NEW TYPE BWTAR_D.


IF ERR <> 'X' AND LV_ERCODE <> 'X' AND LV_ERCODE1 <> 'X' AND LV_ERCODE3 <> 'X'.
 SELECT SINGLE KDGRP FROM KNVV INTO CUST_INDENT-
KDGRP WHERE KUNNR = LV_KUNNR.  "#EC CI_NOORDER " #S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
 CUST_INDENT-BEGDA = LV_BEGDA.
 CUST_INDENT-VEHICLE = LV_VEHICLE.
 CUST_INDENT-DEPOT = LV_PLANT.
 CUST_INDENT-INDENT_DATE = SY-DATUM.
 CUST_INDENT-INDENT_TIME = SY-UZEIT.
 CUST_INDENT-CUST_USER_ID = SY-UNAME.
 CUST_INDENT-KUNNR = LV_KUNNR.
 IF VEH_TYPE = 'TTV' OR VEH_TYPE = 'ATF' OR VEH_TYPE = 'WOIL' OR VEH_TYPE = 'TTVM'.
  CUST_INDENT-SHTYPE = '1111'.
 ELSE.
  CUST_INDENT-SHTYPE = '1112'.
 ENDIF.

 IF LV_PLANT = '3100'. LV_VAL = 'OWN-BOND'.
  ELSE. LV_VAL = 'OWN-DTPD'.
 ENDIF.
 IF LV_PLANT <> '3100' AND LV_VBELN = 'EP'.
  LV_VAL = 'OWN-NILDTY'.
 ENDIF.
 IF LV_PLANT <> '3100' AND LV_VBELN = 'IN'.
  LV_VAL = 'OWN-DTPDIN'.
 ENDIF.

 SELECT SINGLE * FROM ZSD_CUST_USR_MAP INTO ZCUST_MAP WHERE
  CUST_USER_ID = SY-UNAME AND DEPOT = LV_PLANT.

 CUST_INDENT-PROD_CMP1 = LV_PROD1.
 IF LV_PROD1 IS NOT INITIAL.
  CLEAR: LV_VAL_NEW.
  SELECT SINGLE VALUATION FROM ZDESP_STOCK_PLNT INTO LV_VAL_NEW WHERE
   PLANT = LV_PLANT AND CUSTOMER = ZCUST_MAP-KUNNR AND MATNR = LV_PROD1.
  IF LV_VAL_NEW <> ' '.
   CUST_INDENT-VAL_COMP1 = LV_VAL_NEW.
   ELSE.
    CUST_INDENT-VAL_COMP1 = LV_VAL.
  ENDIF.
  ELSE.
   CLEAR CUST_INDENT-VAL_COMP1.
 ENDIF.
```

```
 CUST_INDENT-QUAN_COMP1 = CAP1.
 CUST_INDENT-
PROD_CMP2 = LV_PROD2.*"#EC CI_FLDEXT_OK[2215424] " #S/4 - OSS Note# - Start of Change- Z_SHSHRUK
HA – 2021/08/13*
 IF LV_PROD2 IS NOT INITIAL.
  CLEAR: LV_VAL_NEW.
  SELECT SINGLE VALUATION FROM ZDESP_STOCK_PLNT INTO LV_VAL_NEW WHERE
   PLANT = LV_PLANT AND CUSTOMER = ZCUST_MAP-KUNNR AND MATNR = LV_PROD2.
  IF LV_VAL_NEW <> ' '.
   CUST_INDENT-VAL_COMP2 = LV_VAL_NEW.
   ELSE.
    CUST_INDENT-VAL_COMP2 = LV_VAL.
  ENDIF.
  ELSE.
   CLEAR CUST_INDENT-VAL_COMP2.
 ENDIF.
 CUST_INDENT-QUAN_COMP2 = CAP2.
 CUST_INDENT-
PROD_CMP3 = LV_PROD3.*"#EC CI_FLDEXT_OK[2215424] " #S/4 - OSS Note# - Start of Change- Z_SHSHRUK
HA – 2021/08/13*
 IF LV_PROD3 IS NOT INITIAL.
  CLEAR: LV_VAL_NEW.
  SELECT SINGLE VALUATION FROM ZDESP_STOCK_PLNT INTO LV_VAL_NEW WHERE
   PLANT = LV_PLANT AND CUSTOMER = ZCUST_MAP-KUNNR AND MATNR = LV_PROD3.
  IF LV_VAL_NEW <> ' '.
   CUST_INDENT-VAL_COMP3 = LV_VAL_NEW.
   ELSE.
    CUST_INDENT-VAL_COMP3 = LV_VAL.
  ENDIF.
  ELSE.
   CLEAR CUST_INDENT-VAL_COMP3.
 ENDIF.
 CUST_INDENT-QUAN_COMP3 = CAP3.
 CUST_INDENT-
PROD_CMP4 = LV_PROD4.*"#EC CI_FLDEXT_OK[2215424] " #S/4 - OSS Note# - Start of Change- Z_SHSHRUK
HA – 2021/08/13*
 IF LV_PROD4 IS NOT INITIAL.
  CLEAR: LV_VAL_NEW.
  SELECT SINGLE VALUATION FROM ZDESP_STOCK_PLNT INTO LV_VAL_NEW WHERE
   PLANT = LV_PLANT AND CUSTOMER = ZCUST_MAP-KUNNR AND MATNR = LV_PROD4.
  IF LV_VAL_NEW <> ' '.
   CUST_INDENT-VAL_COMP4 = LV_VAL_NEW.
   ELSE.
    CUST_INDENT-VAL_COMP4 = LV_VAL.
  ENDIF.
  ELSE.
   CLEAR CUST_INDENT-VAL_COMP4.
 ENDIF.
 CUST_INDENT-QUAN_COMP4 = CAP4.
 CUST_INDENT-
PROD_CMP5 = LV_PROD5.*"#EC CI_FLDEXT_OK[2215424] " #S/4 - OSS Note# - Start of Change- Z_SHSHRUK
HA – 2021/08/13*
 IF LV_PROD5 IS NOT INITIAL.
  CLEAR: LV_VAL_NEW.
```

```
  SELECT SINGLE VALUATION FROM ZDESP_STOCK_PLNT INTO LV_VAL_NEW WHERE
   PLANT = LV_PLANT AND CUSTOMER = ZCUST_MAP-KUNNR AND MATNR = LV_PROD5.
  IF LV_VAL_NEW <> ' '.
   CUST_INDENT-VAL_COMP5 = LV_VAL_NEW.
   ELSE.
    CUST_INDENT-VAL_COMP5 = LV_VAL.
  ENDIF.
  ELSE.
   CLEAR CUST_INDENT-VAL_COMP5.
 ENDIF.
 CUST_INDENT-QUAN_COMP5 = CAP5.

 CUST_INDENT-
PROD_CMP6 = LV_PROD6."#EC CI_FLDEXT_OK[2215424] " #S/4 - OSS Note# - Start of Change- Z_SHSHRUK
HA – 2021/08/13
 IF LV_PROD6 IS NOT INITIAL.
  CLEAR: LV_VAL_NEW.
  SELECT SINGLE VALUATION FROM ZDESP_STOCK_PLNT INTO LV_VAL_NEW WHERE
   PLANT = LV_PLANT AND CUSTOMER = ZCUST_MAP-KUNNR AND MATNR = LV_PROD6.
  IF LV_VAL_NEW <> ' '.
   CUST_INDENT-VAL_COMP6 = LV_VAL_NEW.
   ELSE.
    CUST_INDENT-VAL_COMP6 = LV_VAL.
  ENDIF.
  ELSE.
   CLEAR CUST_INDENT-VAL_COMP6.
 ENDIF.
 CUST_INDENT-QUAN_COMP6 = CAP6.

 CUST_INDENT-
PROD_CMP7 = LV_PROD7."#EC CI_FLDEXT_OK[2215424] " #S/4 - OSS Note# - Start of Change- Z_SHSHRUK
HA – 2021/08/13
 IF LV_PROD7 IS NOT INITIAL.
  CLEAR: LV_VAL_NEW.
  SELECT SINGLE VALUATION FROM ZDESP_STOCK_PLNT INTO LV_VAL_NEW WHERE
   PLANT = LV_PLANT AND CUSTOMER = ZCUST_MAP-KUNNR AND MATNR = LV_PROD7.
  IF LV_VAL_NEW <> ' '.
   CUST_INDENT-VAL_COMP7 = LV_VAL_NEW.
   ELSE.
    CUST_INDENT-VAL_COMP7 = LV_VAL.
  ENDIF.
  ELSE.
   CLEAR CUST_INDENT-VAL_COMP7.
 ENDIF.
 CUST_INDENT-QUAN_COMP7 = CAP7.

 CUST_INDENT-
PROD_CMP8 = LV_PROD8."#EC CI_FLDEXT_OK[2215424] " #S/4 - OSS Note# - Start of Change- Z_SHSHRUK
HA – 2021/08/13
 IF LV_PROD8 IS NOT INITIAL.
  CLEAR: LV_VAL_NEW.
  SELECT SINGLE VALUATION FROM ZDESP_STOCK_PLNT INTO LV_VAL_NEW WHERE
   PLANT = LV_PLANT AND CUSTOMER = ZCUST_MAP-KUNNR AND MATNR = LV_PROD8.
  IF LV_VAL_NEW <> ' '.
```

```
   CUST_INDENT-VAL_COMP8 = LV_VAL_NEW.
   ELSE.
    CUST_INDENT-VAL_COMP8 = LV_VAL.
 ENDIF.
 ELSE.
   CLEAR CUST_INDENT-VAL_COMP8.
ENDIF.
CUST_INDENT-QUAN_COMP8 = CAP8.

CUST_INDENT-ATF_FLUSH = LV_FLUSH.
CUST_INDENT-ZTT_STATUS = '4'.
CUST_INDENT-ZTT_STATUS_DESC = 'Indent Saved'.
CUST_INDENT-INDENT_TYPE = 'PORTAL'.
CONDENSE LV_VBELN.
IF LV_VBELN <> 'IN' AND LV_VBELN <> 'EP'.
 CLEAR CUST_INDENT-KONDM.
 ELSE.
   CUST_INDENT-KONDM = LV_VBELN.
ENDIF.

DATA: IT_VALTYPE TYPE TABLE OF ZSET_MAT_VALTYPE,
    Z_VALTYPE TYPE ZSET_MAT_VALTYPE.

CLEAR: IT_VALTYPE, IT_VALTYPE[], Z_VALTYPE.
SELECT * FROM ZSET_MAT_VALTYPE INTO Z_VALTYPE WHERE PLANT
 = CUST_INDENT-DEPOT.
 APPEND Z_VALTYPE TO IT_VALTYPE.
ENDSELECT.
READ TABLE IT_VALTYPE INTO Z_VALTYPE WITH KEY MATNR = CUST_INDENT-PROD_CMP1.
IF SY-SUBRC = 0.
 CUST_INDENT-VAL_COMP1 = Z_VALTYPE-VAL_TYPE.
ENDIF.
READ TABLE IT_VALTYPE INTO Z_VALTYPE WITH KEY MATNR = CUST_INDENT-PROD_CMP2.
IF SY-SUBRC = 0.
 CUST_INDENT-VAL_COMP2 = Z_VALTYPE-VAL_TYPE.
ENDIF.
READ TABLE IT_VALTYPE INTO Z_VALTYPE WITH KEY MATNR = CUST_INDENT-PROD_CMP3.
IF SY-SUBRC = 0.
 CUST_INDENT-VAL_COMP3 = Z_VALTYPE-VAL_TYPE.
ENDIF.
READ TABLE IT_VALTYPE INTO Z_VALTYPE WITH KEY MATNR = CUST_INDENT-PROD_CMP4.
IF SY-SUBRC = 0.
 CUST_INDENT-VAL_COMP4 = Z_VALTYPE-VAL_TYPE.
ENDIF.
READ TABLE IT_VALTYPE INTO Z_VALTYPE WITH KEY MATNR = CUST_INDENT-PROD_CMP5.
IF SY-SUBRC = 0.
 CUST_INDENT-VAL_COMP5 = Z_VALTYPE-VAL_TYPE.
ENDIF.
READ TABLE IT_VALTYPE INTO Z_VALTYPE WITH KEY MATNR = CUST_INDENT-PROD_CMP6.
IF SY-SUBRC = 0.
 CUST_INDENT-VAL_COMP6 = Z_VALTYPE-VAL_TYPE.
ENDIF.
READ TABLE IT_VALTYPE INTO Z_VALTYPE WITH KEY MATNR = CUST_INDENT-PROD_CMP7.
IF SY-SUBRC = 0.
```

```
  CUST_INDENT-VAL_COMP7 = Z_VALTYPE-VAL_TYPE.
 ENDIF.
 READ TABLE IT_VALTYPE INTO Z_VALTYPE WITH KEY MATNR = CUST_INDENT-PROD_CMP8.
 IF SY-SUBRC = 0.
  CUST_INDENT-VAL_COMP8 = Z_VALTYPE-VAL_TYPE.
 ENDIF.

 DATA LO_ND_Z_CHECK TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_EL_Z_CHECK TYPE REF TO IF_WD_CONTEXT_ELEMENT.
 DATA LS_Z_CHECK TYPE WD_THIS->ELEMENT_Z_CHECK.
 DATA LV_Z_ERROR TYPE WD_THIS->ELEMENT_Z_CHECK-Z_ERROR.

* navigate from <CONTEXT> to <Z_CHECK> via lead selection
 LO_ND_Z_CHECK = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_Z_CHECK ).
* get element via lead selection
 LO_EL_Z_CHECK = LO_ND_Z_CHECK->GET_ELEMENT( ).
 IF LO_EL_Z_CHECK IS INITIAL.
 ENDIF.
* set single attribute
 CLEAR LV_Z_ERROR.
 LO_EL_Z_CHECK->SET_ATTRIBUTE(
  NAME = `Z_ERROR`
  VALUE = LV_Z_ERROR ).



 DATA LO_ND_ITAB_INDENT TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LT_ITAB_INDENT TYPE WD_THIS->ELEMENTS_ITAB_INDENT.

* navigate from <CONTEXT> to <ITAB_INDENT> via lead selection
 LO_ND_ITAB_INDENT = WD_CONTEXT-
>PATH_GET_NODE( PATH = `Z_GET_CUSTOMER_INDEN.CHANGING_1.ITAB_INDENT` ).

 APPEND CUST_INDENT TO LT_ITAB_INDENT.
 LO_ND_ITAB_INDENT-
>BIND_TABLE( NEW_ITEMS = LT_ITAB_INDENT SET_INITIAL_ELEMENTS = ABAP_TRUE ).



DATA ZLO_COMPONENTCONTROLLER TYPE REF TO IG_COMPONENTCONTROLLER .
ZLO_COMPONENTCONTROLLER =  WD_THIS->GET_COMPONENTCONTROLLER_CTR( ).

 ZLO_COMPONENTCONTROLLER->EXECUTE_Z_INDENT_QTY_CHECK(
 ).


 LO_ND_ITAB_INDENT = WD_CONTEXT-
>PATH_GET_NODE( PATH = `Z_GET_CUSTOMER_INDEN.CHANGING_1.ITAB_INDENT` ).
 LO_ND_ITAB_INDENT-
>GET_STATIC_ATTRIBUTES_TABLE( IMPORTING TABLE = LT_ITAB_INDENT ).  "#EC CI_FLDEXT_OK[2215424] "
 #S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
 CLEAR CUST_INDENT.
 READ TABLE LT_ITAB_INDENT INTO CUST_INDENT INDEX 1.
```

```
* navigate from <CONTEXT> to <Z_CHECK> via lead selection
 LO_ND_Z_CHECK = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_Z_CHECK ).
 LO_EL_Z_CHECK = LO_ND_Z_CHECK->GET_ELEMENT( ).
 IF LO_EL_Z_CHECK IS INITIAL.
 ENDIF.

* get single attribute
 LO_EL_Z_CHECK->GET_ATTRIBUTE(
  EXPORTING
   NAME =  `Z_ERROR`
  IMPORTING
   VALUE = LV_Z_ERROR ).

CLEAR LV_ERCODE4.
LV_ERCODE4 = LV_Z_ERROR.

DATA: D_VAL TYPE DOMVALUE_L,
    PR_GRP_MS TYPE KONDM,
    PRD_VAL_MS TYPE BWTAR_D,
    PR_GRP_HSD TYPE KONDM,
    PRD_VAL_HSD TYPE BWTAR_D.

 CLEAR: D_VAL, PR_GRP_MS, PRD_VAL_MS.
 IF LV_IND_USE_MS IS NOT INITIAL.
  SELECT SINGLE DOMVALUE_L FROM DD07T INTO D_VAL WHERE
   DOMNAME = 'ZIND_END_USE' AND DDTEXT = LV_IND_USE_MS+3(97).
  IF D_VAL+0(4) = 'NORM'.
   IF CUST_INDENT-DEPOT = '3100'.
    PRD_VAL_MS = 'OWN-BOND'.
    ELSEIF CUST_INDENT-DEPOT = '3202'.
    PRD_VAL_MS = 'OWN-DTPD'.
   ENDIF.
  ENDIF.

  IF D_VAL+0(4) = 'BRND'.
   IF CUST_INDENT-DEPOT = '3100'.
    PRD_VAL_MS = 'OWN-BONDBR'.
    ELSEIF CUST_INDENT-DEPOT = '3202'.
    PRD_VAL_MS = 'OWN-DTPDBR'.
   ENDIF.
  ENDIF.

  IF D_VAL+5(5) = 'BLND'.
   CLEAR PR_GRP_MS.
   ELSEIF D_VAL+5(5) = 'NBLND'.
    PR_GRP_MS = 'AD'.
  ENDIF.
 ENDIF.

 CLEAR: D_VAL, PR_GRP_HSD, PRD_VAL_HSD.
 IF LV_IND_USE_HSD IS NOT INITIAL.
  SELECT SINGLE DOMVALUE_L FROM DD07T INTO D_VAL WHERE
   DOMNAME = 'ZIND_END_USE' AND DDTEXT = LV_IND_USE_HSD+4(96).
  IF D_VAL+0(4) = 'NORM'.
```

```
    IF CUST_INDENT-DEPOT = '3100'.
      PRD_VAL_HSD = 'OWN-BOND'.
      ELSEIF CUST_INDENT-DEPOT = '3202'.
      PRD_VAL_HSD = 'OWN-DTPD'.
    ENDIF.
   ENDIF.

   IF D_VAL+0(4) = 'BRND'.
    IF CUST_INDENT-DEPOT = '3100'.
      PRD_VAL_HSD = 'OWN-BONDBR'.
      ELSEIF CUST_INDENT-DEPOT = '3202'.
      PRD_VAL_HSD = 'OWN-DTPDBR'.
    ENDIF.
   ENDIF.

   IF D_VAL+5(5) = 'BLND'.
    CLEAR PR_GRP_HSD.
    ELSEIF D_VAL+5(5) = 'NBLND'.
      PR_GRP_HSD = 'AD'.
   ENDIF.
  ENDIF.

DATA ZRAISE_ERR(1).
CLEAR ZRAISE_ERR.
IF CUST_INDENT-KONDM = ' '.
  IF CUST_INDENT-BEGDA >= '20221101'.
   IF LV_IND_USE_MS IS INITIAL AND ( CUST_INDENT-PROD_CMP1  = 'MS06' OR CUST_INDENT-
PROD_CMP2  = 'MS06'
     OR CUST_INDENT-PROD_CMP3  = 'MS06' OR CUST_INDENT-PROD_CMP4  = 'MS06'
     OR CUST_INDENT-PROD_CMP5  = 'MS06' OR CUST_INDENT-PROD_CMP6  = 'MS06'
     OR CUST_INDENT-PROD_CMP7  = 'MS06' OR CUST_INDENT-PROD_CMP8  = 'MS06' ).
     LV_ERCODE4  = 'X'.
     ZRAISE_ERR  = 'X'.
     ELSE.
     CUST_INDENT-KONDM = PR_GRP_MS.
   ENDIF.
   DATA: VAL_CHANGE(1).
   CLEAR VAL_CHANGE.
   SELECT COUNT(*) FROM ZDESP_STOCK_PLNT WHERE PLANT = CUST_INDENT-DEPOT
    AND MATNR = 'MS06' AND CUSTOMER = ZCUST_MAP-KUNNR.
   IF SY-SUBRC = 0.
    VAL_CHANGE = 'N'.
   ENDIF.

   IF CUST_INDENT-PROD_CMP1  = 'MS06' AND VAL_CHANGE = ' '.
    CUST_INDENT-VAL_COMP1   = PRD_VAL_MS.
   ENDIF.
   IF CUST_INDENT-PROD_CMP2  = 'MS06' AND VAL_CHANGE = ' '.
    CUST_INDENT-VAL_COMP2   = PRD_VAL_MS.
   ENDIF.
   IF CUST_INDENT-PROD_CMP3  = 'MS06' AND VAL_CHANGE = ' '.
    CUST_INDENT-VAL_COMP3   = PRD_VAL_MS.
   ENDIF.
   IF CUST_INDENT-PROD_CMP4  = 'MS06' AND VAL_CHANGE = ' '.
```

```
   CUST_INDENT-VAL_COMP4  = PRD_VAL_MS.
  ENDIF.
  IF CUST_INDENT-PROD_CMP5  = 'MS06' AND VAL_CHANGE = ' '.
   CUST_INDENT-VAL_COMP5  = PRD_VAL_MS.
  ENDIF.
  IF CUST_INDENT-PROD_CMP6  = 'MS06' AND VAL_CHANGE = ' '.
   CUST_INDENT-VAL_COMP6  = PRD_VAL_MS.
  ENDIF.
  IF CUST_INDENT-PROD_CMP7  = 'MS06' AND VAL_CHANGE = ' '.
   CUST_INDENT-VAL_COMP7  = PRD_VAL_MS.
  ENDIF.
  IF CUST_INDENT-PROD_CMP8  = 'MS06' AND VAL_CHANGE = ' '.
   CUST_INDENT-VAL_COMP8  = PRD_VAL_MS.
  ENDIF.
 ENDIF.

 IF CUST_INDENT-BEGDA >= '20280401'. " Earlier date 20230401
  IF LV_IND_USE_HSD IS INITIAL AND ( CUST_INDENT-PROD_CMP1  = 'HSD06' OR CUST_INDENT-
PROD_CMP2  = 'HSD06'
   OR CUST_INDENT-PROD_CMP3  = 'HSD06' OR CUST_INDENT-PROD_CMP4  = 'HSD06'
   OR CUST_INDENT-PROD_CMP5  = 'HSD06' OR CUST_INDENT-PROD_CMP6  = 'HSD06'
   OR CUST_INDENT-PROD_CMP7  = 'HSD06' OR CUST_INDENT-PROD_CMP8  = 'HSD06' ).
   LV_ERCODE4  = 'X'.
   ZRAISE_ERR  = 'X'.
   ELSE.
    CUST_INDENT-KONDM2 = PR_GRP_HSD.
  ENDIF.
  CLEAR VAL_CHANGE.
  SELECT COUNT(*) FROM ZDESP_STOCK_PLNT WHERE PLANT = CUST_INDENT-DEPOT
   AND MATNR = 'HSD06' AND CUSTOMER = ZCUST_MAP-KUNNR.
  IF SY-SUBRC = 0.
   VAL_CHANGE = 'N'.
  ENDIF.

  IF CUST_INDENT-PROD_CMP1  = 'HSD06' AND VAL_CHANGE = ' '.
    CUST_INDENT-VAL_COMP1  = PRD_VAL_HSD.
  ENDIF.
  IF CUST_INDENT-PROD_CMP2  = 'HSD06' AND VAL_CHANGE = ' '.
   CUST_INDENT-VAL_COMP2  = PRD_VAL_HSD.
  ENDIF.
  IF CUST_INDENT-PROD_CMP3  = 'HSD06' AND VAL_CHANGE = ' '.
   CUST_INDENT-VAL_COMP3  = PRD_VAL_HSD.
  ENDIF.
  IF CUST_INDENT-PROD_CMP4  = 'HSD06' AND VAL_CHANGE = ' '.
   CUST_INDENT-VAL_COMP4  = PRD_VAL_HSD.
  ENDIF.
  IF CUST_INDENT-PROD_CMP5  = 'HSD06' AND VAL_CHANGE = ' '.
   CUST_INDENT-VAL_COMP5  = PRD_VAL_HSD.
  ENDIF.
  IF CUST_INDENT-PROD_CMP6  = 'HSD06' AND VAL_CHANGE = ' '.
   CUST_INDENT-VAL_COMP6  = PRD_VAL_HSD.
  ENDIF.
  IF CUST_INDENT-PROD_CMP7  = 'HSD06' AND VAL_CHANGE = ' '.
   CUST_INDENT-VAL_COMP7  = PRD_VAL_HSD.
```

```
  ENDIF.
  IF CUST_INDENT-PROD_CMP8  = 'HSD06' AND VAL_CHANGE = ' '.
   CUST_INDENT-VAL_COMP8   = PRD_VAL_HSD.
  ENDIF.
 ENDIF.
ENDIF.
IF ZRAISE_ERR = 'X'.
 CLEAR ls_string. CLEAR lt_string. REFRESH lt_string.
 ls_string = 'Please select end use'.
 insert ls_string into table lt_string.
 LO_API_COMPONENT        = WD_COMP_CONTROLLER->WD_GET_API( ).
 LO_WINDOW_MANAGER       = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).

 LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
        text         = lt_string
        button_kind    = if_wd_window=>co_buttons_ok
        message_type   = if_wd_window=>CO_MSG_TYPE_STOPP
        window_title   = 'Please check!'
        window_position = if_wd_window=>co_center
        WINDOW_WIDTH   = '20%'
        WINDOW_HEIGHT  = '20%' ).
 LO_WINDOW->open( ).
ENDIF.

IF LV_ERCODE4 <> 'X'.
 CUST_INDENT-CONTRACT = LV_VBELN.
 IF CUST_INDENT-CONTRACT = 'IN' OR CUST_INDENT-CONTRACT = 'EP'.
  CLEAR CUST_INDENT-CONTRACT.
 ENDIF.
 CUST_INDENT-TPT_GSTN = LV_GSTN+0(15).
 CUST_INDENT-FLUSH_REASON = LV_FLSH_REASON.
 CUST_INDENT-ZDELETE = 'Y'.


data: zstk_err(1),
    ZERR_MSG TYPE STRING.
****Added for multiple source Ethanol check *****
 data: zebms_srouce type ZEBMS_SRS,
      zknvv type knvv,
      zmat_map type ZEBMS_MAT_MAP.

 clear: zknvv, zebms_srouce.
 select single * from knvv into zknvv where kunnr = CUST_INDENT-kunnr
  and vtweg = '10' and spart = '25'.
 select single * from  ZEBMS_SRS into zebms_srouce where werks_d = CUST_INDENT-depot
  and kdgrp =  zknvv-kdgrp.
 if sy-subrc <> 0.
  zebms_srouce-zebms_source = 'HOSP'.
 endif.

 if zebms_srouce-zebms_source <> 'HOSP'.
  move zebms_srouce-zebms_source to CUST_INDENT-ZTRAN.
  select single * from ZEBMS_MAT_MAP into zmat_map where hosp_mat = CUST_INDENT-PROD_CMP1.
  if sy-subrc = 0.
```

```
    move zmat_map-own_mat to CUST_INDENT-PROD_CMP1.
    if zebms_srouce-zebms_source = 'ABRPL'.
     CUST_INDENT-VAL_COMP1 = 'OWN-ABRPL'.
    endif.
    if zebms_srouce-zebms_source = 'NRL' and ZMAT_MAP-HOSP_MAT+0(6) = '(BRND)'.
     concatenate CUST_INDENT-VAL_COMP1 'BR' into CUST_INDENT-VAL_COMP1.
    endif.

    else.
     select single * from ZEBMS_MAT_MAP into zmat_map where hosp_mat = CUST_INDENT-
PROD_CMP1+6(12).
      if sy-subrc = 0.
       if CUST_INDENT-PROD_CMP1+0(6) = '(BRND)'.
        move zmat_map-own_mat to CUST_INDENT-PROD_CMP1.
        CUST_INDENT-VAL_COMP1 = 'OWN-ABRPLBR'.
       endif.
      endif.
   endif.

   select single * from ZEBMS_MAT_MAP into zmat_map where hosp_mat = CUST_INDENT-PROD_CMP2.
   if sy-subrc = 0.
    move zmat_map-own_mat to CUST_INDENT-PROD_CMP2.
    if zebms_srouce-zebms_source = 'ABRPL'.
     CUST_INDENT-VAL_COMP2 = 'OWN-ABRPL'.
    endif.
    if zebms_srouce-zebms_source = 'NRL' and ZMAT_MAP-HOSP_MAT+0(6) = '(BRND)'.
     concatenate CUST_INDENT-VAL_COMP2 'BR' into CUST_INDENT-VAL_COMP2.
    endif.

    else.
     select single * from ZEBMS_MAT_MAP into zmat_map where hosp_mat = CUST_INDENT-
PROD_CMP2+6(12).
      if sy-subrc = 0.
       if CUST_INDENT-PROD_CMP2+0(6) = '(BRND)'.
        move zmat_map-own_mat to CUST_INDENT-PROD_CMP2.
        CUST_INDENT-VAL_COMP2 = 'OWN-ABRPLBR'.
       endif.
      endif.
   endif.
   select single * from ZEBMS_MAT_MAP into zmat_map where hosp_mat = CUST_INDENT-PROD_CMP3.
   if sy-subrc = 0.
    move zmat_map-own_mat to CUST_INDENT-PROD_CMP3.
    if zebms_srouce-zebms_source = 'ABRPL'.
     CUST_INDENT-VAL_COMP3 = 'OWN-ABRPL'.
    endif.
    if zebms_srouce-zebms_source = 'NRL' and ZMAT_MAP-HOSP_MAT+0(6) = '(BRND)'.
     concatenate CUST_INDENT-VAL_COMP3 'BR' into CUST_INDENT-VAL_COMP3.
    endif.

    else.
     select single * from ZEBMS_MAT_MAP into zmat_map where hosp_mat = CUST_INDENT-
PROD_CMP3+6(12).
      if sy-subrc = 0.
       if CUST_INDENT-PROD_CMP3+0(6) = '(BRND)'.
```

```
        move zmat_map-own_mat to CUST_INDENT-PROD_CMP3.
        CUST_INDENT-VAL_COMP3 = 'OWN-ABRPLBR'.
       endif.
      endif.
    endif.
    select single * from ZEBMS_MAT_MAP into zmat_map where hosp_mat = CUST_INDENT-PROD_CMP4.
    if sy-subrc = 0.
     move zmat_map-own_mat to CUST_INDENT-PROD_CMP4.
     if zebms_srouce-zebms_source = 'ABRPL'.
      CUST_INDENT-VAL_COMP4 = 'OWN-ABRPLBR'.
     endif.
     if zebms_srouce-zebms_source = 'NRL' and ZMAT_MAP-HOSP_MAT+0(6) = '(BRND)'.
      concatenate CUST_INDENT-VAL_COMP4 'BR' into CUST_INDENT-VAL_COMP4.
     endif.

     else.
      select single * from ZEBMS_MAT_MAP into zmat_map where hosp_mat = CUST_INDENT-
PROD_CMP4+6(12).
      if sy-subrc = 0.
       if CUST_INDENT-PROD_CMP4+0(6) = '(BRND)'.
        move zmat_map-own_mat to CUST_INDENT-PROD_CMP4.
        CUST_INDENT-VAL_COMP4 = 'OWN-ABRPLBR'.
       endif.
      endif.
    endif.
    select single * from ZEBMS_MAT_MAP into zmat_map where hosp_mat = CUST_INDENT-PROD_CMP5.
    if sy-subrc = 0.
     move zmat_map-own_mat to CUST_INDENT-PROD_CMP5.
     if zebms_srouce-zebms_source = 'ABRPL'.
      CUST_INDENT-VAL_COMP5 = 'OWN-ABRPL'.
     endif.
     if zebms_srouce-zebms_source = 'NRL' and ZMAT_MAP-HOSP_MAT+0(6) = '(BRND)'.
      concatenate CUST_INDENT-VAL_COMP5 'BR' into CUST_INDENT-VAL_COMP5.
     endif.

     else.
      select single * from ZEBMS_MAT_MAP into zmat_map where hosp_mat = CUST_INDENT-
PROD_CMP5+6(12).
      if sy-subrc = 0.
       if CUST_INDENT-PROD_CMP5+0(6) = '(BRND)'.
        move zmat_map-own_mat to CUST_INDENT-PROD_CMP5.
        CUST_INDENT-VAL_COMP5 = 'OWN-ABRPLBR'.
       endif.
      endif.
    endif.
    select single * from ZEBMS_MAT_MAP into zmat_map where hosp_mat = CUST_INDENT-PROD_CMP6.
    if sy-subrc = 0.
     move zmat_map-own_mat to CUST_INDENT-PROD_CMP6.
     if zebms_srouce-zebms_source = 'ABRPL'.
      CUST_INDENT-VAL_COMP6 = 'OWN-ABRPL'.
     endif.
     if zebms_srouce-zebms_source = 'NRL' and ZMAT_MAP-HOSP_MAT+0(6) = '(BRND)'.
      concatenate CUST_INDENT-VAL_COMP6 'BR' into CUST_INDENT-VAL_COMP6.
     endif.
```

```
    else.
     select single * from ZEBMS_MAT_MAP into zmat_map where hosp_mat = CUST_INDENT-
PROD_CMP6+6(12).
      if sy-subrc = 0.
       if CUST_INDENT-PROD_CMP6+0(6) = '(BRND)'.
        move zmat_map-own_mat to CUST_INDENT-PROD_CMP6.
        CUST_INDENT-VAL_COMP6 = 'OWN-ABRPLBR'.
       endif.
      endif.
   endif.
   select single * from ZEBMS_MAT_MAP into zmat_map where hosp_mat = CUST_INDENT-PROD_CMP7.
   if sy-subrc = 0.
    move zmat_map-own_mat to CUST_INDENT-PROD_CMP7.
    if zebms_srouce-zebms_source = 'ABRPL'.
     CUST_INDENT-VAL_COMP7 = 'OWN-ABRPL'.
    endif.
    if zebms_srouce-zebms_source = 'NRL' and ZMAT_MAP-HOSP_MAT+0(6) = '(BRND)'.
     concatenate CUST_INDENT-VAL_COMP7 'BR' into CUST_INDENT-VAL_COMP7.
    endif.

    else.
     select single * from ZEBMS_MAT_MAP into zmat_map where hosp_mat = CUST_INDENT-
PROD_CMP7+6(12).
      if sy-subrc = 0.
       if CUST_INDENT-PROD_CMP7+0(6) = '(BRND)'.
        move zmat_map-own_mat to CUST_INDENT-PROD_CMP7.
        CUST_INDENT-VAL_COMP7 = 'OWN-ABRPLBR'.
       endif.
      endif.
   endif.
   select single * from ZEBMS_MAT_MAP into zmat_map where hosp_mat = CUST_INDENT-PROD_CMP8.
   if sy-subrc = 0.
    move zmat_map-own_mat to CUST_INDENT-PROD_CMP8.
    if zebms_srouce-zebms_source = 'ABRPL'.
     CUST_INDENT-VAL_COMP8 = 'OWN-ABRPL'.
    endif.
    if zebms_srouce-zebms_source = 'NRL' and ZMAT_MAP-HOSP_MAT+0(6) = '(BRND)'.
     concatenate CUST_INDENT-VAL_COMP8 'BR' into CUST_INDENT-VAL_COMP8.
    endif.

    else.
     select single * from ZEBMS_MAT_MAP into zmat_map where hosp_mat = CUST_INDENT-
PROD_CMP8+6(12).
      if sy-subrc = 0.
       if CUST_INDENT-PROD_CMP8+0(6) = '(BRND)'.
        move zmat_map-own_mat to CUST_INDENT-PROD_CMP8.
        CUST_INDENT-VAL_COMP8 = 'OWN-ABRPLBR'.
       endif.
      endif.
   endif.

  endif.
```

```
 if zebms_srouce-zebms_source = 'HOSP'.
  CALL FUNCTION 'Z_ETHANOL_INDENT_STK_CHECK'
      EXPORTING
        CUST_INDENT = CUST_INDENT
        C1_QT     = C1_QT
        C2_QT     = C2_QT
        C3_QT     = C3_QT
        C4_QT     = C4_QT
        C5_QT     = C5_QT
        C6_QT     = C6_QT
        C7_QT     = C7_QT
        C8_QT     = C8_QT
      IMPORTING
        ERR_MSG    = ZERR_MSG
        ZSTK_ERR   = ZSTK_ERR .

  CUST_INDENT-ZTRAN   = 'HOSP'.

  IF ZSTK_ERR = 'X'.
   insert ZERR_MSG into table lt_string.
   LO_API_COMPONENT         = WD_COMP_CONTROLLER->WD_GET_API( ).
   LO_WINDOW_MANAGER        = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).

    LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
         text         = lt_string
         button_kind    = if_wd_window=>co_buttons_ok
*          button_kind    = 4
         message_type    = if_wd_window=>CO_MSG_TYPE_STOPP
         window_title    = 'Please check!'
         window_position = if_wd_window=>co_center
         WINDOW_WIDTH    = '20%'
         WINDOW_HEIGHT   = '20%' ).
    LO_WINDOW->open( ).
  ENDIF.
 endif.
*****Ethanol stock check for OMCs END****

 MOVE LV_IND_USE_MS TO CUST_INDENT-MS_END_USE.
 MOVE LV_IND_USE_HSD TO CUST_INDENT-HSD_END_USE.
 IF PR_GRP_MS = 'AD' OR LV_IND_USE_HSD = 'AD'.
  CLEAR: IT_STK, IT_STK[], LN_STK.
  SELECT * FROM ZINDENT_CUSSTOCK INTO LN_STK
   WHERE PLANT = CUST_INDENT-DEPOT AND KDGRP = CUST_INDENT-KDGRP.
   APPEND LN_STK  TO IT_STK.
  ENDSELECT.
  READ TABLE IT_STK INTO LN_STK WITH KEY MATNR = CUST_INDENT-PROD_CMP1.
  IF SY-SUBRC = 0.
   CUST_INDENT-VAL_COMP1 = LN_STK-VAL_TYPE.
  ENDIF.
  READ TABLE IT_STK INTO LN_STK WITH KEY MATNR = CUST_INDENT-PROD_CMP2.
  IF SY-SUBRC = 0.
   CUST_INDENT-VAL_COMP2 = LN_STK-VAL_TYPE.
  ENDIF.
  READ TABLE IT_STK INTO LN_STK WITH KEY MATNR = CUST_INDENT-PROD_CMP3.
```

```
 IF SY-SUBRC = 0.
  CUST_INDENT-VAL_COMP3 = LN_STK-VAL_TYPE.
 ENDIF.
 READ TABLE IT_STK INTO LN_STK WITH KEY MATNR = CUST_INDENT-PROD_CMP4.
 IF SY-SUBRC = 0.
  CUST_INDENT-VAL_COMP4 = LN_STK-VAL_TYPE.
 ENDIF.
 READ TABLE IT_STK INTO LN_STK WITH KEY MATNR = CUST_INDENT-PROD_CMP5.
 IF SY-SUBRC = 0.
  CUST_INDENT-VAL_COMP5 = LN_STK-VAL_TYPE.
 ENDIF.
 READ TABLE IT_STK INTO LN_STK WITH KEY MATNR = CUST_INDENT-PROD_CMP6.
 IF SY-SUBRC = 0.
  CUST_INDENT-VAL_COMP6 = LN_STK-VAL_TYPE.
 ENDIF.
 READ TABLE IT_STK INTO LN_STK WITH KEY MATNR = CUST_INDENT-PROD_CMP7.
 IF SY-SUBRC = 0.
  CUST_INDENT-VAL_COMP7 = LN_STK-VAL_TYPE.
 ENDIF.
 READ TABLE IT_STK INTO LN_STK WITH KEY MATNR = CUST_INDENT-PROD_CMP8.
 IF SY-SUBRC = 0.
  CUST_INDENT-VAL_COMP8 = LN_STK-VAL_TYPE.
 ENDIF.
ENDIF.

IF LV_IND_USE_MS IS INITIAL.
 IF CUST_INDENT-PROD_CMP1+0(6) = '(BRND)'.
  CUST_INDENT-PROD_CMP1 = CUST_INDENT-PROD_CMP1+6(20).
  CONCATENATE CUST_INDENT-VAL_COMP1 'BR' INTO CUST_INDENT-VAL_COMP1.
 ENDIF.
 IF CUST_INDENT-PROD_CMP2+0(6) = '(BRND)'.
  CUST_INDENT-PROD_CMP2 = CUST_INDENT-PROD_CMP2+6(20).
  CONCATENATE CUST_INDENT-VAL_COMP2 'BR' INTO CUST_INDENT-VAL_COMP2.
 ENDIF.
 IF CUST_INDENT-PROD_CMP3+0(6) = '(BRND)'.
  CUST_INDENT-PROD_CMP3 = CUST_INDENT-PROD_CMP3+6(20).
  CONCATENATE CUST_INDENT-VAL_COMP3 'BR' INTO CUST_INDENT-VAL_COMP3.
 ENDIF.
 IF CUST_INDENT-PROD_CMP4+0(6) = '(BRND)'.
  CUST_INDENT-PROD_CMP4 = CUST_INDENT-PROD_CMP4+6(20).
  CONCATENATE CUST_INDENT-VAL_COMP4 'BR' INTO CUST_INDENT-VAL_COMP4.
 ENDIF.
 IF CUST_INDENT-PROD_CMP5+0(6) = '(BRND)'.
  CUST_INDENT-PROD_CMP5 = CUST_INDENT-PROD_CMP5+6(20).
  CONCATENATE CUST_INDENT-VAL_COMP5 'BR' INTO CUST_INDENT-VAL_COMP5.
 ENDIF.
 IF CUST_INDENT-PROD_CMP6+0(6) = '(BRND)'.
  CUST_INDENT-PROD_CMP6 = CUST_INDENT-PROD_CMP6+6(20).
  CONCATENATE CUST_INDENT-VAL_COMP6 'BR' INTO CUST_INDENT-VAL_COMP6.
 ENDIF.
 IF CUST_INDENT-PROD_CMP7+0(6) = '(BRND)'.
  CUST_INDENT-PROD_CMP7 = CUST_INDENT-PROD_CMP7+6(20).
  CONCATENATE CUST_INDENT-VAL_COMP7 'BR' INTO CUST_INDENT-VAL_COMP7.
 ENDIF.
```

```
 IF CUST_INDENT-PROD_CMP8+0(6) = '(BRND)'.
  CUST_INDENT-PROD_CMP8 = CUST_INDENT-PROD_CMP8+6(20).
  CONCATENATE CUST_INDENT-VAL_COMP8 'BR' INTO CUST_INDENT-VAL_COMP8.
 ENDIF.
ENDIF.

IF ZNVEH_IND-ORDER_NO IS NOT INITIAL.
 CLEAR CAP8.
 CALL FUNCTION 'CEVA_CONVERT_FLOAT_TO_CHAR'
  EXPORTING
   float_imp = LV_QTY1
   format_imp = W_NUMBER
   round_imp = ' '
  IMPORTING
   char_exp = CAP8.
 condense CAP8.
 CAP8  = CAP8 / 1000.
 ZNVEH_IND-BL_QTY1  = ZNVEH_IND-BL_QTY1 - CAP8.
 CUST_INDENT-BULK_QTY = CAP8.
ENDIF.

IF zstk_err = ' '.
 MOVE ZBLK_ORD TO CUST_INDENT-BULK_ORDER.
 INSERT ZSD_CUST_INDENT FROM CUST_INDENT.
 IF SY-SUBRC = 0.
  MODIFY ZSD_INDENT_NVEH FROM ZNVEH_IND.
 ENDIF.
ENDIF.

 DATA LO_ND_GSTN TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_EL_GSTN TYPE REF TO IF_WD_CONTEXT_ELEMENT.
 DATA LS_GSTN TYPE WD_THIS->ELEMENT_GSTN.
 DATA LV_TPT_GSTN TYPE WD_THIS->ELEMENT_GSTN-TPT_GSTN.
 LO_ND_GSTN = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_GSTN ).
 LO_EL_GSTN = LO_ND_GSTN->GET_ELEMENT( ).
 IF LO_EL_GSTN IS INITIAL.
 ENDIF.
 CLEAR LV_TPT_GSTN.
 LO_EL_GSTN->SET_ATTRIBUTE(
  NAME =  `TPT_GSTN`
  VALUE = LV_TPT_GSTN ).

 CLEAR LO_IMPORTING.
 LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_HEADER_DATA ).
 LO_IMPORTING->SET_ATTRIBUTE(
   NAME  = `KUNNR`
   VALUE = LV_KUNNR ).


LO_ND_ENABLE2 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_ENABLE2 ).
LO_EL_ENABLE2 = LO_ND_ENABLE2->GET_ELEMENT( ).
CLEAR LV_ZT2.
LO_EL_ENABLE2->SET_ATTRIBUTE(
  NAME =  `ZT2`
```

```
    VALUE = LV_ZT2 ).

 LO_ND_ATF_FLUSH = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_ATF_FLUSH ).
 LO_EL_ATF_FLUSH = LO_ND_ATF_FLUSH->GET_ELEMENT( ).
 IF LO_EL_ATF_FLUSH IS NOT INITIAL.
  CLEAR LV_FLUSH.
  LO_EL_ATF_FLUSH->SET_ATTRIBUTE(
     NAME =  `FLUSH`
     VALUE = LV_FLUSH ).
 ENDIF.

 DATA LO_ND_ENABLE1 TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_EL_ENABLE1 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
 DATA LS_ENABLE1 TYPE WD_THIS->ELEMENT_ENABLE1.
 DATA LV_ET2 TYPE WD_THIS->ELEMENT_ENABLE1-ET2.
 LO_ND_ENABLE1 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_ENABLE1 ).
 LO_EL_ENABLE1 = LO_ND_ENABLE1->GET_ELEMENT( ).
 CLEAR LV_ET2.
 LO_EL_ENABLE1->SET_ATTRIBUTE(
    NAME =  `ET2`
    VALUE = LV_ET2 ).



LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_IMPORTING ).
CLEAR LV_VEHICLE.
LO_IMPORTING->SET_ATTRIBUTE(
  EXPORTING
    NAME  = `VEHICLE`
    VALUE = LV_VEHICLE ).

IF LV_FLSH_REASON <> ' '.
  LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_FLUSH_REASON ).
  CLEAR LV_FLSH_REASON.
  LO_IMPORTING->SET_ATTRIBUTE(
     EXPORTING
      NAME  = `FLSH_REASON`
      VALUE = LV_FLSH_REASON ).
ENDIF.

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_PROD1 ).
CLEAR LV_PROD1.
LO_IMPORTING->SET_ATTRIBUTE(
  EXPORTING
    NAME  = `PRODUCT`
    VALUE = LV_PROD1 ).

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_PROD2 ).
CLEAR LV_PROD2.
LO_IMPORTING->SET_ATTRIBUTE(
  EXPORTING
    NAME  = `PRODUCT`
```

```
    VALUE = LV_PROD2 ).

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_PROD3 ).
CLEAR LV_PROD3.
LO_IMPORTING->SET_ATTRIBUTE(
  EXPORTING
    NAME  = `PRODUCT`
    VALUE = LV_PROD3 ).

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_PROD4 ).
CLEAR LV_PROD4.
LO_IMPORTING->SET_ATTRIBUTE(
  EXPORTING
    NAME  = `PRODUCT`
    VALUE = LV_PROD4 ).

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_PROD5 ).
CLEAR LV_PROD5.
LO_IMPORTING->SET_ATTRIBUTE(
  EXPORTING
    NAME  = `PRODUCT`
    VALUE = LV_PROD5 ).

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_PROD6 ).
CLEAR LV_PROD6.
LO_IMPORTING->SET_ATTRIBUTE(
  EXPORTING
    NAME  = `PRODUCT`
    VALUE = LV_PROD6 ).

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_PROD7 ).
CLEAR LV_PROD7.
LO_IMPORTING->SET_ATTRIBUTE(
  EXPORTING
    NAME  = `PRODUCT`
    VALUE = LV_PROD7 ).

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_PROD8 ).
CLEAR LV_PROD8.
LO_IMPORTING->SET_ATTRIBUTE(
  EXPORTING
    NAME  = `PRODUCT`
    VALUE = LV_PROD8 ).

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_EXPORTING ).
CLEAR LV_QTY1.
LO_IMPORTING->SET_ATTRIBUTE(
```

```
    EXPORTING
     NAME  = `CAPACITY1`
     VALUE = LV_QTY1 ).

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_EXPORTING ).
CLEAR LV_QTY2.
LO_IMPORTING->SET_ATTRIBUTE(
    EXPORTING
     NAME  = `CAPACITY2`
     VALUE = LV_QTY2 ).

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_EXPORTING ).
CLEAR LV_QTY3.
LO_IMPORTING->SET_ATTRIBUTE(
    EXPORTING
     NAME  = `CAPACITY3`
     VALUE = LV_QTY3 ).

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_EXPORTING ).
CLEAR LV_QTY4.
LO_IMPORTING->SET_ATTRIBUTE(
    EXPORTING
     NAME  = `CAPACITY4`
     VALUE = LV_QTY4 ).

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_EXPORTING ).
CLEAR LV_QTY5.
LO_IMPORTING->SET_ATTRIBUTE(
    EXPORTING
     NAME  = `CAPACITY5`
     VALUE = LV_QTY5 ).

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_EXPORTING ).
CLEAR LV_QTY6.
LO_IMPORTING->SET_ATTRIBUTE(
    EXPORTING
     NAME  = `CAPACITY6`
     VALUE = LV_QTY6 ).
CLEAR LO_IMPORTING.

LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_EXPORTING ).
CLEAR LV_QTY7.
LO_IMPORTING->SET_ATTRIBUTE(
    EXPORTING
     NAME  = `CAPACITY7`
     VALUE = LV_QTY7 ).
CLEAR LO_IMPORTING.

LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_EXPORTING ).
```

```
CLEAR LV_QTY8.
LO_IMPORTING->SET_ATTRIBUTE(
  EXPORTING
   NAME  = `CAPACITY8`
   VALUE = LV_QTY8 ).
CLEAR LO_IMPORTING.

CLEAR LO_EXPORT_CHECK. CLEAR LO_EXPORT_CHECK_ELEMENT.
LO_EXPORT_CHECK = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_EXPORT ).
LO_EXPORT_CHECK_ELEMENT = LO_EXPORT_CHECK->GET_ELEMENT( ).
IF LO_EXPORT_CHECK_ELEMENT IS NOT INITIAL.
 CLEAR LV_VBELN.
  LO_EXPORT_CHECK_ELEMENT->SET_ATTRIBUTE(
     EXPORTING
      NAME  = `VBELN`
      VALUE = LV_VBELN ).
ENDIF.

if VEH_TYPE = 'ATF'.
 LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_ATF_FLUSH ).
 CLEAR LV_DESC.
 LO_IMPORTING->SET_ATTRIBUTE(
  EXPORTING
   NAME  = `DESC`
   VALUE = LV_DESC ).
endif.

DATA LO_EXPORTING TYPE REF TO IF_WD_CONTEXT_NODE.
LO_EXPORTING = wd_context->get_child_node( name = wd_this->WDCTX_ENABLE ).
LO_EXPORTING->set_attribute(
       name = 'SAVE'
       value = ' ' ).
LO_EXPORTING->set_attribute(
       name = 'CONT'
       value = ' ' ).

ENDIF.
ENDIF.

CLEAR LO_EXPORTING.
LO_EXPORTING = wd_context->get_child_node( name = wd_this->WDCTX_ENABLE ).
LO_EXPORTING->set_attribute(
       name = 'DELETE'
       value = 'X' ).
LO_EXPORTING->set_attribute(
     name = 'SUBMIT'
     value = 'X' ).

LO_EXPORTING->set_attribute(
     name = 'SUBMIT_ALL'
     value = 'X' ).

DATA LO_COMPONENTCONTROLLER TYPE REF TO IG_COMPONENTCONTROLLER .
LO_COMPONENTCONTROLLER =  WD_THIS->GET_COMPONENTCONTROLLER_CTR( ).
```

```
LO_COMPONENTCONTROLLER->EXECUTE_Z_GET_CUSTOMER_INDENT(
).

endmethod.
```

- method ONACTIONSAVE_NOV_VEH_INDENT .

```
DATA: imat_data type REF TO IF_WD_CONTEXT_NODE,
    imat_data1 TYPE wd_this->Elements_material1,
    lmat_data1 LIKE LINE OF imat_data1,
    imat_data2 TYPE wd_this->Elements_material2.
DATA: ZKDGRP TYPE KDGRP.

DATA: ORDC_QTY1 TYPE ZSD_INDENT_NVEH-QUANTITY1,
    ORDC_QTY2 TYPE ZSD_INDENT_NVEH-QUANTITY1.
DATA: IT_ITEM type standard table of ZINDT_NVEH_ITEM.
DATA: LN_ITEM type ZINDT_NVEH_ITEM.
DATA: IT_HDR type standard table of ZSD_INDENT_NVEH.
DATA: LN_HDR type ZSD_INDENT_NVEH.


CLEAR ZKDGRP.

DATA LO_ND_HEADER_DATA TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_EL_HEADER_DATA TYPE REF TO IF_WD_CONTEXT_ELEMENT.
DATA LS_HEADER_DATA TYPE WD_THIS->ELEMENT_HEADER_DATA.
DATA LV_LOAD_DATE TYPE WD_THIS->ELEMENT_HEADER_DATA-LOAD_DATE.
DATA LV_KUNNR TYPE WD_THIS->ELEMENT_HEADER_DATA-KUNNR.
 LO_ND_HEADER_DATA = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS-
>WDCTX_HEADER_DATA ).
 LO_EL_HEADER_DATA = LO_ND_HEADER_DATA->GET_ELEMENT( ).
 IF LO_EL_HEADER_DATA IS NOT INITIAL.
  LO_EL_HEADER_DATA->GET_ATTRIBUTE(
   EXPORTING
    NAME =  `LOAD_DATE`
   IMPORTING
    VALUE = LV_LOAD_DATE ).

  LO_EL_HEADER_DATA->GET_ATTRIBUTE(
   EXPORTING
    NAME =  `KUNNR`
   IMPORTING
    VALUE = LV_KUNNR ).
 ENDIF.
 SELECT SINGLE KDGRP FROM KNVV INTO ZKDGRP WHERE KUNNR = LV_KUNNR.  "#EC CI_NOORDER " #
S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18

  DATA LO_ND_NO_VEH_CONTRACT TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LO_EL_NO_VEH_CONTRACT TYPE REF TO IF_WD_CONTEXT_ELEMENT.
  DATA LS_NO_VEH_CONTRACT TYPE WD_THIS->ELEMENT_NO_VEH_CONTRACT.
  DATA LV_VBELN1 TYPE WD_THIS->ELEMENT_NO_VEH_CONTRACT-VBELN1.
  LO_ND_NO_VEH_CONTRACT = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS-
>WDCTX_NO_VEH_CONTRACT ).
  LO_EL_NO_VEH_CONTRACT = LO_ND_NO_VEH_CONTRACT->GET_ELEMENT( ).
  IF LO_EL_NO_VEH_CONTRACT IS NOT INITIAL.
```

```
 LO_EL_NO_VEH_CONTRACT->GET_ATTRIBUTE(
  EXPORTING
   NAME = `VBELN1`
  IMPORTING
   VALUE = LV_VBELN1 ).
ENDIF.

 DATA LO_ND_MATERIAL1 TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_EL_MATERIAL1 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
 DATA LS_MATERIAL1 TYPE WD_THIS->ELEMENT_MATERIAL1.
 DATA LV_MATNR1 TYPE WD_THIS->ELEMENT_MATERIAL1-MATNR.
 LO_ND_MATERIAL1 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_MATERIAL1 ).
 LO_EL_MATERIAL1 = LO_ND_MATERIAL1->GET_ELEMENT( ).
 IF LO_EL_MATERIAL1 IS NOT INITIAL.
  LO_EL_MATERIAL1->GET_ATTRIBUTE(
   EXPORTING
    NAME = `MATNR`
   IMPORTING
    VALUE = LV_MATNR1 ).
 ENDIF.

  DATA LO_ND_EXPORTING TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LO_EL_EXPORTING TYPE REF TO IF_WD_CONTEXT_ELEMENT.
  DATA LS_EXPORTING TYPE WD_THIS->ELEMENT_EXPORTING.
  DATA LV_CAPACITY1 TYPE WD_THIS->ELEMENT_EXPORTING-CAPACITY1.
  DATA LV_CAPACITY2 TYPE WD_THIS->ELEMENT_EXPORTING-CAPACITY2.
  LO_ND_EXPORTING = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_EXPORTING ).
  LO_EL_EXPORTING = LO_ND_EXPORTING->GET_ELEMENT( ).

  IF LO_EL_EXPORTING IS NOT INITIAL.
   LO_EL_EXPORTING->GET_ATTRIBUTE(
    EXPORTING
     NAME = `CAPACITY1`
    IMPORTING
     VALUE = LV_CAPACITY1 ).
  ENDIF.


   DATA LO_ND_MATERIAL2 TYPE REF TO IF_WD_CONTEXT_NODE.
   DATA LO_EL_MATERIAL2 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
   DATA LS_MATERIAL2 TYPE WD_THIS->ELEMENT_MATERIAL2.
   DATA LV_MATNR2 TYPE WD_THIS->ELEMENT_MATERIAL2-MATNR.
   LO_ND_MATERIAL2 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_MATERIAL2 ).
   LO_EL_MATERIAL2 = LO_ND_MATERIAL2->GET_ELEMENT( ).
   IF LO_EL_MATERIAL2 IS NOT INITIAL.
    LO_EL_MATERIAL2->GET_ATTRIBUTE(
     EXPORTING
      NAME = `MATNR`
     IMPORTING
      VALUE = LV_MATNR2 ).
   ENDIF.


    LO_ND_EXPORTING = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS-
```

```
>WDCTX_EXPORTING ).
        LO_EL_EXPORTING = LO_ND_EXPORTING->GET_ELEMENT( ).
        IF LO_EL_EXPORTING IS NOT INITIAL.
         LO_EL_EXPORTING->GET_ATTRIBUTE(
          EXPORTING
           NAME = `CAPACITY2`
          IMPORTING
           VALUE = LV_CAPACITY2 ).
        ENDIF.

         DATA LO_ND_GSTN TYPE REF TO IF_WD_CONTEXT_NODE.
         DATA LO_EL_GSTN TYPE REF TO IF_WD_CONTEXT_ELEMENT.
         DATA LS_GSTN TYPE WD_THIS->ELEMENT_GSTN.
         DATA LV_TPT_GSTN TYPE WD_THIS->ELEMENT_GSTN-TPT_GSTN.
         LO_ND_GSTN = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_GSTN ).
         LO_EL_GSTN = LO_ND_GSTN->GET_ELEMENT( ).
         IF LO_EL_GSTN IS NOT INITIAL.
          LO_EL_GSTN->GET_ATTRIBUTE(
           EXPORTING
            NAME = `TPT_GSTN`
           IMPORTING
            VALUE = LV_TPT_GSTN ).
         ENDIF.
DATA: CUST_KDGRP TYPE KDGRP,
    MAT_SPART TYPE SPART,
    ZCHK(10),
    LV_ERCODE3(1) TYPE C.

DATA LO_WINDOW_MANAGER TYPE REF TO IF_WD_WINDOW_MANAGER.
DATA LO_API_COMPONENT  TYPE REF TO IF_WD_COMPONENT.
DATA LO_WINDOW       TYPE REF TO IF_WD_WINDOW.
DATA: lt_string type table of string,
    ls_string like line of lt_string.
DATA l_api           TYPE REF TO if_wd_view_controller.
DATA: ERR.

CLEAR: CUST_KDGRP, MAT_SPART, ZCHK.
SELECT SINGLE KDGRP FROM KNVV INTO CUST_KDGRP WHERE KUNNR = LV_KUNNR.  "#EC CI_NOORDER
" #S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
SELECT SINGLE SPART FROM MARA INTO MAT_SPART WHERE MATNR = LV_MATNR1.  "#EC CI_NOORDER "
#S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
IF MAT_SPART <> '40' AND MAT_SPART <> '25' AND MAT_SPART <> '30'.
 ZCHK = 'X'.
ENDIF.
IF ( CUST_KDGRP = 'DI' OR CUST_KDGRP = 'EX' ) AND LV_TPT_GSTN IS INITIAL AND ZCHK = 'X'.
 LV_ERCODE3 = 'X'.
    CLEAR ls_string. CLEAR lt_string. REFRESH lt_string.
    ls_string = 'Please enter GSTN of Transporter'.
    insert ls_string into table lt_string.
    LO_API_COMPONENT        = WD_COMP_CONTROLLER->WD_GET_API( ).
    LO_WINDOW_MANAGER       = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).
    LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
          text         = lt_string
          button_kind    = if_wd_window=>co_buttons_ok
```

```
*            button_kind    = 4
             message_type   = if_wd_window=>CO_MSG_TYPE_STOPP
             window_title   = 'Please check!'
             window_position = if_wd_window=>co_center
             WINDOW_WIDTH   = '20%'
             WINDOW_HEIGHT  = '20%' ).
    LO_WINDOW->open( ).
ENDIF.


IF LV_KUNNR = ' '.
 LV_ERCODE3 = 'X'.
    CLEAR ls_string. CLEAR lt_string. REFRESH lt_string.
    ls_string = 'Please select customer'.
    insert ls_string into table lt_string.
    LO_API_COMPONENT        = WD_COMP_CONTROLLER->WD_GET_API( ).
    LO_WINDOW_MANAGER       = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).
    LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
             text         = lt_string
             button_kind    = if_wd_window=>co_buttons_ok
*            button_kind    = 4
             message_type   = if_wd_window=>CO_MSG_TYPE_STOPP
             window_title   = 'Please check!'
             window_position = if_wd_window=>co_center
             WINDOW_WIDTH   = '20%'
             WINDOW_HEIGHT  = '20%' ).
    LO_WINDOW->open( ).
ENDIF.


IF LV_MATNR1 = ' '.
 LV_ERCODE3 = 'X'.
    CLEAR ls_string. CLEAR lt_string. REFRESH lt_string.
    ls_string = 'Please select material'.
    insert ls_string into table lt_string.
    LO_API_COMPONENT        = WD_COMP_CONTROLLER->WD_GET_API( ).
    LO_WINDOW_MANAGER       = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).
    LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
             text         = lt_string
             button_kind    = if_wd_window=>co_buttons_ok
*            button_kind    = 4
             message_type   = if_wd_window=>CO_MSG_TYPE_STOPP
             window_title   = 'Please check!'
             window_position = if_wd_window=>co_center
             WINDOW_WIDTH   = '20%'
             WINDOW_HEIGHT  = '20%' ).
    LO_WINDOW->open( ).
ENDIF.


IF LV_CAPACITY1 = ' '.
 LV_ERCODE3 = 'X'.
    CLEAR ls_string. CLEAR lt_string. REFRESH lt_string.
    ls_string = 'Please enter indent quantity in KL or MT'.
```

```
    insert ls_string into table lt_string.
    LO_API_COMPONENT        = WD_COMP_CONTROLLER->WD_GET_API( ).
    LO_WINDOW_MANAGER       = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).
    LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
            text         = lt_string
            button_kind    = if_wd_window=>co_buttons_ok
*            button_kind    = 4
            message_type   = if_wd_window=>CO_MSG_TYPE_STOPP
            window_title   = 'Please check!'
            window_position = if_wd_window=>co_center
            WINDOW_WIDTH   = '20%'
            WINDOW_HEIGHT  = '20%' ).
    LO_WINDOW->open( ).
ENDIF.


IF LV_MATNR2 <> ' ' AND LV_CAPACITY2 = ' '.
 LV_ERCODE3 = 'X'.
    CLEAR ls_string. CLEAR lt_string. REFRESH lt_string.
    ls_string = 'Please enter indent quantity for 2nd material in KL or MT'.
    insert ls_string into table lt_string.
    LO_API_COMPONENT        = WD_COMP_CONTROLLER->WD_GET_API( ).
    LO_WINDOW_MANAGER       = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).
    LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
            text         = lt_string
            button_kind    = if_wd_window=>co_buttons_ok
*            button_kind    = 4
            message_type   = if_wd_window=>CO_MSG_TYPE_STOPP
            window_title   = 'Please check!'
            window_position = if_wd_window=>co_center
            WINDOW_WIDTH   = '20%'
            WINDOW_HEIGHT  = '20%' ).
    LO_WINDOW->open( ).
ENDIF.

************************************
DATA: DTCONV1(13).
MOVE LV_CAPACITY1 TO DTCONV1.
REPLACE ALL OCCURRENCES OF '.' IN DTCONV1 WITH ' '.
CONDENSE DTCONV1.

DATA lv_type TYPE DD01V-DATATYPE.
CALL FUNCTION 'NUMERIC_CHECK'
   EXPORTING
    string_in  = DTCONV1
   IMPORTING
    string_out = DTCONV1
    htype      = lv_type.

IF lv_type <> 'NUMC'.
 LV_ERCODE3 = 'X'.
 CLEAR ls_string. CLEAR lt_string. REFRESH lt_string.
    ls_string = 'Please enter quantity only in numeric'.
    insert ls_string into table lt_string.
```

```
    LO_API_COMPONENT        = WD_COMP_CONTROLLER->WD_GET_API( ).
    LO_WINDOW_MANAGER       = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).
    LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
          text         = lt_string
          button_kind    = if_wd_window=>co_buttons_ok
          message_type   = if_wd_window=>CO_MSG_TYPE_STOPP
          window_title   = 'Please check!'
          window_position = if_wd_window=>co_center
          WINDOW_WIDTH   = '20%'
          WINDOW_HEIGHT  = '20%' ).
    LO_WINDOW->open( ).
ENDIF.

IF LV_CAPACITY2 <> ' '.
  CLEAR DTCONV1.
  MOVE LV_CAPACITY2 TO DTCONV1.
  REPLACE ALL OCCURRENCES OF '.' IN DTCONV1 WITH ' '.
  CONDENSE DTCONV1.

  CLEAR lv_type.
  CALL FUNCTION 'NUMERIC_CHECK'
     EXPORTING
      string_in  = DTCONV1
     IMPORTING
      string_out = DTCONV1
      htype     = lv_type.

  IF lv_type <> 'NUMC'.
   LV_ERCODE3 = 'X'.
   CLEAR ls_string. CLEAR lt_string. REFRESH lt_string.
      ls_string = 'Please enter quantity only in numeric'.
      insert ls_string into table lt_string.
      LO_API_COMPONENT        = WD_COMP_CONTROLLER->WD_GET_API( ).
      LO_WINDOW_MANAGER       = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).
      LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
            text         = lt_string
            button_kind    = if_wd_window=>co_buttons_ok
            message_type   = if_wd_window=>CO_MSG_TYPE_STOPP
            window_title   = 'Please check!'
            window_position = if_wd_window=>co_center
            WINDOW_WIDTH   = '20%'
            WINDOW_HEIGHT  = '20%' ).
      LO_WINDOW->open( ).
  ENDIF.
ENDIF.

****************************************


DATA: CUST_IND_NVEH TYPE ZSD_INDENT_NVEH.
DATA: INDENT_NUMBER TYPE CHAR10.
CLEAR CUST_IND_NVEH.
CLEAR INDENT_NUMBER.
DATA: ZTQTY TYPE VBAP-ZMENG,
```

```
    ZTUOM TYPE VBAP-ZIEME,
    ZPOSNR TYPE VBAP-POSNR,
    ZEQTY TYPE VBFA-RFMNG_FLO,
    ZRQTY TYPE VBAP-ZMENG,
    ZORD_QTY TYPE ZSD_INDENT_NVEH-QUANTITY1,
    BLQTY TYPE ZSD_INDENT_NVEH-QUANTITY1,
    ETXT(15).

IF LV_MATNR1+0(4) = 'PWAX' OR LV_MATNR2+0(4) = 'PWAX' OR LV_MATNR1+0(4) = 'RPC0'  "#EC CI_USAGE_
OK[2270335] " #S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
  OR LV_MATNR1+0(4) = 'CPC0' OR LV_MATNR1+0(4) = 'SUL0'.
 CLEAR: ZTQTY, ZTUOM, ZPOSNR, ZEQTY, ZRQTY, ZORD_QTY, BLQTY, ETXT.
 SELECT SINGLE ZMENG ZIEME POSNR FROM VBAP INTO ( ZTQTY, ZTUOM, ZPOSNR )  "#EC CI_NOORDER "
#S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
   WHERE VBELN = LV_VBELN1 AND MATNR = LV_MATNR1.

 SELECT SUM( RFMNG_FLO ) FROM VBFA INTO ZEQTY WHERE VBELV = LV_VBELN1
   AND VBTYP_N = 'C' AND POSNV = ZPOSNR.
   ZRQTY = ZTQTY - ZEQTY.

 SELECT SUM( BL_QTY1 ) FROM ZSD_INDENT_NVEH INTO ZORD_QTY WHERE CONTRACT1 = LV_VBELN1
  AND ORD_CLOSED = ' '.

 SELECT * FROM ZSD_INDENT_NVEH INTO LN_HDR WHERE CONTRACT1 = LV_VBELN1
  AND ORD_CLOSED = ' '.
  APPEND LN_HDR TO IT_HDR.
 ENDSELECT.
 CLEAR LN_HDR.

 LOOP AT IT_HDR INTO LN_HDR.
  CLEAR ORDC_QTY1.
  SELECT SUM( QUANTITY1 ) FROM ZINDT_NVEH_ITEM INTO ORDC_QTY1 WHERE ORDER_NO
   = LN_HDR-ORDER_NO AND SHIPMENT = ' '.
   ZORD_QTY = ZORD_QTY + ORDC_QTY1.
 ENDLOOP.

   IF ZTUOM = 'KG'. ZORD_QTY = ZORD_QTY * 1000. ENDIF.
   ZRQTY = ZRQTY - ZORD_QTY. "To account for the open order balance quantities
DATA: Z_TX1(70).

   BLQTY = LV_CAPACITY1 * 1000.
   IF ZRQTY < BLQTY.
    LV_ERCODE3 = 'X'.
    ZRQTY = ZRQTY / 1000.
    MOVE ZRQTY TO ETXT.
    CLEAR Z_TX1.
    IF ZRQTY < 0.
     MOVE 'No quantity is available in contract' to Z_TX1.
     ELSE.
      CONCATENATE 'Maximum quantity available in contract is' ETXT 'MT' INTO Z_TX1
       SEPARATED BY SPACE.
    ENDIF.

    CLEAR ls_string. CLEAR lt_string. REFRESH lt_string.
```

```
    IF LV_VBELN1 <> ' '.
     MOVE Z_TX1 TO ls_string.
     ELSE.
      MOVE 'Please select contract number' TO ls_string.
    ENDIF.
     insert ls_string into table lt_string.
     LO_API_COMPONENT         = WD_COMP_CONTROLLER->WD_GET_API( ).
     LO_WINDOW_MANAGER        = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).
     LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
           text         = lt_string
           button_kind    = if_wd_window=>co_buttons_ok
           message_type   = if_wd_window=>CO_MSG_TYPE_STOPP
           window_title   = 'Please check!'
           window_position = if_wd_window=>co_center
           WINDOW_WIDTH   = '20%'
           WINDOW_HEIGHT  = '20%' ).
     LO_WINDOW->open( ).
   ENDIF.
ENDIF.

IF LV_ERCODE3 = ' '.
 CUST_IND_NVEH-MANDT = '300'.
 CUST_IND_NVEH-BEGDA = LV_LOAD_DATE.
 SELECT SINGLE DEPOT FROM ZSD_CUST_USR_MAP INTO CUST_IND_NVEH-
DEPOT WHERE CUST_USER_ID = SY-
UNAME. "#EC CI_NOORDER " #S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
 CUST_IND_NVEH-INDENT_DATE = SY-DATUM.
 CUST_IND_NVEH-INDENT_TIME = SY-UZEIT.
 CUST_IND_NVEH-KUNNR = LV_KUNNR.
 SELECT SINGLE NAME1 FROM KNA1 INTO CUST_IND_NVEH-KUNNR_DESC WHERE KUNNR = LV_KUNNR.
 CUST_IND_NVEH-
MATNR1   = LV_MATNR1."#EC CI_FLDEXT_OK[2215424] " #S/4 - OSS Note# - Start of Change- Z_SHSHRUKH
A – 2021/08/13
 CUST_IND_NVEH-QUANTITY1 = LV_CAPACITY1.
 CUST_IND_NVEH-
MATNR2   = LV_MATNR2."#EC CI_FLDEXT_OK[2215424] " #S/4 - OSS Note# - Start of Change- Z_SHSHRUKH
A – 2021/08/13
 CUST_IND_NVEH-QUANTITY2 = LV_CAPACITY2.
 CUST_IND_NVEH-CONTRACT1 = LV_VBELN1.
 CUST_IND_NVEH-TPT_GSTN  = LV_TPT_GSTN.


 CALL FUNCTION 'NUMBER_GET_NEXT'
  EXPORTING
   NR_RANGE_NR          = '01'
   OBJECT               = 'ZINDNT_HDR'
  IMPORTING
   NUMBER               = INDENT_NUMBER.

 CUST_IND_NVEH-ORDER_NO  = INDENT_NUMBER.
 CUST_IND_NVEH-BL_QTY1 = CUST_IND_NVEH-QUANTITY1.
 CUST_IND_NVEH-BL_QTY2 = CUST_IND_NVEH-QUANTITY2.
 CUST_IND_NVEH-CUST_USER_ID = SY-UNAME.
 INSERT ZSD_INDENT_NVEH FROM CUST_IND_NVEH.
```

```
 IF SY-SUBRC = 0.
  CLEAR ls_string. CLEAR lt_string. REFRESH lt_string.
  CONCATENATE 'Thank you, your order number is' INDENT_NUMBER INTO ls_string separated by space.
  insert ls_string into table lt_string.
  LO_API_COMPONENT         = WD_COMP_CONTROLLER->WD_GET_API( ).
  LO_WINDOW_MANAGER        = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).
  LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
          text         = lt_string
          button_kind    = if_wd_window=>co_buttons_ok
*          message_type    = if_wd_window=>CO_MSG_TYPE_STOPP
          window_title    = 'Please check!'
          window_position = if_wd_window=>co_center
          WINDOW_WIDTH    = '20%'
          WINDOW_HEIGHT   = '20%' ).
  LO_WINDOW->open( ).
 ENDIF.
ENDIF.

 DATA LO_ND_WV_ENABLE TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_EL_WV_ENABLE TYPE REF TO IF_WD_CONTEXT_ELEMENT.
 DATA LS_WV_ENABLE TYPE WD_THIS->ELEMENT_WV_ENABLE.
 DATA LV_VW_SV1 TYPE WD_THIS->ELEMENT_WV_ENABLE-VW_SV1.
 LO_ND_WV_ENABLE = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_WV_ENABLE ).
 LO_EL_WV_ENABLE = LO_ND_WV_ENABLE->GET_ELEMENT( ).
 CLEAR LV_VW_SV1.
 LO_EL_WV_ENABLE->SET_ATTRIBUTE(
  NAME =  `VW_SV1`
  VALUE = LV_VW_SV1 ).

* LO_ND_HEADER_DATA = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS-
>WDCTX_HEADER_DATA ).
* LO_EL_HEADER_DATA = LO_ND_HEADER_DATA->GET_ELEMENT( ).
* CLEAR LV_KUNNR.
* LO_EL_HEADER_DATA->SET_ATTRIBUTE(
*     NAME =  `KUNNR`
*     VALUE = LV_KUNNR ).

 LO_ND_MATERIAL2 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_MATERIAL2 ).
 LO_EL_MATERIAL2 = LO_ND_MATERIAL2->GET_ELEMENT( ).
 IF LO_EL_MATERIAL2 IS NOT INITIAL.
  CLEAR LV_MATNR2.
  LO_EL_MATERIAL2->SET_ATTRIBUTE(
     NAME =  `MATNR`
     VALUE = LV_MATNR2 ).
 ENDIF.

 LO_ND_MATERIAL1 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_MATERIAL1 ).
 LO_EL_MATERIAL1 = LO_ND_MATERIAL1->GET_ELEMENT( ).
 IF LO_EL_MATERIAL1 IS NOT INITIAL.
  CLEAR LV_MATNR1.
  LO_EL_MATERIAL1->SET_ATTRIBUTE(
       NAME =  `MATNR`
       VALUE = LV_MATNR1 ).
 ENDIF.
```

```
  LO_ND_NO_VEH_CONTRACT = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS-
>WDCTX_NO_VEH_CONTRACT ).
  LO_EL_NO_VEH_CONTRACT = LO_ND_NO_VEH_CONTRACT->GET_ELEMENT( ).
  IF LO_EL_NO_VEH_CONTRACT IS NOT INITIAL.
   CLEAR LV_VBELN1.
   LO_EL_NO_VEH_CONTRACT->SET_ATTRIBUTE(
          NAME = `VBELN1`
          VALUE = LV_VBELN1 ).
  ENDIF.

  LO_ND_EXPORTING = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_EXPORTING ).
  LO_EL_EXPORTING = LO_ND_EXPORTING->GET_ELEMENT( ).
  IF LO_EL_EXPORTING IS NOT INITIAL.
   CLEAR LV_CAPACITY1. CLEAR LV_CAPACITY2.
   LO_EL_EXPORTING->SET_ATTRIBUTE(
     NAME = `CAPACITY1`
     VALUE = LV_CAPACITY1 ).

   LO_EL_EXPORTING->SET_ATTRIBUTE(
     NAME = `CAPACITY2`
     VALUE = LV_CAPACITY2 ).
  ENDIF.

  LO_ND_GSTN = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_GSTN ).
  LO_EL_GSTN = LO_ND_GSTN->GET_ELEMENT( ).
  IF LO_EL_GSTN IS NOT INITIAL.
   CLEAR LV_TPT_GSTN.
   LO_EL_GSTN->SET_ATTRIBUTE(
      NAME = `TPT_GSTN`
      VALUE = LV_TPT_GSTN ).
  ENDIF.



 imat_data = wd_context->path_get_node( path = `MATERIAL1` ).
 select * into corresponding fields of table imat_data1 from ZSD_INDT_NO_VEH WHERE CUST_GROUP = ZK
DGRP.
 insert lmat_data1 into imat_data1 index 1.
 imat_data->bind_table( new_items = imat_data1 set_initial_elements = abap_true ).

 if ZKDGRP <> 'DI' AND ZKDGRP <> 'EX'.
    clear imat_data.
    imat_data = wd_context->path_get_node( path = `MATERIAL2` ).
    select * into corresponding fields of table imat_data2 from ZSD_INDT_NO_VEH WHERE CUST_GROUP =
ZKDGRP.
    insert lmat_data1 into imat_data2 index 1.
    imat_data->bind_table( new_items = imat_data2 set_initial_elements = abap_true ).
 endif.

DATA lr_event1 TYPE REF TO cl_wd_custom_event.
DATA lr_event TYPE REF TO cl_wd_custom_event.
*  IF LV_VBELN IS INITIAL.
```

```
CREATE OBJECT lr_event
  EXPORTING
    name = 'ACTIVATE_GSTN'.
wd_this->ONACTIONACTIVATE_GSTN(
 wdevent = lr_event1                    " ref to cl_wd_custom_event
).


DATA LO_COMPONENTCONTROLLER TYPE REF TO IG_COMPONENTCONTROLLER .
LO_COMPONENTCONTROLLER =   WD_THIS->GET_COMPONENTCONTROLLER_CTR( ).

 LO_COMPONENTCONTROLLER->EXECUTE_Z_GET_CUST_INDENT_NO_V(
 ).
```

```
endmethod.
```

- method ONACTIONSELECT_CONTRACT_NVEH .

```
 TYPES:  BEGIN OF SCONTRACT ,
     VBELN TYPE VBAK-VBELN,
     KUNNR TYPE KUNNR,
     MATNR TYPE MATNR,
     VAL TYPE BWTAR_D,
     KONDM TYPE KONDM,
     TQTY TYPE VBFA-RFMNG,
     TUOM TYPE VBAP-ZIEME,
     POSNR TYPE VBAP-POSNR,
     EQTY TYPE VBFA-RFMNG,
     RQTY TYPE VBFA-RFMNG,
     END OF SCONTRACT.

DATA: IT_CONTRACT type
   standard table of SCONTRACT.
```

```
DATA: LN_CONTRACT TYPE SCONTRACT.
DATA LO_VBELN TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_EXPORTING TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_VBELN1 TYPE wd_this->ELEMENTS_NO_VEH_CONTRACT.
DATA LS_VBELN TYPE IG_COMPONENTCONTROLLER=>Element_NO_VEH_CONTRACT.
DATA ZDEPOT TYPE WERKS_D.
DATA: IT_ITEM type standard table of ZINDT_NVEH_ITEM.
DATA: LN_ITEM type ZINDT_NVEH_ITEM.
DATA: IT_HDR type standard table of ZSD_INDENT_NVEH.
DATA: LN_HDR type ZSD_INDENT_NVEH.


 DATA LO_ND_EXPORTING TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_EL_EXPORTING TYPE REF TO IF_WD_CONTEXT_ELEMENT.
 DATA LS_EXPORTING TYPE WD_THIS->ELEMENT_EXPORTING.
 DATA LV_CAPACITY1 TYPE WD_THIS->ELEMENT_EXPORTING-CAPACITY1.
 DATA LV_CAPACITY2 TYPE WD_THIS->ELEMENT_EXPORTING-CAPACITY2.
 DATA LV_CAPACITY3 TYPE WD_THIS->ELEMENT_EXPORTING-CAPACITY3.
 DATA LV_CAPACITY4 TYPE WD_THIS->ELEMENT_EXPORTING-CAPACITY4.
 DATA LV_CAPACITY5 TYPE WD_THIS->ELEMENT_EXPORTING-CAPACITY5.
 LO_ND_EXPORTING = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_EXPORTING ).
 LO_EL_EXPORTING = LO_ND_EXPORTING->GET_ELEMENT( ).
 IF LO_EL_EXPORTING IS NOT INITIAL.
  CLEAR: LV_CAPACITY1, LV_CAPACITY2, LV_CAPACITY3, LV_CAPACITY4, LV_CAPACITY5.
  LO_EL_EXPORTING->SET_ATTRIBUTE(
      NAME =  `CAPACITY1`
      VALUE = LV_CAPACITY1 ).

  LO_EL_EXPORTING->SET_ATTRIBUTE(
      NAME =  `CAPACITY2`
      VALUE = LV_CAPACITY2 ).

  LO_EL_EXPORTING->SET_ATTRIBUTE(
      NAME =  `CAPACITY3`
      VALUE = LV_CAPACITY3 ).

  LO_EL_EXPORTING->SET_ATTRIBUTE(
      NAME =  `CAPACITY4`
      VALUE = LV_CAPACITY4 ).

  LO_EL_EXPORTING->SET_ATTRIBUTE(
      NAME =  `CAPACITY5`
      VALUE = LV_CAPACITY5 ).
 ENDIF.


SELECT SINGLE DEPOT FROM zsd_cust_usr_map INTO ZDEPOT WHERE CUST_USER_ID = SY-
UNAME. "#EC CI_NOORDER " #S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18

   DATA LO_ND_HEADER_DATA TYPE REF TO IF_WD_CONTEXT_NODE.
   DATA LO_EL_HEADER_DATA TYPE REF TO IF_WD_CONTEXT_ELEMENT.
   DATA LS_HEADER_DATA TYPE WD_THIS->ELEMENT_HEADER_DATA.
   DATA LV_KUNNR TYPE WD_THIS->ELEMENT_HEADER_DATA-KUNNR.
   LO_ND_HEADER_DATA = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS-
```

```
>WDCTX_HEADER_DATA ).
  LO_EL_HEADER_DATA = LO_ND_HEADER_DATA->GET_ELEMENT( ).
  LO_EL_HEADER_DATA->GET_ATTRIBUTE(
   EXPORTING
    NAME = `KUNNR`
   IMPORTING
    VALUE = LV_KUNNR ).


 DATA LO_ND_MATERIAL1 TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_EL_MATERIAL1 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
 DATA LS_MATERIAL1 TYPE WD_THIS->ELEMENT_MATERIAL1.
 DATA LV_MATNR1 TYPE WD_THIS->ELEMENT_MATERIAL1-MATNR.
 LO_ND_MATERIAL1 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_MATERIAL1 ).
 LO_EL_MATERIAL1 = LO_ND_MATERIAL1->GET_ELEMENT( ).
 IF LO_EL_MATERIAL1 IS NOT INITIAL.
  LO_EL_MATERIAL1->GET_ATTRIBUTE(
   EXPORTING
    NAME = `MATNR`
   IMPORTING
    VALUE = LV_MATNR1 ).
 ENDIF.

  DATA LO_ND_MATERIAL2 TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LO_EL_MATERIAL2 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
  DATA LS_MATERIAL2 TYPE WD_THIS->ELEMENT_MATERIAL2.
  DATA LV_MATNR2 TYPE WD_THIS->ELEMENT_MATERIAL2-MATNR.
  LO_ND_MATERIAL2 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_MATERIAL2 ).
  LO_EL_MATERIAL2 = LO_ND_MATERIAL2->GET_ELEMENT( ).
  IF LO_EL_MATERIAL2 IS NOT INITIAL.
   LO_EL_MATERIAL2->GET_ATTRIBUTE(
    EXPORTING
     NAME = `MATNR`
    IMPORTING
     VALUE = LV_MATNR2 ).
  ENDIF.

   DATA LO_ND_WV_ENABLE TYPE REF TO IF_WD_CONTEXT_NODE.
   DATA LO_EL_WV_ENABLE TYPE REF TO IF_WD_CONTEXT_ELEMENT.
   DATA LS_WV_ENABLE TYPE WD_THIS->ELEMENT_WV_ENABLE.
   DATA LV_VW_SV1 TYPE WD_THIS->ELEMENT_WV_ENABLE-VW_SV1.
   LO_ND_WV_ENABLE = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_WV_ENABLE ).
   LO_EL_WV_ENABLE = LO_ND_WV_ENABLE->GET_ELEMENT( ).
   IF LV_MATNR1 <> ' '.
    LV_VW_SV1 = 'X'.
    ELSE.
     CLEAR LV_VW_SV1.
   ENDIF.

   LO_EL_WV_ENABLE->SET_ATTRIBUTE(
    NAME = `VW_SV1`
    VALUE = LV_VW_SV1 ).

DATA: ORD_QTY TYPE ZSD_INDENT_NVEH-QUANTITY1,
```

```
    ORDC_QTY1 TYPE ZSD_INDENT_NVEH-QUANTITY1,
    ORDC_QTY2 TYPE ZSD_INDENT_NVEH-QUANTITY1.

*   IF LV_MATNR1+0(4) = 'PWAX' OR LV_MATNR2+0(4) = 'PWAX' OR LV_MATNR1+0(4) = 'RPC0'
*    OR LV_MATNR1+0(4) = 'CPC0' OR LV_MATNR1+0(4) = 'SUL0'.
  IF LV_MATNR1 <> ' ' OR  LV_MATNR2 <> ' '.
    CLEAR: LN_CONTRACT, IT_CONTRACT. REFRESH IT_CONTRACT.
    SELECT * INTO CORRESPONDING FIELDS OF LN_CONTRACT FROM VBAK
     INNER JOIN vbpa
     ON vbak~vbeln = vbpa~vbeln
      INNER JOIN vbap ON
       vbak~vbeln = vbap~vbeln WHERE vbak~guebg <= sy-datum
        AND vbak~gueen >= sy-datum AND vbpa~kunnr = LV_KUNNR
         AND vbap~matnr = LV_MATNR1 AND vbap~bwtar = 'OWN-BOND'
          AND vbap~werks = ZDEPOT AND PARVW = 'WE'
           AND vbap~pstyv <> 'ZTAE' AND vbap~abgru = ' '.
     APPEND LN_CONTRACT TO IT_CONTRACT.
     ENDSELECT.
     SORT IT_CONTRACT BY VBELN ASCENDING.

     LOOP AT IT_CONTRACT INTO LN_CONTRACT.
      CLEAR: ORD_QTY, ORDC_QTY1, ORDC_QTY2, LN_ITEM, IT_ITEM, LN_HDR, IT_HDR.
      REFRESH: IT_ITEM, IT_HDR.
      SELECT SINGLE ZMENG ZIEME POSNR FROM VBAP INTO ( LN_CONTRACT-TQTY, LN_CONTRACT-
TUOM, LN_CONTRACT-
POSNR )   "#EC CI_NOORDER " #S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
       WHERE VBELN = LN_CONTRACT-VBELN AND MATNR = LN_CONTRACT-MATNR.
      SELECT SUM( RFMNG_FLO ) FROM VBFA INTO LN_CONTRACT-EQTY WHERE VBELV = LN_CONTRACT-
VBELN
       AND VBTYP_N = 'C' AND POSNV = LN_CONTRACT-POSNR.
      LN_CONTRACT-RQTY = LN_CONTRACT-TQTY - LN_CONTRACT-EQTY.
      SELECT SUM( BL_QTY1 ) FROM ZSD_INDENT_NVEH INTO ORD_QTY WHERE CONTRACT1 = LN_CONTR
ACT-VBELN
       AND ORD_CLOSED = ' '.
      SELECT * FROM ZSD_INDENT_NVEH INTO LN_HDR WHERE CONTRACT1 = LN_CONTRACT-VBELN
       AND ORD_CLOSED = ' '.
       APPEND LN_HDR TO IT_HDR.
      ENDSELECT.
      CLEAR LN_HDR.
      LOOP AT IT_HDR INTO LN_HDR.
       CLEAR ORDC_QTY1.
       SELECT SUM( QUANTITY1 ) FROM ZINDT_NVEH_ITEM INTO ORDC_QTY1 WHERE ORDER_NO
        = LN_HDR-ORDER_NO AND SHIPMENT = ' '.
*       IF LN_CONTRACT-TUOM = 'KG'.  ORDC_QTY1 = ORDC_QTY1 * 1000. ENDIF.
       ORD_QTY = ORD_QTY + ORDC_QTY1.
      ENDLOOP.

*      SELECT SUM( QUANTITY1 ) SUM( BL_QTY1 ) FROM ZSD_INDENT_NVEH INTO (ORDC_QTY1, ORDC_QT
Y2)
*       WHERE CONTRACT1 = LN_CONTRACT-VBELN AND ORD_CLOSED = 'X'.
*       ORD_QTY = ORD_QTY + ORDC_QTY1 - ORDC_QTY2.
       IF LN_CONTRACT-TUOM = 'KG'. ORD_QTY = ORD_QTY * 1000. ENDIF.
       LN_CONTRACT-RQTY = LN_CONTRACT-RQTY - ORD_QTY.
      IF LN_CONTRACT-RQTY <= 0.
```

```
        DELETE IT_CONTRACT.
        ELSE.
         MODIFY IT_CONTRACT FROM LN_CONTRACT.
       ENDIF.
      ENDLOOP.

      LO_VBELN = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_NO_VEH_CONTRACT ).
      CLEAR LO_VBELN1. REFRESH LO_VBELN1.
      CLEAR LS_VBELN.

      CLEAR LN_CONTRACT.
      APPEND LS_VBELN TO LO_VBELN1.
      LOOP AT IT_CONTRACT INTO LN_CONTRACT.
       CLEAR LS_VBELN.
       LS_VBELN = LN_CONTRACT-VBELN.
       APPEND LS_VBELN TO LO_VBELN1.
      ENDLOOP.
      LO_VBELN->bind_table( new_items = LO_VBELN1 set_initial_elements = abap_true ).

      LO_VBELN = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_EXPORT ).
      CLEAR LO_VBELN1. REFRESH LO_VBELN1.
      CLEAR LS_VBELN.
     ENDIF.




endmethod.
```

- method ONACTIONSELECT_EXPORT .

```
 DATA LO_ND_IND_END_USE TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_EL_IND_END_USE TYPE REF TO IF_WD_CONTEXT_ELEMENT.
 DATA LS_IND_END_USE TYPE WD_THIS->Element_ind_end_use.
 DATA LV_IND_USE_MS TYPE WD_THIS->Element_ind_end_use-IND_USE_MS.
 DATA LV_IND_USE_HSD TYPE WD_THIS->Element_ind_end_use-IND_USE_HSD.

 LO_ND_IND_END_USE = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS-
>WDCTX_IND_END_USE ).
 LO_EL_IND_END_USE = LO_ND_IND_END_USE->GET_ELEMENT( ).
 IF LO_EL_IND_END_USE IS INITIAL.
 ENDIF.
* set single attribute
  CLEAR: LV_IND_USE_MS, LV_IND_USE_HSD.
  LO_EL_IND_END_USE->SET_ATTRIBUTE(
   NAME = `IND_USE_MS`
   VALUE = LV_IND_USE_MS ).
  LO_EL_IND_END_USE->SET_ATTRIBUTE(
   NAME = `IND_USE_HSD`
   VALUE = LV_IND_USE_HSD ).

***** Added for Matetial Price Group START ****
TYPES:
begin of ZT178,
   KONDM TYPE KONDM,
```

```
    end of ZT178.

DATA: ZZT178 type
    standard table of ZT178.
DATA: KT178 TYPE ZT178.
***** Added for Matetial Price Group END ****

TYPES:  BEGIN OF SCONTRACT ,
      VBELN TYPE VBAK-VBELN,
      KUNNR TYPE KUNNR,
      MATNR TYPE MATNR,
      VAL TYPE BWTAR_D,
      KONDM TYPE KONDM,
      TQTY TYPE VBFA-RFMNG,
      TUOM TYPE VBAP-ZIEME,
      POSNR TYPE VBAP-POSNR,
      EQTY TYPE VBFA-RFMNG,
      RQTY TYPE VBFA-RFMNG,
      END OF SCONTRACT.

DATA: IT_CONTRACT type
    standard table of SCONTRACT.
DATA: LN_CONTRACT TYPE SCONTRACT.

DATA: LV_PROD1 TYPE CHAR100,
    LV_PROD2 TYPE CHAR100,
    LV_PROD3 TYPE CHAR100,
    LV_PROD4 TYPE CHAR100,
    LV_PROD5 TYPE CHAR100,
    LV_PROD6 TYPE CHAR100,
    LV_PROD7 TYPE CHAR100,
    LV_PROD8 TYPE CHAR100.

DATA: ZPRODUCT TYPE CHAR100.

DATA LO_IMPORTING TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_EXPORTING TYPE REF TO IF_WD_CONTEXT_NODE.

DATA LO_VBELN TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_VBELN1 TYPE wd_this->ELEMENTS_EXPORT.
DATA LS_VBELN TYPE IG_COMPONENTCONTROLLER=>Element_EXPORT.

DATA LO_ND_IMPORTING TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_EL_IMPORTING TYPE REF TO IF_WD_CONTEXT_ELEMENT.
DATA LS_IMPORTING TYPE WD_THIS->ELEMENT_IMPORTING.
DATA LV_VEHICLE TYPE WD_THIS->ELEMENT_IMPORTING-VEHICLE.
LO_ND_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_IMPORTING ).
LO_EL_IMPORTING = LO_ND_IMPORTING->GET_ELEMENT( ).
LO_EL_IMPORTING->GET_ATTRIBUTE(
  EXPORTING
   NAME = `VEHICLE`
  IMPORTING
   VALUE = LV_VEHICLE ).
```

```
DATA CNT TYPE I.
SELECT COUNT(*) FROM OIGCC INTO CNT WHERE TU_NUMBER = LV_VEHICLE.

DATA: LV_KUNNR1 TYPE KNA1-KUNNR,
    ZDEPOT TYPE WERKS_D.

CLEAR LO_IMPORTING. CLEAR ZDEPOT.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_HEADER_DATA ).
LO_IMPORTING->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `KUNNR`
  IMPORTING
   VALUE = LV_KUNNR1 ).

if LV_KUNNR1 is initial.
 DATA: lt_string type table of string,
     ls_string like line of lt_string.
 DATA LO_WINDOW_MANAGER TYPE REF TO IF_WD_WINDOW_MANAGER.
 DATA LO_API_COMPONENT  TYPE REF TO IF_WD_COMPONENT.
 DATA LO_WINDOW       TYPE REF TO IF_WD_WINDOW.

***&&&&&&&&&&&&&&&&&&&&&&&&
 DATA LO_ND_PROD1 TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LT_PROD1 TYPE WD_THIS->Elements_prod1.
 LO_ND_PROD1 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD1 ).
 LO_ND_PROD1->GET_STATIC_ATTRIBUTES_TABLE( IMPORTING TABLE = LT_PROD1 ).
***&&&&&&&&&&&&&&&&&&&&&&&&

*   DATA LO_ND_PROD1 TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LO_EL_PROD1 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
  DATA LS_PROD1 TYPE WD_THIS->Element_prod1.
  DATA LV_PRODUCT1 TYPE WD_THIS->Element_prod1-PRODUCT.
  LO_ND_PROD1 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD1 ).
  LO_EL_PROD1 = LO_ND_PROD1->GET_ELEMENT( ).
*  set single attribute
  clear LV_PRODUCT1.
  LO_EL_PROD1->SET_ATTRIBUTE(
   NAME =  `PRODUCT`
   VALUE = LV_PRODUCT1 ).
LO_ND_PROD1->bind_table( new_items = LT_PROD1 set_initial_elements = abap_true ).


 DATA LO_ND_PROD2 TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LT_PROD2 TYPE WD_THIS->Elements_prod2.
 LO_ND_PROD2 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD2 ).
 LO_ND_PROD2->GET_STATIC_ATTRIBUTES_TABLE( IMPORTING TABLE = LT_PROD2 ).

*   DATA LO_ND_PROD2 TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LO_EL_PROD2 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
  DATA LS_PROD2 TYPE WD_THIS->Element_prod2.
  DATA LV_PRODUCT2 TYPE WD_THIS->Element_prod2-PRODUCT.
  LO_ND_PROD2 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD2 ).
  LO_EL_PROD2 = LO_ND_PROD2->GET_ELEMENT( ).
*  set single attribute
```

```
  clear LV_PRODUCT2.
  LO_EL_PROD2->SET_ATTRIBUTE(
   NAME = `PRODUCT`
   VALUE = LV_PRODUCT2 ).
LO_ND_PROD2->bind_table( new_items = LT_PROD2 set_initial_elements = abap_true ).

  DATA LO_ND_PROD3 TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LT_PROD3 TYPE WD_THIS->Elements_prod3.
  LO_ND_PROD3 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD3 ).
  LO_ND_PROD3->GET_STATIC_ATTRIBUTES_TABLE( IMPORTING TABLE = LT_PROD3 ).
  DATA LO_EL_PROD3 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
  DATA LS_PROD3 TYPE WD_THIS->Element_prod3.
  DATA LV_PRODUCT3 TYPE WD_THIS->Element_prod3-PRODUCT.
  LO_ND_PROD3 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD3 ).
  LO_EL_PROD3 = LO_ND_PROD3->GET_ELEMENT( ).
*  set single attribute
  clear LV_PRODUCT3.
  LO_EL_PROD3->SET_ATTRIBUTE(
   NAME = `PRODUCT`
   VALUE = LV_PRODUCT3 ).
LO_ND_PROD3->bind_table( new_items = LT_PROD3 set_initial_elements = abap_true ).


  DATA LO_ND_PROD4 TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LT_PROD4 TYPE WD_THIS->Elements_prod4.
  LO_ND_PROD4 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD4 ).
  LO_ND_PROD4->GET_STATIC_ATTRIBUTES_TABLE( IMPORTING TABLE = LT_PROD4 ).
  DATA LO_EL_PROD4 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
  DATA LS_PROD4 TYPE WD_THIS->Element_prod4.
  DATA LV_PRODUCT4 TYPE WD_THIS->Element_prod4-PRODUCT.
  LO_ND_PROD4 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD4 ).
  LO_EL_PROD4 = LO_ND_PROD4->GET_ELEMENT( ).
*  set single attribute
  clear LV_PRODUCT4.
  LO_EL_PROD4->SET_ATTRIBUTE(
   NAME = `PRODUCT`
   VALUE = LV_PRODUCT4 ).
LO_ND_PROD4->bind_table( new_items = LT_PROD4 set_initial_elements = abap_true ).

  DATA LO_ND_PROD5 TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LT_PROD5 TYPE WD_THIS->Elements_prod5.
  LO_ND_PROD5 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD5 ).
  LO_ND_PROD5->GET_STATIC_ATTRIBUTES_TABLE( IMPORTING TABLE = LT_PROD5 ).
  DATA LO_EL_PROD5 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
  DATA LS_PROD5 TYPE WD_THIS->Element_prod5.
  DATA LV_PRODUCT5 TYPE WD_THIS->Element_prod5-PRODUCT.
  LO_ND_PROD5 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD5 ).
  LO_EL_PROD5 = LO_ND_PROD5->GET_ELEMENT( ).
*  set single attribute
  clear LV_PRODUCT5.
  LO_EL_PROD5->SET_ATTRIBUTE(
   NAME = `PRODUCT`
   VALUE = LV_PRODUCT5 ).
LO_ND_PROD5->bind_table( new_items = LT_PROD5 set_initial_elements = abap_true ).
```

```
  DATA LO_ND_PROD6 TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LT_PROD6 TYPE WD_THIS->Elements_prod6.
  LO_ND_PROD6 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD6 ).
  LO_ND_PROD6->GET_STATIC_ATTRIBUTES_TABLE( IMPORTING TABLE = LT_PROD6 ).
  DATA LO_EL_PROD6 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
  DATA LS_PROD6 TYPE WD_THIS->Element_prod6.
  DATA LV_PRODUCT6 TYPE WD_THIS->Element_prod6-PRODUCT.
  LO_ND_PROD6 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD6 ).
  LO_EL_PROD6 = LO_ND_PROD6->GET_ELEMENT( ).
*  set single attribute
  clear LV_PRODUCT6.
  LO_EL_PROD6->SET_ATTRIBUTE(
   NAME =  `PRODUCT`
   VALUE = LV_PRODUCT6 ).
LO_ND_PROD6->bind_table( new_items = LT_PROD6 set_initial_elements = abap_true ).

  DATA LO_ND_PROD7 TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LT_PROD7 TYPE WD_THIS->Elements_prod7.
  LO_ND_PROD7 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD7 ).
  LO_ND_PROD7->GET_STATIC_ATTRIBUTES_TABLE( IMPORTING TABLE = LT_PROD7 ).
  DATA LO_EL_PROD7 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
  DATA LS_PROD7 TYPE WD_THIS->Element_prod7.
  DATA LV_PRODUCT7 TYPE WD_THIS->Element_prod7-PRODUCT.
  LO_ND_PROD7 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD7 ).
  LO_EL_PROD7 = LO_ND_PROD7->GET_ELEMENT( ).
*  set single attribute
  clear LV_PRODUCT7.
  LO_EL_PROD7->SET_ATTRIBUTE(
   NAME =  `PRODUCT`
   VALUE = LV_PRODUCT7 ).
LO_ND_PROD7->bind_table( new_items = LT_PROD7 set_initial_elements = abap_true ).

  DATA LO_ND_PROD8 TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LT_PROD8 TYPE WD_THIS->Elements_prod8.
  LO_ND_PROD8 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD8 ).
  LO_ND_PROD8->GET_STATIC_ATTRIBUTES_TABLE( IMPORTING TABLE = LT_PROD8 ).
  DATA LO_EL_PROD8 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
  DATA LS_PROD8 TYPE WD_THIS->Element_prod8.
  DATA LV_PRODUCT8 TYPE WD_THIS->Element_prod8-PRODUCT.
  LO_ND_PROD8 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD8 ).
  LO_EL_PROD8 = LO_ND_PROD8->GET_ELEMENT( ).
*  set single attribute
  clear LV_PRODUCT8.
  LO_EL_PROD8->SET_ATTRIBUTE(
   NAME =  `PRODUCT`
   VALUE = LV_PRODUCT8 ).
LO_ND_PROD8->bind_table( new_items = LT_PROD8 set_initial_elements = abap_true ).

  CLEAR ls_string. CLEAR lt_string. REFRESH lt_string.
  ls_string = 'Please select the customer first'.
  insert ls_string into table lt_string.
  LO_API_COMPONENT        = WD_COMP_CONTROLLER->WD_GET_API( ).
  LO_WINDOW_MANAGER       = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).
```

```
  LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
        text         = lt_string
        button_kind    = if_wd_window=>co_buttons_ok
        message_type   = if_wd_window=>CO_MSG_TYPE_STOPP
        window_title   = 'Please check!'
        window_position = if_wd_window=>co_center
        WINDOW_WIDTH   = '20%'
        WINDOW_HEIGHT  = '20%' ).
  LO_WINDOW->open( ).
endif.

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_PROD1 ).
LO_IMPORTING->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `PRODUCT`
  IMPORTING
   VALUE = LV_PROD1 ).

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_PROD2 ).
LO_IMPORTING->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `PRODUCT`
  IMPORTING
   VALUE = LV_PROD2 ).

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_PROD3 ).
LO_IMPORTING->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `PRODUCT`
  IMPORTING
   VALUE = LV_PROD3 ).

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_PROD4 ).
LO_IMPORTING->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `PRODUCT`
  IMPORTING
   VALUE = LV_PROD4 ).


CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_PROD5 ).
LO_IMPORTING->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `PRODUCT`
  IMPORTING
   VALUE = LV_PROD5 ).
```

```
CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_PROD6 ).
LO_IMPORTING->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `PRODUCT`
  IMPORTING
   VALUE = LV_PROD6 ).

CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_PROD7 ).
LO_IMPORTING->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `PRODUCT`
  IMPORTING
   VALUE = LV_PROD7 ).


CLEAR LO_IMPORTING.
LO_IMPORTING = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_PROD8 ).
LO_IMPORTING->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `PRODUCT`
  IMPORTING
   VALUE = LV_PROD8 ).


SELECT SINGLE DEPOT FROM zsd_cust_usr_map INTO ZDEPOT WHERE CUST_USER_ID = SY-
UNAME.  "#EC CI_NOORDER " #S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18

SELECT KONDM FROM T178 INTO KT178.
 APPEND KT178 TO ZZT178.
ENDSELECT.


IF LV_PROD1+0(4) = 'PWAX' OR LV_PROD2+0(4) = 'PWAX' OR LV_PROD3+0(4) = 'PWAX'
 OR LV_PROD4+0(4) = 'PWAX' OR LV_PROD5+0(4) = 'PWAX' OR LV_PROD1+0(4) = 'RPC0'
 OR LV_PROD1+0(4) = 'CPC0' OR LV_PROD1+0(4) = 'SUL0' OR
 LV_PROD1+0(4) = 'GWAX' OR LV_PROD2+0(4) = 'GWAX' OR LV_PROD3+0(4) = 'GWAX'
 OR LV_PROD4+0(4) = 'GWAX' OR LV_PROD5+0(4) = 'GWAX'.

 DATA: IT_INDENT_NVEH TYPE standard table of ZSD_INDENT_NVEH,
   LN_INDENT_NVEH TYPE ZSD_INDENT_NVEH.

 DATA LO_ND_EXPORTING TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_EL_EXPORTING TYPE REF TO IF_WD_CONTEXT_ELEMENT.
 DATA LS_EXPORTING TYPE WD_THIS->Element_exporting.
 DATA LV_CAPACITY1 TYPE WD_THIS->Element_exporting-CAPACITY1.
 LO_ND_EXPORTING = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_EXPORTING ).
 LO_EL_EXPORTING = LO_ND_EXPORTING->GET_ELEMENT( ).

 clear LV_CAPACITY1.
 LO_EL_EXPORTING->GET_ATTRIBUTE(
  EXPORTING
   NAME =  `CAPACITY1`
```

```
  IMPORTING
    VALUE = LV_CAPACITY1 ).

 CLEAR: LN_CONTRACT, IT_CONTRACT, IT_INDENT_NVEH, LN_INDENT_NVEH.
 REFRESH: IT_CONTRACT, IT_INDENT_NVEH.
 SELECT * INTO CORRESPONDING FIELDS OF LN_CONTRACT FROM VBAK
  INNER JOIN vbpa
   ON vbak~vbeln = vbpa~vbeln
    INNER JOIN vbap ON
     vbak~vbeln = vbap~vbeln WHERE vbak~guebg <= sy-datum
      AND vbak~gueen >= sy-datum AND vbpa~kunnr = LV_KUNNR1
       AND vbap~matnr = LV_PROD1 AND vbap~bwtar = 'OWN-BOND'
        AND vbap~werks = ZDEPOT AND PARVW = 'WE'
         AND vbap~pstyv <> 'ZTAE'.
  APPEND LN_CONTRACT TO IT_CONTRACT.
  ENDSELECT.
  SORT IT_CONTRACT BY VBELN ASCENDING.

  LOOP AT IT_CONTRACT INTO LN_CONTRACT.
   SELECT SINGLE ZMENG ZIEME POSNR FROM VBAP INTO ( LN_CONTRACT-TQTY, LN_CONTRACT-
TUOM, LN_CONTRACT-
POSNR ) "#EC CI_NOORDER " #S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
     WHERE VBELN = LN_CONTRACT-VBELN AND MATNR = LN_CONTRACT-MATNR.
   SELECT SUM( RFMNG_FLO ) FROM VBFA INTO LN_CONTRACT-EQTY WHERE VBELV = LN_CONTRACT-
VBELN
     AND VBTYP_N = 'C' AND POSNV = LN_CONTRACT-POSNR.
   LN_CONTRACT-RQTY = LN_CONTRACT-TQTY - LN_CONTRACT-EQTY.
   MODIFY IT_CONTRACT FROM LN_CONTRACT.
  ENDLOOP.

  LOOP AT IT_CONTRACT INTO LN_CONTRACT.
   SELECT SINGLE * FROM ZSD_INDENT_NVEH INTO LN_INDENT_NVEH
    WHERE CONTRACT1 = LN_CONTRACT-VBELN AND ORD_CLOSED = ' '.
   IF SY-SUBRC = 0.
    APPEND LN_INDENT_NVEH TO IT_INDENT_NVEH.
   ENDIF.
  ENDLOOP.

  IF IT_INDENT_NVEH[] IS NOT INITIAL.
   READ TABLE IT_CONTRACT INTO LN_CONTRACT INDEX 1.
   CLEAR: IT_CONTRACT, IT_CONTRACT[].
   LOOP AT IT_INDENT_NVEH INTO LN_INDENT_NVEH.
    LN_CONTRACT-VBELN = LN_INDENT_NVEH-ORDER_NO.
    LN_CONTRACT-TQTY  = LN_INDENT_NVEH-QUANTITY1.
    LN_CONTRACT-RQTY  = LN_INDENT_NVEH-BL_QTY1.
    APPEND LN_CONTRACT TO IT_CONTRACT.
   ENDLOOP.
  ENDIF.

 LO_VBELN = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_EXPORT ).
 CLEAR LO_VBELN1. REFRESH LO_VBELN1.
 CLEAR LS_VBELN.

 CLEAR LN_CONTRACT.
```

```
 APPEND LS_VBELN TO LO_VBELN1.
 LOOP AT IT_CONTRACT INTO LN_CONTRACT.
  CLEAR LS_VBELN.
  LS_VBELN = LN_CONTRACT-VBELN.
  APPEND LS_VBELN TO LO_VBELN1.
 ENDLOOP.
 LO_VBELN->bind_table( new_items = LO_VBELN1 set_initial_elements = abap_true ).

 LO_EXPORTING = wd_context->get_child_node( name = wd_this->WDCTX_ENABLE ).
    LO_EXPORTING->set_attribute(
      name = 'CONT'
      value = 'X' ).

ELSE.
 LO_VBELN = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_EXPORT ).
 CLEAR LO_VBELN1. REFRESH LO_VBELN1.
 CLEAR LS_VBELN.
*  CLEAR LO_VBELN. "*** Commented for Matetial Price Group

***** Added for Matetial Price Group START ****
 APPEND LS_VBELN TO LO_VBELN1.
 LOOP AT ZZT178 INTO KT178 WHERE ( KONDM = 'IN' OR KONDM = 'EP' ).
  LS_VBELN = KT178-KONDM.
  APPEND LS_VBELN TO LO_VBELN1.
 ENDLOOP.
***** Added for Matetial Price Group END ****

 LO_VBELN = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_EXPORT ).
 LO_VBELN->bind_table( new_items = LO_VBELN1 set_initial_elements = abap_true ).
 LO_EXPORTING = wd_context->get_child_node( name = wd_this->WDCTX_ENABLE ).
    LO_EXPORTING->set_attribute(
      name = 'CONT'
      value = 'X' ).

ENDIF.

DATA: ACTIVATE_USE(1),
    ACTIVATE_MS(1),
    ACTIVATE_HSD(1).

DATA LO_ND_HEADER_DATA TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_EL_HEADER_DATA TYPE REF TO IF_WD_CONTEXT_ELEMENT.
DATA LS_HEADER_DATA TYPE WD_THIS->Element_header_data.
DATA LV_LOAD_DATE TYPE WD_THIS->Element_header_data-LOAD_DATE.
 LO_ND_HEADER_DATA = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS-
>WDCTX_HEADER_DATA ).
 LO_EL_HEADER_DATA = LO_ND_HEADER_DATA->GET_ELEMENT( ).
* get single attribute
 LO_EL_HEADER_DATA->GET_ATTRIBUTE(
  EXPORTING
   NAME =  `LOAD_DATE`
  IMPORTING
   VALUE = LV_LOAD_DATE ).
```

```
DATA: CHK_DATE_MS TYPE DATUM,
    CHK_DATE_HSD TYPE DATUM.
MOVE '20221101' TO CHK_DATE_MS.
MOVE '20280401' TO CHK_DATE_HSD. "Earlier date was '20230401'

CLEAR: ACTIVATE_USE, ACTIVATE_MS, ACTIVATE_HSD.
IF CNT = 1 AND LV_PROD1 IS NOT INITIAL.
 ACTIVATE_USE = 'X'.
ENDIF.
IF CNT = 2 AND LV_PROD1 IS NOT INITIAL AND LV_PROD2 IS NOT INITIAL.
 ACTIVATE_USE = 'X'.
ENDIF.
IF CNT = 3 AND LV_PROD1 IS NOT INITIAL AND LV_PROD2 IS NOT INITIAL AND LV_PROD3 IS NOT INITIAL.
 ACTIVATE_USE = 'X'.
ENDIF.
IF CNT = 4 AND LV_PROD1 IS NOT INITIAL AND LV_PROD2 IS NOT INITIAL AND LV_PROD3 IS NOT INITIAL
 AND LV_PROD4 IS NOT INITIAL.
 ACTIVATE_USE = 'X'.
ENDIF.
IF CNT = 5 AND LV_PROD1 IS NOT INITIAL AND LV_PROD2 IS NOT INITIAL AND LV_PROD3 IS NOT INITIAL
 AND LV_PROD4 IS NOT INITIAL AND LV_PROD5 IS NOT INITIAL.
 ACTIVATE_USE = 'X'.
ENDIF.
IF CNT = 6 AND LV_PROD1 IS NOT INITIAL AND LV_PROD2 IS NOT INITIAL AND LV_PROD3 IS NOT INITIAL
 AND LV_PROD4 IS NOT INITIAL AND LV_PROD5 IS NOT INITIAL AND LV_PROD6 IS NOT INITIAL.
 ACTIVATE_USE = 'X'.
ENDIF.
IF CNT = 7 AND LV_PROD1 IS NOT INITIAL AND LV_PROD2 IS NOT INITIAL AND LV_PROD3 IS NOT INITIAL
 AND LV_PROD4 IS NOT INITIAL AND LV_PROD5 IS NOT INITIAL AND LV_PROD6 IS NOT INITIAL
 AND LV_PROD7 IS NOT INITIAL.
 ACTIVATE_USE = 'X'.
ENDIF.
IF CNT = 8 AND LV_PROD1 IS NOT INITIAL AND LV_PROD2 IS NOT INITIAL AND LV_PROD3 IS NOT INITIAL
 AND LV_PROD4 IS NOT INITIAL AND LV_PROD5 IS NOT INITIAL AND LV_PROD6 IS NOT INITIAL
 AND LV_PROD7 IS NOT INITIAL AND LV_PROD8 IS NOT INITIAL.
 ACTIVATE_USE = 'X'.
ENDIF.
IF LV_PROD1 = 'MS06' OR LV_PROD2 = 'MS06' OR LV_PROD3 = 'MS06' OR LV_PROD4 = 'MS06'
 OR LV_PROD5 = 'MS06' OR LV_PROD6 = 'MS06' OR LV_PROD7 = 'MS06' OR LV_PROD8 = 'MS06'.
 IF LV_LOAD_DATE >= CHK_DATE_MS.
  SELECT COUNT(*) FROM KNVV WHERE VTWEG = '10' AND SPART = '25' AND KUNNR = LV_KUNNR1.
  IF SY-SUBRC = 0.
   ACTIVATE_MS = 'X'.
  ENDIF.
 ENDIF.
ENDIF.
IF LV_PROD1 = 'HSD06' OR LV_PROD2 = 'HSD06' OR LV_PROD3 = 'HSD06' OR LV_PROD4 = 'HSD06'
 OR LV_PROD5 = 'HSD06' OR LV_PROD6 = 'HSD06' OR LV_PROD7 = 'HSD06' OR LV_PROD8 = 'HSD06'.
 IF LV_LOAD_DATE >= CHK_DATE_HSD.
  SELECT COUNT(*) FROM KNVV WHERE VTWEG = '10' AND SPART = '40' AND KUNNR = LV_KUNNR1.
  IF SY-SUBRC = 0.
   ACTIVATE_HSD = 'X'.
  ENDIF.
```

```
 ENDIF.
ENDIF.

IF ACTIVATE_MS = 'X' OR ACTIVATE_HSD = 'X'.
 ACTIVATE_USE = 'X'.
 ELSE.
  CLEAR ACTIVATE_USE.
ENDIF.


DATA LO_ND_IND_CAT TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_EL_IND_CAT TYPE REF TO IF_WD_CONTEXT_ELEMENT.
DATA LV_IND_CAT TYPE wd_this->ELEMENTS_IND_CAT.
DATA LS_IND_CAT TYPE WD_THIS->Element_ind_cat.
DATA LV_IND_CATEGORY TYPE WD_THIS->Element_ind_cat-IND_CATEGORY.

 LO_ND_IND_CAT = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_IND_CAT ).
 LO_EL_IND_CAT = LO_ND_IND_CAT->GET_ELEMENT( ).
 IF LO_EL_IND_CAT IS INITIAL.
 ENDIF.

 TYPES: BEGIN OF ZIND_USE,
      TEXT(100).
 TYPES  END OF ZIND_USE.
 DATA: IT_IND_USE TYPE TABLE OF ZIND_USE.
 DATA: ZEND_USE TYPE ZIND_USE.

 IF ACTIVATE_USE = 'X'.
  APPEND LV_IND_CATEGORY TO LV_IND_CAT.
  if ZDEPOT = '3100'.
   SELECT DDTEXT FROM DD07T INTO LS_IND_CAT-IND_CATEGORY WHERE
    DOMNAME = 'ZIND_END_USE' AND DOMVALUE_L <> 'NORM_OBLND'
     AND DOMVALUE_L <> 'BRND_OBLND'.
    APPEND LS_IND_CAT TO LV_IND_CAT.
   ENDSELECT.
  endif.
  if ZDEPOT <> '3100'.
   SELECT DDTEXT FROM DD07T INTO LS_IND_CAT-IND_CATEGORY WHERE
    DOMNAME = 'ZIND_END_USE' .
    APPEND LS_IND_CAT TO LV_IND_CAT.
   ENDSELECT.
  endif.
  SORT LV_IND_CAT BY IND_CATEGORY.

  LOOP AT LV_IND_CAT INTO LS_IND_CAT.
   IF LS_IND_CAT IS NOT INITIAL AND ACTIVATE_MS = 'X'.
    CONCATENATE 'MS-' LS_IND_CAT INTO LS_IND_CAT.
    MOVE LS_IND_CAT TO ZEND_USE-TEXT.
    APPEND ZEND_USE TO IT_IND_USE.
   ENDIF.
  ENDLOOP.
** Activate whenever required for HSD from April'2023 START **
  LOOP AT LV_IND_CAT INTO LS_IND_CAT.
   IF LS_IND_CAT IS NOT INITIAL AND ACTIVATE_HSD = 'X'.
```

```
      CONCATENATE 'HSD-' LS_IND_CAT INTO LS_IND_CAT.
      MOVE LS_IND_CAT TO ZEND_USE-TEXT.
      APPEND ZEND_USE TO IT_IND_USE.
     ENDIF.
    ENDLOOP.
** Activate whenever required for HSD from April'2023 END **
    CLEAR: LV_IND_CAT, LV_IND_CAT[], LS_IND_CAT.
    APPEND LS_IND_CAT TO LV_IND_CAT.
    LOOP AT IT_IND_USE INTO ZEND_USE.
     MOVE ZEND_USE-TEXT TO LS_IND_CAT-IND_CATEGORY.
     APPEND LS_IND_CAT TO LV_IND_CAT.
    ENDLOOP.

    LO_ND_IND_CAT->bind_table( new_items = LV_IND_CAT set_initial_elements = abap_true ).
   ENDIF.

   IF ACTIVATE_USE <> 'X'.
    CLEAR: LV_IND_CAT, LV_IND_CAT[].
    LO_ND_IND_CAT->bind_table( new_items = LV_IND_CAT set_initial_elements = abap_true ).
   ENDIF.


  endmethod.
```

- method ONACTIONSELECT_IND_END_USE .

```
   DATA LO_ND_IND_CAT TYPE REF TO IF_WD_CONTEXT_NODE.
    DATA LO_EL_IND_CAT TYPE REF TO IF_WD_CONTEXT_ELEMENT.
    DATA LS_IND_CAT TYPE WD_THIS->Element_ind_cat.
    DATA LV_IND_CATEGORY TYPE WD_THIS->Element_ind_cat-IND_CATEGORY.
    LO_ND_IND_CAT = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_IND_CAT ).
    LO_EL_IND_CAT = LO_ND_IND_CAT->GET_ELEMENT( ).

    LO_EL_IND_CAT->GET_ATTRIBUTE(
     EXPORTING
      NAME = `IND_CATEGORY`
     IMPORTING
      VALUE = LV_IND_CATEGORY ).


     DATA LO_ND_IND_END_USE TYPE REF TO IF_WD_CONTEXT_NODE.
     DATA LO_EL_IND_END_USE TYPE REF TO IF_WD_CONTEXT_ELEMENT.
     DATA LS_IND_END_USE TYPE WD_THIS->Element_ind_end_use.
     DATA LV_IND_USE_MS TYPE WD_THIS->Element_ind_end_use-IND_USE_MS.
     DATA LV_IND_USE_HSD TYPE WD_THIS->Element_ind_end_use-IND_USE_HSD.

     LO_ND_IND_END_USE = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS-
>WDCTX_IND_END_USE ).
     LO_EL_IND_END_USE = LO_ND_IND_END_USE->GET_ELEMENT( ).
     IF LO_EL_IND_END_USE IS INITIAL.
     ENDIF.
*   get single attribute
     LO_EL_IND_END_USE->GET_ATTRIBUTE(
      EXPORTING
       NAME = `IND_USE_MS`
      IMPORTING
```

```
      VALUE = LV_IND_USE_MS ).
    LO_EL_IND_END_USE->GET_ATTRIBUTE(
     EXPORTING
      NAME = `IND_USE_HSD`
     IMPORTING
      VALUE = LV_IND_USE_HSD ).

*   set single attribute
    IF LV_IND_USE_MS IS INITIAL AND LV_IND_CATEGORY+0(3) = 'MS-'.
     MOVE LV_IND_CATEGORY TO LV_IND_USE_MS.
     LO_EL_IND_END_USE->SET_ATTRIBUTE(
      NAME = `IND_USE_MS`
      VALUE = LV_IND_USE_MS ).
    ENDIF.

    IF LV_IND_USE_HSD IS INITIAL AND LV_IND_CATEGORY+0(4) = 'HSD-'.
     MOVE LV_IND_CATEGORY TO LV_IND_USE_HSD.
     LO_EL_IND_END_USE->SET_ATTRIBUTE(
      NAME = `IND_USE_HSD`
      VALUE = LV_IND_USE_HSD ).
    ENDIF.


***To get the products START ***
   DATA LO_ND_PROD1 TYPE REF TO IF_WD_CONTEXT_NODE.
   DATA LO_EL_PROD1 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
   DATA LS_PROD1 TYPE WD_THIS->Element_prod1.
   DATA LV_PRODUCT1 TYPE WD_THIS->Element_prod1-PRODUCT.
   LO_ND_PROD1 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD1 ).
   LO_EL_PROD1 = LO_ND_PROD1->GET_ELEMENT( ).
   LO_EL_PROD1->GET_ATTRIBUTE(
    EXPORTING
     NAME = `PRODUCT`
    IMPORTING
     VALUE = LV_PRODUCT1 ).

   DATA LO_ND_PROD2 TYPE REF TO IF_WD_CONTEXT_NODE.
   DATA LO_EL_PROD2 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
   DATA LS_PROD2 TYPE WD_THIS->Element_prod2.
   DATA LV_PRODUCT2 TYPE WD_THIS->Element_prod2-PRODUCT.
   LO_ND_PROD2 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD2 ).
   LO_EL_PROD2 = LO_ND_PROD2->GET_ELEMENT( ).
   LO_EL_PROD2->GET_ATTRIBUTE(
    EXPORTING
     NAME = `PRODUCT`
    IMPORTING
     VALUE = LV_PRODUCT2 ).

   DATA LO_ND_PROD3 TYPE REF TO IF_WD_CONTEXT_NODE.
   DATA LO_EL_PROD3 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
   DATA LS_PROD3 TYPE WD_THIS->Element_prod3.
   DATA LV_PRODUCT3 TYPE WD_THIS->Element_prod3-PRODUCT.
   LO_ND_PROD3 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD3 ).
   LO_EL_PROD3 = LO_ND_PROD3->GET_ELEMENT( ).
```

```
LO_EL_PROD3->GET_ATTRIBUTE(
 EXPORTING
  NAME = `PRODUCT`
 IMPORTING
  VALUE = LV_PRODUCT3 ).


DATA LO_ND_PROD4 TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_EL_PROD4 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
DATA LS_PROD4 TYPE WD_THIS->Element_prod4.
DATA LV_PRODUCT4 TYPE WD_THIS->Element_prod4-PRODUCT.
LO_ND_PROD4 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD4 ).
LO_EL_PROD4 = LO_ND_PROD4->GET_ELEMENT( ).
LO_EL_PROD4->GET_ATTRIBUTE(
 EXPORTING
  NAME = `PRODUCT`
 IMPORTING
  VALUE = LV_PRODUCT4 ).


DATA LO_ND_PROD5 TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_EL_PROD5 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
DATA LS_PROD5 TYPE WD_THIS->Element_prod5.
DATA LV_PRODUCT5 TYPE WD_THIS->Element_prod5-PRODUCT.
LO_ND_PROD5 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD5 ).
LO_EL_PROD5 = LO_ND_PROD5->GET_ELEMENT( ).
LO_EL_PROD5->GET_ATTRIBUTE(
 EXPORTING
  NAME = `PRODUCT`
 IMPORTING
  VALUE = LV_PRODUCT5 ).


DATA LO_ND_PROD6 TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_EL_PROD6 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
DATA LS_PROD6 TYPE WD_THIS->Element_prod6.
DATA LV_PRODUCT6 TYPE WD_THIS->Element_prod6-PRODUCT.
LO_ND_PROD6 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD6 ).
LO_EL_PROD6 = LO_ND_PROD6->GET_ELEMENT( ).
LO_EL_PROD6->GET_ATTRIBUTE(
 EXPORTING
  NAME = `PRODUCT`
 IMPORTING
  VALUE = LV_PRODUCT6 ).


DATA LO_ND_PROD7 TYPE REF TO IF_WD_CONTEXT_NODE.
DATA LO_EL_PROD7 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
DATA LS_PROD7 TYPE WD_THIS->Element_prod7.
DATA LV_PRODUCT7 TYPE WD_THIS->Element_prod7-PRODUCT.
LO_ND_PROD7 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD7 ).
LO_EL_PROD7 = LO_ND_PROD7->GET_ELEMENT( ).
LO_EL_PROD7->GET_ATTRIBUTE(
 EXPORTING
```

```
    NAME = `PRODUCT`
   IMPORTING
    VALUE = LV_PRODUCT7 ).

  DATA LO_ND_PROD8 TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LO_EL_PROD8 TYPE REF TO IF_WD_CONTEXT_ELEMENT.
  DATA LS_PROD8 TYPE WD_THIS->Element_prod8.
  DATA LV_PRODUCT8 TYPE WD_THIS->Element_prod8-PRODUCT.
  LO_ND_PROD8 = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_PROD8 ).
  LO_EL_PROD8 = LO_ND_PROD8->GET_ELEMENT( ).
  LO_EL_PROD8->GET_ATTRIBUTE(
   EXPORTING
    NAME = `PRODUCT`
   IMPORTING
    VALUE = LV_PRODUCT8 ).
```

****To get the products END ****

```
  DATA LV_IND_CAT TYPE wd_this->ELEMENTS_IND_CAT.
  LO_ND_IND_CAT = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_IND_CAT ).
  LO_EL_IND_CAT = LO_ND_IND_CAT->GET_ELEMENT( ).
  IF LO_EL_IND_CAT IS INITIAL.
  ENDIF.

  TYPES: BEGIN OF ZIND_USE,
      TEXT(100).
  TYPES  END OF ZIND_USE.
  DATA: IT_IND_USE TYPE TABLE OF ZIND_USE.
  DATA: ZEND_USE TYPE ZIND_USE.

  DATA LO_ND_HEADER_DATA TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LO_EL_HEADER_DATA TYPE REF TO IF_WD_CONTEXT_ELEMENT.
  DATA LS_HEADER_DATA TYPE WD_THIS->Element_header_data.
  DATA LV_LOAD_DATE TYPE WD_THIS->Element_header_data-LOAD_DATE.

  LO_ND_HEADER_DATA = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS-
>WDCTX_HEADER_DATA ).
  LO_EL_HEADER_DATA = LO_ND_HEADER_DATA->GET_ELEMENT( ).

  LO_EL_HEADER_DATA->GET_ATTRIBUTE(
    EXPORTING
     NAME = `LOAD_DATE`
    IMPORTING
     VALUE = LV_LOAD_DATE ).


  SELECT DDTEXT FROM DD07T INTO LS_IND_CAT-IND_CATEGORY WHERE
   DOMNAME = 'ZIND_END_USE'.
   APPEND LS_IND_CAT TO LV_IND_CAT.
  ENDSELECT.

  LOOP AT LV_IND_CAT INTO LS_IND_CAT.
```

```
    IF LS_IND_CAT IS NOT INITIAL.
      CONCATENATE 'MS-' LS_IND_CAT INTO LS_IND_CAT.
      MOVE LS_IND_CAT TO ZEND_USE-TEXT.
      APPEND ZEND_USE TO IT_IND_USE.
    ENDIF.
  ENDLOOP.
  LOOP AT LV_IND_CAT INTO LS_IND_CAT.
    IF LS_IND_CAT IS NOT INITIAL AND LV_LOAD_DATE >= '20240401'. " Earlier date 20230401
      CONCATENATE 'HSD-' LS_IND_CAT INTO LS_IND_CAT.
      MOVE LS_IND_CAT TO ZEND_USE-TEXT.
      APPEND ZEND_USE TO IT_IND_USE.
    ENDIF.
  ENDLOOP.
  CLEAR: LV_IND_CAT, LV_IND_CAT[], LS_IND_CAT.
  APPEND LS_IND_CAT TO LV_IND_CAT.
  LOOP AT IT_IND_USE INTO ZEND_USE.
    MOVE ZEND_USE-TEXT TO LS_IND_CAT-IND_CATEGORY.
    IF ( LS_IND_CAT-IND_CATEGORY+0(3) <> LV_IND_USE_MS+0(3) ) AND
      ( LS_IND_CAT-IND_CATEGORY+0(4) <> LV_IND_USE_HSD+0(4) ).
      APPEND LS_IND_CAT TO LV_IND_CAT.
    ENDIF.
  ENDLOOP.

 LO_ND_IND_CAT->bind_table( new_items = LV_IND_CAT set_initial_elements = abap_true ).




endmethod.
```

- method ONACTIONSELECT_MAT_AND_GSTN .

```
  DATA LO_ND_HEADER_DATA TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LO_EL_HEADER_DATA TYPE REF TO IF_WD_CONTEXT_ELEMENT.
  DATA LS_HEADER_DATA TYPE WD_THIS->ELEMENT_HEADER_DATA.
  DATA LV_KUNNR TYPE WD_THIS->ELEMENT_HEADER_DATA-KUNNR.
  LO_ND_HEADER_DATA = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS-
>WDCTX_HEADER_DATA ).
  LO_EL_HEADER_DATA = LO_ND_HEADER_DATA->GET_ELEMENT( ).
  LO_EL_HEADER_DATA->GET_ATTRIBUTE(
   EXPORTING
    NAME = `KUNNR`
   IMPORTING
    VALUE = LV_KUNNR ).

  DATA: ZKDGRP TYPE KDGRP.
  CLEAR ZKDGRP.
  SELECT SINGLE KDGRP FROM KNVV INTO ZKDGRP WHERE KUNNR = LV_KUNNR. "#EC CI_NOORDER " #
S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
  IF ZKDGRP <> 'DI' AND ZKDGRP <> 'EX'.
```

```
   SELECT SINGLE KDGRP FROM KNVV INTO ZKDGRP WHERE KUNNR = LV_KUNNR
     AND ( KDGRP = 'DI' OR KDGRP = 'EX' OR KDGRP = 'OI' OR KDGRP = 'TG' OR KDGRP = 'ON' ).
  ENDIF.
 DATA LO_ND_WV_ENABLE TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_EL_WV_ENABLE TYPE REF TO IF_WD_CONTEXT_ELEMENT.
 DATA LS_WV_ENABLE TYPE WD_THIS->ELEMENT_WV_ENABLE.
 DATA LV_VWEN1 TYPE WD_THIS->ELEMENT_WV_ENABLE-VWEN1.
 DATA LV_VWEN2 TYPE WD_THIS->ELEMENT_WV_ENABLE-VWEN2.
 DATA LV_VWEN3 TYPE WD_THIS->ELEMENT_WV_ENABLE-VWEN3.
 DATA LV_VWEN4 TYPE WD_THIS->ELEMENT_WV_ENABLE-VWEN4.
 LO_ND_WV_ENABLE = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_WV_ENABLE ).
 LO_EL_WV_ENABLE = LO_ND_WV_ENABLE->GET_ELEMENT( ).

 IF ZKDGRP = 'DI' OR ZKDGRP = 'EX' OR ZKDGRP = 'OI' OR ZKDGRP = 'TG' OR ZKDGRP = 'ON'.
  LV_VWEN1 = 'X'. LV_VWEN2 = 'X'.
  ELSE.
   CLEAR LV_VWEN1. CLEAR LV_VWEN2. CLEAR LV_VWEN3. CLEAR LV_VWEN4.
 ENDIF.

 IF ZKDGRP = 'OI' OR ZKDGRP = 'TG' OR ZKDGRP = 'ON'.
  LV_VWEN3 = 'X'. LV_VWEN4 = 'X'.
  ELSE.
   CLEAR LV_VWEN3. CLEAR LV_VWEN4.
 ENDIF.


  LO_EL_WV_ENABLE->SET_ATTRIBUTE(
   NAME = `VWEN1`
   VALUE = LV_VWEN1 ).

  LO_EL_WV_ENABLE->SET_ATTRIBUTE(
   NAME = `VWEN2`
   VALUE = LV_VWEN2 ).

  LO_EL_WV_ENABLE->SET_ATTRIBUTE(
   NAME = `VWEN3`
   VALUE = LV_VWEN3 ).

  LO_EL_WV_ENABLE->SET_ATTRIBUTE(
   NAME = `VWEN4`
   VALUE = LV_VWEN4 ).




 DATA:  imat_data type REF TO IF_WD_CONTEXT_NODE,
      imat_data1 TYPE wd_this->Elements_material1,
      lmat_data1 LIKE LINE OF imat_data1,
      imat_data2 TYPE wd_this->Elements_material2.

 imat_data = wd_context->path_get_node( path = `MATERIAL1` ).
 select * into corresponding fields of table imat_data1 from ZSD_INDT_NO_VEH WHERE CUST_GROUP = ZK
DGRP.
 insert lmat_data1 into imat_data1 index 1.
```

```
 imat_data->bind_table( new_items = imat_data1 set_initial_elements = abap_true ).

 if ZKDGRP <> 'DI' AND ZKDGRP <> 'EX'.
    clear imat_data.
    imat_data = wd_context->path_get_node( path = `MATERIAL2` ).
    select * into corresponding fields of table imat_data2 from ZSD_INDT_NO_VEH WHERE CUST_GROUP =
ZKDGRP.
    insert lmat_data1 into imat_data2 index 1.
    imat_data->bind_table( new_items = imat_data2 set_initial_elements = abap_true ).
 endif.

 DATA lr_event1 TYPE REF TO cl_wd_custom_event.
 DATA lr_event TYPE REF TO cl_wd_custom_event.
*  IF LV_VBELN IS INITIAL.
    CREATE OBJECT lr_event
      EXPORTING
       name = 'ACTIVATE_GSTN'.
    wd_this->ONACTIONACTIVATE_GSTN(
     wdevent = lr_event1                    " ref to cl_wd_custom_event
    ).

endmethod.
```

- method ONACTIONSET_VEHCILE_DATE .

```
  DATA LO_ND_INBND_HEADER TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LO_EL_INBND_HEADER TYPE REF TO IF_WD_CONTEXT_ELEMENT.
  DATA LS_INBND_HEADER TYPE WD_THIS->Element_inbnd_header.
  DATA LV_UNLD_ACT1 TYPE WD_THIS->Element_inbnd_header-UNLD_ACT1.
  DATA LV_UNLD_MATNR TYPE WD_THIS->Element_inbnd_header-UNLD_MATNR.
  DATA LV_UNLD_STOCK TYPE WD_THIS->Element_inbnd_header-UNLD_STOCK.
  LO_ND_INBND_HEADER = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS-
>WDCTX_INBND_HEADER ).
  LO_EL_INBND_HEADER = LO_ND_INBND_HEADER->GET_ELEMENT( ).
  LO_EL_INBND_HEADER->GET_ATTRIBUTE(
    EXPORTING
     NAME =  `UNLD_MATNR`
    IMPORTING
     VALUE = LV_UNLD_MATNR ).

  IF LV_UNLD_MATNR <> ' '.
   LV_UNLD_ACT1  = 'X'.
  ENDIF.
  LO_EL_INBND_HEADER->SET_ATTRIBUTE(
   NAME =  `UNLD_ACT1`
   VALUE = LV_UNLD_ACT1 ).

*   IF LV_UNLD_MATNR = ' '.
    LO_EL_INBND_HEADER->GET_ATTRIBUTE(
     EXPORTING
      NAME =  `UNLD_STOCK`
     IMPORTING
      VALUE = LV_UNLD_STOCK ).
```

```
    CLEAR LV_UNLD_STOCK.
    LO_EL_INBND_HEADER->SET_ATTRIBUTE(
      NAME =  `UNLD_STOCK`
      VALUE = LV_UNLD_STOCK ).
*   ENDIF.
```

```
  endmethod.
```

- method ONACTIONSHORT_CLOSE_ORDER .

```
   DATA LO_WINDOW_MANAGER TYPE REF TO IF_WD_WINDOW_MANAGER.
    DATA LO_API_COMPONENT  TYPE REF TO IF_WD_COMPONENT.
    DATA LO_WINDOW       TYPE REF TO IF_WD_WINDOW.
    data: lt_string type table of string,
        ls_string like line of lt_string.
    DATA l_api          TYPE REF TO if_wd_view_controller.
    DATA: CNT TYPE I.

    DATA LO_ND_CHANGING_2 TYPE REF TO IF_WD_CONTEXT_NODE.
    DATA LT_CHANGING_2 TYPE WD_THIS->ELEMENTS_CHANGING_2.
    DATA LS_CHANGING_2 TYPE WD_THIS->ELEMENT_CHANGING_2.
    LO_ND_CHANGING_2 = WD_CONTEXT-
>PATH_GET_NODE( PATH = `ZSD_INDENT_NON_VEH.CHANGING_2` ).
    LO_ND_CHANGING_2-
>GET_STATIC_ATTRIBUTES_TABLE( IMPORTING TABLE = LT_CHANGING_2 ).  "#EC CI_FLDEXT_OK[2215424]
" #S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18

    CLEAR CNT.
    LOOP AT LT_CHANGING_2 INTO LS_CHANGING_2 WHERE CHK = 'X'.
     CNT = CNT + 1.
    ENDLOOP.

    LO_API_COMPONENT        = WD_COMP_CONTROLLER->WD_GET_API( ).
    LO_WINDOW_MANAGER       = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).
    CLEAR: ls_string, lt_string.
    REFRESH lt_string.

    IF CNT = 0.
     ls_string = 'Please select the order to be closed'.
     insert ls_string into table lt_string.

     ELSEIF CNT > 1.
      ls_string = 'Please select only one order'.
      insert ls_string into table lt_string.
    ENDIF.


    IF CNT <> 1.
     LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
           text        = lt_string
```

```
            button_kind    = if_wd_window=>co_buttons_ok
            message_type   = if_wd_window=>CO_MSG_TYPE_STOPP
            window_title   = 'Please check!'
            window_position = if_wd_window=>co_center ).

     LO_WINDOW->open( ).
    ENDIF.

    IF CNT = 1 .
     CLEAR: ls_string, lt_string. REFRESH lt_string.
     ls_string = 'Selected order will be closed?'.
     insert ls_string into table lt_string.

      LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
            text         = lt_string
            button_kind    = 4
            message_type   = if_wd_window=>CO_MSG_TYPE_QUESTION
            window_title   = 'Please check!'
            window_position = if_wd_window=>co_center ).

    DATA LO_API_START_VIEW TYPE REF TO IF_WD_VIEW_CONTROLLER.
    LO_API_START_VIEW = WD_THIS->WD_GET_API( ).

    CALL METHOD LO_WINDOW->SUBSCRIBE_TO_BUTTON_EVENT
      EXPORTING
       BUTTON    = if_wd_window=>co_button_yes
       ACTION_NAME = 'REACT_TO_CLOSE_ORDER'
       ACTION_VIEW = LO_API_START_VIEW.

     LO_WINDOW->open( ).

     CALL METHOD LO_WINDOW_MANAGER->CREATE_POPUP_TO_CONFIRM
       EXPORTING
        TEXT             = lt_string
        BUTTON_KIND        = 4
       RECEIVING
        RESULT           = lo_window.


    ENDIF.
endmethod.
```

- method ONACTIONSUBMIT_INDENT .

```
 DATA lt_indent TYPE ig_componentcontroller=>Elements_itab_indent.
 DATA ls_indent TYPE ig_componentcontroller=>Element_itab_indent.
 DATA LO_Z_GET_CUSTOMER_INDEN TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_Z_GET_CUSTOMER_INDEN_CHANG TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA LO_IMPORTING TYPE REF TO IF_WD_CONTEXT_NODE.
 DATA: ITABWGT TYPE ZSAUTOMATETT_LPG,
     ITABVOL TYPE ZSAUTOMATETT_TBL.
 DATA VEH_TYPE TYPE OIGV-VEH_TYPE.
 DATA: QUAN1 TYPE ZSD_CUST_INDENT-QUAN_COMP1,
     QUAN2 TYPE ZSD_CUST_INDENT-QUAN_COMP2,
     QUAN3 TYPE ZSD_CUST_INDENT-QUAN_COMP3,
     QUAN4 TYPE ZSD_CUST_INDENT-QUAN_COMP4,
     QUAN5 TYPE ZSD_CUST_INDENT-QUAN_COMP5,
     QUAN6 TYPE ZSD_CUST_INDENT-QUAN_COMP6,
     QUAN7 TYPE ZSD_CUST_INDENT-QUAN_COMP7,
     QUAN8 TYPE ZSD_CUST_INDENT-QUAN_COMP8.

 DATA: CT TYPE I.
 DATA LV_ALL_SUBMIT TYPE CHAR1.
 DATA: LO_ERROR TYPE REF TO IF_WD_CONTEXT_NODE.

 LO_ERROR = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_ERROR ).
 LO_ERROR->GET_ATTRIBUTE(
  EXPORTING
   NAME  = `ALL_SUBMIT`
  IMPORTING
   VALUE = LV_ALL_SUBMIT ).

 LO_Z_GET_CUSTOMER_INDEN = WD_CONTEXT->GET_CHILD_NODE( WD_THIS-
>WDCTX_Z_GET_CUSTOMER_INDEN ).
 LO_Z_GET_CUSTOMER_INDEN_CHANG = LO_Z_GET_CUSTOMER_INDEN->GET_CHILD_NODE( WD_THIS-
>WDCTX_CHANGING_1 ).
 LO_IMPORTING = LO_Z_GET_CUSTOMER_INDEN_CHANG->GET_CHILD_NODE( WD_THIS-
>WDCTX_ITAB_INDENT ).  "#EC CI_FLDEXT_OK[2215424] " #S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2
021/08/18
 LO_IMPORTING-
>get_static_attributes_table( IMPORTING table = lt_indent ).  "#EC CI_FLDEXT_OK[2215424] " #S/4 - OSS No
te# - Added by - Z_SHSHRUKHA – 2021/08/18
 DATA: INDX TYPE I.

 IF LV_ALL_SUBMIT = 'X'.
  LOOP AT LT_INDENT INTO LS_INDENT.
   CLEAR LS_INDENT-CHK.
   MODIFY LT_INDENT FROM LS_INDENT.
  ENDLOOP.
 ENDIF.

 CLEAR INDX.
 LOOP AT LT_INDENT INTO LS_INDENT WHERE ZTT_STATUS = '4' AND CHK <> LV_ALL_SUBMIT.
```

```
  INDX = INDX + 1.

  DATA LO_ND_Z_CHECK TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LO_EL_Z_CHECK TYPE REF TO IF_WD_CONTEXT_ELEMENT.
  DATA LS_Z_CHECK TYPE WD_THIS->ELEMENT_Z_CHECK.
  DATA LV_Z_ERROR TYPE WD_THIS->ELEMENT_Z_CHECK-Z_ERROR.

  LO_ND_Z_CHECK = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_Z_CHECK ).
  LO_EL_Z_CHECK = LO_ND_Z_CHECK->GET_ELEMENT( ).
  IF LO_EL_Z_CHECK IS INITIAL.
  ENDIF.
  CLEAR LV_Z_ERROR.
  LO_EL_Z_CHECK->SET_ATTRIBUTE(
   NAME = `Z_ERROR`
   VALUE = LV_Z_ERROR ).

   DATA LO_ND_WV_ENABLE TYPE REF TO IF_WD_CONTEXT_NODE.
   DATA LO_EL_WV_ENABLE TYPE REF TO IF_WD_CONTEXT_ELEMENT.
   DATA LS_WV_ENABLE TYPE WD_THIS->ELEMENT_WV_ENABLE.
   DATA LV_VW_SV2 TYPE WD_THIS->ELEMENT_WV_ENABLE-VW_SV2.
   LO_ND_WV_ENABLE = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS->WDCTX_WV_ENABLE ).
   LO_EL_WV_ENABLE = LO_ND_WV_ENABLE->GET_ELEMENT( ).
   IF LO_EL_WV_ENABLE IS NOT INITIAL.
   LV_VW_SV2 = 'X'.
   LO_EL_WV_ENABLE->SET_ATTRIBUTE(
    NAME = `VW_SV2`
    VALUE = LV_VW_SV2 ).
   ENDIF.



  DATA LO_ND_ITAB_INDENT TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LT_ITAB_INDENT TYPE WD_THIS->ELEMENTS_ITAB_INDENT.
  LO_ND_ITAB_INDENT = WD_CONTEXT-
>PATH_GET_NODE( PATH = `Z_GET_CUSTOMER_INDEN.CHANGING_1.ITAB_INDENT` ).
  CLEAR LT_ITAB_INDENT. REFRESH LT_ITAB_INDENT.
  APPEND LS_INDENT TO LT_ITAB_INDENT.
  LO_ND_ITAB_INDENT-
>BIND_TABLE( NEW_ITEMS = LT_ITAB_INDENT SET_INITIAL_ELEMENTS = ABAP_TRUE ).

  DATA ZLO_COMPONENTCONTROLLER TYPE REF TO IG_COMPONENTCONTROLLER .
  ZLO_COMPONENTCONTROLLER =   WD_THIS->GET_COMPONENTCONTROLLER_CTR( ).

  ZLO_COMPONENTCONTROLLER->EXECUTE_Z_IND_QTY_CHECK_SUBMIT(
  ).



  LO_ND_ITAB_INDENT = WD_CONTEXT-
>PATH_GET_NODE( PATH = `Z_GET_CUSTOMER_INDEN.CHANGING_1.ITAB_INDENT` ).  "#EC CI_FLDEXT_
OK[2215424] " #S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
  LO_ND_ITAB_INDENT-
>GET_STATIC_ATTRIBUTES_TABLE( IMPORTING TABLE = LT_ITAB_INDENT ).   "#EC CI_FLDEXT_OK[2215424]
" #S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
  CLEAR LS_INDENT.
```

```
*   READ TABLE LT_ITAB_INDENT INTO LS_INDENT INDEX INDX.
  READ TABLE LT_ITAB_INDENT INTO LS_INDENT INDEX 1.

  LO_EL_Z_CHECK->GET_ATTRIBUTE(
  EXPORTING
   NAME =  `Z_ERROR`
  IMPORTING
   VALUE = LV_Z_ERROR ).

  DATA: LV_ERCODE4 TYPE CHAR1.
  LV_ERCODE4 = LV_Z_ERROR.

 IF LV_ERCODE4 <> 'X'.
  CLEAR: QUAN1, QUAN2, QUAN3, QUAN4, QUAN5, QUAN6, QUAN7, QUAN8.
  QUAN1 = LS_INDENT-QUAN_COMP1.
  QUAN2 = LS_INDENT-QUAN_COMP2.
  QUAN3 = LS_INDENT-QUAN_COMP3.
  QUAN4 = LS_INDENT-QUAN_COMP4.
  QUAN5 = LS_INDENT-QUAN_COMP5.
  QUAN6 = LS_INDENT-QUAN_COMP6.
  QUAN7 = LS_INDENT-QUAN_COMP7.
  QUAN8 = LS_INDENT-QUAN_COMP8.

  CT = STRLEN( LS_INDENT-QUAN_COMP1 ). CT = CT - 2.
  LS_INDENT-QUAN_COMP1 = LS_INDENT-QUAN_COMP1(CT). CLEAR CT.
  CT = STRLEN( LS_INDENT-QUAN_COMP2 ). CT = CT - 2.
  LS_INDENT-QUAN_COMP2 = LS_INDENT-QUAN_COMP2(CT). CLEAR CT.
  CT = STRLEN( LS_INDENT-QUAN_COMP3 ). CT = CT - 2.
  LS_INDENT-QUAN_COMP3 = LS_INDENT-QUAN_COMP3(CT). CLEAR CT.
  CT = STRLEN( LS_INDENT-QUAN_COMP4 ). CT = CT - 2.
  LS_INDENT-QUAN_COMP4 = LS_INDENT-QUAN_COMP4(CT). CLEAR CT.
  CT = STRLEN( LS_INDENT-QUAN_COMP5 ). CT = CT - 2.
  LS_INDENT-QUAN_COMP5 = LS_INDENT-QUAN_COMP5(CT). CLEAR CT.
  CT = STRLEN( LS_INDENT-QUAN_COMP6 ). CT = CT - 2.
  LS_INDENT-QUAN_COMP6 = LS_INDENT-QUAN_COMP6(CT). CLEAR CT.
  CT = STRLEN( LS_INDENT-QUAN_COMP7 ). CT = CT - 2.
  LS_INDENT-QUAN_COMP7 = LS_INDENT-QUAN_COMP7(CT). CLEAR CT.
  CT = STRLEN( LS_INDENT-QUAN_COMP8 ). CT = CT - 2.
  LS_INDENT-QUAN_COMP8 = LS_INDENT-QUAN_COMP8(CT). CLEAR CT.

* To check license details for indents received in B2B START ***
  DATA LV_ERROR TYPE CHAR1.
  DATA: LT_C_RETURN TYPE TABLE OF BAPIRET2.
  DATA: ZRETURN LIKE LINE OF LT_C_RETURN.
  data: lt_string type table of string,
      ls_string like line of lt_string.
  DATA LO_WINDOW_MANAGER TYPE REF TO IF_WD_WINDOW_MANAGER.
  DATA LO_API_COMPONENT  TYPE REF TO IF_WD_COMPONENT.
  DATA LO_WINDOW       TYPE REF TO IF_WD_WINDOW.

  IF LS_INDENT-INDENT_TYPE = 'B2B'.
   CALL FUNCTION 'Z_CHECK_VEHICLE_LICENSE'
      EXPORTING
        VEHICLE  =  LS_INDENT-VEHICLE
```

```
    IMPORTING
      ERROR   =  LV_ERROR
    TABLES
      RETURN   =  LT_C_RETURN.

  if LV_ERROR = 'X' .
   LOOP AT LT_C_RETURN INTO ZRETURN.
    ls_string = ZRETURN-MESSAGE.
    insert ls_string into table lt_string.
   ENDLOOP.

   LO_API_COMPONENT        = WD_COMP_CONTROLLER->WD_GET_API( ).
   LO_WINDOW_MANAGER       = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).
   LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
               text         = lt_string
               button_kind    = if_wd_window=>co_buttons_ok
               message_type    = if_wd_window=>CO_MSG_TYPE_ERROR
               window_title    = 'Please check!'
               window_position = if_wd_window=>co_center
               WINDOW_WIDTH    = '50%'
               WINDOW_HEIGHT   = '50%' ).
   LO_WINDOW->open( ).
  endif.

 ENDIF.
* To check license details for indents received in B2B END ***


  SELECT SINGLE VEH_TYPE FROM OIGV INTO VEH_TYPE WHERE VEHICLE = LS_INDENT-VEHICLE.
  IF VEH_TYPE = 'TTV' OR VEH_TYPE = 'ATF' OR VEH_TYPE = 'WOIL' OR VEH_TYPE = 'TTVM'.
    ITABVOL-BEGDA = LS_INDENT-BEGDA.
    ITABVOL-INDENT_DATE = LS_INDENT-INDENT_DATE.
    ITABVOL-INDENT_TIME = LS_INDENT-INDENT_TIME.
    ITABVOL-VEHICLE = LS_INDENT-VEHICLE.
    ITABVOL-DEPOT = LS_INDENT-DEPOT.
    ITABVOL-CUST_USER_ID = LS_INDENT-CUST_USER_ID.
    ITABVOL-KUNNR = LS_INDENT-KUNNR.
    ITABVOL-SHTYPE = LS_INDENT-SHTYPE.
    ITABVOL-PROD_CMP1 = LS_INDENT-PROD_CMP1.
    ITABVOL-VAL_COMP1 = LS_INDENT-VAL_COMP1.
    ITABVOL-QUAN_COMP1 = LS_INDENT-QUAN_COMP1.
    ITABVOL-PROD_CMP2 = LS_INDENT-PROD_CMP2.
    ITABVOL-VAL_COMP2 = LS_INDENT-VAL_COMP2.
    ITABVOL-QUAN_COMP2 = LS_INDENT-QUAN_COMP2.
    ITABVOL-PROD_CMP3 = LS_INDENT-PROD_CMP3.
    ITABVOL-VAL_COMP3 = LS_INDENT-VAL_COMP3.
    ITABVOL-QUAN_COMP3 = LS_INDENT-QUAN_COMP3.
    ITABVOL-PROD_CMP4 = LS_INDENT-PROD_CMP4.
    ITABVOL-VAL_COMP4 = LS_INDENT-VAL_COMP4.
    ITABVOL-QUAN_COMP4 = LS_INDENT-QUAN_COMP4.
    ITABVOL-PROD_CMP5 = LS_INDENT-PROD_CMP5.
    ITABVOL-VAL_COMP5 = LS_INDENT-VAL_COMP5.
    ITABVOL-QUAN_COMP5 = LS_INDENT-QUAN_COMP5.
    ITABVOL-PROD_CMP6 = LS_INDENT-PROD_CMP6.
```

```
ITABVOL-VAL_COMP6 = LS_INDENT-VAL_COMP6.
ITABVOL-QUAN_COMP6 = LS_INDENT-QUAN_COMP6.
ITABVOL-PROD_CMP7 = LS_INDENT-PROD_CMP7.
ITABVOL-VAL_COMP7 = LS_INDENT-VAL_COMP7.
ITABVOL-QUAN_COMP7 = LS_INDENT-QUAN_COMP7.
ITABVOL-PROD_CMP8 = LS_INDENT-PROD_CMP8.
ITABVOL-VAL_COMP8 = LS_INDENT-VAL_COMP8.
ITABVOL-QUAN_COMP8 = LS_INDENT-QUAN_COMP8.
ITABVOL-ZTRAN     = LS_INDENT-ZTRAN.

ITABVOL-ZTT_STATUS = '5'.
ITABVOL-CONTRACT = LS_INDENT-CONTRACT.
IF LS_INDENT-KONDM = '00'. CLEAR LS_INDENT-KONDM. ENDIF.
ITABVOL-KONDM = LS_INDENT-KONDM.
ITABVOL-KONDM2 = LS_INDENT-KONDM2.
ITABVOL-ATF_FLASH = LS_INDENT-ATF_FLUSH.

data: zstk_err(1),
    ZERR_MSG TYPE STRING.
DATA: C1_QT TYPE LABST,
    C2_QT TYPE LABST,
    C3_QT TYPE LABST,
    C4_QT TYPE LABST,
    C5_QT TYPE LABST,
    C6_QT TYPE LABST,
    C7_QT TYPE LABST,
    C8_QT TYPE LABST.

 MOVE LS_INDENT-QUAN_COMP1 TO C1_QT.
 MOVE LS_INDENT-QUAN_COMP2 TO C2_QT.
 MOVE LS_INDENT-QUAN_COMP3 TO C3_QT.
 MOVE LS_INDENT-QUAN_COMP4 TO C4_QT.
 MOVE LS_INDENT-QUAN_COMP5 TO C5_QT.
 MOVE LS_INDENT-QUAN_COMP6 TO C6_QT.
 MOVE LS_INDENT-QUAN_COMP7 TO C7_QT.
 MOVE LS_INDENT-QUAN_COMP8 TO C8_QT.

 CLEAR ZSTK_ERR.
 IF LS_INDENT-INDENT_TYPE = 'B2B'.
  CALL FUNCTION 'Z_ETHANOL_INDENT_STK_CHECK'
      EXPORTING
        CUST_INDENT = LS_INDENT
        C1_QT     = C1_QT
        C2_QT     = C2_QT
        C3_QT     = C3_QT
        C4_QT     = C4_QT
        C5_QT     = C5_QT
        C6_QT     = C6_QT
        C7_QT     = C7_QT
        C8_QT     = C8_QT
      IMPORTING
        ERR_MSG    = ZERR_MSG
        ZSTK_ERR   = ZSTK_ERR .
```

```
        if ZSTK_ERR = 'X'.
         insert ZERR_MSG into table lt_string.
         LO_API_COMPONENT         = WD_COMP_CONTROLLER->WD_GET_API( ).
         LO_WINDOW_MANAGER        = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).

         LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
         text         = lt_string
         button_kind    = if_wd_window=>co_buttons_ok
*         button_kind    = 4
         message_type   = if_wd_window=>CO_MSG_TYPE_STOPP
         window_title   = 'Please check!'
         window_position = if_wd_window=>co_center
         WINDOW_WIDTH   = '20%'
         WINDOW_HEIGHT  = '20%' ).
         LO_WINDOW->open( ).
        endif.

       ENDIF.


      IF ( LV_ALL_SUBMIT = 'X' OR ( LV_ALL_SUBMIT = ' ' AND LS_INDENT-CHK = 'X' ) )
       AND LV_ERROR <> 'X' AND ZSTK_ERR = ' '.
        INSERT ZSAUTOMATETT_TBL FROM ITABVOL.
        IF SY-SUBRC = 0.
         clear LS_INDENT-ERROR.
         LS_INDENT-QUAN_COMP1 = QUAN1.
         LS_INDENT-QUAN_COMP2 = QUAN2.
         LS_INDENT-QUAN_COMP3 = QUAN3.
         LS_INDENT-QUAN_COMP4 = QUAN4.
         LS_INDENT-QUAN_COMP5 = QUAN5.
         LS_INDENT-QUAN_COMP6 = QUAN6.
         LS_INDENT-QUAN_COMP7 = QUAN7.
         LS_INDENT-QUAN_COMP8 = QUAN8.
         LS_INDENT-ZTT_STATUS = '5'.
         LS_INDENT-ZTT_STATUS_DESC = 'Indent Submitted'.
         CLEAR LS_INDENT-CHK.
         MODIFY ZSD_CUST_INDENT FROM LS_INDENT.
        ENDIF.
       ENDIF.

     ELSE.
      ITABWGT-BEGDA = LS_INDENT-BEGDA.
      ITABWGT-INDENT_DATE = LS_INDENT-INDENT_DATE.
      ITABWGT-INDENT_TIME = LS_INDENT-INDENT_TIME.
      ITABWGT-VEHICLE = LS_INDENT-VEHICLE.
      ITABWGT-DEPOT = LS_INDENT-DEPOT.
      ITABWGT-CUST_USER_ID = LS_INDENT-CUST_USER_ID.
      ITABWGT-KUNNR = LS_INDENT-KUNNR.
      ITABWGT-SHTYPE = LS_INDENT-SHTYPE.
      ITABWGT-PROD_CMP1 = LS_INDENT-PROD_CMP1.
      ITABWGT-PROD_QUAN1 = LS_INDENT-QUAN_COMP1.
      ITABWGT-PROD_CMP2 = LS_INDENT-PROD_CMP2.
      ITABWGT-PROD_QUAN2 = LS_INDENT-QUAN_COMP2.
      ITABWGT-PROD_CMP3 = LS_INDENT-PROD_CMP3.
```

```
      ITABWGT-PROD_QUAN3 = LS_INDENT-QUAN_COMP3.
      ITABWGT-PROD_CMP4 = LS_INDENT-PROD_CMP4.
      ITABWGT-PROD_QUAN4 = LS_INDENT-QUAN_COMP4.
      ITABWGT-PROD_CMP5 = LS_INDENT-PROD_CMP5.
      ITABWGT-PROD_QUAN5 = LS_INDENT-QUAN_COMP5.
      ITABWGT-PROD_CMP6 = LS_INDENT-PROD_CMP6.
      ITABWGT-PROD_QUAN6 = LS_INDENT-QUAN_COMP6.
      ITABWGT-PROD_CMP7 = LS_INDENT-PROD_CMP7.
      ITABWGT-PROD_QUAN7 = LS_INDENT-QUAN_COMP7.
      ITABWGT-PROD_CMP8 = LS_INDENT-PROD_CMP8.
      ITABWGT-PROD_QUAN8 = LS_INDENT-QUAN_COMP8.

      ITABWGT-ZTT_STATUS = '5'.
      ITABWGT-CONTRACT = LS_INDENT-CONTRACT.
      IF LS_INDENT-KONDM = '00'. CLEAR LS_INDENT-KONDM. ENDIF.
      ITABWGT-KONDM = LS_INDENT-KONDM.
      IF LV_ALL_SUBMIT = 'X' OR ( LV_ALL_SUBMIT = ' ' AND LS_INDENT-CHK = 'X' ).
       INSERT ZSAUTOMATETT_LPG FROM ITABWGT.
       IF SY-SUBRC = 0.
        LS_INDENT-QUAN_COMP1 = QUAN1.
        LS_INDENT-QUAN_COMP2 = QUAN2.
        LS_INDENT-QUAN_COMP3 = QUAN3.
        LS_INDENT-QUAN_COMP4 = QUAN4.
        LS_INDENT-QUAN_COMP5 = QUAN5.
        LS_INDENT-QUAN_COMP6 = QUAN6.
        LS_INDENT-QUAN_COMP7 = QUAN7.
        LS_INDENT-QUAN_COMP8 = QUAN8.
        LS_INDENT-ZTT_STATUS = '5'.
        LS_INDENT-ZTT_STATUS_DESC = 'Indent Submitted'.
        CLEAR LS_INDENT-CHK.
        MODIFY ZSD_CUST_INDENT FROM LS_INDENT.
       ENDIF.
      ENDIF.
    ENDIF.
   ENDIF.
  ENDLOOP.

  DATA LO_COMPONENTCONTROLLER TYPE REF TO IG_COMPONENTCONTROLLER .
  LO_COMPONENTCONTROLLER =   WD_THIS->GET_COMPONENTCONTROLLER_CTR( ).

  LO_COMPONENTCONTROLLER->EXECUTE_Z_GET_CUSTOMER_INDENT(
  ).
 endmethod.
```

- method ONACTIONSUBMIT_INDENT_ALL .

```
  DATA LV_ALL_SUBMIT TYPE CHAR1.
  DATA: LO_ERROR TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA lr_event TYPE REF TO cl_wd_custom_event.

  LO_ERROR = WD_CONTEXT->GET_CHILD_NODE( WD_THIS->WDCTX_ERROR ).
  LV_ALL_SUBMIT = 'X'.
  LO_ERROR->SET_ATTRIBUTE(
    EXPORTING
```

```
      NAME  = `ALL_SUBMIT`
      VALUE = LV_ALL_SUBMIT ).

 CREATE OBJECT lr_event
   EXPORTING
    name = 'SUBMIT_INDENT'.

  WD_THIS->ONACTIONSUBMIT_INDENT(
   WDEVENT = lr_event
  ).

 CLEAR LV_ALL_SUBMIT .
 LO_ERROR->SET_ATTRIBUTE(
   EXPORTING
    NAME  = `ALL_SUBMIT`
    VALUE = LV_ALL_SUBMIT ).

 DATA LO_COMPONENTCONTROLLER TYPE REF TO IG_COMPONENTCONTROLLER .
 LO_COMPONENTCONTROLLER =   WD_THIS->GET_COMPONENTCONTROLLER_CTR( ).

 LO_COMPONENTCONTROLLER->EXECUTE_Z_GET_CUSTOMER_INDENT(
 ).
endmethod.
```

- method ONACTIONUNLD_ALL_INDENTS .

```
  DATA LO_ND_INBND_HEADER TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LO_EL_INBND_HEADER TYPE REF TO IF_WD_CONTEXT_ELEMENT.
  DATA LS_INBND_HEADER TYPE WD_THIS->Element_inbnd_header.
  DATA LV_UNLD_DATE TYPE WD_THIS->Element_inbnd_header-UNLD_DATE.
  DATA LV_UNLD_TDATE TYPE WD_THIS->Element_inbnd_header-UNLD_TDATE.
  DATA LV_UNLD_ACT3 TYPE WD_THIS->Element_inbnd_header-UNLD_ACT3.
  DATA LV_UNLD_ACT2 TYPE WD_THIS->Element_inbnd_header-UNLD_ACT2.
  DATA LV_MOD_QTY_KL TYPE WD_THIS->Element_inbnd_header-MOD_QTY_KL.
  DATA LV_MOD_QTY_KL15 TYPE WD_THIS->Element_inbnd_header-MOD_QTY_KL15.
  DATA LV_MOD_QTY_MT TYPE WD_THIS->Element_inbnd_header-MOD_QTY_MT.

  LO_ND_INBND_HEADER = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS-
>WDCTX_INBND_HEADER ).
  LO_EL_INBND_HEADER = LO_ND_INBND_HEADER->GET_ELEMENT( ).

  DATA LO_WINDOW_MANAGER TYPE REF TO IF_WD_WINDOW_MANAGER.
  DATA LO_API_COMPONENT  TYPE REF TO IF_WD_COMPONENT.
  DATA LO_WINDOW       TYPE REF TO IF_WD_WINDOW.
  DATA: lt_string type table of string,
     ls_string like line of lt_string.
  DATA l_api          TYPE REF TO if_wd_view_controller.
  DATA: ERR.

* get single attribute
  LO_EL_INBND_HEADER->GET_ATTRIBUTE(
   EXPORTING
    NAME =  `UNLD_DATE`
   IMPORTING
    VALUE = LV_UNLD_DATE ).
```

```
LO_EL_INBND_HEADER->GET_ATTRIBUTE(
 EXPORTING
  NAME = `UNLD_TDATE`
 IMPORTING
  VALUE = LV_UNLD_TDATE ).

CLEAR LV_UNLD_ACT3.
LO_EL_INBND_HEADER->SET_ATTRIBUTE(
  NAME = `UNLD_ACT3`
  VALUE = LV_UNLD_ACT3 ).

CLEAR LV_UNLD_ACT2.
LO_EL_INBND_HEADER->SET_ATTRIBUTE(
  NAME = `UNLD_ACT2`
  VALUE = LV_UNLD_ACT2 ).

CLEAR: LV_MOD_QTY_KL, LV_MOD_QTY_KL15, LV_MOD_QTY_MT.
LO_EL_INBND_HEADER->SET_ATTRIBUTE(
  NAME = `MOD_QTY_KL`
  VALUE = LV_MOD_QTY_KL ).

LO_EL_INBND_HEADER->SET_ATTRIBUTE(
  NAME = `MOD_QTY_KL15`
  VALUE = LV_MOD_QTY_KL15 ).

LO_EL_INBND_HEADER->SET_ATTRIBUTE(
  NAME = `MOD_QTY_MT`
  VALUE = LV_MOD_QTY_MT ).

IF LV_UNLD_DATE IS INITIAL OR LV_UNLD_TDATE IS INITIAL.
 ls_string = 'Please enter date range'.
 ERR = 'X'.
ENDIF.

IF LV_UNLD_DATE > LV_UNLD_TDATE .
 ls_string = 'From date cannot be greater than To date'.
 ERR = 'X'.
ENDIF.


IF  ERR = 'X'.
 insert ls_string into table lt_string.
 LO_API_COMPONENT        = WD_COMP_CONTROLLER->WD_GET_API( ).
 LO_WINDOW_MANAGER       = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).

 LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
        text         = lt_string
        button_kind    = if_wd_window=>co_buttons_ok
*         button_kind    = 4
        message_type   = if_wd_window=>CO_MSG_TYPE_STOPP
        window_title   = 'Please check!'
        window_position = if_wd_window=>co_center
        WINDOW_WIDTH   = '20%'
        WINDOW_HEIGHT  = '20%' ).
```

```
    LO_WINDOW->open( ).
  ENDIF.

  DATA LV_UNLD_MATNR TYPE WD_THIS->Element_inbnd_header-UNLD_MATNR.
  LO_ND_INBND_HEADER = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS-
>WDCTX_INBND_HEADER ).
  LO_EL_INBND_HEADER = LO_ND_INBND_HEADER->GET_ELEMENT( ).

* get single attribute
  LO_EL_INBND_HEADER->GET_ATTRIBUTE(
   EXPORTING
    NAME = `UNLD_MATNR`
   IMPORTING
    VALUE = LV_UNLD_MATNR ).


 DATA lo_COMPONENTCONTROLLER TYPE REF TO IG_COMPONENTCONTROLLER .
 lo_COMPONENTCONTROLLER =  WD_THIS->GET_COMPONENTCONTROLLER_CTR( ).

  LO_COMPONENTCONTROLLER->EXECUTE_Z_GET_CUST_UNLOAD_INDE(
*   ind_date =                  " begda
*   kunnr =                    " kunnr
    cust_id = SYST-UNAME                  " char10
    matnr  = LV_UNLD_MATNR                    " matnr
  ).


endmethod.
```

- method ONACTIONUNLD_GET_STOCK .

```
  DATA: ZCUST_DTL TYPE ZSD_CUST_USR_MAP,
      zmchbo1 type mchbo1,
      zkdgrp type kdgrp,
      zcharg type charg.
  DATA LO_ND_INBND_HEADER TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LO_EL_INBND_HEADER TYPE REF TO IF_WD_CONTEXT_ELEMENT.
  DATA LS_INBND_HEADER TYPE WD_THIS->Element_inbnd_header.
  DATA LV_UNLD_STOCK TYPE WD_THIS->Element_inbnd_header-UNLD_STOCK.
  DATA LV_UNLD_STOCK15 TYPE WD_THIS->Element_inbnd_header-UNLD_STOCK15.
  DATA LV_UNLD_STOCKMT TYPE WD_THIS->Element_inbnd_header-UNLD_STOCKMT.

  DATA LV_LN_HLD_KL TYPE WD_THIS->Element_inbnd_header-LN_HLD_KL.
  DATA LV_LN_HLD_KL15 TYPE WD_THIS->Element_inbnd_header-LN_HLD_KL15.
  DATA LV_LN_HLD_MT TYPE WD_THIS->Element_inbnd_header-LN_HLD_MT.
  DATA LV_OPEN_INDT TYPE WD_THIS->Element_inbnd_header-OPEN_INDT.
  DATA LV_BAL_QTY TYPE WD_THIS->Element_inbnd_header-BAL_QTY.

  DATA LV_UNLD_MATNR TYPE WD_THIS->Element_inbnd_header-UNLD_MATNR.
  LO_ND_INBND_HEADER = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS-
>WDCTX_INBND_HEADER ).
  LO_EL_INBND_HEADER = LO_ND_INBND_HEADER->GET_ELEMENT( ).
```

```
*  set single attribute
  LO_EL_INBND_HEADER->GET_ATTRIBUTE(
     EXPORTING
      NAME =  `UNLD_MATNR`
     IMPORTING
      VALUE = LV_UNLD_MATNR ).

  clear: zmchbo1, ZCUST_DTL, zkdgrp, zcharg.
  select single * from zsd_cust_usr_map into ZCUST_DTL
   where CUST_USER_ID = sy-uname.
  select single kdgrp from knvv into zkdgrp where kunnr =
    ZCUST_DTL-kunnr and spart = '25'.
  IF ZKDGRP = 'BP'.
    zcharg = 'BPCL-NOVAL'.
  ENDIF.
  IF ZKDGRP = 'HP'.
    zcharg = 'HPCL-NOVAL'.
  ENDIF.
  IF ZKDGRP = 'EO'.
    zcharg = 'ESSR-NOVAL'.
  ENDIF.
  IF ZKDGRP = 'RI'.
    zcharg = 'RIL-NOVAL'.
  ENDIF.
  IF ZKDGRP = 'SH'.
    zcharg = 'SHEL-NOVAL'.
  ENDIF.
  IF ZKDGRP = 'IO'.
    zcharg = 'IOCL-NOVAL'.
  ENDIF.

  data: it_mchbo1 type standard table of mchbo1.
  data: ztank_stk type mchbo1-clabs,
      zline_stk type mchbo1-clabs.

  clear: zmchbo1, it_mchbo1, it_mchbo1[].
  select * from mchbo1 into zmchbo1 where
   matnr = LV_UNLD_MATNR and werks = ZCUST_DTL-DEPOT and
    charg = zcharg and msehi = 'L'.
   if sy-subrc = 0.
     select count(*) from T001L where werks = ZCUST_DTL-DEPOT
       and lgort = zmchbo1-lgort and oib_tnkassign = 'T'.
     if sy-subrc = 0.
       zmchbo1-lgort = 'TANK'.
       else.
         zmchbo1-lgort = 'LINE'.
     endif.
     append zmchbo1 to it_mchbo1.
   endif.
  endselect.
  clear: ztank_stk, zline_stk.
  loop at it_mchbo1 into zmchbo1.
   if zmchbo1-lgort = 'TANK'.
     ztank_stk = ztank_stk + zmchbo1-clabs + zmchbo1-cinsm.
```

```
 endif.
 if zmchbo1-lgort = 'LINE'.
  zline_stk = zline_stk + zmchbo1-clabs + zmchbo1-cinsm.
 endif.
endloop.
ztank_stk = ztank_stk / 1000.
zline_stk = zline_stk / 1000.
move ztank_stk to LV_UNLD_STOCK.
clear LV_BAL_QTY.
move ztank_stk to LV_BAL_QTY.
condense: LV_UNLD_STOCK, LV_BAL_QTY.
concatenate LV_UNLD_STOCK 'KL' into LV_UNLD_STOCK
 separated by space.
condense LV_UNLD_STOCK.
if zline_stk > 0.
 move zline_stk to LV_LN_HLD_KL.
 condense LV_LN_HLD_KL.
 concatenate LV_LN_HLD_KL 'KL' into LV_LN_HLD_KL
  separated by space.
 condense LV_LN_HLD_KL.
endif.
IF LV_UNLD_MATNR <> ' '.
 LO_EL_INBND_HEADER->SET_ATTRIBUTE(
  NAME = `UNLD_STOCK`
  VALUE = LV_UNLD_STOCK ).
 LO_EL_INBND_HEADER->SET_ATTRIBUTE(
  NAME = `LN_HLD_KL`
  VALUE = LV_LN_HLD_KL ).
ENDIF.

clear: zmchbo1, it_mchbo1, it_mchbo1[].
select * from mchbo1 into zmchbo1 where
 matnr = LV_UNLD_MATNR and werks = ZCUST_DTL-DEPOT and
  charg = zcharg and msehi = 'L15'.
 if sy-subrc = 0.
  select count(*) from T001L where werks = ZCUST_DTL-DEPOT
   and lgort = zmchbo1-lgort and oib_tnkassign = 'T'.
  if sy-subrc = 0.
   zmchbo1-lgort = 'TANK'.
   else.
    zmchbo1-lgort = 'LINE'.
  endif.
  append zmchbo1 to it_mchbo1.
 endif.
endselect.
clear: ztank_stk, zline_stk.
loop at it_mchbo1 into zmchbo1.
 if zmchbo1-lgort = 'TANK'.
  ztank_stk = ztank_stk + zmchbo1-clabs + zmchbo1-cinsm.
 endif.
 if zmchbo1-lgort = 'LINE'.
  zline_stk = zline_stk + zmchbo1-clabs + zmchbo1-cinsm.
 endif.
endloop.
```

```
ztank_stk = ztank_stk / 1000.
zline_stk = zline_stk / 1000.
move ztank_stk to LV_UNLD_STOCK15.
condense LV_UNLD_STOCK15.
concatenate LV_UNLD_STOCK15 'KL15' into LV_UNLD_STOCK15
 separated by space.
condense LV_UNLD_STOCK15.
if zline_stk > 0.
 move zline_stk to LV_LN_HLD_KL15.
 condense LV_LN_HLD_KL15.
 concatenate LV_LN_HLD_KL15 'KL15' into LV_LN_HLD_KL15
  separated by space.
 condense LV_LN_HLD_KL15.
endif.
IF LV_UNLD_MATNR <> ' '.
 LO_EL_INBND_HEADER->SET_ATTRIBUTE(
  NAME = `UNLD_STOCK15`
  VALUE = LV_UNLD_STOCK15 ).
 LO_EL_INBND_HEADER->SET_ATTRIBUTE(
  NAME = `LN_HLD_KL15`
  VALUE = LV_LN_HLD_KL15 ).
ENDIF.

clear: zmchbo1, it_mchbo1, it_mchbo1[].
select * from mchbo1 into zmchbo1 where
 matnr = LV_UNLD_MATNR and werks = ZCUST_DTL-DEPOT and
  charg = zcharg and msehi = 'KG'.
 if sy-subrc = 0.
  select count(*) from T001L where werks = ZCUST_DTL-DEPOT
   and lgort = zmchbo1-lgort and oib_tnkassign = 'T'.
  if sy-subrc = 0.
   zmchbo1-lgort = 'TANK'.
   else.
    zmchbo1-lgort = 'LINE'.
  endif.
  append zmchbo1 to it_mchbo1.
 endif.
endselect.
clear: ztank_stk, zline_stk.
loop at it_mchbo1 into zmchbo1.
 if zmchbo1-lgort = 'TANK'.
  ztank_stk = ztank_stk + zmchbo1-clabs + zmchbo1-cinsm.
 endif.
 if zmchbo1-lgort = 'LINE'.
  zline_stk = zline_stk + zmchbo1-clabs + zmchbo1-cinsm.
 endif.
endloop.
ztank_stk = ztank_stk / 1000.
zline_stk = zline_stk / 1000.
move ztank_stk to LV_UNLD_STOCKMT.
condense LV_UNLD_STOCKMT.
concatenate LV_UNLD_STOCKMT 'MT' into LV_UNLD_STOCKMT
 separated by space.
condense LV_UNLD_STOCKMT.
```

```
  if zline_stk > 0.
   move zline_stk to LV_LN_HLD_MT.
   condense LV_LN_HLD_MT.
   concatenate LV_LN_HLD_MT 'MT' into LV_LN_HLD_MT
    separated by space.
   condense LV_LN_HLD_MT.
  endif.
  IF LV_UNLD_MATNR <> ' '.
   LO_EL_INBND_HEADER->SET_ATTRIBUTE(
    NAME =  `UNLD_STOCKMT`
    VALUE = LV_UNLD_STOCKMT ).
   LO_EL_INBND_HEADER->SET_ATTRIBUTE(
    NAME =  `LN_HLD_MT`
    VALUE = LV_LN_HLD_MT ).
  ENDIF.

DATA: Z_IND TYPE ZSD_CUST_INDENT,
    Z_EBMS TYPE ZEBMS_SHIPMENT.
DATA: IT_CUST_IND TYPE STANDARD TABLE OF ZSD_CUST_INDENT,
    IT_EBMS TYPE STANDARD TABLE OF ZEBMS_SHIPMENT,
    ZIND_TOT_QTY TYPE LABST.
DATA ZBRND_CHK TYPE ZREBRAND.
DATA IT_REBRAND TYPE STANDARD TABLE OF ZREBRAND.
DATA IT_REBRAND1 TYPE STANDARD TABLE OF ZREBRAND.
DATA: ZIND_KL(12),
    ZIND_QTY TYPE LABST,
    ZIND_UOM(3).

clear: ZBRND_CHK, IT_REBRAND, IT_REBRAND[], IT_REBRAND1, IT_REBRAND1[].
select * from ZREBRAND into ZBRND_CHK.
 append ZBRND_CHK to IT_REBRAND.
endselect.
IT_REBRAND1[] = IT_REBRAND[].
loop at IT_REBRAND1 into ZBRND_CHK.
 concatenate '(BRND)' ZBRND_CHK-FPROD INTO ZBRND_CHK-FPROD.
 append ZBRND_CHK to IT_REBRAND.
endloop.

CLEAR: IT_CUST_IND, IT_CUST_IND[], IT_EBMS, IT_EBMS[], Z_IND, Z_EBMS.
*select * from zsd_cust_indent into Z_IND where
*  begda >= SY-DATUM and kdgrp = ZKDGRP and ztt_status <> 'D'.
select * from zsd_cust_indent into Z_IND where begda >= SY-DATUM
  and kdgrp = ZKDGRP and depot = ZCUST_DTL-DEPOT and ztt_status <> 'D'.
 if sy-subrc = 0 .
  append Z_IND to IT_CUST_IND.
 endif.
endselect.
select * from zebms_shipment into Z_EBMS where
    lddate = sy-datum and gi_doc <> ' '.
 if sy-subrc = 0.
  select count(*) from mseg where smbln = Z_EBMS-gi_doc.
   if sy-subrc <> 0.
    append Z_EBMS to IT_EBMS.
   endif.
```

```
 endif.
endselect.
loop at IT_EBMS into Z_EBMS.
 delete IT_CUST_IND where shnumber = Z_EBMS-shnumber.
endloop.
clear ZIND_TOT_QTY.
loop at IT_CUST_IND into Z_IND.
  read table IT_REBRAND into ZBRND_CHK with key fprod = Z_IND-PROD_CMP1.
  if sy-subrc = 0.
   clear: ZIND_KL, ZIND_UOM, ZIND_QTY.
   split Z_IND-QUAN_COMP1 at 'KL' into ZIND_KL ZIND_UOM.
   MOVE ZIND_KL TO ZIND_QTY.
   ZIND_QTY = ( ZIND_QTY * ZBRND_CHK-PERC2 ) / 100.
   ZIND_TOT_QTY = ZIND_TOT_QTY + ZIND_QTY.
  endif.
  read table IT_REBRAND into ZBRND_CHK with key fprod = Z_IND-PROD_CMP2.
  if sy-subrc = 0.
   clear: ZIND_KL, ZIND_UOM, ZIND_QTY.
   split Z_IND-QUAN_COMP2 at 'KL' into ZIND_KL ZIND_UOM.
   MOVE ZIND_KL TO ZIND_QTY.
   ZIND_QTY = ( ZIND_QTY * ZBRND_CHK-PERC2 ) / 100.
   ZIND_TOT_QTY = ZIND_TOT_QTY + ZIND_QTY.
  endif.
  read table IT_REBRAND into ZBRND_CHK with key fprod = Z_IND-PROD_CMP3.
  if sy-subrc = 0.
   clear: ZIND_KL, ZIND_UOM, ZIND_QTY.
   split Z_IND-QUAN_COMP3 at 'KL' into ZIND_KL ZIND_UOM.
   MOVE ZIND_KL TO ZIND_QTY.
   ZIND_QTY = ( ZIND_QTY * ZBRND_CHK-PERC2 ) / 100.
   ZIND_TOT_QTY = ZIND_TOT_QTY + ZIND_QTY.
  endif.
  read table IT_REBRAND into ZBRND_CHK with key fprod = Z_IND-PROD_CMP4.
  if sy-subrc = 0.
   clear: ZIND_KL, ZIND_UOM, ZIND_QTY.
   split Z_IND-QUAN_COMP4 at 'KL' into ZIND_KL ZIND_UOM.
   MOVE ZIND_KL TO ZIND_QTY.
   ZIND_QTY = ( ZIND_QTY * ZBRND_CHK-PERC2 ) / 100.
   ZIND_TOT_QTY = ZIND_TOT_QTY + ZIND_QTY.
  endif.
  read table IT_REBRAND into ZBRND_CHK with key fprod = Z_IND-PROD_CMP5.
  if sy-subrc = 0.
   clear: ZIND_KL, ZIND_UOM, ZIND_QTY.
   split Z_IND-QUAN_COMP5 at 'KL' into ZIND_KL ZIND_UOM.
   MOVE ZIND_KL TO ZIND_QTY.
   ZIND_QTY = ( ZIND_QTY * ZBRND_CHK-PERC2 ) / 100.
   ZIND_TOT_QTY = ZIND_TOT_QTY + ZIND_QTY.
  endif.
  read table IT_REBRAND into ZBRND_CHK with key fprod = Z_IND-PROD_CMP6.
  if sy-subrc = 0.
   clear: ZIND_KL, ZIND_UOM, ZIND_QTY.
   split Z_IND-QUAN_COMP6 at 'KL' into ZIND_KL ZIND_UOM.
   MOVE ZIND_KL TO ZIND_QTY.
   ZIND_QTY = ( ZIND_QTY * ZBRND_CHK-PERC2 ) / 100.
   ZIND_TOT_QTY = ZIND_TOT_QTY + ZIND_QTY.
```

```
  endif.
  read table IT_REBRAND into ZBRND_CHK with key fprod = Z_IND-PROD_CMP7.
  if sy-subrc = 0.
   clear: ZIND_KL, ZIND_UOM, ZIND_QTY.
   split Z_IND-QUAN_COMP7 at 'KL' into ZIND_KL ZIND_UOM.
   MOVE ZIND_KL TO ZIND_QTY.
   ZIND_QTY = ( ZIND_QTY * ZBRND_CHK-PERC2 ) / 100.
   ZIND_TOT_QTY = ZIND_TOT_QTY + ZIND_QTY.
  endif.
  read table IT_REBRAND into ZBRND_CHK with key fprod = Z_IND-PROD_CMP8.
  if sy-subrc = 0.
   clear: ZIND_KL, ZIND_UOM, ZIND_QTY.
   split Z_IND-QUAN_COMP8 at 'KL' into ZIND_KL ZIND_UOM.
   MOVE ZIND_KL TO ZIND_QTY.
   ZIND_QTY = ( ZIND_QTY * ZBRND_CHK-PERC2 ) / 100.
   ZIND_TOT_QTY = ZIND_TOT_QTY + ZIND_QTY.
  endif.
 endloop.
 CLEAR: LV_OPEN_INDT.
 MOVE ZIND_TOT_QTY TO LV_OPEN_INDT.
 LV_BAL_QTY = LV_BAL_QTY - ZIND_TOT_QTY.
 CONDENSE: LV_OPEN_INDT, LV_BAL_QTY.

 CONCATENATE LV_OPEN_INDT 'KL' INTO LV_OPEN_INDT SEPARATED BY SPACE.
 CONDENSE LV_OPEN_INDT.
 CONCATENATE LV_BAL_QTY 'KL' INTO LV_BAL_QTY SEPARATED BY SPACE.
 CONDENSE LV_BAL_QTY.
 IF LV_UNLD_MATNR <> ' '.
   LO_EL_INBND_HEADER->SET_ATTRIBUTE(
    NAME = `OPEN_INDT`
    VALUE = LV_OPEN_INDT ).
   LO_EL_INBND_HEADER->SET_ATTRIBUTE(
    NAME = `BAL_QTY`
    VALUE = LV_BAL_QTY ).
 ENDIF.



endmethod.
```

- method ONACTIONUNLD_MODIFY_QTY .

```
  DATA LO_ND_INBND_HEADER TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LO_EL_INBND_HEADER TYPE REF TO IF_WD_CONTEXT_ELEMENT.
  DATA LS_INBND_HEADER TYPE WD_THIS->Element_inbnd_header.
  DATA LV_UNLD_ACT2 TYPE WD_THIS->Element_inbnd_header-UNLD_ACT2.
  DATA LV_UNLD_ACT3 TYPE WD_THIS->Element_inbnd_header-UNLD_ACT3.
  DATA LV_MOD_QTY_KL TYPE WD_THIS->Element_inbnd_header-MOD_QTY_KL.
  DATA LV_MOD_QTY_KL15 TYPE WD_THIS->Element_inbnd_header-MOD_QTY_KL15.
  DATA LV_MOD_QTY_MT TYPE WD_THIS->Element_inbnd_header-MOD_QTY_MT.
  DATA LV_MOD_VEH TYPE WD_THIS->Element_inbnd_header-MOD_VEHICLE.
  DATA LV_MOD_INV TYPE WD_THIS->Element_inbnd_header-MOD_INV.
```

```
  LO_ND_INBND_HEADER = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS-
>WDCTX_INBND_HEADER ).
  LO_EL_INBND_HEADER = LO_ND_INBND_HEADER->GET_ELEMENT( ).
  DATA ZINB_IND TYPE ZINB_INDENT_TAB.

  LO_EL_INBND_HEADER->GET_ATTRIBUTE(
   EXPORTING
    NAME = `UNLD_ACT3`
   IMPORTING
    VALUE = LV_UNLD_ACT3 ).


  IF LV_UNLD_ACT3 = 'X'.
   LV_UNLD_ACT2 = 'X'.
   LO_EL_INBND_HEADER->SET_ATTRIBUTE(
    NAME = `UNLD_ACT2`
    VALUE = LV_UNLD_ACT2 ).
  ENDIF.

  DATA LO_ND_ZINDENT_DATA TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LO_EL_ZINDENT_DATA TYPE REF TO IF_WD_CONTEXT_ELEMENT.
  DATA LS_ZINDENT_DATA TYPE WD_THIS->Element_zindent_data.
  LO_ND_ZINDENT_DATA = WD_CONTEXT-
>PATH_GET_NODE( PATH = `Z_GET_CUST_UNLOAD_IN.CHANGING_3.ZINDENT_DATA` ).
  LO_EL_ZINDENT_DATA = LO_ND_ZINDENT_DATA->GET_ELEMENT( ).

  IF NOT LO_EL_ZINDENT_DATA IS INITIAL.

*   get all declared attributes
    LO_EL_ZINDENT_DATA->GET_STATIC_ATTRIBUTES(
     IMPORTING
      STATIC_ATTRIBUTES = LS_ZINDENT_DATA ).

    clear ZINB_IND.
    select single * from ZINB_INDENT_TAB into ZINB_IND where
     ind_date = LS_ZINDENT_DATA-ind_date and ind_matnr = LS_ZINDENT_DATA-ind_matnr
      and ind_veh = LS_ZINDENT_DATA-ind_veh.

    LV_MOD_QTY_KL    = ZINB_IND-IND_QTY.
    LV_MOD_QTY_KL15  = ZINB_IND-IND_QTY15.
    LV_MOD_QTY_MT    = ZINB_IND-IND_QTYMT.
    LV_MOD_VEH       = ZINB_IND-IND_VEH.
    LV_MOD_INV       = ZINB_IND-IND_INV.

    LO_EL_INBND_HEADER->SET_ATTRIBUTE(
     NAME = `MOD_QTY_KL`
     VALUE = LV_MOD_QTY_KL ).

    LO_EL_INBND_HEADER->SET_ATTRIBUTE(
     NAME = `MOD_QTY_KL15`
     VALUE = LV_MOD_QTY_KL15 ).

    LO_EL_INBND_HEADER->SET_ATTRIBUTE(
     NAME = `MOD_QTY_MT`
```

```
      VALUE = LV_MOD_QTY_MT ).

     LO_EL_INBND_HEADER->SET_ATTRIBUTE(
      NAME = `MOD_VEHICLE`
      VALUE = LV_MOD_VEH ).
*
     LO_EL_INBND_HEADER->SET_ATTRIBUTE(
      NAME = `MOD_INV`
      VALUE = LV_MOD_INV ).
   ENDIF.
endmethod.
```

- method ONACTIONUNLD_OPEN_INDENTS .

```
   DATA LO_ND_INBND_HEADER TYPE REF TO IF_WD_CONTEXT_NODE.
   DATA LO_EL_INBND_HEADER TYPE REF TO IF_WD_CONTEXT_ELEMENT.
   DATA LS_INBND_HEADER TYPE WD_THIS->Element_inbnd_header.
   DATA LV_UNLD_MATNR TYPE WD_THIS->Element_inbnd_header-UNLD_MATNR.
   DATA LV_UNLD_ACT3 TYPE WD_THIS->Element_inbnd_header-UNLD_ACT3.


   LO_ND_INBND_HEADER = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS-
>WDCTX_INBND_HEADER ).
   LO_EL_INBND_HEADER = LO_ND_INBND_HEADER->GET_ELEMENT( ).

*  get single attribute
   LO_EL_INBND_HEADER->GET_ATTRIBUTE(
    EXPORTING
     NAME = `UNLD_MATNR`
    IMPORTING
     VALUE = LV_UNLD_MATNR ).


   DATA LV_UNLD_DATE TYPE WD_THIS->Element_inbnd_header-UNLD_DATE.
   DATA LV_UNLD_TDATE TYPE WD_THIS->Element_inbnd_header-UNLD_TDATE.
   LO_ND_INBND_HEADER = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS-
>WDCTX_INBND_HEADER ).
   LO_EL_INBND_HEADER = LO_ND_INBND_HEADER->GET_ELEMENT( ).

   CLEAR: LV_UNLD_DATE, LV_UNLD_TDATE.
    LO_EL_INBND_HEADER->SET_ATTRIBUTE(
     NAME = `UNLD_DATE`
     VALUE = LV_UNLD_DATE ).

    LO_EL_INBND_HEADER->SET_ATTRIBUTE(
     NAME = `UNLD_TDATE`
     VALUE = LV_UNLD_TDATE ).

    LV_UNLD_ACT3 = 'X'.
    LO_EL_INBND_HEADER->SET_ATTRIBUTE(
     NAME = `UNLD_ACT3`
     VALUE = LV_UNLD_ACT3 ).
```

```
DATA lo_COMPONENTCONTROLLER TYPE REF TO IG_COMPONENTCONTROLLER .
lo_COMPONENTCONTROLLER =  WD_THIS->GET_COMPONENTCONTROLLER_CTR( ).

  LO_COMPONENTCONTROLLER->EXECUTE_Z_GET_CUST_UNLOAD_INDE(
*   ind_date =                  " begda
*   kunnr =                     " kunnr
    cust_id = SYST-UNAME                  " char10
    matnr   = LV_UNLD_MATNR               " matnr
  ).


endmethod.
```

- method ONACTIONUNLD_SAVE_DATA .

```
  DATA LT_INDT TYPE ZINB_INDENT_TAB.
  DATA LO_ND_INBND_HEADER TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LO_EL_INBND_HEADER TYPE REF TO IF_WD_CONTEXT_ELEMENT.
  DATA LS_INBND_HEADER TYPE WD_THIS->Element_inbnd_header.
  DATA LV_UNLD_VEHICLE TYPE WD_THIS->Element_inbnd_header-UNLD_VEHICLE.
  DATA LV_UNLD_DATE TYPE WD_THIS->Element_inbnd_header-UNLD_DATE.
  DATA LV_UNLD_KUNNR TYPE WD_THIS->Element_inbnd_header-UNLD_KUNNR.
  DATA LV_UNLD_INV TYPE WD_THIS->Element_inbnd_header-UNLOAD_INV.
  DATA LV_UNLD_MATNR TYPE WD_THIS->Element_inbnd_header-UNLD_MATNR.
  DATA LV_UNLD_PLANT TYPE WD_THIS->Element_inbnd_header-UNLD_PLANT.
  DATA LV_UNLD_QTY TYPE WD_THIS->Element_inbnd_header-UNLD_QTY.
  DATA LV_UNLD_QTY15 TYPE WD_THIS->Element_inbnd_header-UNLD_QTY_L15.
  DATA LV_UNLD_QTYMT TYPE WD_THIS->Element_inbnd_header-UNLD_QTY_KG.


  DATA LO_WINDOW_MANAGER TYPE REF TO IF_WD_WINDOW_MANAGER.
  DATA LO_API_COMPONENT  TYPE REF TO IF_WD_COMPONENT.
  DATA LO_WINDOW       TYPE REF TO IF_WD_WINDOW.
  DATA: lt_string type table of string,
     ls_string like line of lt_string.
  DATA l_api          TYPE REF TO if_wd_view_controller.
  DATA: er_txt type string,
     zgo type sy-subrc.
  data: num_str(200) type C,
     num_out type DD01V-DATATYPE,
     err_string_l like line of lt_string,
```

```
      err_string_15 like line of lt_string,
      err_string_mt like line of lt_string.

  LO_ND_INBND_HEADER = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS-
>WDCTX_INBND_HEADER ).
  LO_EL_INBND_HEADER = LO_ND_INBND_HEADER->GET_ELEMENT( ).

  IF LO_EL_INBND_HEADER IS INITIAL.
  ENDIF.

* get single attribute
  LO_EL_INBND_HEADER->GET_ATTRIBUTE(
   EXPORTING
    NAME = `UNLD_VEHICLE`
   IMPORTING
    VALUE = LV_UNLD_VEHICLE ).

  LO_EL_INBND_HEADER->GET_ATTRIBUTE(
   EXPORTING
    NAME = `UNLD_DATE`
   IMPORTING
    VALUE = LV_UNLD_DATE ).

  LO_EL_INBND_HEADER->GET_ATTRIBUTE(
   EXPORTING
    NAME = `UNLD_KUNNR`
   IMPORTING
    VALUE = LV_UNLD_KUNNR ).

  LO_EL_INBND_HEADER->GET_ATTRIBUTE(
   EXPORTING
    NAME = `UNLD_MATNR`
   IMPORTING
    VALUE = LV_UNLD_MATNR ).

  LO_EL_INBND_HEADER->GET_ATTRIBUTE(
   EXPORTING
    NAME = `UNLD_QTY`
   IMPORTING
    VALUE = LV_UNLD_QTY ).


  LO_EL_INBND_HEADER->GET_ATTRIBUTE(
   EXPORTING
    NAME = `UNLD_QTY_L15`
   IMPORTING
    VALUE = LV_UNLD_QTY15 ).

  LO_EL_INBND_HEADER->GET_ATTRIBUTE(
   EXPORTING
    NAME = `UNLD_QTY_KG`
   IMPORTING
    VALUE = LV_UNLD_QTYMT ).
```

```
LO_EL_INBND_HEADER->GET_ATTRIBUTE(
 EXPORTING
  NAME =  `UNLD_PLANT`
 IMPORTING
  VALUE = LV_UNLD_PLANT ).

LO_EL_INBND_HEADER->GET_ATTRIBUTE(
 EXPORTING
  NAME =  `UNLOAD_INV`
 IMPORTING
  VALUE = LV_UNLD_INV ).

select single depot kunnr from ZSD_CUST_USR_MAP into
 ( LV_UNLD_PLANT, LV_UNLD_KUNNR ) where CUST_USER_ID = sy-uname.

if LV_UNLD_QTY <> ' '.
 clear: num_str, err_string_l.
 move LV_UNLD_QTY to num_str.
 replace first occurrence of '.' in num_str with ' '.
 condense num_str.
 call function 'NUMERIC_CHECK'
    exporting
      STRING_IN = num_str
    importing
      HTYPE    = num_out.
 if num_out <> 'NUMC'.
  move 'KL quantity format is incorrect' to err_string_l.
 endif.
endif.

if LV_UNLD_QTY15 <> ' '.
 clear: num_str, err_string_15.
 move LV_UNLD_QTY15 to num_str.
 replace first occurrence of '.' in num_str with ' '.
 condense num_str.
 call function 'NUMERIC_CHECK'
    exporting
      STRING_IN = num_str
    importing
      HTYPE    = num_out.
 if num_out <> 'NUMC'.
  move 'KL15 quantity format is incorrect' to err_string_15.
 endif.
endif.

if LV_UNLD_QTYMT <> ' '.
 clear: num_str, err_string_mt.
 move LV_UNLD_QTYMT to num_str.
 replace first occurrence of '.' in num_str with ' '.
 condense num_str.
 call function 'NUMERIC_CHECK'
    exporting
      STRING_IN = num_str
    importing
```

```
        HTYPE    = num_out.
   if num_out <> 'NUMC'.
    move 'MT quantity format is incorrect' to err_string_mt.
   endif.
  endif.

  IF err_string_l <> ' ' or err_string_15 <> ' ' or err_string_mt <> ' '.
   insert err_string_l into table lt_string.
   insert err_string_15 into table lt_string.
   insert err_string_mt into table lt_string.
    LO_API_COMPONENT         = WD_COMP_CONTROLLER->WD_GET_API( ).
    LO_WINDOW_MANAGER        = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).

    LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
          text         = lt_string
          button_kind    = if_wd_window=>co_buttons_ok
*          button_kind    = 4
          message_type   = if_wd_window=>CO_MSG_TYPE_ERROR
          window_title   = 'Quantity format error'
          window_position = if_wd_window=>co_center
          WINDOW_WIDTH   = '60%'
          WINDOW_HEIGHT  = '30%' ).
    LO_WINDOW->open( ).
  ENDIF.

 IF err_string_l = ' ' and err_string_15 = ' ' and err_string_mt = ' '.
  CLEAR LT_INDT.
  LT_INDT-IND_DATE  = SY-DATUM.
  LT_INDT-IND_MATNR = LV_UNLD_MATNR.
  LT_INDT-IND_VEH   = LV_UNLD_VEHICLE.
  LT_INDT-IND_QTY   = LV_UNLD_QTY.
  LT_INDT-IND_PLANT = LV_UNLD_PLANT.
  LT_INDT-IND_CUST  = LV_UNLD_KUNNR.
  LT_INDT-IND_INV   = LV_UNLD_INV.
  LT_INDT-IND_QTY15 = LV_UNLD_QTY15.
  LT_INDT-IND_QTYMT = LV_UNLD_QTYMT.

  select single * from ZINB_INDENT_TAB into LT_INDT where
   ind_veh = LV_UNLD_VEHICLE and zstatus <> 'COMPLETE'.

  if sy-subrc <> 0.
   if LV_UNLD_VEHICLE <> ' ' and LV_UNLD_QTY <> ' ' and LV_UNLD_INV <> ' '.
    "and LV_UNLD_QTY15 <> ' ' and LV_UNLD_QTYMT <> ' '.
    INSERT ZINB_INDENT_TAB FROM LT_INDT.
    zgo = sy-subrc.
    else.
     zgo = 4.
   endif.

   else.
   ls_string = | { 'Open Indent available for the vehicle on' } { LT_INDT-IND_DATE+6(2) }{ '.' } |.
   ls_string = | { ls_string }{ LT_INDT-IND_DATE+4(2) }{ '.' }{ LT_INDT-IND_DATE+0(4) } |.
   ls_string = | { ls_string }{ '.' } |.
   zgo = 4.
```

```
    insert ls_string into table lt_string.
   endif.

   clear er_txt.
   if LV_UNLD_VEHICLE = ' '.
    move 'Please enter TT number' to er_txt.
   endif.
   if LV_UNLD_QTY = ' '.
    move 'Please enter KL quantity' to er_txt.
   endif.

*   if LV_UNLD_QTY15 = ' '.
*    move 'Please enter KL-15 quantity' to er_txt.
*   endif.
*   if LV_UNLD_QTYMT = ' '.
*    move 'Please enter MT quantity' to er_txt.
*   endif.
   if LV_UNLD_INV = ' '.
    move 'Please vendor invoice number' to er_txt.
   endif.

   IF zgo = 0.
    ls_string = 'Data updated successfully'.
    insert ls_string into table lt_string.
    LO_API_COMPONENT        = WD_COMP_CONTROLLER->WD_GET_API( ).
    LO_WINDOW_MANAGER       = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).

    LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
          text         = lt_string
          button_kind    = if_wd_window=>co_buttons_ok
*          button_kind    = 4
          message_type   = if_wd_window=>CO_MSG_TYPE_INFORMATION
          window_title   = 'Success'
          window_position = if_wd_window=>co_center
          WINDOW_WIDTH   = '20%'
          WINDOW_HEIGHT  = '20%' ).
    LO_WINDOW->open( ).

    ELSE.
     select count(*) from ZINB_INDENT_TAB where ind_date = sy-datum and
      ind_matnr = LV_UNLD_MATNR and IND_VEH   = LV_UNLD_VEHICLE.
     if sy-subrc = 0.
      ls_string = 'Indent already available for the TT'.
      else.
       ls_string = er_txt.
     endif.
     insert ls_string into table lt_string.
     LO_API_COMPONENT        = WD_COMP_CONTROLLER->WD_GET_API( ).
     LO_WINDOW_MANAGER       = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).

     LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
           text         = lt_string
           button_kind    = if_wd_window=>co_buttons_ok
*           button_kind    = 4
```

```
            message_type    = if_wd_window=>CO_MSG_TYPE_STOPP
            window_title    = 'Update Fail'
            window_position = if_wd_window=>co_center
            WINDOW_WIDTH    = '60%'
            WINDOW_HEIGHT   = '30%' ).
      LO_WINDOW->open( ).
    ENDIF.
   ENDIF.


  endmethod.
```

- ```
  method ONACTIONUNLD_UPD_TABLE .
   DATA LO_ND_INBND_HEADER TYPE REF TO IF_WD_CONTEXT_NODE.
   DATA LO_EL_INBND_HEADER TYPE REF TO IF_WD_CONTEXT_ELEMENT.
   DATA LS_INBND_HEADER TYPE WD_THIS->Element_inbnd_header.
   DATA LV_UNLD_ACT2 TYPE WD_THIS->Element_inbnd_header-UNLD_ACT2.
   DATA LV_UNLD_ACT3 TYPE WD_THIS->Element_inbnd_header-UNLD_ACT3.
   DATA LV_MOD_QTY_KL TYPE WD_THIS->Element_inbnd_header-MOD_QTY_KL.
   DATA LV_MOD_QTY_KL15 TYPE WD_THIS->Element_inbnd_header-MOD_QTY_KL15.
   DATA LV_MOD_QTY_MT TYPE WD_THIS->Element_inbnd_header-MOD_QTY_MT.
   DATA LV_MOD_VEH TYPE WD_THIS->Element_inbnd_header-MOD_VEHICLE.
   DATA LV_MOD_INV TYPE WD_THIS->Element_inbnd_header-MOD_INV.
   DATA DEL_VEH TYPE OIG_VHLNMR.

   DATA LO_WINDOW_MANAGER TYPE REF TO IF_WD_WINDOW_MANAGER.
   DATA LO_API_COMPONENT  TYPE REF TO IF_WD_COMPONENT.
   DATA LO_WINDOW       TYPE REF TO IF_WD_WINDOW.
   DATA: lt_string type table of string,
      ls_string like line of lt_string.
   DATA l_api           TYPE REF TO if_wd_view_controller.


    LO_ND_INBND_HEADER = WD_CONTEXT->GET_CHILD_NODE( NAME = WD_THIS-
  >WDCTX_INBND_HEADER ).
    LO_EL_INBND_HEADER = LO_ND_INBND_HEADER->GET_ELEMENT( ).
    DATA ZINB_IND TYPE ZINB_INDENT_TAB.

    LO_EL_INBND_HEADER->GET_ATTRIBUTE(
     EXPORTING
      NAME = `UNLD_ACT3`
     IMPORTING
      VALUE = LV_UNLD_ACT3 ).


    IF LV_UNLD_ACT3 = 'X'.
     LV_UNLD_ACT2 = 'X'.
     LO_EL_INBND_HEADER->SET_ATTRIBUTE(
      NAME = `UNLD_ACT2`
      VALUE = LV_UNLD_ACT2 ).
    ENDIF.

    DATA LO_ND_ZINDENT_DATA TYPE REF TO IF_WD_CONTEXT_NODE.
  ```
  ```

```
  DATA LO_EL_ZINDENT_DATA TYPE REF TO IF_WD_CONTEXT_ELEMENT.
  DATA LS_ZINDENT_DATA TYPE WD_THIS->Element_zindent_data.
  LO_ND_ZINDENT_DATA = WD_CONTEXT-
>PATH_GET_NODE( PATH = `Z_GET_CUST_UNLOAD_IN.CHANGING_3.ZINDENT_DATA` ).
  LO_EL_ZINDENT_DATA = LO_ND_ZINDENT_DATA->GET_ELEMENT( ).

  IF NOT LO_EL_ZINDENT_DATA IS INITIAL.

*   get all declared attributes
    LO_EL_ZINDENT_DATA->GET_STATIC_ATTRIBUTES(
     IMPORTING
      STATIC_ATTRIBUTES = LS_ZINDENT_DATA ).
  ENDIF.
    clear ZINB_IND.
    select single * from ZINB_INDENT_TAB into ZINB_IND where
     ind_date = LS_ZINDENT_DATA-ind_date and ind_matnr = LS_ZINDENT_DATA-ind_matnr
      and ind_veh = LS_ZINDENT_DATA-ind_veh.


    LO_EL_INBND_HEADER->GET_ATTRIBUTE(
      EXPORTING
       NAME =  `MOD_QTY_KL`
      IMPORTING
       VALUE = LV_MOD_QTY_KL ).

    LO_EL_INBND_HEADER->GET_ATTRIBUTE(
      EXPORTING
       NAME =  `MOD_QTY_KL15`
      IMPORTING
       VALUE = LV_MOD_QTY_KL15 ).

    LO_EL_INBND_HEADER->GET_ATTRIBUTE(
      EXPORTING
       NAME =  `MOD_QTY_MT`
      IMPORTING
       VALUE = LV_MOD_QTY_MT ).

    LO_EL_INBND_HEADER->GET_ATTRIBUTE(
      EXPORTING
       NAME =  `MOD_VEHICLE`
      IMPORTING
       VALUE = LV_MOD_VEH ).

    LO_EL_INBND_HEADER->GET_ATTRIBUTE(
      EXPORTING
       NAME =  `MOD_INV`
      IMPORTING
       VALUE = LV_MOD_INV ).

    ZINB_IND-IND_QTY    = LV_MOD_QTY_KL.
    ZINB_IND-IND_QTY15  = LV_MOD_QTY_KL15.
    ZINB_IND-IND_QTYMT  = LV_MOD_QTY_MT.

    CLEAR DEL_VEH.
```

```
  MOVE ZINB_IND-IND_VEH TO DEL_VEH.
  ZINB_IND-IND_VEH    = LV_MOD_VEH.
  ZINB_IND-IND_INV    = LV_MOD_INV.

 if  LV_UNLD_ACT3 = 'X'.
  if ZINB_IND-ZSTATUS  <> 'COMPLETE'.
   DELETE FROM ZINB_INDENT_TAB WHERE IND_DATE = ZINB_IND-IND_DATE
    AND IND_MATNR = ZINB_IND-IND_MATNR AND IND_VEH = DEL_VEH.
   MODIFY ZINB_INDENT_TAB FROM ZINB_IND.
   else.
    sy-subrc = 4.
  endif.

  IF SY-SUBRC = 0.
   ls_string = 'Data updated successfully'.
   insert ls_string into table lt_string.
   LO_API_COMPONENT         = WD_COMP_CONTROLLER->WD_GET_API( ).
   LO_WINDOW_MANAGER        = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).

   LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
          text         = lt_string
          button_kind    = if_wd_window=>co_buttons_ok
*          button_kind    = 4
          message_type    = if_wd_window=>CO_MSG_TYPE_INFORMATION
          window_title    = 'Success'
          window_position = if_wd_window=>co_center
          WINDOW_WIDTH    = '20%'
          WINDOW_HEIGHT   = '20%' ).
   LO_WINDOW->open( ).

   ELSE.
    ls_string = 'Data update failed'.
   insert ls_string into table lt_string.
   LO_API_COMPONENT         = WD_COMP_CONTROLLER->WD_GET_API( ).
   LO_WINDOW_MANAGER        = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).

   LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
          text         = lt_string
          button_kind    = if_wd_window=>co_buttons_ok
*          button_kind    = 4
          message_type    = if_wd_window=>CO_MSG_TYPE_STOPP
          window_title    = 'Update Fail'
          window_position = if_wd_window=>co_center
          WINDOW_WIDTH    = '20%'
          WINDOW_HEIGHT   = '20%' ).
   LO_WINDOW->open( ).
  ENDIF.

  else.
   ls_string = 'Please click on Open Indents first'.
   insert ls_string into table lt_string.
   LO_API_COMPONENT         = WD_COMP_CONTROLLER->WD_GET_API( ).
   LO_WINDOW_MANAGER        = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).
```

```
            LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
                  text         = lt_string
                  button_kind    = if_wd_window=>co_buttons_ok
          *            button_kind    = 4
                  message_type   = if_wd_window=>CO_MSG_TYPE_INFORMATION
                  window_title   = 'Success'
                  window_position = if_wd_window=>co_center
                  WINDOW_WIDTH   = '20%'
                  WINDOW_HEIGHT  = '20%' ).
            LO_WINDOW->open( ).
          endif.
        endmethod.

method ONACTIONDELETE_ORDER .
  DATA LO_WINDOW_MANAGER TYPE REF TO IF_WD_WINDOW_MANAGER.
  DATA LO_API_COMPONENT  TYPE REF TO IF_WD_COMPONENT.
  DATA LO_WINDOW       TYPE REF TO IF_WD_WINDOW.
  data: lt_string type table of string,
    ls_string like line of lt_string.
  DATA l_api          TYPE REF TO if_wd_view_controller.
  DATA: CNT TYPE I.
  DATA: DEL(1) TYPE C.

  DATA LO_ND_CHANGING_2 TYPE REF TO IF_WD_CONTEXT_NODE.
  DATA LT_CHANGING_2 TYPE WD_THIS->ELEMENTS_CHANGING_2.
  DATA LS_CHANGING_2 TYPE WD_THIS->ELEMENT_CHANGING_2.
  LO_ND_CHANGING_2 = WD_CONTEXT->PATH_GET_NODE( PATH = `ZSD_INDENT_NON_VEH.CHANGING_2` ).
  LO_ND_CHANGING_2-
>GET_STATIC_ATTRIBUTES_TABLE( IMPORTING TABLE = LT_CHANGING_2 ).  "#EC CI_FLDEXT_OK[2215424] " #S/4 - O
SS Note# - Added by - Z_SHSHRUKHA – 2021/08/18

  CLEAR CNT.
  LOOP AT LT_CHANGING_2 INTO LS_CHANGING_2 WHERE CHK = 'X'.
   CNT = CNT + 1.
  ENDLOOP.

  LO_API_COMPONENT       = WD_COMP_CONTROLLER->WD_GET_API( ).
  LO_WINDOW_MANAGER      = LO_API_COMPONENT->GET_WINDOW_MANAGER( ).
  CLEAR: ls_string, lt_string.
  REFRESH lt_string.

  IF CNT = 0.
   ls_string = 'Please select the order to be deleted'.
   insert ls_string into table lt_string.

   ELSEIF CNT > 1.
    ls_string = 'Please select only one order'.
    insert ls_string into table lt_string.
  ENDIF.

  CLEAR DEL.
  IF CNT = 1.
   SELECT SINGLE * FROM ZSD_INDENT_NVEH INTO LS_CHANGING_2 WHERE ORDER_NO = LS_CHANGING_2-
ORDER_NO.  "#EC CI_NOORDER " #S/4 - OSS Note# - Added by - Z_SHSHRUKHA – 2021/08/18
   IF LS_CHANGING_2-QUANTITY1 <> LS_CHANGING_2-BL_QTY1 OR LS_CHANGING_2-
```

```
QUANTITY2 <> LS_CHANGING_2-BL_QTY2.
      DEL = 'N'.
    ENDIF.
  ENDIF.

  IF CNT <> 1.
    LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
          text         = lt_string
          button_kind    = if_wd_window=>co_buttons_ok
          message_type   = if_wd_window=>CO_MSG_TYPE_STOPP
          window_title   = 'Please check!'
          window_position = if_wd_window=>co_center ).

    LO_WINDOW->open( ).
  ENDIF.

  IF DEL = 'N'.
    CLEAR: ls_string, lt_string. REFRESH lt_string.
    ls_string = 'Selected order has been processed and cannot de deleted'.
    insert ls_string into table lt_string.
    LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
          text         = lt_string
          button_kind    = if_wd_window=>co_buttons_ok
          message_type   = if_wd_window=>CO_MSG_TYPE_STOPP
          window_title   = 'Please check!'
          window_position = if_wd_window=>co_center ).

    LO_WINDOW->open( ).
  ENDIF.

  IF CNT = 1 AND DEL = ' '.
    CLEAR: ls_string, lt_string. REFRESH lt_string.
    ls_string = 'Selected order will be deleted?'.
    insert ls_string into table lt_string.

     LO_WINDOW = LO_WINDOW_MANAGER->create_popup_to_confirm(
          text         = lt_string
          button_kind    = 4
          message_type   = if_wd_window=>CO_MSG_TYPE_QUESTION
          window_title   = 'Please check!'
          window_position = if_wd_window=>co_center ).

  DATA LO_API_START_VIEW TYPE REF TO IF_WD_VIEW_CONTROLLER.
  LO_API_START_VIEW = WD_THIS->WD_GET_API( ).

  CALL METHOD LO_WINDOW->SUBSCRIBE_TO_BUTTON_EVENT
     EXPORTING
      BUTTON    = if_wd_window=>co_button_yes
      ACTION_NAME = 'REACT_TO_DEL_ORDER'
      ACTION_VIEW = LO_API_START_VIEW.

    LO_WINDOW->open( ).

    CALL METHOD LO_WINDOW_MANAGER->CREATE_POPUP_TO_CONFIRM
```

```
        EXPORTING
         TEXT              = lt_string
         BUTTON_KIND       = 4
        RECEIVING
         RESULT            = lo_window.


    ENDIF.


endmethod.
```